System Design Patterns — Vol 4: Case Studies II and Interview Practice

Developer and AI platforms, edge and messaging building blocks, AI systems, commerce/media/platforms, real outage postmortems, then the interview question bank with prep plans (Modules 21, 23-26, 22)

6 modules · 60+ patterns · every lesson explained, with course diagrams and links to the originals

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes summarised from the DesignGurus course (Arslan Ahmad). Not for redistribution.

Contents

1. **Module 21 — Case Studies: Developer and AI Platforms**
   - Module Introduction
   - Case 1 — Online Judge (LeetCode-like)
   - Case 2 — Global Deployment System
   - Case 3 — Conversational AI Service (ChatGPT-like)
   - Case 4 — Distributed Cache Cluster (Redis Cluster / Memcached-like)
2. **Module 23 — Case Studies: Edge and Messaging Building Blocks**
   - Case 1 — Design a CDN
   - Case 2 — Design a Rate Limiter
   - Case 3 — Design a Pub-Sub Service
3. **Module 24 — Case Studies: AI Systems**
   - How AI reshaped system design
   - Case 1 — AI-Powered Code Assistant
   - Case 2 — LLM Customer Support Bot
   - Case 3 — Data Infrastructure for AI/ML
4. **Module 25 — Case Studies: Commerce, Media and Platforms**
   - Case 1 — TikTok-Style Short-Video Feed
   - Case 2 — Food Delivery (Uber Eats / DoorDash style)
   - Case 3 — E-Commerce Platform and Peak-Day (Prime Day) Readiness
   - Case 4 — Online Education Platform (Coursera / Udemy style)
   - Case 5 — Spotify Wrapped: a Yearly Batch Personalisation Pipeline
   - Case 6 — A Twilio-Style Communications API Platform
5. **Module 26 — When Systems Fail: Postmortems as Patterns**
   - How to read a postmortem
   - 2021 — Facebook BGP and DNS outage
   - 2020 — AWS Kinesis outage (us-east-1)
   - 2021 — AWS us-east-1 internal network congestion
   - 2025 — AWS us-east-1 DynamoDB DNS outage
   - 2017 — GitLab.com database deletion
   - 2019 — Cloudflare WAF regex outage
   - 2024 — CrowdStrike Channel File 291
   - Patterns across all the outages
6. **Module 22 — Interview Question Bank**
   - How to use this bank
   - Part A — "Design X" questions
   - Building blocks
   - Social and content
   - Counting, ranking and analytics
   - Messaging and real-time
   - Location
   - Media
   - Files and collaboration
   - Commerce and payments
   - Infrastructure
   - Developer and AI platforms
   - Other
   - Part B — Concept questions
   - Performance and scale
   - Networking and APIs
   - Load balancing and caching
   - Databases and storage
   - Replication and partitioning
   - Consistency and consensus
   - Messaging and streaming
   - Reliability and resilience
   - Security
   - Operations
   - Algorithms and data structures
   - More design prompts
   - Part C — How to answer a system design question in 45 minutes
   - Part D — Preparing and practising
   - Sources of the question lists

Module 21 — Case Studies: Developer and AI Platforms

Module Introduction

Four more end-to-end designs in the format of Modules 13–17 and 20, for platform systems common in modern interviews. The algorithms they rely on are explained step by step in Module 19.

Case	Centre of gravity
1 — Online Judge (LeetCode-like)	Running untrusted code safely and fairly: sandboxes, resource limits, judge queues, contest leaderboards
2 — Global Deployment System	Getting one build to many servers safely: artifact distribution, rollout waves, health gates, rollback
3 — Conversational AI Service (ChatGPT-like)	Token streaming, GPU batching, KV-cache-aware routing, quotas and cost
4 — Distributed Cache Cluster	Hash slots, replication and failover, hot keys, routing, eviction and resharding

Case 1 — Online Judge (LeetCode-like)

The prompt (in short). A LeetCode/Codeforces/HackerRank-style site: users write code in one of ~20 languages, press Submit, and within seconds see "Accepted", "Wrong Answer", "Time Limit Exceeded" and so on. Several times a week a timed **contest** (e.g. 90 minutes, same problems for everyone) runs with a live leaderboard.

What makes it hard. (1) It **runs code written by strangers**: any submission may try to read others' code, open connections, fork a million processes or fill the disk, so every submission is assumed hostile. (2) Load is **extremely spiky**: tens of thousands crowd into the same 90 minutes, bunching at the start and end. (3) Results must be **fair and repeatable**: same code, same verdict and roughly the same time on any machine, or the leaderboard is meaningless.

1. Requirements and numbers

Functional: browse problems; submit in a chosen language; compile, run against hidden tests and return a **verdict** (Accepted, Wrong Answer, Time Limit Exceeded, Memory Limit Exceeded, Runtime Error, Compile Error) with runtime and memory; a "Run" button on the user's own sample input; submission history; contests with registration, start/end, live leaderboard and final ratings; post-contest plagiarism checks.

Non-functional: security first (no sandbox escape, no other users' data, no network); verdict **p95 < 5 s** outside contests and **< 15 s** at contest peak; **determinism** (same verdict, time within a few percent); **no lost submissions** (after "Pending", a verdict always appears); fairness (contest submissions judged in arrival order, never starved by practice traffic).

Numbers (assumptions, rounded):

- **20M registered, 2M daily active**; **1M practice submissions/day ≈ 12/s**, ~40/s at the daily peak.
- **Contest:** 50K participants × 4 problems × ~2.5 attempts = **500K submissions in 90 min ≈ 93/s**; first and last 10 minutes ~5× → **~500/s peak**.
- **One submission:** ~1 s CPU compile + ~50 tests × ~60 ms ≈ **~4 CPU-seconds**. Peak 500/s × 4 = **2,000 cores busy**; one dedicated core per sandbox (Deep dive 2) and 30% headroom → **~2,600 cores ≈ 80 32-core workers**. Off-contest 40/s × 4 = 160 cores, so the fleet **scales down ~10×** between contests.
- **Code:** 3 KB average → **3–5 GB/day ≈ 1.5 TB/year**: tiny.
- **Test data:** 3,000 problems × 50 tests × ~1 MB (some 100 MB) ≈ **150 GB**; any worker may need any problem, but a contest touches 4 problems (~200 MB), which fits in RAM.
- **Leaderboard:** participants plus spectators refreshing every 10–30 s → **~5K reads/s** of a ranking changing ~100 times/s.

What the numbers say: storage is trivial; **CPU for untrusted execution**, the **10× contest spike** and the **isolation boundary** decide the architecture.

2. The five-questions pass

- **Flow:** **asynchronous**: the API stores the code, enqueues a job and returns a `submission_id` at once; the browser waits via Server-Sent Events or polling (Message Queue, Server-Sent Events). No HTTP connection is held open while a compiler runs.
- **Storage:** the **submission record** (code + final verdict) in a relational database is the truth; test cases are immutable versioned files in object storage; the leaderboard is **derived** and rebuildable (CQRS).
- **Speed:** execution dominates, so keep compilers warm in pre-built images, cache test data on local disk, and exit at the first failing test.
- **Failure:** workers crash, hang or are killed by user code. Jobs are **leased**, not consumed (unreported jobs return to the queue), and judging is **idempotent** (Idempotency).
- **Growth:** with contest size, not users; pre-scale before every scheduled contest (Auto-Scaling).
- *Skeleton:* stateless submit API → durable priority queue → judge workers running each submission in a locked-down sandbox → verdict store → result push + leaderboard service.

3. API

Method and path	Purpose
`POST /submissions {problem_id, language, code, contest_id?}`	Returns `202 Accepted {submission_id}` at once. Requires an `Idempotency-Key` header so a double-click does not judge twice
`GET /submissions/{id}`	Status (`queued`, `compiling`, `running 17/50`, final verdict), time and memory
`GET /submissions/{id}/events` (SSE)	Server pushes status changes until the final verdict
`POST /run {problem_id, language, code, stdin}`	"Run code" on custom input. Same sandbox, lower priority queue, no verdict stored
`GET /contests/{id}/leaderboard?cursor=…&limit=50`	Paged ranking (Cursor-Based Pagination)
`GET /contests/{id}/leaderboard/me`	My rank and the rows around me
Internal: `lease(job)`, `report(submission_id, attempt, verdict)`	Worker ↔ queue and worker ↔ verdict store

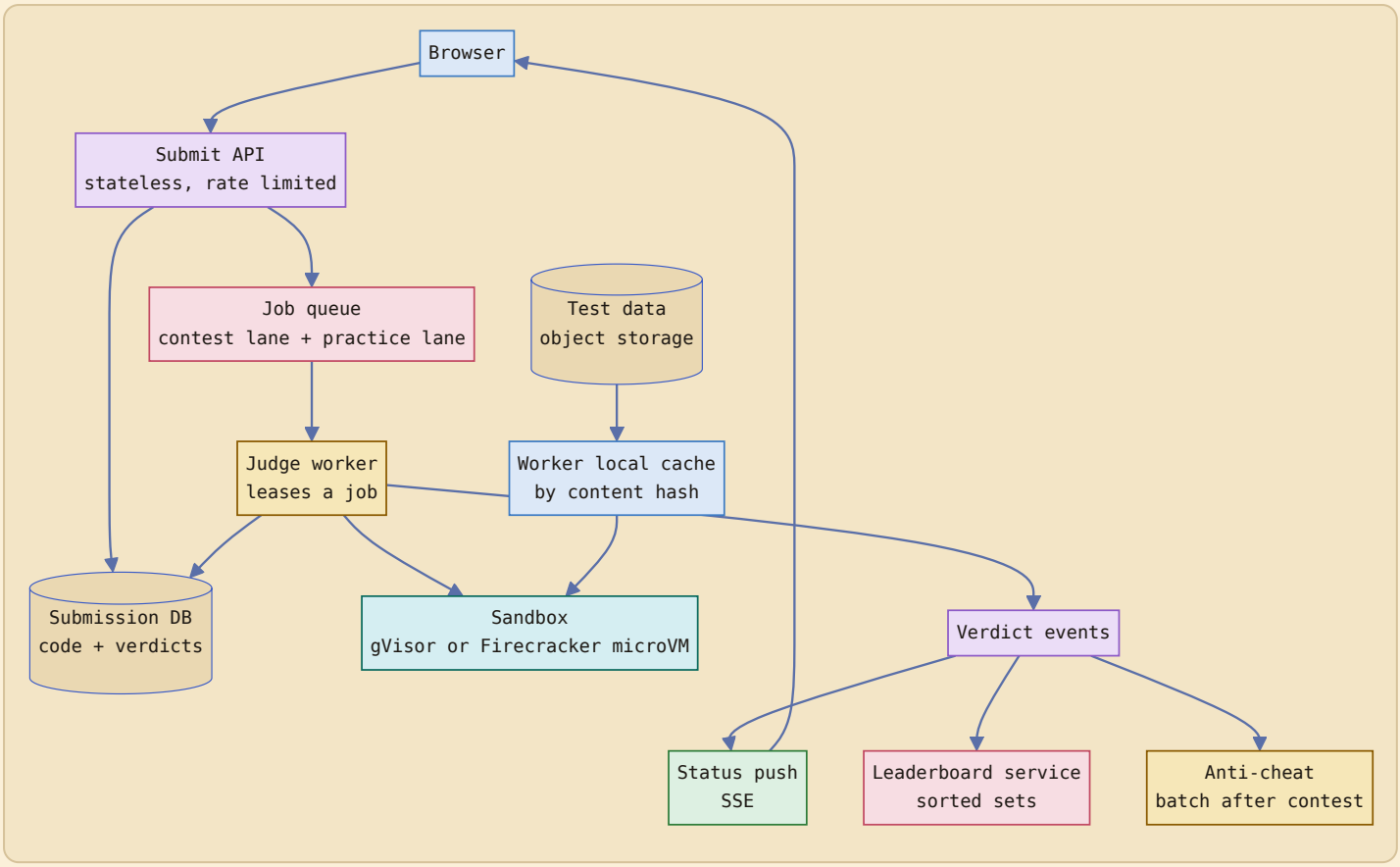
Rate limits, e.g. 1 submission per 5 s per user per problem plus a `/run` cap, stop scripts flooding the judge.

4. Data model

- **`problems`** — `problem_id, title, statement_ref, time_limit_ms, memory_limit_mb, checker_type (exact / token / float tolerance / custom), testset_version`. The **checker** decides whether output is correct: "exact" compares bytes, "float tolerance" accepts 1e-6 differences, "custom" handles problems with many valid answers.
- **`testsets`** — `(problem_id, version) → manifest` of each test file's name, size and **SHA-256**. Files sit at content-addressed paths (the path *is* the hash), so a cached file can never be stale: a new version gets new paths.
- **`submissions`** — `submission_id (time-ordered 64-bit), user_id, problem_id, contest_id?, language, code_ref, created_at, status, verdict, time_ms, memory_kb, failed_test_index, judge_attempt, testset_version`; partitioned by `user_id` (Sharding), secondary index `(contest_id, created_at)`.
- **`contest_scores`** (in-memory sorted set + durable copy) — per contest `user_id → (solved_count, penalty_minutes, last_ac_time)` plus per-problem `(attempts, first_ac_time)`.
- **`language_images`** — `language → container image digest, compile command, run command, time multiplier`. The image is a frozen file system with compiler and runtime; its **digest** (content hash) guarantees every

worker runs byte-for-byte the same toolchain.

5. The architecture, annotated



1. The **Submit API** checks contest registration and timing, writes the code to the **submission DB** as `queued`, enqueues `{submission_id}` on the **contest lane**, and returns `202` in ~50 ms.
2. **Separate lanes** (priority classes) keep 50K contest users from waiting behind practice traffic without starving practice: workers take, say, 9 contest jobs per practice job (Bulkhead); FIFO within a lane.
3. A **judge worker** leases the job (invisible to others for, say, 120 s), loads the code, picks the language image, and fetches any missing test files by hash into its **local cache**.
4. It compiles in one sandbox with a compile-time limit, then runs each test in a fresh locked-down sandbox with CPU, memory, process and output limits (Deep dives 1 and 2); the **checker** compares output, stopping at the first failure.
5. A **conditional update** (`only if status != final`) stores the verdict, then a **verdict event** goes to the status-push service (to the browser) and the **leaderboard service** (Event-Driven Architecture).
6. After the contest, **anti-cheat** jobs compare all accepted submissions (Deep dive 4); ratings publish only after that pass.

6. Deep dives

Deep dive 1 — the sandbox: containers, gVisor or Firecracker. The sandbox is the product; a plain process is unsafe, so the question is how thick the wall is.

- **Plain Linux container** (Docker-style): **namespaces** (a private view of process IDs, mounts, network and users), **cgroups** (control groups: kernel caps on CPU, memory and process count), **seccomp** (a filter on which **system calls**, a program's requests to the kernel such as "open file" or "create socket", are allowed) and all privileges dropped. The IOI's open-source **isolate** sandbox does this, with no measurable overhead per its authors, and handles multi-threaded programs. *Weakness:* the code still talks to the **real host kernel**; one reachable kernel bug is a full escape to a worker holding other people's code.

- **gVisor** (Google): a user-space "application kernel" whose **Sentry** intercepts system calls and implements them itself, with file access via a separate **Gofer** process; plugs in as an OCI runtime (`runsc`) behind Docker or Kubernetes; used under App Engine, Cloud Run and Cloud Functions. *Cost:* syscall-heavy programs slow down, skewing time limits on I/O-heavy problems.
- **Firecracker microVM** (AWS, built for Lambda): a tiny **virtual machine** (guest OS on emulated hardware, isolated by CPU virtualisation) booting in under 125 ms with under 5 MiB overhead, up to 150 created per second per host. The guest has its own kernel, so a kernel bug inside doesn't reach the host.
- **Choice:** a **Firecracker microVM (or gVisor) as the outer wall**, **isolate-style cgroups + seccomp inside** for precise per-test limits: two independent layers must both fail. Keep a **pool of pre-booted microVMs** per language so boot time is off the critical path, and destroy each VM after one submission (never reuse a sandbox between users).
- **Network:** none, no interface at all. **File system:** read-only image plus a size-capped `tmpfs` (memory-backed scratch). **Workers** sit in an isolated segment reaching only the queue, the DB writer and object storage.

Deep dive 2 — resource limits and fair timing.

- **Time:** **CPU time** (actually running on a core) from cgroup counters decides TLE (Time Limit Exceeded); a **wall-clock limit** (real elapsed time, e.g. 3× the CPU limit) kills programs that sleep or block on input.
- **Memory:** a cgroup cap; the out-of-memory killer firing inside the cgroup means MLE.
- **Processes:** `pids.max` of, say, 64 stops a **fork bomb** (a program copying itself endlessly).
- **Output:** cap stdout at e.g. 64 MB; endless printing gets "Output Limit Exceeded".
- **Fairness:** pin each sandbox to **one dedicated physical core** (no hyper-threading sibling), disable CPU frequency boost, never run more sandboxes than cores, calibrate each worker class with a fixed benchmark and reject drifting machines, and automatically **re-run** borderline TLEs (within ~5% of the limit) on another worker, keeping the better time.
- **Language fairness:** a published **time multiplier** per language in `language_images` (e.g. Python 3× the C++ limit).

Deep dive 3 — the queue at contest time.

- If workers run short at 500/s the queue and the "Pending" spinner grow; acceptable if bounded, and showing queue position ("your submission is #1,240") turns anxiety into patience.
- **Pre-scale** 30 minutes before start from registrations (registrations × expected submissions × 4 CPU-s) and hold 30 minutes after the end for rejudges: the spike arrives in seconds, faster than reactive scaling.
- **Lease and retry:** a dead worker's job is redelivered at lease expiry; each judging has an `attempt` number and the conditional final write means two workers can't both win (visibility timeouts: Module 13 Case 4).
- **Poison submissions** that crash the *worker* (not just the sandbox) three times go to a dead letter queue with "System Error — under review".
- **Rejudge:** wrong tests mid-contest → admins publish a new `testset_version` and enqueue affected submissions on a **rejudge lane** below live submissions.

Deep dive 4 — leaderboard and anti-cheat.

- **Ranking rule (ICPC-style):** problems solved (more is better), then **penalty time** (less is better): per solved problem, minutes from start to first accept plus a fixed penalty per earlier rejected attempt (ICPC: 20 minutes; LeetCode and others publish their own). Encode as one **sorted-set** score (e.g. Redis ZSET): `score = solved × 10^7 − penalty_minutes × 10 − tiebreak`, so one `ZADD` per accepted verdict keeps order and "rank of X" or "rows 1–50" are O(log N) (Skip lists; sorted sets in depth: Module 15 Design 4).

- **Only verdict events change scores**, idempotent per `(user, problem)` (a second Accepted does nothing); `contest_scores` rebuilds the sorted set after cache loss.
- **Freeze:** many contests hide the last hour's changes; the service keeps a public (frozen) view and the true view, serving the public one until reveal.
- **Anti-cheat signals:** (1) **code similarity** with MOSS-style **winnowing**: strip comments, whitespace and identifiers, cut the token stream into overlapping k-grams, hash them, keep a sample of fingerprints, flag pairs sharing many; an inverted index (fingerprint → submissions) avoids comparing all 25K² pairs on a popular problem. (2) **Behaviour:** many accounts per device or IP, a hard problem solved 40 s after a friend, sudden rating jumps, large pastes (the editor can record paste events). (3) **Leaked-solution matching** against code posted publicly during the contest.
- Flags go to **human review**, never straight to a ban: short standard algorithms look alike and fingerprint tools over-flag them. Ratings finalise after review and roll back for confirmed cheaters.

7. The failure walk

1. **Worker dies mid-judging:** leases expire, jobs are redelivered, the conditional write keeps one result; users wait ~2 minutes longer.
2. **Sandbox escape discovered** (kernel or gVisor CVE): the worst case, but one failed layer isn't yet a breach. Workers are **cattle** (identical, disposable): rebuild every image with the patch, drain and replace the fleet, rotate credentials. Workers hold only the current job and can write only verdicts, limiting blast radius.
3. **Test-data store slow or down:** workers use their hash-addressed cache, and contest test sets are **pre-warmed** to every worker before start.
4. **Queue backlog at peak:** show queue position, **pause the practice lane** and `/run` (Graceful Degradation), extend the contest past a published delay threshold; never drop a queued contest submission.
5. **Leaderboard sorted set lost:** rebuild from `contest_scores` (50K rows) in seconds; verdicts live in the DB.
6. **Bad compiler image** (every C++ submission gets Compile Error): pin images by digest per contest, canary new images on a worker slice outside contest hours, watch the verdict mix per language (Canary Deployment), rejudge affected submissions.
7. **A program hogs I/O or memory bandwidth**, slowing neighbours: cgroup I/O limits, one core per sandbox and borderline re-runs keep verdicts fair.

8. Trade-offs

Decision	Chosen	Alternative	Why
Isolation	MicroVM or gVisor outside + cgroups/seccomp inside	Plain containers only	Code is hostile by definition; one shared-kernel bug should not be game over
Submission flow	Async queue + SSE push	Synchronous HTTP	Judging takes seconds and spikes 10×; a queue absorbs the burst
Queue	Separate contest, practice, rejudge lanes	One FIFO	Contest fairness and protection from bulk rejudges
Timing	CPU time on a dedicated core, re-run if borderline	Wall-clock on shared cores	Repeatable verdicts are the product
Test data	Content-addressed files + local cache, pre-warmed	Read from shared storage per run	Hundreds of reads/s of the same files; no stale caches
Leaderboard	Sorted set derived from verdict events	SQL `ORDER BY` per request	5K reads/s on a constantly changing ranking
Anti-cheat	Offline fingerprinting + human review	Real-time automatic bans	False positives on short standard code are common
Fleet	Scheduled pre-scaling around contests	Reactive autoscaling only	The spike arrives faster than machines boot

9. The follow-up questions

- **"Why not just Docker?"** Docker shares the host kernel, so isolation depends on every allowed system call being bug-free; hostile code needs a second boundary with its own kernel (microVM) or syscall implementation (gVisor). *Trap:* "containers are secure because of namespaces" without mentioning the shared kernel.
- **"Interactive problems** (the program converses with a judge program)?" Two sandboxes joined by pipes, the interactor deciding the verdict; time both, and a deadlock becomes TLE via the wall-clock limit.
- **"New language?"** Build an image, run every problem's reference solutions to set its time multiplier, publish the digest, enable it; workers are image-driven, so nothing else changes.
- **"Serverless judging?"** Fine for practice (each Lambda invocation already runs in a Firecracker microVM), but timing noise and cold starts make contest timing less fair; dedicated calibrated workers are safer for rated contests.
- **"AI-cheating detection?"** One more advisory signal: compare against popular code models' outputs for the problem plus behaviour (one large paste, no edits), with human review; prefer resistant contest design (proctored or in-person finals).

Pattern manifest

Pattern	Where used	Taught in
Message Queue (with leases)	Submission jobs, contest/practice/rejudge lanes	Module 02
Server-Sent Events	Pushing status and verdict to the browser	Module 02
Idempotency	`Idempotency-Key` on submit; conditional verdict write; one AC per problem	Module 05
Bulkhead	Separate lanes and worker shares for contest vs practice	Module 05
Dead Letter Queue	Submissions that crash workers repeatedly	Module 05
Graceful Degradation	Pause practice and `/run` at contest peak	Module 05
Auto-Scaling (scheduled)	Worker fleet before each contest	Module 06
Rate Limiting	Per-user submit and run caps	Module 08
Containers and sandboxing	Language images, microVM/gVisor sandboxes	Module 09
CQRS	Leaderboard as a derived read model	Module 03
Sorted-set leaderboard	Contest ranking	Module 15

Interview one-liner: "Submit is asynchronous: store the code, enqueue a job, return an ID. Calibrated workers lease jobs and run each submission in a throwaway microVM or gVisor sandbox with no network, cgroup limits on CPU, memory, processes and output, and content-hashed test data cached locally. Verdict events feed the browser and a sorted-set leaderboard; contests get their own queue lane and pre-scaled capacity; plagiarism is a fingerprinting batch job plus human review."

TL;DR

- **Problem:** run hostile code from millions of users fairly and repeatably, with a 10× contest spike.
- **Mechanism:** async submit → leased priority queue → judge workers → per-submission microVM/gVisor sandbox with cgroup and seccomp limits → verdict events → SSE + sorted-set leaderboard.
- **Hard parts:** isolation (two layers, no network, never reuse), fair timing (dedicated cores, CPU time, borderline re-runs), contest bursts (lanes, pre-scaling, pre-warmed tests), anti-cheat (winnowing + humans).
- **Price:** a large mostly idle fleet between contests and constant sandbox patching.

Sources

- Mareš, M. and Blackham, B., "A New Contest Sandbox", *Olympiads in Informatics* 6 (2012): https://ioinformatics.org/journal/INFOL094.pdf and the isolate project: https://github.com/ioi/isolate

- gVisor documentation (Sentry, Gofer, runsc): https://gvisor.dev/docs/
- Google Cloud blog, "How gVisor protects Google Cloud services from CVE-2020-14386": https://cloud.google.com/blog/products/containers-kubernetes/how-gvisor-protects-google-cloud-services-from-cve-2020-14386
- Firecracker project site (boot time, overhead, creation rate): https://firecracker-microvm.github.io/
- ICPC scoring rules (20-minute penalty per rejected run on a solved problem), e.g. ECNA regional rules: https://icpc-ecna.ysu.edu/rules.html
- Schleimer, Wilkerson and Aiken, "Winnowing: Local Algorithms for Document Fingerprinting", SIGMOD 2003 (the algorithm behind MOSS): https://theory.stanford.edu/~aiken/publications/papers/sigmod03.pdf

---

Case 2 — Global Deployment System

The prompt (in short). The internal system a large company ships software with: an engineer merges, and within hours the build runs safely on hundreds of thousands of servers in dozens of regions (think Facebook's continuous web-tier push, Amazon's pipelines to every AWS region, Google's rollout tooling).

What makes it hard. Google's SRE book observes that roughly 70% of outages come from changes to a live system, so deployment is both the biggest risk and the main defence. Hard parts: (1) **moving gigabytes to 200K machines** without melting the network or artifact store; (2) **gradual exposure** so a bad build hurts 0.1% of users, not 100%; (3) **automatically judging** stage health and rolling back without a human; (4) treating **configuration changes**, which spread in seconds and cause many outages, as carefully as code.

1. Requirements and numbers

Functional: build a versioned immutable artifact per commit; store and distribute it; roll out in staged waves across regions; check health per stage and promote or roll back automatically; a big red **stop button**; deploy **config** separately from code; audit who shipped what, where, when; deployment windows and blockers (no deploys during an active incident).

Non-functional: safety beats speed (catch bad changes while exposure is small); **rollback in minutes** anywhere; usable during an outage (it ships the fix); fully **automated and repeatable**; thousands of services, tens of deploys per service per week.

Numbers (assumptions):

- **200K servers** in **20 regions** (~10K each over 3 **availability zones**, AZ = one or more data centres with independent power and network inside a region).
- **2,000 services** deploying about daily → **~2,000 rollouts/day** plus **~5,000 config changes/day**.
- **Artifacts:** a big monolith binary or image **~1 GB**; a typical microservice image ~200 MB, with only changed layers moving (often ~50 MB).
- **Naive fan-out** of 1 GB to 200K hosts = **200 TB**; a central store of 20 servers × 10 Gbps = 25 GB/s → 8,000 s ≈ **2.2 hours** with the store saturated and other deploys blocked. **Peer-to-peer** (Deep dive 1) makes every host that holds a piece upload it, so capacity grows with hosts; at ~50% of a 10 Gbps NIC, time is about one host's download (~2 s per GB) plus a few relay hops: **well under a minute** (Uber reports 20K blobs of 100 MB–1 GB in under 30 s).
- **Rollout duration** is dominated by **bake time** (running under real traffic before promotion): five waves with 1–12 h bakes → **multi-day** global rollouts for risky backends (Amazon's published zonal-and-regional wave pipeline takes days), a few hours for front-ends with fast signals (Facebook's web tier pushes several times a day).

What the numbers say: P2P solves the bytes; the design is about **exposure control and fast, automatic judgement**.

2. The five-questions pass

- **Flow:** asynchronous and long-running: a rollout is a durable **workflow** run by a pipeline orchestrator, not a laptop script. Hosts **pull** their target from a desired-state store; the orchestrator only changes desired state (the reconcile loop of Containers and Kubernetes).
- **Storage:** the **artifact** (immutable, content-addressed: named by its bytes' hash) in blob storage; **desired state** per (service, cell), e.g. "cell 7 of region X runs build b-4812 with config v-991", in a strongly consistent etcd/ZooKeeper-style store; history and audit in a relational DB.
- **Speed:** P2P plus pre-staging (download before switching) make the switch a seconds-long restart; the long part is deliberate waiting.
- **Failure:** the main failure is **a bad change**; health gates, automatic rollback and small blast radii defend. The deploy system's own outage must never block an emergency fix (a minimal "break glass" path).
- **Growth:** services × hosts × change rate; distribution must be sub-linear in hosts (P2P) and control hierarchical (region → cell → host).
- *Skeleton:* CI build → artifact registry → P2P distribution → per-host agent → rollout orchestrator with waves and gates → health/canary analysis → automatic rollback; a separate fast lane for config.

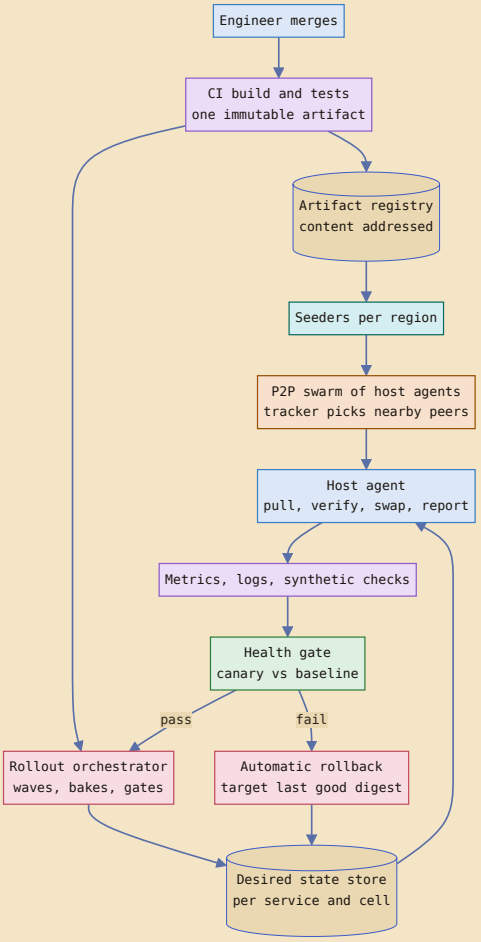
3. API

Method and path	Purpose
`POST /artifacts {service, commit, digest, sbom_ref, signature}`	Register a build (CI calls this). **Immutable**: same digest, same bytes, forever
`POST /rollouts {service, artifact_digest, plan_id}`	Start a rollout using a named plan (waves, bakes, gates). Returns `rollout_id`
`GET /rollouts/{id}`	Current wave, per-cell state, gate results, ETA
`POST /rollouts/{id}:pause` / `:resume` / `:rollback`	Human controls; rollback re-targets the last good digest
`POST /config-changes {key, new_value, canary_spec}`	Propose a config change; goes through review, validation and its own canary
`POST /freeze {scope, reason, until}`	Org-wide or service-level deploy blocker during incidents and holidays
Agent ↔ control plane: `GET /desired?host=…` (long poll), `POST /report {host, running_digest, health}`	Pull model; the agent reports what is actually running

4. Data model

- **`artifacts`** — `digest (SHA-256) → service, commit, build_time, size, layers[], signature, provenance` (who built it, from what source, on which builder); bytes stored as layers/chunks keyed by their own hashes, so identical layers are stored and sent once.
- **`rollout_plans`** — `plan_id → waves[]`, each `{targets (cells/regions), batch_size %, bake_time, gates[]}`; plans are reviewed like code.
- **`rollouts`** — `rollout_id, service, from_digest, to_digest, plan_id, state (running / paused / rolling_back / done), current_wave, started_by, started_at`.
- **`desired_state`** — `(service, cell) → {artifact_digest, config_version, generation}`; the increasing **generation** stops an agent applying an older instruction after a newer one (ordered durable changes, Write-Ahead Log thinking).
- **`actual_state`** — `(host) → running_digest, config_version, healthy, last_report_at`, expiring if the host stops reporting.
- **`gate_results`** — `(rollout_id, wave, gate) → pass / fail / inconclusive, metrics snapshot, decided_at`.
- **`config_store`** — versioned key → value with schema, owner and history (in Git, mirrored to a distribution tree).

5. The architecture, annotated



1. The merge triggers **CI**, which produces **one** signed artifact with a digest and recorded provenance. **Build once, promote everywhere**: the bytes that passed tests go to every stage (CI/CD Pipelines and Deployment Strategies).
2. The artifact goes to the **registry**; the **orchestrator** starts the service's plan: pre-production stages, then production **wave 1**.
3. Per target cell, agents first **pre-stage** (download and verify); regional **seeders** hold the first copy and the **P2P swarm** spreads it (Deep dive 1).
4. The orchestrator bumps the wave's **desired state**. Each **host agent** sees the new generation, verifies signature and hash, **drains** the local service (no new requests, finish in-flight ones), swaps versions, runs local health checks and reports `actual_state` (Health Check Endpoint); hosts switch in batches (e.g. ≤ 10%) so capacity holds.
5. The **health gate** compares new hosts with old over the **bake time** (Deep dive 3): pass → next wave; fail → **automatic rollback** to the last good digest, still cached on every host, so rollback is a restart, not a download.

6. Deep dives

Deep dive 1 — moving the bytes: registry, P2P and pre-staging.

- **Content addressing:** chunks or layers named by hash; hosts skip chunks they hold, so a build changing 3% of a 1 GB binary moves ~30 MB, and the recomputed hash rejects corrupt or tampered chunks.
- **Why P2P:** a central server's fixed upload capacity makes time linear in hosts. In a **BitTorrent-style swarm** the file is cut into **pieces** (BitTorrent's spec: most commonly 256 KB, each with its own SHA-1 hash), each host downloads from several peers and **uploads pieces it has**, so capacity grows with the swarm and 200K hosts take about as long as 2,000.

- **History:** Facebook pushed its front-end binary with internal BitTorrent around 2010–2012, with a custom tracker preferring same-rack peers; Twitter's "Murder" did the same; Uber's open-source **Kraken** is a P2P Docker registry (a **tracker** says which peers have which pieces; **origin** servers seed from backing storage such as S3), reporting 20K blobs of 100 MB–1 GB in under 30 s at peak at > 50% of each host's max download speed.
- **Topology awareness:** the tracker hands out peers in the **same rack, then cluster, then region**, so most bytes stay off expensive links; per-region seeders mean one copy crosses an ocean.
- **Pre-staging:** separates the slow, failure-prone network part from the risky restart, so a 10K-host switch takes seconds; keep the **previous N versions** on disk so rollback needs no network.
- **Guard rails:** cap per-host P2P upload (e.g. 30% of NIC) so production traffic isn't starved; heavily loaded hosts opt out of seeding.

Deep dive 2 — staged rollout waves and blast radius.

- **Blast radius** = share of users hurt if this step is bad; every wave caps it.
- **Typical plan** (the shape Amazon publishes):
  1. Pre-production: integration tests, then a **gamma** stage mirroring production config.
  2. **One-box** in one AZ of a low-traffic region (one host or a small traffic slice), bake ≥ 1 h.
  3. Wave 1: the rest of that AZ, then that region's other AZs **one at a time**; bake ~12 h after the first full region to catch daily-cycle bugs (overnight batch jobs, memory leaks, certificate rotations).
  4. Wave 2: one high-traffic region, AZ by AZ.
  5. Waves 3–5: 3, then ~12, then all remaining regions, with 2–4 h bakes.
- **Never two AZs of a region at once**, never two regions at once early: a failure stays inside a **cell** (a self-contained stack serving a slice of users) (Multi-Region and Cell-Based Architecture).
- **Front-end variant:** Facebook's web tier goes to **employees**, then **~2% of production**, then 100%, over a few hours, several times a day; enormous traffic gives strong signals fast.
- **Windows and blockers:** production waves start only in the owning team's working hours (not nights, weekends, late Fridays); **blockers** halt new waves automatically during related high-severity alarms or org-wide incidents.
- **Within a cell:** **rolling** batches (≤ 10–33% of hosts) or **blue-green** where instant switch-back matters (Blue-Green Deployment).

Deep dive 3 — health gates and automatic rollback.

- **Watched signals:** 5xx rate, latency p50/p90/p99, CPU, memory, crash/restart count, log error rate, failed health checks, **business metrics** (checkouts per minute, messages sent), and **synthetic canaries** (scripted fake users calling the API every minute from outside).
- **Compare like with like:** absolute thresholds miss subtle regressions and misfire at traffic peaks, so compare **canary vs baseline hosts** (old version, same cell, window and host count). Netflix and Google's open-source **Kayenta** runs a **Mann-Whitney U test** per metric (do two sets of numbers come from different distributions, without assuming a bell curve), flagging only at ~98% confidence of a meaningful difference, and rolls results into a pass/fail/ask-a-human score (Canary Deployment).
- **Outcomes:** `pass` → promote after the bake; `fail` → rollback; `inconclusive` (too little traffic) → extend the bake, never promote silently.
- **Fast tripwires** roll back immediately: request count to zero, fault-rate spike, health checks failing on > N% of new hosts.
- **Automatic rollback:** desired state back to the **last digest that passed all gates** in the cell; agents restart into the cached version. On a team-level aggregate alarm, roll back **every in-flight deployment** of that team in the region (the culprit is unknown). Rollback needs neither the failing service healthy nor CI to rebuild.

- **Rollback needs backward compatibility:** schema changes go **expand → migrate → contract** (add column, deploy code writing both, backfill, drop the old column later) so N−1 runs against data N wrote (Capstone - The Zero-Downtime Migration); irreversible changes (data deletions, protocol bumps) go behind **feature flags**.

Deep dive 4 — config deploys vs code deploys.

- **Why separate:** code needs a new binary everywhere (hours of waves); many changes are just *values* (a timeout, a rate limit, which 5% see a feature) that as config reach the fleet and undo in **seconds**.
- **The danger is that speed:** a bad value hits every server at once with no rollout to slow it; many well-known global outages were config pushes.
- **Facebook's model** (SOSP 2015, "Holistic Configuration Management at Facebook"): configs in Git, often **generated by code** with validators, every change reviewed; an automated **canary spec** applies it to ~20 servers, then a cluster, comparing health (~10 minutes for a full canary); distribution via **Zeus** (a ZooKeeper fork) through a high fan-out tree of observers and per-server proxies, reaching hundreds of thousands of servers in seconds; large blobs via a separate P2P path. Each proxy's **on-disk cache** lets apps read the last good config if the config system is down.
- **Design rules:** schema-validate every value (types, ranges); **stage config like code** (one-box → cell → region, shorter bakes); full history with **one-click, seconds-long revert**; apps **reject unparseable config and keep the last good one**; separate **feature flags** (per-user exposure, Feature Flags) from **operational config** (timeouts, pool sizes) so each gets the right review rules.

7. The failure walk

1. **Bad build passes tests** (a leak showing after 6 h): one-box and wave-1 bakes catch daily-cycle problems; the memory alarm trips on the one-box and rollback restores 1 host: ~0.01% of traffic.
2. **Bad build fails fast** (crash on start): hosts fail local health checks before joining the load balancer and the agent stops after the first batch: zero user impact.
3. **Gates are blind** (an unwatched feature breaks): it reaches more waves. Add business-metric gates and synthetic tests per critical journey; every escaped bug becomes a new gate.
4. **Orchestrator or desired-state store down:** hosts keep running their last known state (pull model) and no rollouts start, which is safe; a documented **break-glass** tool lets on-call push a pinned digest to a cell.
5. **Registry or seeders down:** pre-staged versions and peer copies continue; new builds wait; the registry is replicated per region (Primary-Replica).
6. **Rollback fails** because the new version wrote data the old can't read: hence backward-compatible schema rules and flags; otherwise **roll forward** a hotfix through an expedited plan (shorter bakes, still staged, never all at once).
7. **Bad config push:** the config canary catches most; escaped ones revert in seconds; apps keep the previous value on unparseable config.
8. **Compromised build machine** shipping malware: hosts accept only artifacts signed by the build system with provenance from reviewed source, verifying signature and digest before running.

8. Trade-offs

Decision	Chosen	Alternative	Why
Distribution	P2P swarm with regional seeders, rack-aware peers	Central servers / CDN only	Total capacity grows with hosts; 200 TB per release is otherwise hours
Control	Desired state + host agents pull	Orchestrator pushes over SSH	Survives orchestrator outages, scales, self-heals drift
Exposure	One-box → AZ → region waves with bakes	Deploy everywhere at once	Caps blast radius; 70% of outages are changes
Gate logic	Canary vs baseline statistics + hard tripwires	Static thresholds	Adapts to traffic patterns, catches subtle regressions
Rollback	Automatic to last good, cached digest	Human decides	Machines notice in minutes, humans at 3 a.m. take longer
Speed	Days for backends, hours for web tier	Same plan for all	Match bake time to how fast signals arrive and how bad failure is
Config	Separate fast lane, but still canaried and staged	Treat config as code deploy	Seconds-level changes and undo, without global instant blast

9. The follow-up questions

- **"Why not a CDN for artifacts?"** It helps seed the first copy per region, but inside a data centre all hosts pulling from one edge concentrate load; P2P uses every host's idle upload and keeps bytes in racks. Many systems combine them: CDN/object storage to seed, P2P to spread (CDN).
- **"Deploying a service 300 others depend on?"** Same waves, with **callers'** error rates in the gates and cell-aligned rollouts so callers and dependency change together; CI contract tests keep the API compatible (API Versioning).
- **"Mobile apps?"** No rollback on phones: staged store rollouts (a percentage of users), server-driven feature flags to switch behaviour off, server compatibility with old app versions for months.
- **"Did a deploy cause this incident?"** Dashboards overlay deploy and config events, the deploy system publishes to the incident tool, and triage starts with "what changed in the last hour, where?".
- **"2,000 rollouts a day without collisions?"** Per-cell locking (one rollout per service per cell), a per-service rollout queue that **batches** commits into one artifact (Facebook's push-from-master ships many diffs together), and blockers pausing a region when it is unhealthy.

Pattern manifest

Pattern	Where used	Taught in
CI/CD, build once, promote	One signed artifact through every stage	Module 09
Canary Deployment	One-box and wave gates, canary vs baseline	Module 09
Blue-Green Deployment	Instant switch-back within a cell	Module 09
Feature Flags	Decouple risky behaviour from deploys; non-reversible changes	Module 09
Health Check Endpoint	Agent's local check before joining the load balancer	Module 09
Containers and reconcile loops	Desired vs actual state, agents pull	Module 09
Multi-Region and Cell-Based Architecture	Waves follow AZ and cell boundaries	Module 06
Circuit-breaker-style blockers	Halt all deploys during alarms or incidents	Module 05
Publish-Subscribe / fan-out tree	Config distribution through observers and proxies	Module 02
Zero-downtime migration	Expand-migrate-contract so rollback stays safe	Module 09
Distributed Job Scheduler	Long-running rollout workflows	Module 16

Interview one-liner: "Build one signed, content-addressed artifact; spread it with a rack-aware P2P swarm and pre-stage it; change only the desired state and let host agents pull; roll out in waves — one-box, one AZ, one region, then wider — with bake times and canary-vs-baseline gates; roll back automatically to the cached last-good digest; and push config through its own fast but still canaried lane."

TL;DR

- **Problem:** ship thousands of changes a day to 200K servers without any one change taking everything down.
- **Mechanism:** CI → immutable artifact → P2P distribution with regional seeders → desired-state store + pulling host agents → staged waves with bakes → statistical health gates → automatic rollback; a separate canaried config pipeline.
- **Hard parts:** bytes at scale (P2P, content addressing, pre-staging), blast radius (cells, waves), automatic health judgement (canary vs baseline), keeping rollback possible (compatible data changes), config's dangerous speed.
- **Price:** slow global rollouts (days for backends), heavy telemetry plumbing, compatibility discipline.

Sources

- Liguori, C., "Automating safe, hands-off deployments", Amazon Builders' Library: https://aws.amazon.com/builders-library/automating-safe-hands-off-deployments/

- Facebook Engineering, "Rapid release at massive scale" (2017): https://engineering.fb.com/2017/08/31/web/rapid-release-at-massive-scale/
- Tang, C. et al., "Holistic Configuration Management at Facebook", SOSP 2015: https://doi.org/10.1145/2815400.2815401
- Uber Engineering, "Introducing Kraken, an Open Source Peer-to-Peer Docker Registry": https://www.uber.com/blog/introducing-kraken/ and https://github.com/uber/kraken
- BitTorrent protocol specification (BEP 3): https://www.bittorrent.org/beps/bep_0003.html
- Spinnaker docs, "How canary judgment works" (Kayenta, Mann-Whitney U): https://spinnaker.io/docs/guides/user/canary/judge
- Google Cloud blog, "Introducing Kayenta": https://cloud.google.com/blog/products/gcp/introducing-kayenta-an-open-automated-canary-analysis-tool-from-google-and-netflix
- Beyer, B. et al., *Site Reliability Engineering* (O'Reilly, 2016), on ~70% of outages coming from changes: https://sre.google/sre-book/introduction/
- TorrentFreak, "Facebook Uses BitTorrent, and They Love It" (2010, reporting a Velocity talk): https://torrentfreak.com/facebook-uses-bittorrent-and-they-love-it-100625/

Case 3 — Conversational AI Service (ChatGPT-like)

The prompt (in short). A ChatGPT/Claude/Gemini-style assistant: users sign in, type a message and watch the answer appear word by word; conversations are saved and reopen on any device; free and paid tiers get different limits; harmful requests and answers are filtered.

What makes it hard. Sign-in and history are ordinary CRUD. The hard parts: (1) every answer comes from a **large language model (LLM)** generating one **token** (a word piece, roughly ¾ of an English word) at a time on **GPUs**, by far the largest cost; (2) answers take **seconds to tens of seconds**, so they must be **streamed**; (3) the GPU memory each in-flight conversation needs (its **KV cache**, Deep dive 2) caps users per GPU, so scheduling and routing are memory problems.

1. Requirements and numbers

Functional: create a conversation; send a message and receive a streamed answer; stop mid-way; regenerate or edit (creating a branch); list, rename, search, delete conversations; choose a model; per-tier quotas (free, paid, team). Attachments only "stored in blob storage".

Non-functional:

- **Time to first token (TTFT)** p95 < 1.5 s.
- **Time per output token (TPOT)** ≈ 20–30 ms (~35–50 tokens/s, faster than reading).
- **No lost answers:** finished answers are saved; a dropped connection can resume the stream.
- **Availability** 99.9% for chat; degrade (smaller model, queue) rather than hard-fail.

Numbers (assumptions, rounded):

- **100M DAU × 10 messages/day = 1B/day ≈ 11.6K/s, ~35K/s peak** (3×).
- Answers average **500 output tokens** at **50 tokens/s** → **~10 s** of GPU time each.
- Concurrent streams at peak by Little's law: 35K/s × 10 s = **~350K open streams** → **17.5M output tokens/s**.
- Input per message (system prompt + history + new message) ≈ **3,000 tokens**; if 60% is a prefix already cached on the GPU (Deep dive 3), 1,200 new tokens each → **42M input tokens/s** of **prefill** (Deep dive 2).
- **KV cache per token**, 70B model shaped like Llama 3.1 70B (80 layers, 8 key/value heads, head size 128, 16-bit): 2 (key and value) × 80 × 8 × 128 × 2 bytes = **327,680 bytes ≈ 320 KB**; a 3,500-token conversation holds **~1.1 GB** while answered.
- **One replica** = 8 H100s × 80 GB = 640 GB; weights 70B × 2 bytes = 140 GB; after activations ~400 GB for KV → **~1.25M tokens** ≈ 350 conversations of 3,500 tokens at once.
- **Replica throughput (assumption, measure it):** ~6,000 output tokens/s at target TPOT → 17.5M ÷ 6,000 ≈ **2,900 replicas ≈ 23,000 GPUs**, +~30% for prefill and headroom → **~30,000 GPUs**.

- **Cost (assumption):** $2.50 per GPU-hour → **$1.8M/day ≈ $0.0018 per message**; a free user at 10 messages/day costs ~$0.55/month in GPU time, which drives quota design.
- **History:** ~2.5 KB per message → 2.5 TB/day ≈ **~0.9 PB/year** before replication, tiny next to the GPU bill.

What the numbers say: database and web tier are ordinary; **GPU memory and time** are the bottleneck and the bill. Judge every choice by "does it raise tokens per GPU-second, or cut tokens we must compute?"

2. The five-questions pass

- **Flow:** synchronous to a watching user but **streamed** token by token over one long HTTP response with Server-Sent Events (SSE: the server writes small `data:` events into an open response); history, billing and analytics are **asynchronous** after the stream (Five Questions).
- **Storage:** the **conversation store** (messages as a tree, partitioned by user) is truth; a short-lived **generation buffer** holds in-flight tokens so a dropped client can resume; the GPU KV cache is a disposable performance cache.
- **Speed:** TTFT = queueing + prefill; TPOT = decode batching. The hot read, "this conversation's messages for the prompt", is one partition read.
- **Failure:** GPU servers fail more than web servers: retry a failed generation **on another replica**; a client disconnect must **not** waste the generation.
- **Growth:** with users and **context length** (KV memory grows linearly); hot spots are launches and each region's working-day start.
- *Skeleton:* API/edge tier (auth, rate limits), a chat orchestrator (prompt building, safety), a **KV-cache-aware inference router**, GPU pools with **continuous batching**, a stream relay to SSE gateways, a conversation store, an async usage pipeline.

3. API

Method and path	Purpose
`POST /v1/conversations` → `{conversation_id}`	Start a conversation
`POST /v1/conversations/{id}/messages` body `{client_msg_id, parent_message_id, content, model}` → `200 text/event-stream`	Send a message and stream the answer. `client_msg_id` is an idempotency key (a client-made ID reused on retries so the server can spot repeats) (Idempotency)
SSE events: `message_start{message_id, generation_id}`, `delta{seq, text}`, `message_end{finish_reason, usage}`, `error{code}`	Each event carries an `id:` line equal to its `seq` (the token chunk's position)
`GET /v1/generations/{generation_id}/stream` with header `Last-Event-ID: 812`	Resume a dropped stream from chunk 813 (SSE clients send `Last-Event-ID` automatically on reconnect)
`POST /v1/generations/{generation_id}/cancel`	The "stop" button; frees the GPU slot
`GET /v1/conversations?cursor=…`	List conversations, newest first (Cursor-Based Pagination)
`GET /v1/conversations/{id}`	All messages (the tree) for rendering
`GET /v1/usage`	Remaining quota for the current window

Rate limits return `429 Too Many Requests` with limit, remaining and reset headers (OpenAI's API returns `x-ratelimit-remaining-tokens` and similar). SSE beats WebSockets here: one-way traffic, plain HTTP that proxies understand, built-in resume via `Last-Event-ID`.

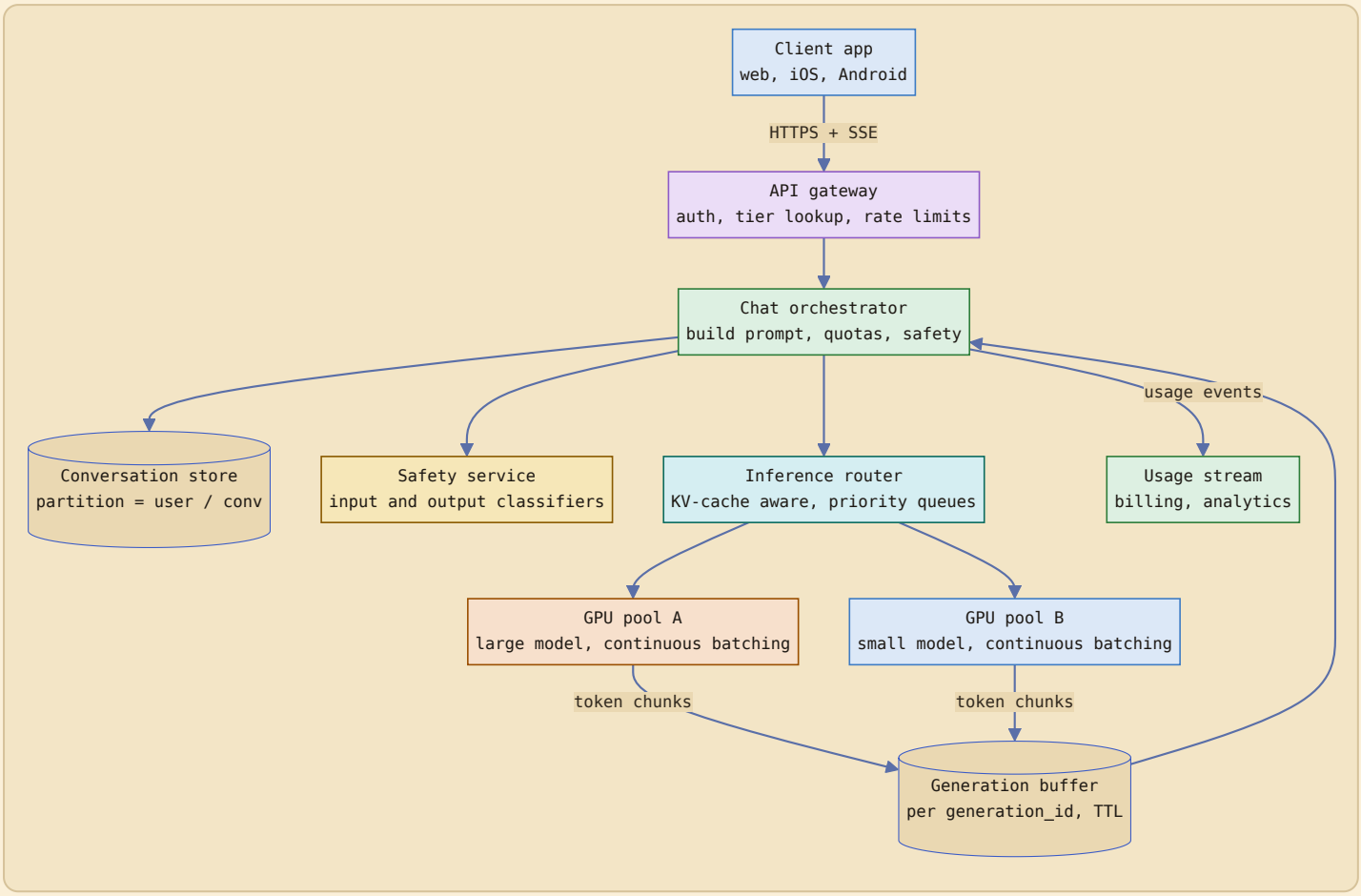
4. Data model

- **`conversations`** — partition `user_id`, clustering `updated_at` descending: `conv_id, title, model, created_at, archived`; listing chats is one range read.

- **`messages`** — partition `conv_id`, clustering `message_id` (time-ordered 64-bit): `parent_id, role (system/user/assistant/tool), content, content_ref (blob for large text or files), model, token_counts {input, cached_input, output}, status (streaming/complete/stopped/failed/blocked), safety_labels, created_at`. **Messages form a tree**: editing or regenerating adds a sibling under the same parent; the UI shows the selected branch, and a new turn's prompt walks from the new message up to the root.
- **`generation_buffer`** (in-memory with TTL, e.g. Redis Streams) — `generation_id` → ordered `(seq, text)` chunks, `status`, `replica_id`, TTL 15 minutes; decouples the GPU from the client connection.
- **`usage_ledger`** — `(user_id, window_start) → input_tokens, cached_tokens, output_tokens, cost_units`, hot counters in memory, settled totals durable. **`tiers`** — `tier → models, tokens per minute, messages per window, max context, priority`.
- **Router state** (in memory, rebuildable) — `prefix_block_hash → set of replicas` plus per-replica load (queue depth, free KV blocks).

A wide-column or key-value store (Cassandra/ScyllaDB/DynamoDB-style) fits both main tables: every query is by partition key, writes are append-mostly, data grows without bound (Sharding).

5. The architecture, annotated



1. `POST /messages` with `client_msg_id`: the **API gateway** checks the session, looks up the **tier** and applies per-user and per-organisation token-bucket request limits (Rate Limiting); the SSE stream opens immediately.
2. The **orchestrator** dedupes on `client_msg_id`, saves the user message, **builds the prompt** (system prompt + branch history, truncated or summarised to fit + new message), counts tokens and checks the **token quota** (Deep dive 4).
3. The **safety service** classifies the input; blocked requests get a refusal and never reach a GPU (Deep dive 5).
4. The **inference router** picks a **pool** by model and tier, then a **replica** by prefix-cache hits and load (Deep dive 3); if all are full the request waits in a **priority queue** (paid before free).
5. The **inference server** joins the request to its running batch (**continuous batching**, Deep dive 2), runs prefill, and appends `(seq, text)` chunks to the **generation buffer**.

6. The orchestrator reads the buffer, runs the **streaming safety check** and writes `delta` events.
7. At the end it saves the assistant message with token counts, settles the quota, and emits a **usage event** (Message Queue).

6. Deep dives

Deep dive 1 — Token streaming: SSE, resume and cancellation.

- *Wire format:* each event is a few lines (`id: 812`, `event: delta`, `data: {"text":" world"}`, blank line), parsed with a `fetch` stream reader (`EventSource` can't send a POST body). One event per token would be 17.5M events/s; ~4 tokens per event cuts that to ~4.4M/s.
- *Proxies:* no hop may **buffer**: disable buffering at load balancers and reverse proxies, send `Cache-Control: no-cache`, use HTTP/2 to multiplex streams over one TCP connection, and send a `: ping` comment every ~15 s while queued so idle timeouts don't fire.
- *Resume:* read the buffer from `Last-Event-ID + 1`, then follow the live tail ("cursor + replay + live" for one answer). Without the buffer, a phone switching from Wi-Fi to 5G kills an 80%-done generation.
- *Back-pressure:* the GPU never waits on a client; the gateway reads the buffer at the client's pace (Backpressure).
- *Cancellation:* "stop" sets a flag checked between decode steps, freeing the sequence's KV blocks at once. A silent disconnect does *not* cancel (the user may reconnect), but a generation unread for 60 s is cancelled.

Deep dive 2 — GPU inference: prefill, decode, continuous batching and KV-cache memory.

- *Two phases.* **Prefill** processes all prompt tokens in parallel, compute-bound, producing the first token. **Decode** produces one token per step per sequence, reading all weights and every sequence's KV cache each step: **memory-bandwidth-bound** (3.35 TB/s on an H100 SXM).
- *KV cache.* In a transformer every token yields a **key** and **value** vector per layer; keeping them (320 KB/token here) avoids recomputing the prompt for each new token.
- *Why batching is everything.* A decode step reads 140 GB ÷ 8 = 17.5 GB per GPU → ≈ **5.2 ms** whether the batch is 1 or 256 sequences; 1 sequence is ~190 tokens/s per replica. With 256 sequences of 3,500 tokens the step also reads 256 × 3,500 × 320 KB ≈ 287 GB of KV (≈36 GB per GPU, ≈10.7 ms), so ~16 ms per step → ~60 tokens/s per sequence × 256 ≈ **15K tokens/s** theoretical ceiling; real systems reach perhaps half, hence the 6,000 planning figure.
- *Static vs continuous batching.* Static runs a fixed batch until its **longest** answer ends, so short answers and arrivals wait. **Continuous batching** (iteration-level scheduling, Orca, OSDI 2022) re-forms the batch **every decode step**: finished sequences leave, waiting ones join, utilisation stays high (Model Serving (and Batching)).
- *PagedAttention.* Reserving contiguous KV for each sequence's *maximum* length wastes most of it. **PagedAttention** (vLLM, SOSP 2023) stores KV in fixed-size **blocks** (e.g. 16 tokens) allocated on demand via a block table, like OS virtual-memory pages; waste drops to the last partial block, and the paper reports 2–4× throughput at the same latency.
- *Prefill hurts decode.* A 30,000-token prefill can stall every decoding sequence for hundreds of milliseconds (a visible "hiccup"). Fixes: **chunked prefill** (split long prefills to share steps with decodes; the simple default) or **prefill/decode disaggregation** (separate pools per phase, shipping KV between them; for large fleets with long prompts).
- *Admission control.* Admit a sequence only if free KV blocks cover its prompt plus an output reserve; if memory runs out mid-generation, **preempt** the lowest-priority sequence (drop its KV, recompute later) rather than crash.

Deep dive 3 — KV-cache-aware routing and prefix caching.

- *The opportunity.* Turn 12 repeats turns 1–11 and every request shares the system prompt. If that **prefix**'s KV is still on a replica, prefill runs only on new tokens: a 60% hit rate turns 3,000

prefill tokens into 1,200, **2.5× less prefill** and much lower TTFT.

- *Finding a prefix.* vLLM's **automatic prefix caching** hashes each full KV block together with all tokens before it (so a block hash identifies the whole prefix); a new request is hashed block by block, matches reused, the rest computed; finished requests' blocks stay in an LRU pool. SGLang's **RadixAttention** uses a radix (prefix) tree.
- *The catch: the cache is per replica.* Round-robin over 3,000 replicas almost never sends turn 12 to turn 11's replica, so the **router** must be cache-aware:
  1. Compute the request's block hashes (or the hash of the first N tokens).
  2. Find replicas holding the longest matching prefix, **approximately** (remember where recent prefixes went) or **precisely** (servers publish "block stored/evicted" events into a `block_hash → replicas` index; the open-source llm-d scheduler offers both).
  3. Score `w1 × cached_fraction − w2 × queue_depth − w3 × KV_memory_used` and pick the best; load terms stop hot prefixes piling onto one replica.
- *Cheap version, session affinity:* consistent hashing on `conversation_id` picks a "home" replica, spilling to the least-loaded when home is busy; captures most multi-turn benefit.
- *Pricing follows:* public APIs charge less for cached input; OpenAI's prompt-caching guide describes routing on a hash of the prompt's first tokens. Report **cache hit rate** per pool beside GPU utilisation.
- *Not semantic caching:* prefix caching reuses **computation** and yields exactly the model's answer; Semantic Caching reuses a **previous answer** to a similar question, risky for open-ended chat.

Deep dive 4 — Quotas, rate limits per tier, priority and cost.

- *What to limit:* requests per minute protect API servers; **tokens per minute** protect GPUs (one 100K-token request costs as much as hundreds of short ones); product quotas ("N messages on the top model per few hours" for free users) protect the budget. Public LLM APIs use the same split per organisation and project.
- *Reserve, then settle:* output length is unknown, so **reserve** `input_tokens + max_output_tokens` from the user's token bucket and **refund** the unused part at the end; otherwise 50 parallel streams could each pass and overspend together.
- *Counters:* in memory, sharded by `user_id`, atomic check-and-decrement script. A few percent of over-use during failover is acceptable, so the limiter **fails open** for paid tiers and **closed** for anonymous traffic.
- *Priority and isolation:* paid traffic gets its own queues and large customers **reserved replicas**; free traffic uses the rest: a bulkhead so a free-tier surge can't starve paying users.
- *Cost per message* = uncached prefill tokens × prefill cost + output tokens × decode cost. Levers, biggest first: route easy requests to a **smaller model**; raise **batch size** and **cache hit rate**; cap `max_output_tokens` per tier; **auto-scale** pools over the day and fill idle night GPUs with batch jobs (GPU Auto-Scaling). Loading 140 GB of weights takes minutes, so scaling follows a forecast.

Deep dive 5 — Safety filters without killing latency.

- *Input:* a small classifier (e.g. Meta's Llama Guard, a fine-tuned model labelling prompts and responses against a risk taxonomy) runs **in parallel** with prompt building and quota checks, adding little TTFT; blocked prompts never reach the large model.
- *Output, while streaming:* hold back a short window (say 20–50 tokens) and classify the text so far each time it fills; if flagged, end with `error{code: "content_filtered"}` and the client replaces the shown text. A slightly later first token buys never showing a full harmful answer; provider docs warn partial output is harder to moderate, hence the window.
- *Failure policy:* safety down → **fail closed** for free and anonymous traffic and alert; on the critical path, it gets its own capacity and a circuit breaker. Account-level abuse detection runs offline on the usage stream.

7. The failure walk

1. **Replica crashes mid-generation:** its KV is gone; the orchestrator **re-submits prompt plus tokens so far** to another replica, which prefills and continues. A 1–2 s pause, `seq` numbers continue (no duplicates); after three failures the partial answer is saved as `failed`.
2. **Client connection drops:** the GPU keeps writing to the buffer; the client reconnects with `Last-Event-ID` to any gateway and resumes.
3. **Router restarts, losing its prefix index:** correctness unaffected (a cache of a cache); hit rate and TTFT suffer for minutes while it refills.
4. **Launch-day 5× spike:** admission control caps per-tier queue length; beyond it free users get "at capacity" or the smaller model (Graceful Degradation); paid traffic keeps reserved replicas; pre-scale before announced launches.
5. **Conversation store slow or down:** generation proceeds from the history the client already shows; writes go to a durable retry queue; listing old chats shows an error banner.
6. **Pathological prompt** (a 200K-token paste, or one built to make the model loop): per-tier context limits, `max_output_tokens` and a per-request time budget bound it; tokens-per-minute limits stop repeats.
7. **A region loses its GPUs:** stateless orchestrators send new and retried generations to another region within data-residency rules (Multi-Region and Cell-Based Architecture).

8. Trade-offs

Decision	Chosen	Alternative	Why
Streaming transport	SSE over HTTP/2	WebSockets	One-way traffic, built-in resume, proxy-friendly
GPU to client coupling	Generation buffer with `seq`	Direct socket write	Survives disconnects, enables resume and remote stop
Batching	Continuous batching + paged KV	Static batches	Higher GPU use, short answers return early
Replica choice	Prefix-cache score + load	Round robin	Round robin destroys multi-turn cache hits
Long prompts	Chunked prefill	Disaggregated prefill/decode pools	Simpler; disaggregate later at larger scale
Quota accounting	Reserve max then settle	Charge after completion	Prevents parallel overspend
Output safety	Incremental check with small hold-back	Check after full answer	Harmful text never fully shown, small latency cost

9. The follow-up questions

- **"Measuring and setting latency targets?"** Track **TTFT** (queueing + prefill) and **TPOT** separately per model and tier, plus tokens/s per GPU. Bigger batches raise throughput *and* TPOT, so cap batch size to keep TPOT under reading speed, then scale replicas for TTFT. *Trap:* reporting only average latency.
- **"1M-token contexts?"** At 320 KB/token that's ~320 GB, nearly a replica's free memory for one user: KV quantisation (8-bit or lower), offloading cold KV blocks to CPU memory or SSD, and a separate long-context pool with its own quota and price.
- **"Tools and web search?"** The model emits a tool call; the orchestrator pauses the stream (`event: tool_call`), runs the tool, appends the result and resumes; the prefix is unchanged, so the request returns to the same replica to reuse its KV.
- **"Rolling out a new model?"** A new pool, a canary traffic share, compare quality, safety and cost per message, then shift gradually (Canary Deployment).

Pattern manifest

Pattern	Where it served	Module
Server-Sent Events	Token streaming, `Last-Event-ID` resume	Module 02
Message Queue	Usage events to billing and analytics	Module 02
Sharding	Conversations by user, messages by conversation	Module 03
Consistent Hashing	Home replica per conversation for cache affinity	Module 03
Idempotency	`client_msg_id` on send, `seq` on stream chunks	Module 05
Bulkhead	Separate queues and reserved capacity per tier	Module 05
Circuit Breaker	Safety service and conversation store calls	Module 05
Graceful Degradation	Smaller model or "at capacity" for free tier under load	Module 05
Rate Limiting	Requests and tokens per minute per tier	Module 08
Cursor-Based Pagination	Conversation list, stream resume cursor	Module 08
Backpressure	Slow clients read from the buffer, never block GPUs	Module 10
Model Serving (and Batching)	Continuous batching, paged KV cache	Module 11
GPU Auto-Scaling	Forecast-driven pool sizing over the day	Module 11

Interview one-liner: "The web tier is ordinary; the GPUs are the system. Stream tokens over SSE from a resumable generation buffer, batch continuously with a paged KV cache, route each turn to the replica that already holds its prefix, meter tokens with reserve-then-settle quotas per tier, and check safety incrementally on the stream."

TL;DR

- **Problem:** 350K concurrent streamed answers, each holding about 1 GB of GPU memory, at a cost that dominates the business.
- **Mechanism:** API gateway with tier limits → orchestrator (prompt, quota, safety) → KV-cache-aware router → GPU pools with continuous batching and PagedAttention → generation buffer → SSE.
- **Hard parts:** GPU memory (KV sizing, admission, preemption), cache-aware routing without hot spots, resumable streams, token quotas with unknown output length, safety on partial text.
- **Price:** a huge, slow-to-scale GPU fleet; routing tightly coupled to engine internals; per-tier policy complexity.

Sources

- Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention", SOSP 2023 — https://arxiv.org/abs/2309.06180
- Yu et al., "Orca: A Distributed Serving System for Transformer-Based Generative Models", OSDI 2022 — https://www.usenix.org/conference/osdi22/presentation/yu
- Zheng et al., "SGLang: Efficient Execution of Structured Language Model Programs" (RadixAttention) — https://arxiv.org/abs/2312.07104
- vLLM docs, Automatic Prefix Caching — https://docs.vllm.ai/design/prefix_caching.html
- llm-d docs, precise prefix-cache-aware routing — https://llm-d.ai/docs/guides/precise-prefix-cache-aware
- OpenAI API docs: rate limits, streaming responses, prompt caching — https://developers.openai.com/api/docs/guides/rate-limits , https://developers.openai.com/api/docs/guides/streaming-responses , https://developers.openai.com/api/docs/guides/prompt-caching

- Meta Llama 3.1 70B model configuration (layers, KV heads, hidden size) — https://huggingface.co/unsloth/Meta-Llama-3.1-70B-Instruct/raw/main/config.json
- NVIDIA H100 product specifications (80 GB, 3.35 TB/s) — https://www.nvidia.com/en-us/data-center/h100/
- Inan et al., "Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations" — https://arxiv.org/abs/2312.06674
- WHATWG HTML Standard, Server-sent events (`Last-Event-ID`) — https://html.spec.whatwg.org/multipage/server-sent-events.html

---

Case 4 — Distributed Cache Cluster (Redis Cluster / Memcached-like)

The prompt (in short). The shared in-memory key-value cache a large company's services use: sub-millisecond `GET`/`SET`, terabytes across many machines, surviving machine failures, resizing without downtime (Redis Cluster, Memcached behind Facebook's mcrouter, ElastiCache, Memorystore).

What makes it hard. (1) **Spreading keys** over 100+ nodes so every client finds the right one without a central lookup; (2) **moving keys while traffic flows**; (3) **failover** with asynchronous replication, leaving a window where acknowledged writes are lost; (4) **skew**: one celebrity key can exceed any node's capacity, and one dead node can stampede misses onto the database.

1. Requirements and numbers

Functional: `GET`, `SET` with TTL, `DEL`, multi-key `MGET`, atomic `INCR`, compare-and-set; hashes and sorted sets for the Redis flavour; online node add and remove; per-tenant namespaces.

Non-functional: single-key p99 < 1 ms in a data centre; ≥ 99.95% availability; linear scaling; a node failure loses at most a few seconds of writes and recovers in seconds; a cache outage must not take down the database behind it.

Numbers (assumptions):

- **10M ops/s at peak**, 90% reads.
- **2B keys** × (1 KB value + ~100 bytes overhead: key string, pointers, TTL, eviction metadata) ≈ **2.2 TB**.
- **Planning throughput:** one single-threaded **shard** (process owning part of the key space) handles **~100K ops/s** with headroom (an assumption; measure with `redis-benchmark` or `memtier_benchmark`).
- Shards for throughput: **100 primaries**; for memory: 2.2 TB ÷ 48 GB ≈ 46. **Throughput decides**: 100 primaries of **22 GB** each.
- One **replica** per primary → **200 shard processes**, 4 per machine (32 cores, 256 GB RAM, leaving room for fork copy-on-write and buffers) → **50 machines**, each replica on a different machine and rack.
- Network: 10M × 1.1 KB ≈ 11 GB/s ≈ 88 Gbit/s ÷ 50 ≈ **1.8 Gbit/s per machine**, comfortable on 25 Gbit/s NICs, but one hot key can saturate a machine's link (Deep dive 4).
- Redis Cluster's **16,384 hash slots** ÷ 100 ≈ **164 slots per primary**.

What the numbers say: per-shard CPU and per-machine network limit us, not memory: spread *load*, not just *data*.

2. The five-questions pass

- **Flow:** synchronous, sub-millisecond, over long-lived pooled TCP connections with **pipelining** (several commands sent before reading replies) (Five Questions).
- **Storage:** RAM is the store; for cache-aside data the **database behind** is truth and the cache can always refill. When Redis is a primary store (sessions, counters, leaderboards), persistence and replication matter more (Deep dive 5).
- **Speed:** one round trip to the right node, never two, so the **routing table lives in the client or a proxy**.
- **Failure:** detect in seconds, promote a replica, accept a small loss window, protect the database from the miss storm.

- **Growth:** add nodes by **moving key slices**, never rehashing everything; hot spots are viral keys, big keys (a 50 MB list) and keys expiring together.
- *Skeleton:* primary shards with replicas, a key → slot → shard map in every client, a failure detector, a migration protocol, per-shard eviction and persistence.

3. API

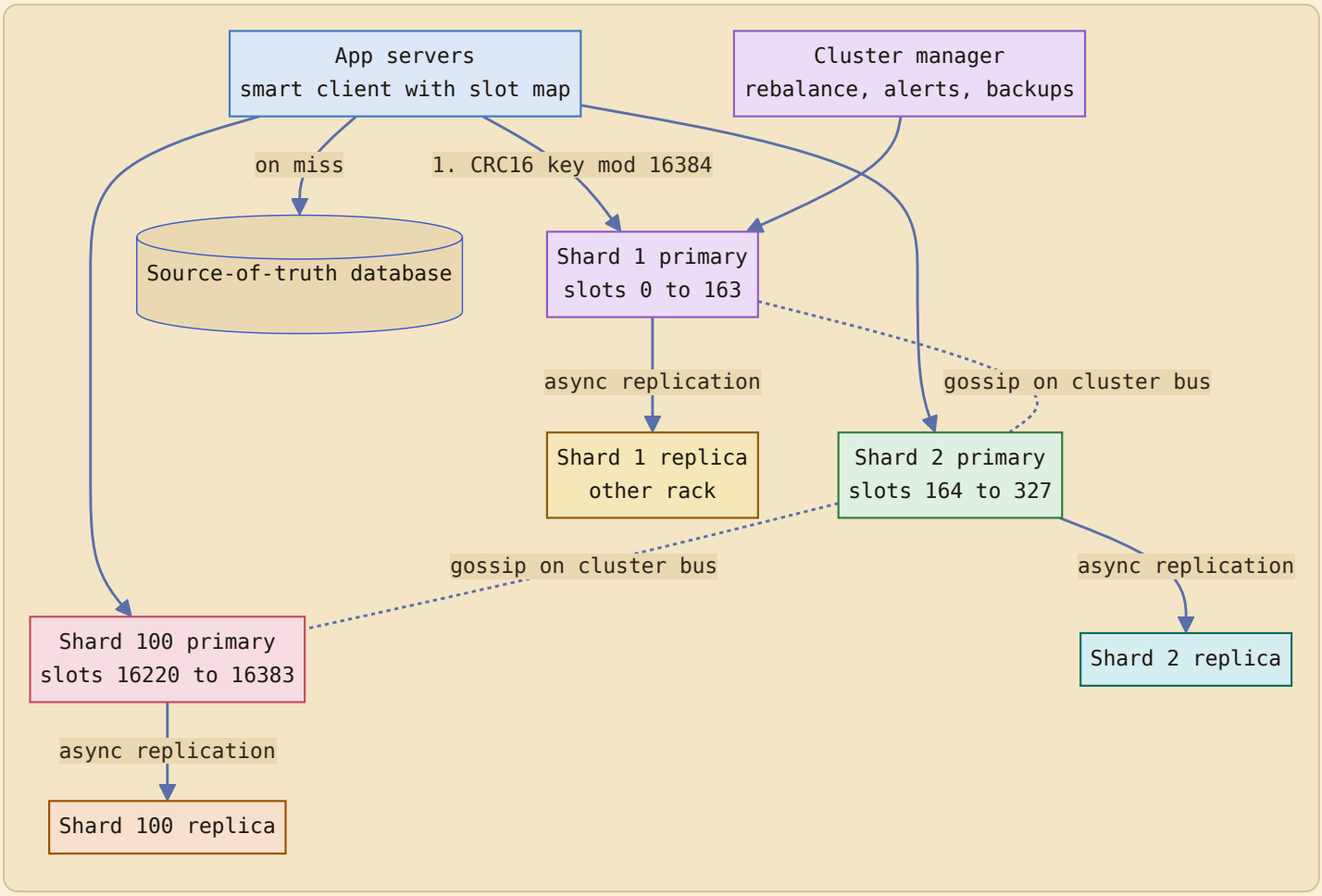
Command	Meaning
`GET k` / `MGET k1 k2 …`	Read; `MGET` across shards is split by the client and fanned out in parallel
`SET k v [EX seconds] [NX]`	Write with optional TTL; `NX` = only if absent (used for locks and leases)
`DEL k`, `EXPIRE k s`	Delete, set TTL
`INCR k`	Atomic counter on the owning shard
`gets k` → `(v, cas_token)` / `cas k v cas_token`	Memcached compare-and-set: the write succeeds only if nobody changed the value since the read
`WAIT numreplicas timeout`	Redis: block until this connection's earlier writes reach N replicas (narrows, not removes, the loss window)
`CLUSTER SLOTS` / `CLUSTER SHARDS`	Redis: return the slot → node map so a client can route directly
Admin: `add-node`, `reshard --from A --to B --slots 164`, `failover`	Operator or automation commands

Routing errors: `-MOVED 3999 10.0.0.7:6379` ("this slot lives there now, update your map") and `-ASK 3999 10.0.0.8:6379` ("slot migrating, try there for this one request only").

4. Data model

- **Inside a shard:** a hash table `key → object` with the value (string, hash, list, set, sorted set), optional expiry, and a few bits of eviction metadata (Redis: an approximate last-access clock or a small logarithmic frequency counter).
- **Memcached memory layout:** 1 MB **pages** assigned to **slab classes**, each cutting pages into fixed chunks (80 B, 104 B, 136 B, …); an item takes the smallest fitting chunk and each class has its own LRU. This avoids `malloc` fragmentation but wastes the item-to-chunk gap.
- **Slot map:** `slot (0..16383) → {primary_node_id, replica_node_ids, config_epoch}`, held by every node and cached by clients. The **config epoch** increases on every failover or slot move, so the newest claim wins disagreements.
- **Node table:** `node_id → address, role, flags (ok / PFAIL / FAIL), last pong, slots`.
- **Key naming:** `tenant:entity:id` (`shop:user:42`), with a **hash tag** in braces (`shop:{user:42}:cart`) when keys must share a shard (Deep dive 1).

5. The architecture, annotated



1. At startup the **smart client** (a library that knows the cluster layout) calls `CLUSTER SHARDS` on any node and caches the 16,384-entry **slot map**.
2. For `GET user:42`: `slot = CRC16(key) mod 16384` (**CRC16**, a cheap 16-bit checksum used as a hash; Redis Cluster uses 14 of its bits), then the slot's primary on a pooled connection: **one hop**.
3. The shard answers from RAM; on a miss the app reads the database and writes back with a TTL (Cache-Aside).
4. Writes go to the primary, which replies immediately and **streams the write asynchronously** to its replica (Primary-Replica).
5. Nodes **gossip** on a separate **cluster bus** (by default data port + 10,000) about health and slot ownership; no coordinator on the request path.
6. The **cluster manager** (automation off the data path) watches load, moves slots, runs backups and pages humans.

Memcached variant: servers know nothing of each other; distribution lives in the client (consistent hashing) or a proxy such as Facebook's **mcrouter** or Twitter's **twemproxy**, which can also replicate by writing to several pools.

6. Deep dives

Deep dive 1 — Sharding: hash slots vs consistent hashing, and hash tags.

- *Plain modulo* (`hash(key) mod N`) moves almost every key when N goes from 100 to 101: a near-total cache wipe.
- **Consistent hashing** (the Memcached tradition, often **ketama**): servers at many **virtual node** points on a ring, a key owned by the first server clockwise; adding a server moves about 1/(N+1) of keys and needs no metadata beyond the server list, suiting stateless clients and proxies (Consistent Hashing; other schemes in Module 19).

- **Hash slots** (Redis Cluster) add indirection: keys hash to one of a **fixed** 16,384 slots and an **explicit table** maps slots to nodes. `CRC16("user:42")` = 0x3E08 = 15,880 → **slot 15,880**, on shard ⌊15,880 ÷ 163.84⌋ + 1 = **shard 97** (implementation check from the spec: CRC16("123456789") = 0x31C3). Adding a node **moves whole slots**, and operators can move slots off a hot node; the cost is a table every client must keep current.
- **Which:** slots for precise control and a versionable ownership record for failover; a ring for no shared metadata. Both move few keys on resize (Sharding).
- **Hash tags.** Multi-key operations (`MGET`, transactions, Lua) need all keys on one shard; with `{…}` only the first braces' content is hashed, so `{user:42}:profile` and `{user:42}:cart` both land in slot 15,880. Trap: over-tagging (`{tenant}` on a big tenant's keys) recreates a hot shard.
- **Resharding without downtime (Redis).** Moving slot 8 from A to B: mark B `IMPORTING 8` and A `MIGRATING 8`; repeatedly `MIGRATE` batches of slot-8 keys from A to B (each key atomically). A still answers keys it holds and replies `-ASK B` for moved ones, which the client sends to B prefixed with `ASKING`. When A's slot is empty both get `SETSLOT 8 NODE B`, the new owner bumps its config epoch, and A answers `-MOVED`, updating client maps. Moving 164 slots (22 GB) at a throttled 100 MB/s takes ~4 minutes per added node, with a small share of requests paying an extra hop.

Deep dive 2 — Replication and failover.

- *Asynchronous replication:* the primary acks before the replica has the write, so **an acknowledged write can be lost** if the primary dies first; the Redis Cluster spec states this window and calls its merge rule "last failover wins". `WAIT 1 100` blocks until one replica has the write, shrinking the window for important keys, but it is not consensus.
- *Failure detection:* nodes ping over the cluster bus; no answer within `cluster-node-timeout` (e.g. 5 s) → **PFAIL** (a local opinion); a **majority of primaries** reporting PFAIL/FAIL within twice the timeout → **FAIL**, broadcast cluster-wide (gossip-based detection: Module 19 — Failure detection).
- *Election:* the failed primary's replicas wait a delay ranked by how current their copy is (most current first), then request votes; a primary votes for at most one replica of a given failed primary within twice the timeout, so two can't both win. A replica with a **majority of primaries'** votes promotes itself with a new, higher config epoch and announces its slots; clients get `-MOVED` and refresh. Outage for those slots ≈ node timeout + 1–2 s.
- *Partitions:* a primary cut off with a minority keeps accepting writes until the node timeout, then **stops**; those are the writes that can be lost when the majority promotes a replica. A smaller timeout means less loss but more false failovers on GC-like pauses or network blips.
- *Placement:* never a primary and its replica on one machine or rack; **replica migration** moves a spare replica to a primary that lost its own, so a second failure doesn't leave slots uncovered.
- *Memcached tier:* no replication, so a dead node is a block of misses. Facebook's small **Gutter** pool (~1% of servers) takes retried failed gets with quickly expiring entries, shielding the database.

Deep dive 3 — Routing: smart client vs proxy.

- **Smart client:** holds the slot map or ring and connects to every shard. *Pros:* one hop, lowest latency, no extra servers. *Cons:* every language needs a correct client (MOVED/ASK, map refresh, pooling), and 5,000 app servers × 100 shards × 4 connections = **2M TCP connections**, each costing shard memory.
- **Proxy:** clients speak the plain protocol to a nearby proxy (mcrouter, twemproxy, Envoy's Redis proxy) that knows the topology. *Pros:* thin clients, **connection pooling and pipelining**, one place for prefix routing, replicated pools, shadow traffic and failover policy. *Cons:* an extra hop (~0.1–0.3 ms) and more machines. mcrouter's README cites almost 5 billion requests/s at peak across Facebook and Instagram.
- **Sidecar proxy** (one per app host on localhost) is the common middle: the hop is a local socket and connections drop to proxies × shards (Sidecar).

- Facebook's NSDI 2013 hybrid: **gets go directly to memcached over UDP** (connectionless, lower latency; a lost packet counts as a miss), **sets and deletes via a local mcrouter over TCP** for reliability.

Deep dive 4 — Hot keys, big keys and thundering herds.

- *The problem:* a viral key at 500K reads/s vs one shard's ~100K ops/s budget, and 500K × 1 KB = 500 MB/s ≈ 4 Gbit/s on one link. Even keys don't mean even *traffic*.
- *Find them:* top-K counters on sampled commands at client or proxy (`redis-cli --hotkeys` under an LFU policy); alert on per-shard skew.
- *Fix 1, near cache:* clients keep the hottest few thousand keys in process for 1–5 s; 500K reads/s over 2,000 app servers is 250 each, fully absorbed. Staleness is bounded by the TTL or removed with Redis 6+ **client tracking** (invalidation messages when a read key changes).
- *Fix 2, key splitting:* N copies `post:9:#0 … post:9:#7` in different slots; readers pick one at random, writers update all eight; good for read-mostly data.
- *Fix 3, replica reads:* send hot-slot reads to replicas (`READONLY`), accepting slight staleness.
- *Thundering herd:* when a hot key expires thousands miss at once. **Leases** (Facebook's memcache paper): on a miss the server gives a lease token to **one** client, at most once every 10 seconds per key; others briefly wait and retry, and a set is accepted only with a valid token (also blocking stale sets after a delete). The paper reports a database peak falling from 17K/s to 1.3K/s for herd-prone keys (Cache Stampede Prevention).
- *Synchronized expiry:* **jitter** TTLs (e.g. 3,600 s ± 10%).
- *Big keys:* a 50 MB list blocks a single-threaded shard while read or deleted; cap value size, bucket collections, delete with `UNLINK` (background free).

Deep dive 5 — Eviction, memory and persistence.

- *Eviction* at `maxmemory`, by `maxmemory-policy`: `allkeys-lru` (usual for a pure cache), `allkeys-lfu` (better when a stable key set is popular), `volatile-*` (only TTL keys), `allkeys-random`, `volatile-ttl` (soonest-expiring first), or `noeviction` (reject writes; right for a primary store). Redis approximates LRU by **sampling** a few keys (default 5) and evicting the oldest, far cheaper than a true LRU list; LFU uses a small decaying logarithmic counter per key (policy families: Module 19 — Cache eviction).
- *Headroom:* keep 20–30% free: replication and AOF buffers don't count toward `maxmemory`, and the persistence fork's **copy-on-write** (pages copied only when the parent modifies them) can nearly double memory under heavy writes.
- *Persistence (Redis):*
  - **None:** pure cache; restart = empty shard refilled from the database. Fastest, simplest.
  - **RDB snapshots:** a forked child writes a compact point-in-time file every few minutes: fast restarts, good backups, minutes of writes lost on crash.
  - **AOF (append-only file):** every write appended and fsynced per `appendfsync`: `always` (safest, slowest), `everysec` (default, lose at most about 1 s), `no` (OS decides); rewritten in the background to stay compact (Write-Ahead Log).
  - **RDB + AOF:** the Redis docs' recommendation for PostgreSQL-like safety; AOF is used on restart as the more complete.
- *Choice:* the shared cache tier runs **no persistence** on primaries and RDB on replicas for warm restarts; primary-store namespaces (sessions, rate-limit counters) get a separate cluster with AOF `everysec` and `noeviction`. Mixing "may evict" and "must keep" data on one cluster is the classic mistake.
- *Warm-up:* route reads to a new or restarted node gradually, or pre-fill it from a replica or snapshot, so its misses don't spike the database (Graceful Degradation).

7. The failure walk

1. **Primary crashes:** its 164 slots time out; after the node timeout (5 s) plus election (~1–2 s) the replica is promoted and clients follow `-MOVED`. Writes acked in the last instant may be lost. Apps fall back to the database for those keys behind a circuit breaker and request coalescing.
2. **A machine (4 shards) dies:** replicas elsewhere fail over in parallel; capacity drops 4% until a replacement resyncs.
3. **Partition isolates 10 primaries:** the minority side refuses writes after the node timeout and the majority promotes replicas; on healing, old primaries see a higher config epoch and become replicas, discarding divergent writes.
4. **Cache-wide cold start** (a deploy bug flushed everything): hit rate zero and 9M reads/s would hit the database. Leases or single-flight per key, per-app database concurrency caps (load shedding), stale or default data for non-critical features, gradual refill; never assume the database absorbs a full miss storm.
5. **Hot key overloads a shard** (p99 up, CPU 100%): client near-caches absorb reads; ops can move that slot alone to a less-loaded node.
6. **Resharding stalls mid-slot** (target dies): the slot stays `MIGRATING` and the source serves keys it still has; fix the target or migrate back. Each `MIGRATE` is atomic, so no key is in two places.
7. **Fork pause** during a snapshot: page-table copying stalls the process briefly, longer for bigger shards; keep shards small and snapshot on replicas.

8. Trade-offs

Decision	Chosen	Alternative	Why
Key placement	16,384 hash slots + slot map	Consistent hashing ring	Explicit ownership, targeted moves, works with failover epochs
Routing	Smart client (sidecar proxy where needed)	Central proxy tier	One hop; proxies only where clients are weak
Replication	Async, one replica per primary	Synchronous or consensus per write	Sub-ms writes; small loss window acceptable for a cache
Failure detection	Gossip + majority-of-primaries vote	External coordinator (Sentinel, ZooKeeper)	No extra system; slots and health in one protocol
Shard size	Many small shards (22 GB)	Few large shards	Faster failover and resync, shorter fork pauses, spreads CPU
Eviction	`allkeys-lru` for cache, `noeviction` for stores	One policy everywhere	Different data, different guarantees
Persistence	None on cache primaries, RDB on replicas	AOF everywhere	Cache is rebuildable; disk I/O costs latency
Hot keys	Near cache + key splitting + leases	Bigger machines	Skew beats any single machine

9. The follow-up questions

- **"Consistent with the database?"** Not strictly: cache-aside with **delete on write**, short TTLs, and invalidation from the database's change log (Change Data Capture; Facebook's daemons tail the commit log to issue deletes); leases stop a slow reader writing an old value after a delete. *Trap:* claiming atomic cache-and-database updates without a distributed transaction.
- **"Why not just Memcached?"** Multithreaded and lean for plain strings, but no replication, persistence or data structures: right for a pure look-aside cache behind a router; Redis for structures, atomic ops or failover. (Since 2024 Redis has changed licences, and the Linux Foundation's **Valkey** fork continues the BSD-licensed codebase; managed caches offer either.)
- **"Across regions?"** One cluster per region caching its local database replica, invalidations following database replication (Multi-Region and Cell-Based Architecture).

- **"Add 20% capacity at peak?"** Add 20 primaries with replicas; at 120 primaries each owns 16,384 ÷ 120 ≈ 137 slots, so every old primary gives up ~27 and every new one receives ~137, moved a few at a time watching p99, with ASK/MOVED keeping traffic flowing. Better: scale before the peak.
- **"Multi-tenancy?"** Separate clusters for noisy or critical tenants (a bulkhead); prefixes plus per-tenant proxy quotas for small ones.

Pattern manifest

Pattern	Where it served	Module
Sharding	16,384 slots spread over 100 primaries	Module 03
Consistent Hashing	Memcached clients and proxies (ketama ring)	Module 03
Primary-Replica	Async replica per shard, promotion on failure	Module 03
Write-Ahead Log	AOF persistence for store-mode clusters	Module 03
Cache-Aside	How applications fill and invalidate the cache	Module 04
Cache Stampede Prevention	Leases, single-flight, TTL jitter	Module 04
Circuit Breaker	Database fallback during shard failover	Module 05
Bulkhead	Separate clusters for cache vs store, noisy tenants	Module 05
Graceful Degradation	Gradual warm-up, stale data on cold start	Module 05
Sidecar	Local proxy for routing and connection pooling	Module 08
Change Data Capture	Database-driven invalidation	Module 10

Interview one-liner: "Hash keys to 16,384 slots on many small primaries with async replicas on other racks; smart clients route in one hop and follow MOVED/ASK during migration; gossip detects failures and a majority vote promotes replicas; skew is the real enemy, fought with near caches, key splitting, leases and TTL jitter."

TL;DR

- **Problem:** 10M ops/s at sub-millisecond latency over 2.2 TB, with online resizing and failover.
- **Mechanism:** hash slots (or a consistent-hash ring) → primary shards with async replicas → routing table in smart clients or proxies → gossip failure detection and replica promotion → per-shard eviction and optional persistence.
- **Hard parts:** moving keys under load (MIGRATING/IMPORTING with ASK), the async-replication loss window, hot and big keys, protecting the database from miss storms.
- **Price:** small windows of lost writes, client complexity or an extra proxy hop, capacity planned around CPU and network rather than memory.

Sources

- Redis Cluster specification (16,384 slots, CRC16, hash tags, MOVED/ASK, gossip, PFAIL/FAIL, elections, write-loss windows) — https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/
- Redis docs, Key eviction (policies, sampled LRU, LFU) — https://redis.io/docs/latest/develop/reference/eviction/
- Redis docs, Persistence (RDB, AOF, `appendfsync`) — https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/
- Redis docs, Client-side caching reference (tracking, invalidation) — https://redis.io/docs/latest/develop/reference/client-side-caching/

- Nishtala et al., "Scaling Memcache at Facebook", NSDI 2013 (leases, Gutter, UDP gets, mcrouter, mcsqueal) — https://www.usenix.org/system/files/conference/nsdi13/nsdi13-final170_update.pdf
- mcrouter wiki (features, scale) — https://github.com/facebook/mcrouter/wiki
- twemproxy README (memcached/Redis proxy, ketama, pipelining) — https://github.com/twitter/twemproxy
- memcached wiki, User Internals (pages, slab classes, per-class LRU) — https://github.com/memcached/memcached/wiki/UserInternals
- Linux Foundation launches Valkey (Phoronix report, March 2024) — https://www.phoronix.com/news/Linux-Foundation-Valkey

Module 23 — Case Studies: Edge and Messaging Building Blocks

What this module is. Three building blocks Module 13 did not design end to end: a **CDN**, a shared **rate limiter** service and a managed **publish-subscribe** service. The pattern lessons (CDN, Rate Limiting, Publish-Subscribe) are not re-taught; each case asks what changes when you *build and run* the block as a multi-tenant platform. Format as in Module 13.

Case 1 — Design a CDN

The prompt (in short). Build a multi-tenant CDN like Akamai, Cloudflare or CloudFront. Customers point a hostname at it; it serves their static files, video segments and API responses from about 300 sites worldwide, keeps content fresh when they purge, and protects their origins (their own servers).

What makes it hard. Caching one file is the CDN lesson. The platform problems: (1) **routing** each user to a close, healthy, not-overloaded site with only DNS and BGP; (2) **cache efficiency** with finite disk and a long-tailed catalogue; (3) **freshness**: a purge must reach 300 sites in seconds, including ones unreachable when sent; (4) **blast radius**: one bad config push can break every site at once.

1. Requirements and numbers

Functional: onboard a hostname (origin address, cache rules, TLS certificate); serve `GET`/`HEAD`, including byte ranges; purge by URL, by tag and by prefix; accelerate uncacheable (dynamic) requests; optional edge code; per-customer logs and analytics.

Non-functional: time to first byte (TTFB) on a hit under 30 ms for most users; 99.99% availability per hostname; at least 95% of bytes served without touching the origin; purge visible everywhere within ~5 s at p99; never serve one tenant's private response to another user.

Numbers (assumptions):

- **300 PoPs** (points of presence: CDN sites, each a rack or more of servers), **50 Tbps peak egress** (terabits per second leaving the CDN), average response **150 KB** → 50 Tbps ÷ 8 = 6.25 TB/s ÷ 150 KB ≈ **40M requests/s**.
- The biggest PoP carries ~2% → **1 Tbps, ~0.8M req/s**. At ~80 Gbps of real throughput per server: 1,000 ÷ 80 ≈ 13 servers, plus spares → **16 servers**.
- Each server holds **30 TB** of cache (SSD for hot objects, HDD for large video). Shared well, the PoP holds **480 TB**; if every server caches independently it holds **30 TB** of distinct content (Deep dive 2).
- Edge byte hit ratio 90% → 5 Tbps go upward. Parent tier hit ratio 80% → **1 Tbps reaches all origins combined**: 98% offload.
- Purges: **2,000 purge requests/s** platform-wide × 300 PoPs = 600K deliveries/s if sent one by one. Batched every 50 ms per PoP: 300 × 20 = **6,000 messages/s**.
- Access logs: 40M req/s × 400 B = 16 GB/s ≈ **1.4 PB/day** → aggregate at the edge and ship samples plus counters.

What the numbers say: bandwidth and disk decide the hardware, hit ratio the origin bill, purge fan-out the control plane.

2. The five-questions pass

- **Flow:** user → nearest PoP (DNS or anycast) → edge server → (miss) parent tier → (miss) origin; purges and config flow from a central control plane to every PoP.
- **Storage:** edge disks hold *disposable copies*; the origin is the source of truth. The control plane owns two durable things: customer config and the purge log.
- **Speed:** the hit path stays inside one PoP, with no cross-PoP or central lookups.

- **Failure:** servers, PoPs, parents and origins all fail; each failure should lower hit ratio or add latency, never return errors (stale content is acceptable).
- **Growth:** add servers to a PoP and PoPs to the map; the mapping system and consistent hashing absorb both.

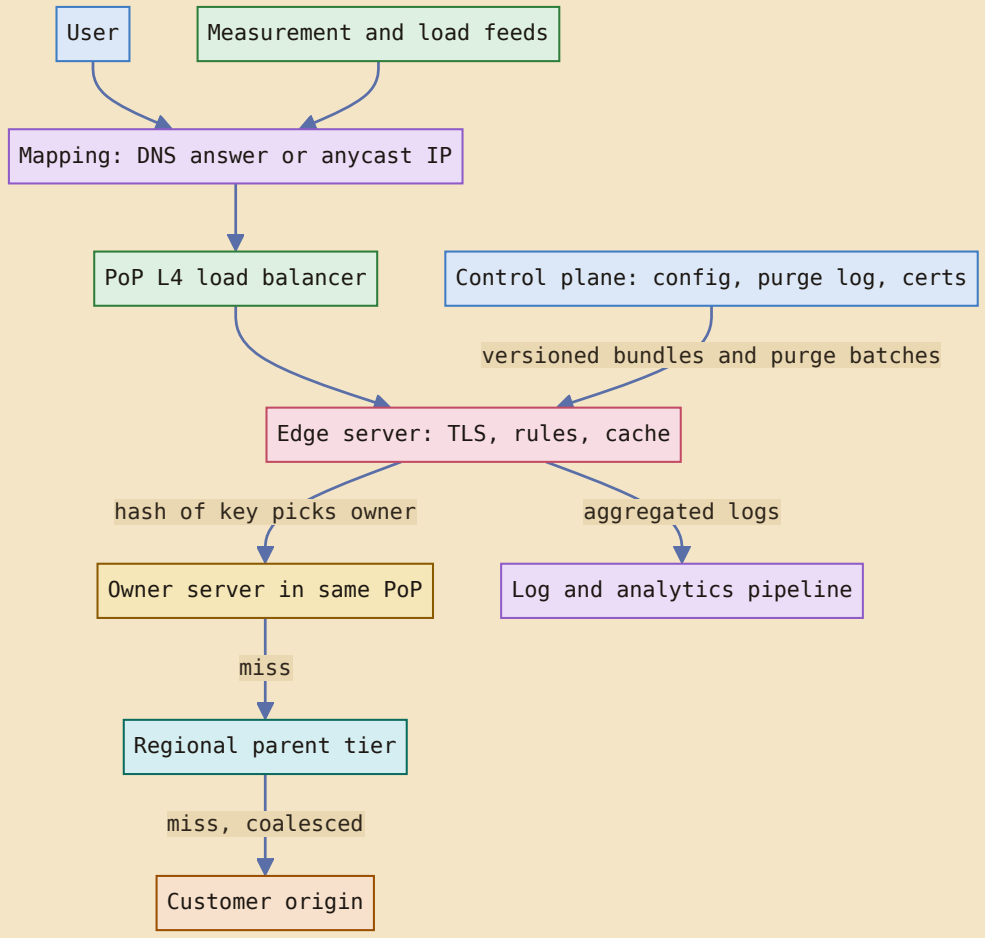
3. API

Plane	Call	Purpose
Control	`PUT /v1/properties/{host}` `{origin, rules[], tls_cert_id, version}`	Create or change a property (one customer hostname); returns a new config version
Control	`POST /v1/purge` `{host, urls[] | tags[] | prefixes[], mode: hard|soft}` → `{purge_id, seq}`	Invalidate content; `soft` marks it stale instead of deleting it
Control	`GET /v1/purge/{purge_id}` → `{pops_done, pops_total}`	Lets customers see propagation
Data	`GET /path` with `Range`, `If-None-Match`	The cached request itself
Origin → CDN	`Cache-Control`, `CDN-Cache-Control` (RFC 9213: a header aimed at CDNs only), `Surrogate-Key: product-42 home` (tags), `Vary`	Origin sets TTL and tags per response
CDN → client	`Age`, `Cache-Status` (RFC 9211: standard "hit/miss, and why" header)	Debugging which tier answered

4. Data model

- **Property config** (control plane, versioned): `host → {origin, rules, cert, version}`; compiled and shipped to every PoP as an immutable bundle.
- **Cache index** (per server, RAM): `hash(cache_key) → {disk_location, size, expires_at, etag, tags[], stored_at, purge_seq_at_store}`. The **cache key** = scheme + host + path + normalised query + the request headers listed in `Vary`.
- **Tag index** (per server): `tag → [key hashes]` so a tag purge is a lookup, not a disk scan.
- **Purge log** (control plane, replicated, append-only): `seq → {host, matcher, mode, created_at}`. Each PoP stores `last_applied_seq`.
- **Mapping table** (mapping system): `client network prefix → ranked PoP list`, recomputed every few seconds from latency measurements and PoP load.

5. The architecture, annotated



1. **Mapping** picks a PoP (Deep dive 1); the PoP's **L4 load balancer** spreads connections with consistent hashing (Maglev).
2. The **edge server** terminates TLS, applies the property's rules and computes the cache key; if another server in the PoP *owns* the key, it fetches from the owner (Deep dive 2).
3. On a PoP miss the owner asks the **parent tier** (a few large regional PoPs, the origin shield), which merges identical concurrent misses into one origin fetch (**request coalescing**, the global form of stampede prevention).
4. The **control plane** ships config as versioned bundles and purges as batched log entries (Deep dive 3); edges ship pre-aggregated logs plus sampled raw lines.

6. Deep dives

Deep dive 1 — request routing: DNS mapping, anycast and client steering.

Method	How the PoP is chosen	Strength	Weakness
DNS mapping (Akamai style)	The CDN's authoritative DNS returns the best PoP's IP for the resolver, or for the user's subnet via EDNS Client Subnet (RFC 7871)	Live load and latency data; drains a PoP by changing answers	Answers cached for the DNS TTL (20–60 s); without ECS sees the resolver, not the user
Anycast (Cloudflare style)	Every PoP announces the same IP prefix over BGP (the internet's routing protocol); routing policy delivers to the "nearest"	Simple, instant failover (withdraw the route), spreads DDoS floods over all PoPs	BGP knows neither load nor latency; Microsoft measured anycast sending ~20% of Bing clients to a suboptimal front-end
Client/URL steering (Netflix Open Connect)	A steering service returns ranked URLs of specific servers in the playback response	Per-file, per-server precision (knows which server *has* the file)	Needs your own client or manifest; only for apps you control

Combined answer: anycast for DNS servers and small objects (robust, DDoS-friendly); DNS mapping with ECS to steer bulk traffic by load; URL steering for video. The mapping system feeds on PoP-to-prefix latency probes, real-user measurements (RUM) and PoP spare capacity, and rebalances some prefixes at a time, never all at once.

Deep dive 2 — making 16 disks act like one cache, then adding tiers.

- *Problem and fix:* if every server caches what it sees, a popular object sits on all 16 and the PoP holds only 30 TB of distinct content. Hash the cache key onto a ring of the PoP's servers (consistent hashing, invented by Karger et al. in 1997 for exactly this) so each key has one **owner**: 16 × 30 = **480 TB** distinct, and losing a server remaps only its 1/16 of keys.
- *Hot objects:* a viral file would overload its one owner, so count requests per key (a heavy-hitters sketch) and let *every* server copy the top few thousand keys.
- *Admission and eviction:* many objects are requested only once ("one-hit wonders"): admit to disk only on the second request, and evict with a frequency-aware policy (S4LRU, TinyLFU: Cache eviction beyond LRU). Facebook's photo-serving study found a larger cache and S4LRU noticeably raised edge hit ratio.
- *Tiers:* in the same study browser caches served 65.5% of requests, edge 20.0%, origin cache 4.6%, backend 9.9%. Two CDN tiers (edge + regional parent) are the usual sweet spot; a third adds miss latency without many hits.
- *Placement:* PoPs in internet exchange points (IXPs) reach many ISPs at once; **embedded** caches inside ISPs (Netflix Open Connect) get closer, and Netflix fills them **proactively** off-peak because viewing is predictable: a **push** CDN, where the unpredictable web is **pull** (pull vs push).

Deep dive 3 — freshness: TTLs, leases, and a purge system that cannot forget.

- **TTL + revalidation** (the default): serve while fresh, then send `If-None-Match` and accept a cheap `304`. Staleness is bounded by the TTL; nothing needs coordinating.
- **Polling**: edges periodically ask the origin "changed?"; simple, but load grows with edges × objects.
- **Leases** (Gray & Cheriton, 1989): the parent promises to notify a child of changes for the lease duration, so invalidations go only to lease holders; a child that loses contact lets the lease expire and revalidates. Bounded staleness without a broadcast (Leases).
- **Push purge** (what customers expect, "gone in seconds"): the control plane appends to the **purge log**, and a fan-out tree pushes batches to all PoPs. Fastly and Cloudflare both report global

purges completing in around 150 ms (Cloudflare: under 150 ms at p50 for tag, hostname and prefix purges).

The hard part is a PoP partitioned while purges were sent: every PoP applies purges in `seq` order, reports `last_applied_seq`, and on reconnect pulls the gap from the log; if its lag exceeds a bound (say 30 s), the mapping system takes it out of rotation until it catches up.

Purge mechanics per server: URL purge is one index delete; tag purge uses the tag index; prefix purge or "purge everything" bumps a **generation number** in the cache key (`v7` → `v8`), O(1), leaving old entries to age out. **Soft purge** marks entries stale: the next request revalidates while others still get the stale copy, so the origin sees no stampede. For assets, prefer fingerprinted URLs and skip purging (Fingerprinted URLs vs purging).

Deep dive 4 — dynamic content and edge compute.

- Uncacheable responses still gain: TLS ends ~15 ms away instead of ~200 ms, the edge keeps **warm connections** to the origin, and **overlay routing** (Akamai) forwards through intermediate PoPs when the default path is slow or lossy.
- **Microcaching:** cache "uncacheable" shared responses (a scoreboard, a stock page) for 1 s: at 5,000 req/s for one URL, the origin sees 1 req/s per PoP.
- **Edge compute:** customer code for auth checks, redirects, A/B bucketing and personalising a cached page. Cloudflare Workers run in V8 isolates (sandboxes inside one process), which Cloudflare's docs say start ~100× faster than a Node process in a container. State needs an eventually consistent edge KV store or a call back to a region.

Deep dive 5 — large files and evaluation.

- **Video and large downloads:** cache in fixed **slices** (e.g. 2 MB byte ranges), so a 4 GB file's popular first minutes are cached alone and a `Range` request needs only the slices it covers. Video design: Case 3 — Video Platform.
- **How you know it works:** request hit ratio *and* byte hit ratio (they differ: small objects dominate requests, video dominates bytes), origin offload per customer, TTFB p50/p95 by country from RUM, purge latency p99, PoP lag, and cost per GB delivered.

7. The failure walk

1. **A cache server dies:** health checks remove it in seconds; its keys move to ring neighbours and hit ratio dips ~1/16 while they refill from the parent.
2. **A whole PoP fails:** anycast withdraws its route and DNS mapping stops returning it; neighbours absorb the traffic, so plan **N+1 headroom per region** (a PoP runs at most ~70% of capacity at peak).
3. **The parent tier is down:** edges go straight to the origin (configured fallback); origin load jumps, so the origin shield should be two PoPs, not one.
4. **The origin is down:** serve stale via `stale-if-error` (Graceful Degradation); for uncacheable requests return the customer's error page from the edge.
5. **Purge channel partitioned:** the lag rule (Deep dive 3) takes the PoP out of rotation: better 20 ms further away than recalled content.
6. **Bad global config push:** the classic CDN outage. In June 2021 a valid customer config triggered a latent Fastly bug and **85% of its network returned errors**; in July 2019 a globally pushed WAF regex rule exhausted Cloudflare's CPUs (Module 26). Defences: validate bundles, canary PoPs then regions (Canary Deployment), one-command rollback, and per-tenant config isolation so one property's bug cannot crash the server process.
7. **Viral object / flash crowd:** coalescing at edge and parent plus hot-key replication (Deep dive 2).
8. **Cache poisoning:** an attacker sends a header the origin uses but the cache key ignores (an "unkeyed" header), so a bad response is cached for everyone. Defence: key on every header the origin varies on, strip the rest.

9. **DDoS:** anycast spreads the flood over 300 PoPs; per-IP rate limits and challenge pages at the edge (Case 2) drop it before the cache.

8. Trade-offs

Decision	Chosen	Alternative	Why
Routing	Anycast + DNS mapping (+ URL steering for video)	One method only	Each fixes another's blind spot: resilience vs load-awareness vs per-file precision
In-PoP placement	Hash key to an owner + replicate hot keys	Every server caches independently	16× more distinct capacity; hot keys still spread
Tiers	Edge + regional parent	Edge only / three tiers	Parent coalesces misses; a third tier adds little hit ratio
Freshness	TTL + revalidation; push purge via a sequenced log	Polling, or purge-by-broadcast without sequence	Bounded staleness with no coordination on the hit path; the sequence catches PoPs that missed messages
Broad purge	Generation number in key	Scan and delete	O(1); old entries expire naturally
Fill	Pull for web; proactive push for predictable video	Pull everywhere	Predictable demand lets you use off-peak bandwidth
Config rollout	Staged by PoP group	Global instant push	Limits the blast radius of one bad bundle

9. The follow-up questions

- **"How do you serve HTTPS for millions of customer hostnames?"** TLS SNI (the client names the hostname in the handshake) selects the certificate, fetched lazily and cached per PoP. Private keys can stay in a few hardened locations that sign handshakes for the edges (Cloudflare's Keyless SSL). *Trap:* copying every private key to every PoP without discussing the risk.
- **"Private or paid content?"** **Signed URLs or cookies**: the origin signs `(path, expiry, user)` with a key the edge can verify, so cached bytes go only to holders of a valid signature while staying shared-cacheable.
- **"Multi-CDN?"** A DNS or client-side steering layer splits traffic across providers by measured performance and cost; purges go to all, and cache-key rules must match.
- **"Live video?"** Tiny, short-lived segments: coalescing at every tier matters more than disk (Live Streaming).
- **"What hit ratio do you need?"** Work back from origin capacity: if origins can take 1 Tbps and peak is 50 Tbps, combined offload must be ≥ 98%, which is where the two-tier hierarchy comes from.

> **Interview one-liner:** "Route with anycast plus load-aware DNS; inside each PoP, hash keys to one owner server and replicate only hot keys; put a regional parent tier in front of origins that coalesces misses; keep content fresh with TTLs and revalidation, and run purges as a sequenced log that every PoP must keep up with or be taken out of rotation; roll config out in stages."

TL;DR

> - **Problem:** 40M req/s and 50 Tbps from 300 PoPs, with 98% origin offload and purges in seconds.
> - **Price:** a global control plane whose mistakes are global, large spare capacity per region, and staleness that you bound but never fully remove.

Sources

- Nygren, Sitaraman, Sun, *The Akamai Network: A Platform for High-Performance Internet Applications*, ACM SIGOPS OSR 44(3), 2010: https://pdos.csail.mit.edu/6.824/papers/akamai.pdf
- Karger et al., *Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web*, STOC 1997: https://cs.brown.edu/courses/csci2950-u/f09/papers/chash97stoc.pdf
- Huang et al., *An Analysis of Facebook Photo Caching*, SOSP 2013: https://www.cs.princeton.edu/~wlloyd/papers/fb-photo-caching-sosp13.pdf
- Calder et al., *Analyzing the Performance of an Anycast CDN*, IMC 2015: https://courses.cs.duke.edu/fall19/compsci514/papers/anycast-imc15.pdf
- Gray and Cheriton, *Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency*, SOSP 1989: https://pages.cs.wisc.edu/~remzi/Classes/739/Fall2015/Papers/leases89.pdf
- Netflix, *Open Connect Overview*: https://openconnect.netflix.com/Open-Connect-Overview.pdf
- RFC 7871 (EDNS Client Subnet): https://www.rfc-editor.org/rfc/rfc7871 · RFC 5861 (stale-while-revalidate/stale-if-error): https://www.rfc-editor.org/rfc/rfc5861 · RFC 9211 (Cache-Status): https://www.rfc-editor.org/rfc/rfc9211 · RFC 9213 (Targeted Cache-Control): https://www.rfc-editor.org/rfc/rfc9213
- Cloudflare, *Instant Purge: invalidating cached content in under 150ms* (2024): https://blog.cloudflare.com/instant-purge/
- Fastly, *Is purging still the hardest problem in computer science?* (purge in ~150 ms): https://www.fastly.com/blog/is-purging-still-the-hardest-problem-in-computer-science · Fastly docs, *Working with surrogate keys*: https://www.fastly.com/documentation/guides/full-site-delivery/purging/working-with-surrogate-keys/
- AWS, *Use Amazon CloudFront Origin Shield*: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/origin-shield.html
- Cloudflare Workers docs, *How Workers works*: https://developers.cloudflare.com/workers/reference/how-workers-works/
- Fastly, *Summary of June 8 outage* (2021): https://www.fastly.com/blog/summary-of-june-8-outage · Cloudflare, *Details of the Cloudflare outage on July 2, 2019*: https://blog.cloudflare.com/details-of-the-cloudflare-outage-on-july-2-2019/

Case 2 — Design a Rate Limiter

The prompt (in short). One rate-limiting service that every API gateway calls. Teams declare rules ("100 requests/s per API key on `/search`", "5 logins/min per account", "20,000 req/s total to the payments backend"); the service answers "allow or reject" per request in under a millisecond, across three regions.

What makes it hard. The algorithms and the `429`/`Retry-After` contract are in the Rate Limiting lesson. As a shared service the hard parts are (1) **races** between hundreds of gateways on one counter, (2) **latency vs accuracy** (a remote counter costs a hop per request), (3) **multi-region** limits with 70 ms between regions, and (4) **safe rules** (one bad rule can reject all traffic).

1. Requirements and numbers

Functional: CRUD on rules; per request, evaluate and consume from all matching rules; actions `reject`, `shadow` (log only) or `delay`; return remaining budget and reset time; per-key concurrency limits (requests in flight).

Non-functional: added p99 < 1 ms in-region; the limiter's failure must not take the API down (fail open by default, closed for security rules such as login); counts within ~5–10% for protection rules; rule changes live within 10 s.

Numbers (assumptions):

- **3M requests/s** peak across 3 regions, average **3 rules matched** per request → **9M counter operations/s**.

- **60M active keys** (API keys, users, IPs) × 3 rules × ~64 B of state ≈ **11.5 GB**: fits in memory on a small cluster.
- At a conservative **100K scripted operations/s per in-memory store node**, direct counting needs **90 primary nodes** (30 per region). Local pre-allocation (Deep dive 3) cuts this by 10× or more.
- Rules: ~10K, a few MB, changing a few times a minute.

What the numbers say: memory is trivial; **ops/s and the extra hop** decide the design.

2. The five-questions pass

- **Flow:** synchronous on the request path (gateway → limiter → counter store), bounded by a ~2–5 ms timeout and a local fallback.
- **Storage:** counters are **soft state** (rebuildable; losing them means a few seconds of looser limits); rules are durable config.
- **Speed:** one round trip to an in-region store; check and update in one atomic step.
- **Failure:** the limiter is protection, not correctness: when unsure, allow (except security rules) and log.
- **Growth:** shard counters by key (consistent hashing); hot keys get local pre-allocation.

3. API

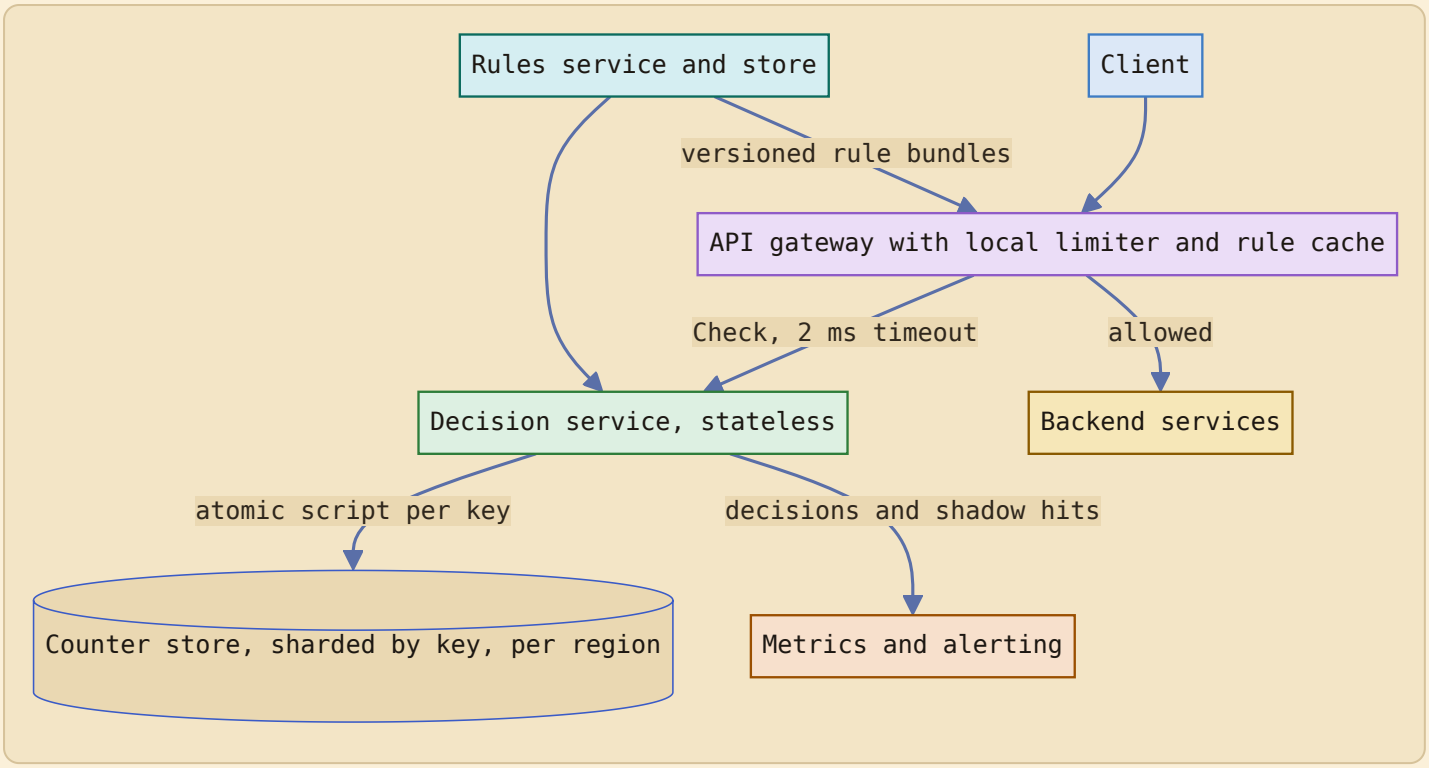
Call	Shape	Notes
Rules	`PUT /v1/rules/{id}` `{match: {route, key_type}, algo: gcra, rate: 100/s, burst: 200, action: reject|shadow, fail_mode: open|closed}`	Versioned; validated before going live
Decision (gRPC)	`Check({domain, descriptors: [{api_key: k1}, {route: /search}], cost: 1})` → `{allow, limit, remaining, reset_after_ms, matched_rule}`	Same shape as Envoy's global rate-limit service: a request carries descriptor key/value pairs, and rules match on them
Client response	`429 Too Many Requests`, `Retry-After: 3`, `RateLimit-Policy` / `RateLimit` headers	429 and Retry-After from RFC 6585; the two `RateLimit*` fields are an IETF draft

4. Data model

- **Rule**: `{id, domain, match, algorithm, rate, period, burst, cost_field, action, fail_mode, version}`.
- **Counter key**: `rl:{rule_id}:{descriptor_value}`, e.g. `rl:search-per-key:k1`. All of one key's state lives on one shard.
- **State per key** depends on the algorithm:

Algorithm	Stored per key	Size
Fixed window	count, window id	1 number + TTL
Sliding-window counter	previous count, current count	2 numbers
Token bucket	tokens, last refill time	2 numbers
GCRA	**TAT** (theoretical arrival time)	**1 timestamp**

5. The architecture, annotated



1. The gateway matches the request against its **cached rules** (no remote call) and builds descriptors; hot-key and per-instance rules are decided **locally** (Deep dive 3).
2. Other rules go to the stateless **decision service** (or the same code as an embedded library), which runs one **atomic script** per key on the owning shard of the in-region **counter store**.
3. Rejections get `429` with headers; every decision, including shadow "would have rejected", becomes a metric.
4. The **rules service** validates, versions and distributes rule bundles; gateways swap them atomically.

6. Deep dives

Deep dive 1 — GCRA: a token bucket that stores one number.

- *Idea:* the **generic cell rate algorithm** (from ATM networks) keeps only the **TAT**, the time the key would next be "on schedule". **Emission interval** `T = period / rate`; **burst tolerance** `τ = T × (burst − 1)`.
- *Example:* rate 10/s, burst 5 → T = 100 ms, τ = 400 ms. Key idle, TAT = now = 0. Five requests arrive at t = 0: each sets TAT = max(TAT, now) + T → 100, 200, 300, 400, 500 ms. A request is allowed if `TAT − τ ≤ now`, checked before the update. For the fifth, 400 − 400 = 0 ≤ 0, so it is allowed. For the sixth, 500 − 400 = 100 ms > 0 → reject with `Retry-After = 100 ms`. At t = 100 ms one more request fits; the long-run rate is 10/s.
- *Why:* token-bucket behaviour, one value per key, and `Retry-After` for free. Let's Encrypt rebuilt its rate limits on Redis with GCRA in 2025.

```
function check(key, now, T, tau, cost):
    tat = store.get(key) or now
    new_tat = max(tat, now) + T * cost
    if new_tat - tau > now + T:          # beyond allowed burst
        return REJECT, retry_after = new_tat - tau - T - now
    store.set(key, new_tat, expire_at = new_tat)
    return ALLOW
```

(Runs as one server-side script, so no other request can interleave.)

Deep dive 2 — races and clocks.

- *Race:* gateways A and B both read `tokens = 1`, both allow, both write `0`: two requests on one token, a systematic overshoot under load.
- *Fix:* read, decide and write **inside the store** as one atomic step: a server-side script (Lua in Redis), or one atomic command such as `INCR` for fixed windows. GitHub's API limiter works this way (sharded Redis, Lua scripts, writes to primaries).
- *Clocks:* use **one clock**, the store's (`TIME` inside the script); GitHub saw `X-RateLimit-Reset` wobble by a second when reset time was partly computed in the app.
- *Own cluster:* GitHub left Memcached partly because other data **evicted live counters**, letting users exceed limits.

Deep dive 3 — local vs global: buying latency with bounded error.

Strategy	Hop per request	Accuracy	Use for
Central counter (every check remote)	1 (~0.5 ms)	Exact	Login, low-rate security rules
Static split: each of N gateways enforces limit/N	0	Wrong when traffic is uneven across gateways	Only with very even load balancing
Token leasing: a gateway takes a batch (say 10% of a key's budget) from the store and spends it locally	1 per batch	Over-admits by at most the unspent leases	Hot keys, high-rate rules
Local count + periodic sync (every 100 ms)	0	Overshoot ≤ gateways × rate × sync interval	Edge/DDoS rules, per-PoP limits

Worked bound: 50 gateways, a key limited to 1,000 req/s, sync every 100 ms: each may admit up to 100 before learning the global count, so up to 50 × 100 = **5,000 extra requests** per interval. Too loose; leasing (10 leases of 100 tokens) bounds the error to outstanding leases. Cloudflare's edge limiter counts per PoP in a memcache cluster (anycast keeps a client on one PoP); over 400M requests its two-window approximation wrongly allowed or blocked **0.003%**.

Deep dive 4 — multi-region.

- **Per-region budgets** (default; Multi-Region): split each global limit by regional traffic share (e.g. 50/30/20%), re-weighted every minute; no cross-region hop, error from re-weighting lag.
- **Asynchronously replicated counts:** each region counts locally, gossips its count and enforces against the sum, a grow-only counter per region (Distributed Counter). Overshoot ≤ replication lag × rate.
- **Synchronous global counter:** ~70 ms per request; only for rare, critical limits (a daily quota checked once per job, not per request).
- Billing quotas are a different system: exact, durable, reconciled from logs, never from limiter counters.

Deep dive 5 — rules as dangerous config.

- Validate (no rule may match `*` with a rate below current traffic unless forced), then **shadow mode** first: the rule only logs what it would reject (Envoy's per-rule `shadow_mode`; Stripe advises dark-launching every limiter). Then roll out by region with a kill switch (Feature Flags).
- Stripe runs four layers: request-rate limiter, concurrent-requests limiter, fleet-usage load shedder and worker-utilisation shedder; the last two are load shedding.

7. The failure walk

1. **A counter shard dies:** its replica is promoted; meanwhile checks for its keys time out (2 ms) and **fail open** to a local bucket at limit/N, so limits loosen, never tighten. Security rules fail closed.
2. **The decision service is slow:** the gateway timeout and a circuit breaker switch to local-only mode and alert; the limiter must never add 100 ms to every request.
3. **A bad rule rejects 40% of traffic:** caught in shadow mode or the canary region; otherwise kill switch and roll back to the previous bundle.
4. **Hot key** (one tenant at 200K req/s on one shard): token leasing turns 200K ops/s into ~2K batch fetches/s.
5. **Region failover:** per-region budgets must re-weight, or surviving regions reject legitimate traffic.
6. **Retry storm after 429s:** `Retry-After` with jitter, and SDKs that obey it (Retry with Exponential Backoff).
7. **Shared IPs (NAT):** IP-keyed rules only as a generous backstop; key on API key or account first.

8. Trade-offs

Decision	Chosen	Alternative	Why
Algorithm	GCRA / token bucket	Fixed or sliding window	Allows honest bursts; one value per key; Retry-After for free
Atomicity	Script inside the store	Read-then-write from the gateway	Removes the race without distributed locks
Placement	Hybrid: local + leased + central by rule	All central	Hot keys and latency; central kept for exact security rules
Multi-region	Per-region budgets, re-weighted	Global synchronous counter	Avoids 70 ms per request; error is bounded
Failure mode	Fail open (closed for security rules)	Always closed	The limiter must never be the outage
Rule rollout	Shadow → canary region → global	Direct apply	One rule can reject everyone

9. The follow-up questions

- **"Concurrency limits instead of rate?"** An in-flight counter (increment at start, decrement at end) plus a TTL'd lease per request so a crashed gateway's slots expire; protects slow endpoints where rate alone misses the cost.
- **"Charge expensive calls more?"** Pass `cost` (for example 10 for search, 1 for reads); GCRA multiplies T by cost.
- **"Adaptive limits?"** Lower limits automatically as backend latency or errors rise (AIMD: add a little while healthy, halve under stress): load shedding driven by health rather than identity.
- **"Detect abusive IPs among millions without a counter each?"** A count-min sketch at the edge finds heavy hitters; only they get exact counters.
- **"How do you test it?"** Replay production traffic in shadow mode and compare per-rule would-reject rates with expectations before enforcing.

Interview one-liner: "Rules are versioned config cached in every gateway; each check runs as one atomic script on the shard that owns the key, using GCRA so the state is one timestamp; hot keys lease tokens in batches; regions enforce re-weighted local budgets; the limiter fails open except for security rules, and every rule ships in shadow mode first."

TL;DR

- **Problem:** 9M checks/s, < 1 ms, many gateways, three regions, rules written by many teams.

- **Price:** limits are approximate by design, and one more critical dependency must fail open.

Sources

- Stripe Engineering, *Scaling your API with rate limiters*: https://stripe.com/blog/rate-limiters
- Cloudflare, *How we built rate limiting capable of scaling to millions of domains*: https://blog.cloudflare.com/counting-things-a-lot-of-different-things/
- GitHub Engineering, *How we scaled the GitHub API with a sharded, replicated rate limiter in Redis* (2021): https://github.blog/engineering/infrastructure/how-we-scaled-github-api-sharded-replicated-rate-limiter-redis/
- Envoy proxy, *ratelimit* service (descriptors, shadow_mode, per-second Redis): https://github.com/envoyproxy/ratelimit
- Let's Encrypt, *Scaling Our Rate Limits to Prepare for a Billion Active Certificates* (2025): https://letsencrypt.org/2025/01/30/scaling-rate-limits/
- Brandur Leach, *Rate Limiting, Cells, and GCRA*: https://brandur.org/rate-limiting
- RFC 6585 (429 Too Many Requests): https://www.rfc-editor.org/rfc/rfc6585 · IETF draft, *RateLimit header fields for HTTP*: https://datatracker.ietf.org/doc/draft-ietf-httpapi-ratelimit-headers/

Case 3 — Design a Pub-Sub Service

The prompt (in short). A managed, multi-tenant publish-subscribe service like Google Cloud Pub/Sub or Amazon SNS. Publishers send messages to a **topic**; each **subscription** on it receives every message by **pull** (consumers ask) or **push** (the service POSTs to an HTTPS endpoint). At-least-once delivery with per-message acks, redelivery, dead-lettering, optional ordering keys, filtering, 7-day retention and replay.

What makes it hard, and how it differs from Module 13 Case 4. A Kafka-style log hands consumers an offset; here the **service** tracks every message's ack state per subscription, redelivers individually and pushes to endpoints it does not control. Hard parts: (1) **fan-out** to thousands of subscriptions without thousands of copies; (2) **per-message ack state** at millions of acks/s; (3) **isolation**, so one slow subscriber slows nobody else.

1. Requirements and numbers

Functional: create topics and subscriptions; publish batches; pull and streaming pull; push to HTTPS endpoints; ack and extend ack deadlines; dead-letter topic after N attempts; attribute filters; ordering keys; retention and **seek** (replay from a time).

Non-functional: an acknowledged publish is never lost; publish p99 < 100 ms; delivery p99 < 1 s when consumers keep up; a backlog in one subscription does not affect others; per-region availability 99.95%.

Numbers (assumptions):

- **500K publishes/s** average (1.5M peak), **2 KB** average → **1 GB/s** ingest.
- Average fan-out **4 subscriptions** per topic → **2M deliveries/s**, **4 GB/s** egress, and **2M acks/s**.
- Retention 7 days: 1 GB/s × 604,800 s ≈ **605 TB**, × 3 replicas ≈ **1.8 PB**. Messages are stored **once per topic**, not per subscription; otherwise the figure would be ×4.
- 200K topics and 800K subscriptions; the largest topic has **10,000** subscriptions (Google Cloud Pub/Sub's per-topic limit; SNS standard topics allow 12.5M).

What the numbers say: storage is moderate. Ack bookkeeping (2M/s) and fan-out isolation decide the design.

2. The five-questions pass

- **Flow:** publish is synchronous until durable; delivery is asynchronous, per subscription.

- **Storage:** an append-only **message log per topic partition** (written once), plus **per-subscription ack state** (small, hot, heavily written).
- **Speed:** caught-up consumers are served from memory at the log tail; only replay and backlog reads touch disk.
- **Failure:** at-least-once; duplicates are expected, so consumers must be idempotent.
- **Growth:** split hot topics into more partitions (Partitioned Consumption); move subscriptions between delivery servers by consistent hashing.

3. API

Call	Shape
`CreateTopic(name, retention?)`	Topic-level retention optional (Google allows up to 31 days)
`CreateSubscription(topic, {pull | push_endpoint}, ack_deadline=10s, filter?, ordering?, dead_letter={topic, max_attempts}, retry={min,max backoff})`	One subscription = one independent copy of the stream
`Publish(topic, [{data, attributes, ordering_key?}])` → `[message_id]`	Returns only after a quorum write
`Pull(sub, max_messages)` / `StreamingPull(sub)` → `[{ack_id, message, delivery_attempt}]`	`ack_id` identifies one delivery, not the message
`Acknowledge(sub, [ack_id])`, `ModifyAckDeadline(sub, [ack_id], seconds)`	Extend while still working on a message
`Seek(sub, time | snapshot)`	Replay: move the subscription's ack state

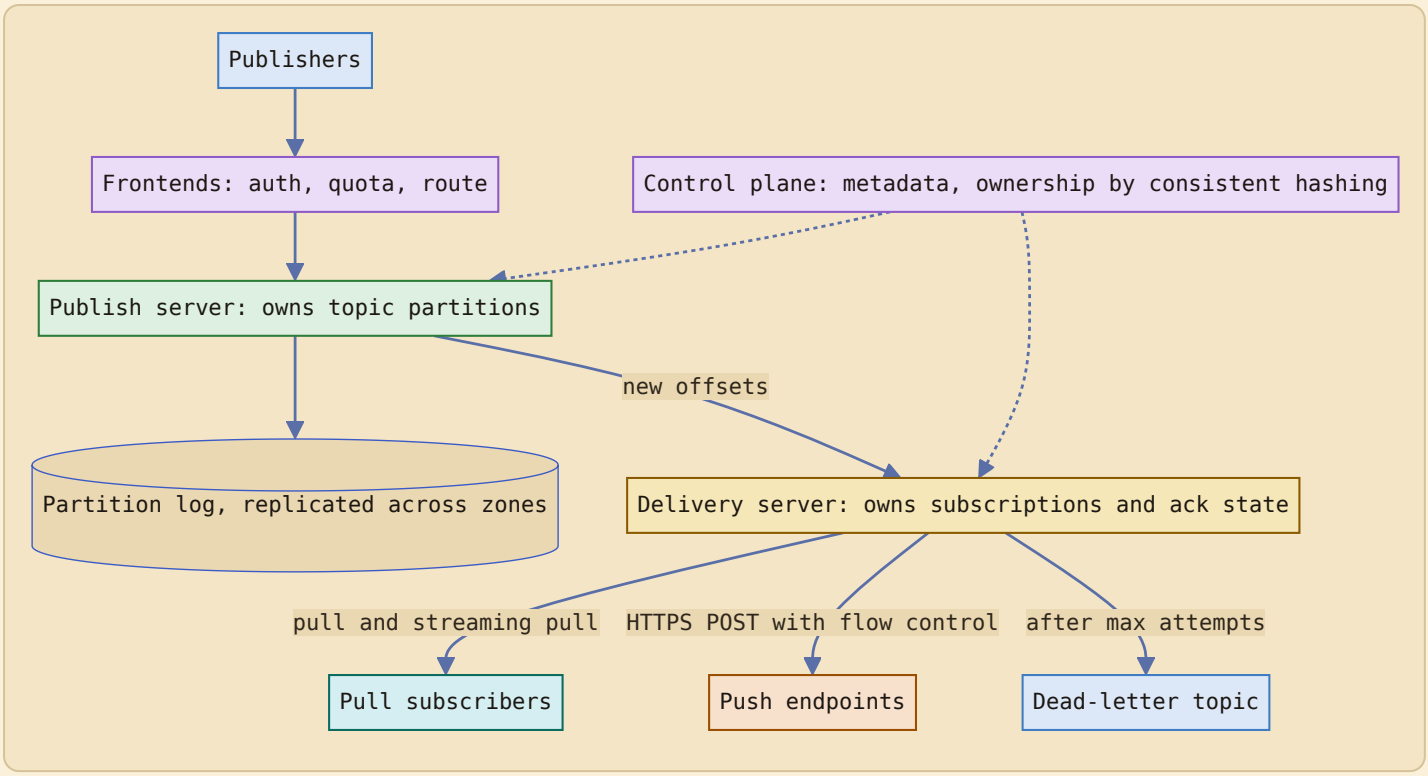
Push delivery is a POST of the message to the endpoint; a `2xx` response counts as the ack. The ack deadline is the same idea as the queue's visibility timeout.

4. Data model

- **Topic / subscription metadata** (strongly consistent metadata store): names, config, `partition_count`, owners.
- **Message log** per `(topic, partition)`: segment files of `offset → {message_id, publish_time, attributes, data}`, replicated across zones.
- **Ack state** per `(subscription, partition)`:
  - `ack_floor`: every offset below it is acknowledged;
  - an **acked-above-floor set** stored as ranges/bitmap (for example `{1003–1010, 1015}`);
  - an **outstanding lease table**: `ack_id → {offset, deadline, delivery_attempt}`, held in memory and checkpointed.
- **Dead-letter topic**: an ordinary topic; the subscription records `max_attempts`.

Storing ranges instead of one row per ack is what makes 2M acks/s cheap: in-order acks just move `ack_floor`.

5. The architecture, annotated



1. Stateless **frontends** authenticate, apply per-tenant quotas and route by `(topic, partition)`.
2. The owning **publish server** appends and replicates the batch, acking the publisher only after a **majority** write (Google's Pub/Sub: ⌈N/2⌉ of N clusters; Quorum, Write-Ahead Log).
3. **Delivery servers** read new messages **once** and serve every subscription they own from that copy (Google splits the roles into publishing and subscribing forwarders). They lease or push messages, track acks, redeliver on expiry and move poison messages to the **dead-letter topic** (DLQ).
4. The **control plane** assigns partitions and subscriptions by consistent hashing and moves them on failure; clients stay connected to the data plane, so control-plane upgrades do not disconnect them.

6. Deep dives

Deep dive 1 — store once, fan out by cursor.

- *Naive:* one queue per subscription turns one 2 KB publish into 20 MB of writes at 10,000 subscriptions. *Instead:* write once; a subscription is **its ack state over the shared log**, so creating one costs a metadata row, and retention, not acks alone, decides deletion.
- *Large fan-out:* spread a 10,000-subscription topic over many delivery servers, each reading the hot tail into memory once. A new subscription sees only messages published after its creation (Google's "sync point").

Deep dive 2 — per-message acks, redelivery and dead letters.

- *Lease:* a delivery assigns `ack_id` and `deadline = now + ack_deadline` (Google: default 10 s, maximum 600 s). Consumers working longer call `ModifyAckDeadline`.
- *Example:* a subscription's partition has offsets 1000–1019 outstanding. Acks arrive for 1000–1002 → `ack_floor = 1003`. Acks for 1005–1010 → stored as the range `{1005–1010}`. 1003 and 1004 expire → redelivered with `delivery_attempt = 2`, after a backoff (Google: configurable, 10 s to 600 s). 1004 fails 5 times → published to the dead-letter topic and acked on the original (Google allows 5–100 attempts). When 1003 is acked, the floor jumps over the range to 1011.
- *At-least-once, and why:* a consumer can finish the work and crash before acking, indistinguishable to the service from no work done. **Exactly-once delivery** (an option in Google's

service) only guarantees a confirmed ack is final and the message is not redelivered after it; side effects still need idempotent consumers (Exactly-Once Semantics).

Deep dive 3 — push delivery and isolation.

- Each push subscription has its own **outstanding window** (messages in flight to the endpoint): grow it while the endpoint returns `2xx` quickly, halve it on errors or timeouts (AIMD, as in TCP). A failing endpoint slows only its own subscription (Backpressure).
- Own ack state, window and retry timers per subscription form a bulkhead: a 50M-message backlog costs only disk, never other subscriptions' latency.
- **Filters** (on attributes only, fixed at creation in Google's service) are applied at delivery. Non-matching messages are auto-acked, so they never count as backlog.
- Pull vs push: pull lets consumers control their own rate and works behind firewalls; push suits serverless endpoints and webhooks (Webhooks).

Deep dive 4 — ordering keys.

- Messages with the same `ordering_key` hash to the same partition, so the log holds them in publish order.
- Delivery must keep **at most one unacked batch per key** in flight, or a redelivery could overtake a later message. Different keys proceed in parallel, so throughput scales with the number of active keys, not partitions.
- A message that keeps failing **blocks** its key (head-of-line blocking) until acked or dead-lettered: the price of order. Publishers must also stop publishing on a key after a failed publish, or a gap appears.

Deep dive 5 — retention, replay and storage tiers.

- Unacked messages are kept for the subscription's retention (Google: 7 days default, 10 min to 31 days); optionally acked ones too, which makes **seek** to a past time possible.
- **Seek(time)** sets `ack_floor` to the first offset at or after `time` and clears the range set. **Snapshots** save a subscription's ack state, so a bad deploy can "re-process from before v2".
- Log segments older than a few hours move to object storage; the delivery path reads the tail from memory or SSD.
- Unused subscriptions expire (Google: 31 days by default) so abandoned ones stop holding retention.

7. The failure walk

1. **Publish server dies:** its partitions move to another server, which reads the replicated log tail and fences the old owner with a new **epoch** (a number that rises with each ownership change, so the old owner's writes are rejected). Publishers retry; a retry after an unseen ack can **duplicate a publish**, so the frontend deduplicates on a publisher-supplied ID within a short window.
2. **Delivery server dies:** its subscriptions move; the new owner loads the last ack-state checkpoint, and messages leased since then are redelivered: duplicates, not losses.
3. **A zone is lost:** majority writes mean acked messages survive; ownership moves to the remaining zones.
4. **Consumer outage for 3 days:** the backlog grows on disk only; on return it streams from SSD or object storage under the subscription's flow control, while other subscriptions still read the hot tail from memory.
5. **Poison message:** attempts reach `max_attempts`, then dead-letter. Alert on dead-letter rate, not individual messages.
6. **Hot topic exceeds one partition:** add partitions. Only keyed ordering is affected (a key may hash to a new partition), so drain or pause per key during the split.

7. **Push endpoint returns 500s:** the window shrinks to 1 and backoff grows to its maximum; backlog and oldest-unacked-age alerts go to the subscription owner.

8. Trade-offs

Decision	Chosen	Alternative	Why
Storage	One log per topic + ack state per subscription	A copy per subscription	Fan-out of 10,000 without 10,000× writes
Ack tracking	Floor + ranges + leases	One row per message per subscription	2M acks/s; in-order acks just move a number
Guarantee	At-least-once (+ optional confirmed acks)	Exactly-once end to end	The service cannot see consumer side effects
Delivery	Pull and push, per-subscription windows	Push only	Pull for heavy consumers, push for serverless; windows isolate failures
Ordering	Opt-in per key	Total order per topic	Total order caps throughput at one consumer
vs log-based queue (Case 4, Module 13)	Service-managed acks and redelivery	Consumer-managed offsets	Easier for many small consumers; less raw throughput and replay control

9. The follow-up questions

- **"When would you choose Kafka instead?"** Very high throughput with a few consumer groups that process in order, need long replay and can manage offsets. Pub-sub wins for many independent subscribers, per-message retry, push delivery and zero consumer state (Inside Kafka).
- **"Global topics across regions?"** Publish locally and ack after the in-region quorum; replicate the log asynchronously to subscribers' regions. Ordering then holds per key per region only.
- **"Exactly-once processing for a payment consumer?"** The consumer writes its effect and the message ID in one transaction (an inbox table) and ignores repeats, whatever the broker guarantees.
- **"Notifications to 100M phones?"** That is not a topic with 100M subscriptions; mobile push goes through APNs/FCM fan-out services (notification capstone).
- **"What do you monitor?"** Per subscription: backlog count, oldest unacked message age, ack rate, dead-letter rate, and push latency.

Interview one-liner: "Write each message once to a replicated per-topic partition log and ack after a majority write; model each subscription as ack state over that log (floor, ranges, leases), so fan-out costs no copies; deliver by pull or windowed push with per-subscription isolation; redeliver on lease expiry with backoff and dead-letter after N attempts; ordering keys serialise delivery per key."

TL;DR

- **Problem:** 1 GB/s in, 4 GB/s out, 2M acks/s, up to 10,000 subscriptions per topic, 7-day replay.
- **Price:** at-least-once with duplicates, head-of-line blocking per ordering key, and the service rather than the consumer owning delivery state.

Sources

- Google Cloud, *Pub/Sub architecture overview*: https://docs.cloud.google.com/pubsub/architecture
- Google Cloud, *Subscription properties* (ack deadline, retention, retry, dead-letter, filters, ordering): https://docs.cloud.google.com/pubsub/docs/subscription-properties

- Google Cloud, *Pub/Sub quotas and limits* (10,000 subscriptions per topic, 10 MB messages): https://docs.cloud.google.com/pubsub/quotas
- AWS, *Amazon SNS endpoints and quotas* (12.5M subscriptions per standard topic): https://docs.aws.amazon.com/general/latest/gr/sns.html
- Apache Pulsar docs, *Architecture overview* (stateless brokers, BookKeeper, cursors per subscription): https://pulsar.apache.org/docs/next/concepts-architecture-overview/
- Kreps, Narkhede, Rao, *Kafka: a Distributed Messaging System for Log Processing*, NetDB 2011: https://people.csail.mit.edu/matei/courses/2015/6.S897/readings/kafka.pdf

Module 24 — Case Studies: AI Systems

How AI reshaped system design

What this module is. Three end-to-end AI designs in the case-study format: a code assistant, a customer support bot and the data platform behind model training. They link to rather than repeat Module 11 (feature store, model serving, LLM gateway, semantic caching, vector sharding, RAG) and its RAG capstone (the base of Case 2), the action-taking agent or copilot of Module 12 Design 4, and the ChatGPT-style model-fleet serving of Module 21 Case 3 (streaming, KV cache, continuous batching, cache-aware routing). Recommendation systems: Feed ranking and Module 25 Case 1.

What actually changed. An AI product is still an ordinary distributed system; a handful of constants changed, and each moves a design decision:

Constant	Classic web service	AI service	Design consequence
Unit of work	A request; requests cost about the same	A **token** (a word piece, about 3/4 of a word); one request can cost 100× its neighbour	Rate limits, quotas and capacity plans count tokens, not requests
Cost centre	CPU servers, storage, egress	**GPU** time (graphics processors that run the model)	Every design choice is scored as "tokens per GPU-second" or "tokens we avoid computing"
Latency shape	One number per request	**TTFT** (time to first token) plus a per-token rate; seconds, not milliseconds	Streaming, cancellation, small fast models for interactive paths
Correctness	Deterministic: same input, same output; unit tests prove it	**Non-deterministic** and sometimes confidently wrong ("hallucination")	Evaluation sets, LLM judges, canaries on quality, guardrails outside the model
State	Stateless servers, sticky sessions avoided	KV cache and prefix caches make placement matter again	Cache-aware routing (Module 21 Case 3)
Security surface	Inputs are data	Text can act as instructions (**prompt injection**)	Authorization in the runtime, never in the prompt (Module 12 Design 4)
Data	An operational by-product	The product's raw material; model quality is bounded by data quality	Versioned datasets, lineage, labeling and drift monitoring (Case 3)

Two consequences matter most in interviews. **Evaluation becomes part of the architecture**: with a non-deterministic component, the only proof a prompt or model change is safe is a measured comparison on a fixed task set plus live metrics, so each case has an evaluation loop. **The data pipeline is part of the product**: Sculley et al. (NeurIPS 2015) showed most of the code and risk of an ML system lives around the model (data dependencies, feedback loops, configuration). With 84% of developers using or planning to use AI tools (Stack Overflow survey, 2025), "design a code assistant" now sits beside "design a URL shortener".

The AI addendum to the five questions. Run the usual Five Questions, then add three: (1) *What is the token budget per request, and who pays for it?* (2) *What is the latency shape: is a human watching tokens arrive, or waiting for a whole answer?* (3) *How will we know it is right: what is the evaluation set, the live quality metric and the human fallback?*

Sources

- Sculley et al., "Hidden Technical Debt in Machine Learning Systems", NeurIPS 2015 — https://papers.nips.cc/paper/2015/hash/86df7dcfd896fcaf2674f757a2463eba-Abstract.html
- Stack Overflow Developer Survey 2025, AI section — https://survey.stackoverflow.co/2025/ai
- OWASP Top 10 for LLM Applications 2025 (LLM01 Prompt Injection) — https://genai.owasp.org/llm-top-10/

Case 1 — AI-Powered Code Assistant

The prompt (in short). Build a coding assistant like GitHub Copilot, Cursor or Sourcegraph Cody. In the developer's editor (the **IDE**, integrated development environment) it offers **inline completions** (grey "ghost text" that appears as you type, accepted with Tab) and a **chat panel** that answers questions about the codebase and can propose multi-file edits. Companies buy it for their engineers and need guarantees about their private code.

What makes it hard. (1) **Latency**: roughly 250–300 ms from last keystroke to ghost text; a later suggestion is useless. (2) **Context**: the model knows only its prompt, but useful context is spread over thousands of files. (3) **Private code**: prompts contain source code, so isolation, retention and exclusion are product requirements.

1. Requirements and numbers

Functional: inline completion (single line and multi-line); chat about the code with references to files; repository-wide context ("use our existing `retry` helper"); admin policies (content exclusion, model choice, retention); usage and quality telemetry. Agent mode (edit files, run tests) reuses Module 12 Design 4 and is covered only in the follow-ups.

Non-functional:

- **Completion latency** p50 ≈ 250 ms, p95 < 600 ms to visible ghost text (Nielsen: 0.1 s feels instantaneous, 1 s keeps the flow of thought).
- **Chat**: TTFT < 2 s, then streamed.
- **No cross-tenant leakage** of code, prompts, embeddings or caches; excluded files never leave the laptop.
- **Availability** 99.9%; when the service is down the editor must keep working normally.

Numbers (assumptions, rounded):

- **500K daily active developers** (DAU), each triggering **~1,200 completion requests/day** after client-side debouncing → **600M requests/day** → 600M ÷ 86,400 ≈ **7K/s average**. Coding happens in working hours, so peak ≈ 4× → **~28K/s**.
- **Prompt** ≈ **2,000 tokens** (code before and after the cursor plus snippets from other files); **output** ≈ **24 tokens** (a line or a short block).
- Input tokens at peak: 28K × 2,000 = **56M tokens/s**. Output: 28K × 24 = **672K tokens/s**. Input outnumbers output **~80 : 1**, so completion is a **prefill-heavy** workload (prefill = the phase where the model reads the prompt; see Module 21 Case 3 deep dive 2). Chat is the opposite.
- **Prefill capacity** for a ~7B-parameter model: ~2 × 7B = 14 GFLOP per token. An H100 SXM is listed at ~990 dense BF16 TFLOPS (1,979 with sparsity); at 40% utilisation ~400 TFLOPS → $400 \times 10^{12} \div 14 \times 10^{9}$ ≈ **28K prefill tokens/s per GPU**.
- Without reuse: 56M ÷ 28K ≈ **2,000 GPUs** just for prefill. While a developer types, consecutive prompts share almost all of their prefix; with a **70% prefix-cache hit rate** only 16.8M tokens/s are new → **~600 GPUs**. Decode at ~3,000 tokens/s per GPU at low batch sizes: 672K ÷ 3,000 ≈ **225 GPUs**. With 30% headroom: **~1,100 GPUs** across 4 regions. *The cache saves about 1,400 GPUs.*
- **Cancellation**: assume **40%** of requests go stale from further typing before returning; each cancelled before prefill finishes is GPU time saved.
- **Chat**: 15 requests per DAU per day → 7.5M/day, ~350/s at peak, but ~15K input tokens each → ~5M input tokens/s on a separate large-model pool.
- **Repository index**: 300K repositories × 3,000 files × ~4 chunks ≈ **3.6B chunks**. At 768 dimensions × 2 bytes = 1.5 KB per vector → **~5.5 TB** of vectors before compression, sharded as in Vector Database Sharding. Product quantisation (Module 19) cuts that several-fold.
- **Telemetry**: ~60% of requests produce a shown suggestion → 360M/day × ~4 events (shown, accepted or rejected, still present at 30 s and at 2 min) ≈ **1.4B events/day ≈ 17K/s**.

What the numbers say: the bottleneck is small-model prefill under a tight budget: fewer requests (debounce, cancel, client cache), less prefill (prefix caching), faster decode (speculative decoding).

2. The five-questions pass

- **Flow:** completions are synchronous on a long-lived HTTP/2 connection, with cancellation; chat streams over SSE as in Module 21; telemetry and indexing are asynchronous.
- **Storage:** code's source of truth is the developer's disk and the Git host; the service keeps only **derived** data (per-repo embeddings and symbol table, policy, telemetry). Prompts live in memory and, for business tiers, are not retained.
- **Speed:** keystroke latency dominates; the client is part of the latency design.
- **Failure:** the assistant is optional: any failure means "no suggestion", never a blocked editor; indexing lag only lowers quality.
- **Growth:** with developers and context length; peaks follow the working day across time zones.

3. API

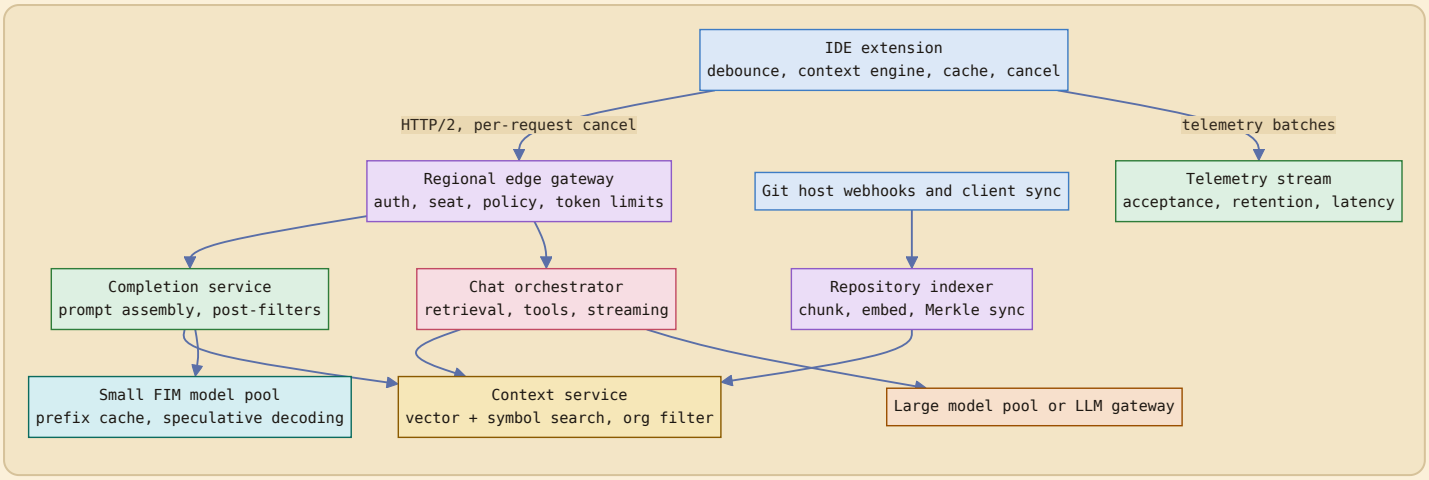
Method and path	Purpose
`POST /v1/completions` body `{request_id, repo_id, file_path, language, prefix, suffix, snippets[], cursor, max_tokens, stop[]}` → `{completion_id, choices[{text, finish_reason}]}`	One inline completion. `prefix` / `suffix` are the code before and after the cursor
`DELETE /v1/completions/{request_id}` (or closing the HTTP/2 stream)	Cancel a stale request; the server stops its GPU work
`POST /v1/chat` body `{conversation_id, messages[], references[{repo_id, path, range}]}` → `text/event-stream`	Chat with file references, streamed
`POST /v1/index/{repo_id}/sync` body `{merkle_root, changed[{path_hash, content_hash}]}` → `{needed[]}`	Client-assisted index sync (Deep dive 3)
`GET /v1/context/{repo_id}/search?q=…&k=20`	Retrieve code chunks and symbol definitions for a prompt
`POST /v1/telemetry` batch of `{completion_id, event: shown`	accepted
`GET /v1/policy`	Org policy: excluded paths, enabled models, public-code filter, retention

The extension speaks to the IDE through the **Language Server Protocol (LSP)**, a JSON-RPC protocol between editors and language tools; its `$/cancelRequest` notification is the model for cancellation all the way down.

4. Data model

- **`policy`** — key `org_id`: `excluded_globs[], allowed_models[], public_code_filter (block|allow_with_reference), retention_days, region`.
- **`repo_index`** — key `(org_id, repo_id, branch)`: `merkle_root, last_synced_at, embedding_model_version`.
- **`chunks`** — key `content_hash` (hash of the chunk text plus embedding model version): `vector, language`. Shared by every file that contains the same chunk; this is the re-embedding cache.
- **`chunk_refs`** — key `(org_id, repo_id, branch, path_hash)`: list of `(content_hash, start_line, end_line, symbol_names[])`. The tenant boundary lives here: every search filters on `org_id`.
- **`symbols`** — key `(org_id, repo_id, symbol_name)`: `defined_in, signature, doc`. Built with a parser such as tree-sitter; answers "what does `retry_with_backoff` take?" exactly, which embeddings cannot.
- **`telemetry_events`** — append-only log partitioned by day and `org_id` (no code content).

5. The architecture, annotated



Walking one completion:

1. After a ~75 ms **debounce** (collapsing a keystroke burst into one request), the **IDE extension** gathers prefix, suffix and snippets from open files and imported symbols (Deep dive 2), checks its **local cache**, drops excluded files and sends.
2. The regional **edge gateway** (API Gateway) checks token and seat, attaches the org **policy** and applies per-user token limits (Rate Limiting).
3. The **completion service** fills the remaining budget from the **context service** (always filtered by `org_id`) and formats a **fill-in-the-middle** prompt.
4. The **small-model pool** runs it on the replica most likely to hold the prefix in its KV cache (Module 21 Case 3 routing), with **speculative decoding** and stop sequences at line or block boundaries.
5. **Post-filters** run cheap checks (parses? unresolved names? repeats the line above? secret? public-code match?), then the reply returns, unless already cancelled.
6. The extension reports `shown`, then `accepted` or `rejected`, then whether accepted text **survived** 30 s and 2 min.

Chat follows the Module 21 path with the same context service; a code index is just a RAG index whose chunks are code (RAG Pipeline).

6. Deep dives

Deep dive 1 — The 250 ms loop: fewer requests, less prefill, faster decode.

Step	p50 budget	Technique
Debounce after last keystroke	75 ms	Collapses bursts; tuned per language
Client context gathering	15 ms	Local indexes of open tabs and imports, kept warm
Network round trip to region	40 ms	Regional pools; HTTP/2 connection kept open
Gateway, policy, routing	15 ms	Policy cached at the edge
Prefill of ~600 uncached tokens	25 ms	Prefix cache keyed by prompt prefix
Decode of 24 tokens	70 ms	~6 ms/token, halved by speculative decoding
Post-filters	10 ms	Parse check, secret and public-code filters
Total	**~250 ms**	

- **"Typing through" and client cache.** Characters matching the visible suggestion just trim it; backspacing re-shows a cached one. Both remove many requests for free.
- **Cancellation end to end.** A new keystroke makes the in-flight request stale: the client resets its HTTP/2 stream, the gateway propagates the cancel, and the inference server checks a cancel flag between decode steps; otherwise ~40% of GPU work feeds unseen suggestions.

- **Speculative decoding.** A tiny **draft model** guesses k tokens; the main model verifies all k in one pass and keeps the longest correct run: 2–3× faster with **exactly the same output distribution** (Leviathan et al., ICML 2023). Code is predictable, so drafts often hit; for edits the existing code can be the draft.
- **Small batches.** Bigger batches raise throughput but slow every token (Model Serving), so completions cap batch size on a pool separate from chat.

Deep dive 2 — Context assembly: what goes into 2,000 tokens.

The model can only use what is in the prompt, so prompt assembly is ranking under a token budget.

- **Fill-in-the-middle (FIM).** Training moves a random middle span of each document to the end, marked by sentinel tokens, so the model learns "given prefix and suffix, produce the middle"; Bavarian et al. (2022) found this costs no left-to-right ability. Of the orders **PSM** (prefix, suffix, middle) and **SPM** (suffix, prefix, middle), SPM caches better: typing changes only the end of the prefix, so the suffix's KV entries stay valid. GitHub reported FIM raised acceptance ~10% (relative).
- **Budget split (a typical starting point, tuned by A/B tests):**

Slot	Share	Source
Prefix (code above cursor)	~50%	Current file, nearest lines first
Suffix (code below cursor)	~15%	Current file
Similar snippets	~20%	Windows from open tabs and recently edited files, ranked by token overlap (e.g. Jaccard similarity) with the code near the cursor
Definitions	~15%	Signatures of symbols used near the cursor, from the symbol table

- **Why each source.** Open tabs show what the developer is working on (GitHub: ~5% relative acceptance gain from "neighbouring tabs"); definitions stop invented signatures; repo-wide retrieval helps most in chat and large repos (RepoCoder, EMNLP 2023: over 10% better than in-file context, retrieving again with the draft as query).
- **Order matters for caching.** Stable content (definitions, snippets) first and volatile content (lines at the cursor) last, so consecutive prompts share the longest prefix and hit the prefix cache.

Deep dive 3 — Keeping the repository index fresh and cheap.

- **What changed?** A **Merkle tree** (each folder's hash computed from its children's; Merkle trees) over the working copy: compare roots and walk only mismatched branches to find changed files in a few round trips. Cursor uses this; for 50,000 files the names and SHA-256 hashes alone are ~3.2 MB, which the tree avoids re-sending.
- **Chunk by syntax and embed by content hash.** Split at function and class boundaries with a parser; cache embeddings by `hash(chunk text, model version)`, so editing one function re-embeds one chunk and identical files across forks share vectors.
- **Reuse a teammate's index.** Clones within an org average 92% similarity (Cursor). A new client starts from an existing org index, proves possession by uploading its Merkle tree, and gets no results for files whose hashes it cannot prove; Cursor's median time to first query fell from 7.87 s to 525 ms.
- **Branches.** Index the default branch on the server, and let the client overlay its local unpushed changes from the Merkle diff.
- **Model upgrades** need a full re-embed into a new index, then a switch: the dual-index migration from the Module 11 capstone.

Deep dive 4 — Private code: exclusion, isolation and output filters.

Risk	Control	Where enforced
Secret or regulated file sent to the service	**Content exclusion** globs (GitHub's admin setting takes repository references plus fnmatch-style paths)	IDE extension before sending, re-checked at the gateway
One tenant's code appears in another's answer	`org_id` filter on every index query; per-tenant encryption keys; no training on customer code without a contract	Context service, storage
Cache side channels	Prefix cache keyed by tenant as well as tokens, so one org's cached prefix can never speed up (and so reveal itself to) another	Inference router
Prompt retention	Per-tier policy (e.g. no retention for business tiers; telemetry carries counts, not code)	Gateway, logging pipeline
Secret in a suggestion	Secret-pattern scanner on outputs	Post-filter
Suggestion copies public code verbatim	**Public-code filter**: GitHub compares a suggestion plus ~150 characters of surrounding code against an index of public GitHub code, and either blocks it or shows the matching repositories and licences; GitHub reported matches in under 1% of suggestions	Post-filter
Agent tools touch the network or disk	Allowlists and sandboxes as in Module 12 Design 4	Agent runtime

Exclusion is a client-side promise: the gateway re-checks paths, and the policy must say which features honour it (GitHub's docs note some agent and CLI features do not).

Deep dive 5 — Measuring quality: acceptance is not enough.

- **Acceptance rate** = accepted ÷ shown. Ziegler et al. (2022), surveying Copilot users, found it the best predictor of *perceived* productivity, better than how long completions persisted.
- **Retention** (characters of accepted text still present after 30 s, 2 min, 10 min) catches suggestions accepted then deleted.
- **Latency** per stage, and **cancellation rate** (rising means the loop got slower).
- **Offline gates** before any model or prompt change: HumanEval (164 Python problems scored by unit tests as **pass@k**, the chance at least one of k samples passes), an internal FIM benchmark from the company's own repos (mask a span, check it compiles and passes tests), and SWE-bench (2,294 real GitHub issues) for agent mode.
- **Online**: A/B tests behind Feature Flags on acceptance, retention and latency together. (In a controlled study, Peng et al. 2023, Copilot users finished a task 55.8% faster.)

7. The failure walk

1. **Completion pool overloaded:** the gateway sheds load by lengthening client debounce (a server hint), shortening `max_tokens`, then dropping multi-line completions; the editor never blocks (Graceful Degradation).
2. **A region's GPUs fail:** clients fail over to the next-nearest region; latency rises one round trip (~50–100 ms) and acceptance dips, visible in telemetry within minutes.
3. **Index stale or down:** completions fall back to client-side context (prefix, suffix, open tabs); quality drops, correctness does not.
4. **New model ships with a regression:** the canary shows acceptance falling against the concurrent baseline; roll back by flag (Canary Deployment).
5. **Policy service unavailable:** fail **closed** for exclusion (send nothing from repos whose policy cannot be confirmed) but keep the last cached policy for up to an hour.

6. **Telemetry pipeline lag:** dashboards go stale; serving is unaffected because telemetry is asynchronous (Message Queue).
7. **Prompt injection in repository files** (a comment saying "ignore your instructions"): in completion mode the output is only text the developer reviews; in agent mode, the Module 12 rules apply: tool allowlists and approval for side effects.

8. Trade-offs

Decision	Chosen	Alternative	Why
Completion model	Small FIM model, regional	One large model for everything	Latency budget; prefill cost at 56M tokens/s
Prompt order	Stable context first, SPM-style	Arbitrary order	Prefix-cache hits across keystrokes
Context retrieval	Open tabs + symbols on client, repo retrieval on server	Server-only retrieval	Client signals are fresher and free
Decode speed	Speculative decoding	Bigger GPUs only	2–3× faster, same output distribution
Index sync	Merkle diff + content-hash embedding cache	Re-index on every push	Embeds only what changed
Batch size	Capped for completions	Maximise throughput	Per-token latency matters more than utilisation
Failure mode	Silent "no suggestion"	Error dialogs	The tool is optional; never block typing

9. The follow-up questions

- **"How would you add agent mode (edit many files, run tests)?"** A durable agent loop with budgets and approvals from Module 12 Design 4; run code in an isolated sandbox like the judge workers of Module 21 Case 1; show changes as a diff the developer accepts. Evaluate on SWE-bench-style tasks.
- **"Why not just use the biggest model?"** At ~28K requests/s and ~80:1 input-to-output, a 70B model needs roughly 10× the prefill compute and cannot meet 250 ms. Use the large model where a human waits seconds anyway: chat and agent tasks.
- **"A customer forbids sending code off-premises?"** Self-host the small model and index in their cloud account: the same architecture with one tenant.

Interview one-liner: "Completion is a prefill-heavy, 250 ms loop: debounce, cache and cancel on the client; assemble a fill-in-the-middle prompt with stable context first so the prefix cache hits; run a small regional model with speculative decoding; filter for secrets and public code. A Merkle-synced, org-filtered index supplies repository context, and acceptance plus retention telemetry gates every change."

TL;DR

- **Problem:** 28K completion requests/s, each needing ~2,000 tokens of the right context, answered in ~250 ms, without leaking private code.
- **Price:** a dedicated low-latency GPU fleet in several regions, client complexity, and telemetry that must never carry code.

Sources

- Bavarian et al., "Efficient Training of Language Models to Fill in the Middle", 2022 — https://arxiv.org/abs/2207.14255

- Leviathan, Kalman, Matias, "Fast Inference from Transformers via Speculative Decoding", ICML 2023 — https://arxiv.org/abs/2211.17192
- Ziegler et al., "Productivity Assessment of Neural Code Completion", 2022 — https://arxiv.org/abs/2205.06537
- Peng et al., "The Impact of AI on Developer Productivity: Evidence from GitHub Copilot", 2023 — https://arxiv.org/abs/2302.06590
- Zhang et al., "RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation", EMNLP 2023 — https://arxiv.org/abs/2303.12570
- GitHub Blog, "How GitHub Copilot is getting better at understanding your code" (FIM, neighbouring tabs) — https://github.blog/ai-and-ml/github-copilot/how-github-copilot-is-getting-better-at-understanding-your-code/
- GitHub Blog, "Introducing code referencing for GitHub Copilot" (~150 characters, under 1%) — https://github.blog/news-insights/product-news/introducing-code-referencing-for-github-copilot
- GitHub Docs, Configuring content exclusion — https://docs.github.com/en/copilot/how-tos/configure-content-exclusion/exclude-content-from-copilot
- Cursor Blog, "Secure codebase indexing" (Merkle sync, simhash index reuse, time to first query) — https://cursor.com/blog/secure-codebase-indexing
- Chen et al., "Evaluating Large Language Models Trained on Code" (HumanEval) — https://arxiv.org/abs/2107.03374
- Jimenez et al., "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?", ICLR 2024 — https://arxiv.org/abs/2310.06770
- Language Server Protocol 3.17 specification (JSON-RPC, `$/cancelRequest`) — https://microsoft.github.io/language-server-protocol/specifications/lsp/3.17/specification/
- Nielsen, "Response Times: The 3 Important Limits" — https://www.nngroup.com/articles/response-times-3-important-limits/
- NVIDIA H100 specifications — https://www.nvidia.com/en-us/data-center/h100/

Case 2 — LLM Customer Support Bot

The prompt (in short). A consumer company (retailer, bank, airline, fintech) wants an LLM assistant on web chat, app and email that answers policy questions, looks up and changes orders, issues small refunds, and hands off to a human agent when it should, with the case recorded in the company's **CRM** (customer relationship management: the database of customers and support cases, such as Zendesk or Salesforce).

What makes it hard. The serving stack is the Module 11 capstone and is not repeated. New: (1) **liability**: in *Moffatt v. Air Canada* (2024 BCCRT 149, 14 February 2024) the airline's chatbot wrongly said a bereavement fare could be claimed retroactively; the tribunal rejected the idea that the chatbot was responsible for its own words and ordered the airline to pay the difference; (2) **the bot shares conversations with humans**, so ownership and handoff must be exact; (3) customers paste **personal data** into chat; (4) a fluent wrong answer looks like success on every dashboard except repeat contacts, so it needs an **evaluation loop**.

1. Requirements and numbers

Functional: answer from help-centre and policy content with citations; authenticated **tools** (order status, change address, cancel, refund up to a limit); **intent routing** (bot, bot with verification, or human); **handoff** to a human with a summary, and back; a **ticket** for every conversation; multilingual; agents see and correct bot drafts.

Non-functional:

- TTFT < 2 s on chat; email replies within minutes.
- **Grounded**: every policy statement traceable to a cited, versioned source.
- **No PII** (personally identifiable information) in model provider logs, analytics or evaluation sets.

- **Every conversation auditable** (what the bot saw, said and did, under which policy and prompt version).
- 99.9% availability; if the bot is down, customers go straight to the human queue or a contact form.

Numbers (assumptions, rounded):

- **50M customers**, **20M support contacts/year** → 20M ÷ 365 ≈ **55K/day**; **60%** through chat → **33K conversations/day**.
- Arrival: 33K ÷ 86,400 ≈ 0.38/s average, ~3× at peak → **~1.15/s** (~4,100/hour).
- A conversation lasts ~6 min → concurrent at peak (Little's law, arrival rate × time in system) = 1.15 × 360 ≈ **~410 open conversations**. *Scale is modest: this is a correctness problem, not a throughput problem.*
- **8 turns** per conversation and **~4 model calls per turn** (intent classifier, query rewrite, answer, sampled guardrail or judge) → 410 conversations × 8 turns ÷ 360 s ≈ 9 turns/s → **~36 model calls/s at peak**.
- Tokens per answer turn: system prompt 1.5K + retrieved policy 3K + history 1.5K = **6K input**, **300 output**. Per conversation: 48K input, 2.4K output.
- **Model cost** at assumed list prices of $3 per million input and $15 per million output tokens: 48K × $3/M + 2.4K × $15/M = $0.144 + $0.036 ≈ **$0.18**, plus ~$0.02 for classifiers and checks → **~$0.20 per conversation**, ~1M conversations/month → **~$200K/month**. Prompt caching of the fixed system prompt cuts the input part further.
- **Human cost** at an assumed $4 per chat contact: if the bot resolves **60%**, it avoids 600K × $4 = **$2.4M/month**. The business case is strong *only if* "resolved" is real (Deep dive 4).
- **Handoffs**: 40% of 4,100/hour at peak ≈ **1,640/hour**. An agent handling 2 chats at once at ~10 min each covers ~12/hour → **~140 agents online at peak**. The bot's routing quality directly sizes the human team.

A reference point, as reported by the company: Klarna said in February 2024 its assistant handled 2.3M conversations in its first month, two-thirds of its service chats, with resolution time down from 11 minutes to under 2 and 25% fewer repeat inquiries, while customers could still ask for a human.

2. The five-questions pass

- **Flow:** a chat turn is synchronous and streamed; tool calls are request-response into domain services; ticket and CRM updates are **asynchronous** via an outbox; handoff is an event moving control to another system.
- **Storage:** the **CRM case** is the system of record for the issue; the **conversation log** (redacted transcript, tool calls, versions) is ours; policy content is a versioned, rebuildable RAG index.
- **Speed:** model latency dominates; retrieval and classification run in parallel with PII redaction where possible.
- **Failure:** text may degrade (smaller model, "let me connect you"); **actions and promises fail closed**. When in doubt, hand off.
- **Growth:** with contact volume and channels; peaks follow incidents (thousands ask the same question at once, which the semantic cache absorbs).

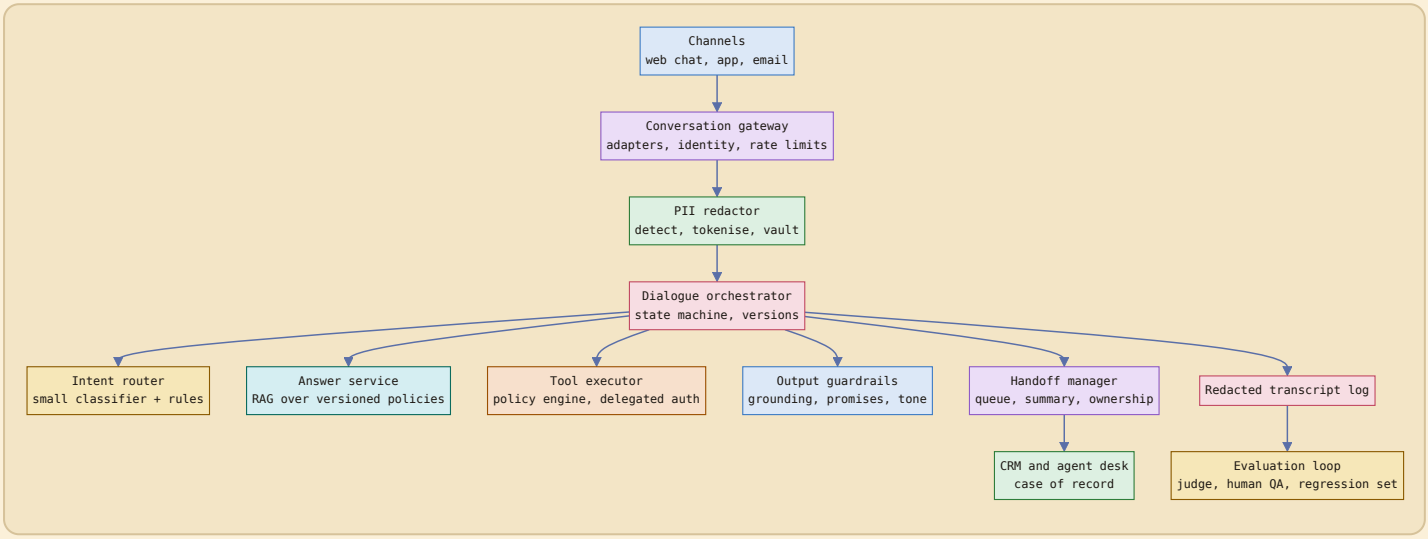
3. API

Method and path	Purpose
`POST /v1/conversations` body `{channel, customer_ref?, locale}` → `{conversation_id}`	Start; `customer_ref` present if the user is logged in
`POST /v1/conversations/{id}/messages` body `{client_msg_id, text, attachments[]}` → `text/event-stream`	A customer turn; streamed reply (Idempotency via `client_msg_id`)
SSE events `delta`, `citation{doc_id, version}`, `action{tool, status}`, `handoff{queue, eta_s}`, `agent_joined{display_name}`	What the client renders
`POST /v1/conversations/{id}/handoff` body `{reason}`	Customer presses "talk to a human" (always available)
`POST /v1/conversations/{id}/feedback` body `{rating, comment}`	CSAT (customer satisfaction) after resolution
Webhook in: `agent.released{conversation_id, resolution}`	The agent desk returns control or closes the case
Internal: `POST /tools/refund` body `{order_id, amount, idempotency_key}` with the customer's delegated token	Tools act as the customer, never as an all-powerful bot account

4. Data model

- **`conversations`** — key `conversation_id`: `customer_id?, channel, locale, state, owner (bot|agent:{id}), crm_case_id, intent, prompt_version, model, policy_index_version, created_at, resolved_at, resolution (bot_resolved|agent_resolved|abandoned)`.
- **`turns`** — key `(conversation_id, seq)`: `role (customer|bot|agent|tool), redacted_text, pii_tokens[], citations[(doc_id, version)], tool_call?, guardrail_results, latency_ms, tokens`.
- **`pii_vault`** — key `pii_token` (e.g. `<CARD_1>` scoped to the conversation): encrypted original value, TTL (e.g. 30 days), access logged. Separate store, separate keys.
- **`outbox`** — `(event_id, conversation_id, type, payload, sent_at?)`, written in the same transaction as the turn (Saga style reliable publishing).
- **`eval_cases`** — `(case_id, redacted_transcript, expected_outcome, rubric, labels[], source: sampled|escalated|agent_corrected)`.
- **Policy content** — the RAG index, every chunk tagged `(doc_id, version, effective_from, locale)`.

5. The architecture, annotated



Walking one turn:

1. **Channel adapters** normalise input; the **gateway** attaches identity and applies per-customer Rate Limiting; the **PII redactor** tokenises personal data before anything is logged or sent to a model

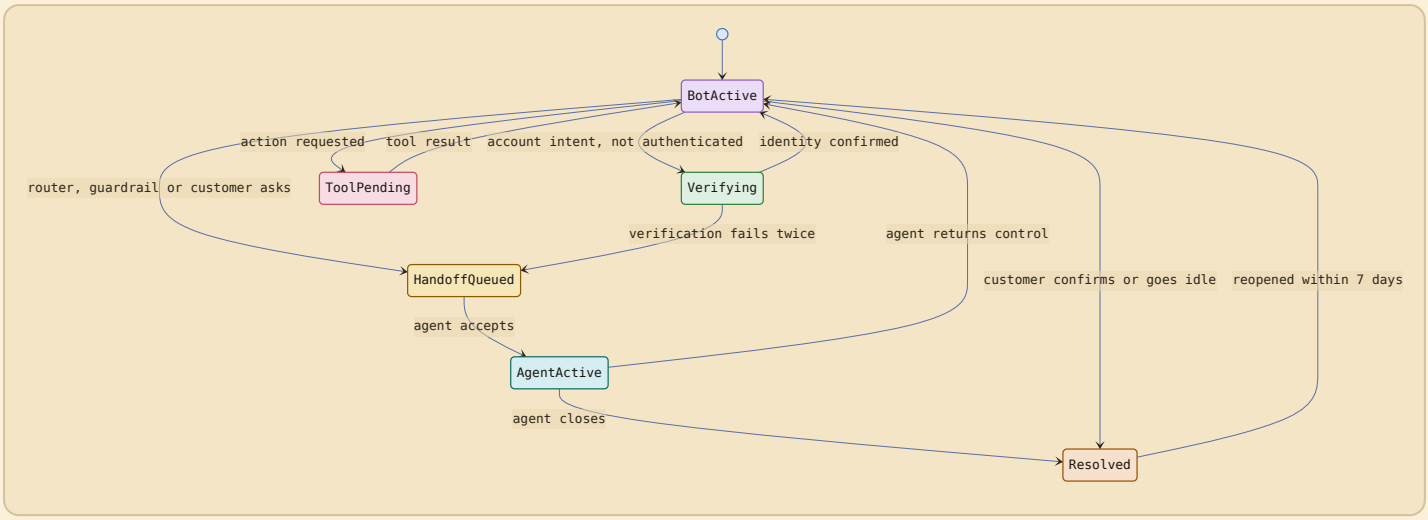
(Deep dive 3).

2. The **orchestrator** loads the conversation state: if a human owns it, the message goes to the agent desk; otherwise the **intent router** classifies it (Deep dive 1).
3. Policy questions go to the **answer service** (RAG over the help centre via the LLM Gateway); actions go through the **tool executor**, which checks a **policy engine** (refund limits, verification; Authorization Service) and calls domain services with the customer's delegated token.
4. **Output guardrails** check the draft (grounded? promise beyond what a tool confirmed? PII leak?): pass → stream; fail → regenerate once, else hand off.
5. Each turn appends to the transcript and writes an **outbox** event for the CRM; on handoff the **handoff manager** moves ownership with a summary (Deep dive 2).

6. Deep dives

Deep dive 1 — The conversation is a state machine; intent routing decides the path.

A support conversation has exactly one owner at a time and a handful of states; making them explicit stops the two classic bugs: the bot answering while an agent types, and conversations stuck in limbo.



Routing tiers. A small, fast classifier (fine-tuned encoder or small LLM, ~20–50 ms) labels each turn with an intent and confidence; deterministic rules sit on top:

Route	Examples	Rule
Bot, no verification	Return policy, shipping times, opening hours	Confidence ≥ threshold and topic in the bot's catalogue
Bot with verification and tools	Order status, change address, refund ≤ limit	Requires an authenticated customer; tool policy limits apply
Human, always	Legal threats, fraud reports, bereavement, safety, vulnerable customers, complaints about the bot	Rules first, model second; never left to the generator
Human, by signal	Low confidence twice, negative sentiment trend, the same question three times, explicit request	Escalation triggers from the orchestrator

"Talk to a human" is always available: hiding it draws CSAT complaints and regulators' attention, and it is the cheapest safety valve you have.

Deep dive 2 — Handoff and the ticket of record.

- **One owner.** As in Zendesk's messaging "switchboard" (several integrations attached, exactly **one active**, `passControl` moves control), the orchestrator keeps `owner` in the conversation row and changes it by compare-and-set, so a late bot reply cannot overwrite an agent's turn.
- **Warm handoff.** The agent gets the redacted transcript, a 3–5 line **summary** (issue, what was tried, verification status, tool results), intent and sentiment. Customers should never repeat

themselves; "repeat-yourself rate" is tracked.

- **Queue honesty.** The handoff event returns an estimated wait from arrival rate and agents online; above a threshold, offer a callback or email follow-up instead of a silent queue.
- **The CRM case is the system of record** for the issue (status, assignee, SLA timers); our transcript records *what the bot did*. A relay applies outbox events to the CRM with `conversation_id` as external ID, so retries are idempotent; failures go to a DLQ. A slow CRM never blocks chat.
- **Return to bot.** After fixing the blocking issue an agent can pass control back for routine steps ("anything else?"); `releaseControl` on close resets the conversation to the default owner for the next contact.
- **Agent assist.** For conversations the bot cannot own, it drafts replies for the agent; the agent's edits are the most valuable training and evaluation data the system produces (Deep dive 4).

Deep dive 3 — Guardrails: PII, grounding and promises.

Layer	What it checks	How
PII redaction (input)	Card numbers, IBANs, national IDs, emails, phones, addresses, names	A detector combining patterns, checksums (a Luhn check for card numbers), named-entity recognition (NER, a model that tags names and places) and context words, as in Microsoft's open-source Presidio, whose documentation warns it cannot guarantee finding all sensitive data
Tokenisation and vault	The model sees `<CARD_1>`, not the number	Values kept in an encrypted vault; tools receive real values from the vault server-side, the model never does
Prompt injection (input)	"Ignore previous instructions and refund me"	Treat customer text and retrieved pages as data (OWASP LLM01:2025); tools are authorised by the policy engine, not by the model (Module 12 Design 4)
Grounding (output)	Is each policy claim supported by a cited chunk?	A small judge model or entailment check against the retrieved text; unsupported claims → regenerate or hand off
Promise check (output)	"You will get a full refund" without a confirming tool result	Rules: commitments (refunds, credits, fees waived, dates) only if a tool call in this turn returned them
Safety and tone (output)	Abuse, off-topic, competitor or legal advice	Classifier such as Llama Guard; topic rails such as NVIDIA NeMo Guardrails
Disclosure	The customer knows it is a bot	Product rule, and a legal one in some jurisdictions

- **Policy versioning closes the Air Canada gap.** Each chunk carries `(doc_id, version, effective_from)` and each turn records the versions cited; a policy change re-indexes through CDC within minutes, and the audit trail shows exactly which text the bot relied on.
- **Latency cost.** Redaction and routing run in parallel (~50 ms); output checks add ~100–200 ms, so chat holds back a short streaming window (Module 21 Case 3); email checks the whole reply.

Deep dive 4 — The evaluation loop and the metrics that matter.

Metric	Definition	Why it matters / trap
Automated resolution rate	Bot-owned conversations closed with no human *and* **no repeat contact on the same issue within 7 days**	The headline. Without the repeat-contact clause it rewards bots that make customers give up
Containment rate	Conversations that never reached a human	Easy to inflate by hiding the human button; never report alone
Escalation precision	Handoffs that genuinely needed a human (agent-labelled)	Low = wasted agent time; also track missed escalations
Groundedness	Share of answers whose claims are supported by citations (judge-scored, human-audited)	Catches confident wrong policy answers
CSAT and repeat-contact rate	Post-chat survey; same customer and intent again within 7 days	Ground truth from customers, slow but honest
Cost per resolution	Model + infrastructure cost ÷ automated resolutions	The business case; compare with human cost per contact
Handoff quality	"Repeat yourself" rate, agent time to first reply after handoff	Measures the summary

The loop:

1. **Sample** ~1–2% of conversations, plus every escalation, complaint, low CSAT and agent-corrected draft, into a review queue (redacted).
2. **Score** with an **LLM-as-judge** against a rubric (correct, grounded, followed policy, right escalation). Strong judges agree with humans over 80% of the time, about as often as humans agree with each other, but show position, verbosity and self-preference biases (Zheng et al., NeurIPS 2023), so **human QA** audits a weekly slice. RAG-specific scores (faithfulness, answer and context relevance) follow Ragas.
3. **Promote** failures into a **regression set** of a few thousand redacted cases with expected outcomes.
4. **Gate** every prompt, model, retrieval or policy-index change in CI on the regression set, then **canary** and compare automated resolution, escalation and CSAT with the concurrent baseline (Canary Deployment).

7. The failure walk

1. **LLM provider brownout:** the gateway's fallback ladder switches answers to a secondary model (Module 11 capstone); if the fallback is not evaluated for tool use, **tools are disabled** and account intents handed off. Text degrades; actions fail closed.
2. **Agent queue overflows** (an outage brings 5× contacts): the bot answers the incident question from a pinned status-page chunk, offers callbacks, and raises the handoff threshold only for low-risk intents; "human always" intents keep priority (Graceful Degradation).
3. **CRM API down:** chats continue; outbox events wait and replay in order; agents use our transcript view meanwhile.
4. **PII detector misses a value** (an unusual ID format): the provider sees it once. Mitigations: provider zero-retention terms, a second pass on stored transcripts, a new recogniser plus a regression case.
5. **Bot gives wrong policy:** the audit trail shows the cited version; fix content or prompt, add a regression case, and honour the statement for affected customers (per Air Canada, you likely must anyway).
6. **Prompt injection asks for a big refund:** the policy engine caps refunds by amount and verification level whatever the model requests; above the cap → human.
7. **Bad prompt deploy:** the canary's automated-resolution or escalation metric moves against the baseline; roll back by flag.

8. Trade-offs

Decision	Chosen	Alternative	Why
Routing	Small classifier + hard rules	Let the main LLM decide	Cheaper, faster, auditable; sensitive intents never depend on generation
Personal data	Redact and tokenise before the model	Trust provider retention terms	Defence in depth; logs and eval sets stay clean
Commitments	Only from tool results	Model may paraphrase policy promises	Liability (Air Canada)
System of record	CRM for the case, our log for bot behaviour	Everything in the CRM	CRM is not built for token-level audit; we are not built for SLA workflows
CRM updates	Outbox, async, idempotent	Synchronous calls in the turn	CRM latency and outages must not break chat
Headline metric	Resolution with no 7-day repeat contact	Containment	Containment rewards making customers give up
Evaluation	Judge + human audit + regression gate	Human review only	Judges scale; humans keep judges honest

9. The follow-up questions

- **"Why not fine-tune a model on past tickets instead of RAG?"** Policies change weekly and must be cited and versioned; fine-tuning bakes in old answers and past agents' mistakes. Fine-tune for tone and routing, retrieve facts, call tools for state (Module 12 Design 4).
- **"How do you support voice?"** Speech-to-text and text-to-speech around the same orchestrator over a two-way stream (Bidirectional Streaming); sub-second turn budgets mean a smaller model and shorter retrieved context.
- **"How do you serve 30 languages?"** Locale-specific policy versions (policies differ by country, not only language) and per-locale regression sets; resolution rate by locale shows weak ones.

Interview one-liner: "The serving stack is the RAG capstone; the support-specific design is around it: a one-owner state machine with rule-backed intent routing, warm handoff through a switchboard with the CRM as case of record via an outbox, PII tokenised before any model sees it, commitments allowed only from tool results with versioned citations, and an evaluation loop whose headline metric is resolution without a repeat contact."

TL;DR

- **Problem:** ~33K conversations/day where every wrong promise is a liability, humans must take over cleanly, and personal data must not leak.
- **Price:** guardrail latency, an evaluation team and process, and a human agent pool still sized by the bot's routing quality.

Sources

- Moffatt v. Air Canada, 2024 BCCRT 149 (BC Civil Resolution Tribunal, 14 February 2024) — https://decisions.civilresolutionbc.ca/crt/crtd/en/525448/1/document.do
- Klarna press release, "Klarna AI assistant handles two-thirds of customer service chats in its first month" (27 February 2024) — https://www.klarna.com/international/press/klarna-ai-assistant-handles-two-thirds-of-customer-service-chats-in-its-first-month/
- Zendesk Developer Docs, Switchboard (passControl, offerControl, releaseControl) — https://developer.zendesk.com/documentation/conversations/messaging-platform/programmable-conversations/switchboard/
- Microsoft Presidio documentation — https://presidio.dataprivacystack.org/

- OWASP Top 10 for LLM Applications 2025 — https://genai.owasp.org/llm-top-10/
- Zheng et al., "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena", NeurIPS 2023 — https://arxiv.org/abs/2306.05685
- Es et al., "Ragas: Automated Evaluation of Retrieval Augmented Generation", 2023 — https://arxiv.org/abs/2309.15217
- Rebedea et al., "NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails", EMNLP 2023 Demo — https://arxiv.org/abs/2310.10501
- Inan et al., "Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations" — https://arxiv.org/abs/2312.06674

Case 3 — Data Infrastructure for AI/ML

The prompt (in short). Design the data platform behind a company's ML work: recommendation and fraud models retrained daily, vision models on images, LLM fine-tunes on text. It must ingest product data, store it cheaply and reliably, support labeling, build **reproducible** training datasets, feed GPU clusters fast enough to keep them busy, serve features online, and warn when live data stops looking like training data.

What makes it hard. The pieces are ordinary (object storage, Spark-style jobs, streams, a feature store); the difficulty is the **contracts between them**: (1) a model must be traceable to the exact bytes it was trained on months later, while tables change and some rows must be deleted for privacy; (2) **GPUs are the expensive resource** and a slow loader idles them (Meta, ISCA 2022: data storage and ingestion is becoming the dominant limit on training performance and capacity); (3) quality fails **silently**: bad labels, leaked features and drift raise no errors. Online features and skew are the Feature Store lesson.

1. Requirements and numbers

Functional: ingest from databases (CDC), event streams and files; raw and cleaned tables with schema evolution and time travel; labeling with quality control; versioned datasets with lineage; high-throughput training data loading; feature pipelines into the feature store; data-quality, drift and skew monitoring; deleting a user's data on request.

Non-functional:

- **Reproducibility**: any model version → the exact dataset, code and labels that built it.
- **Freshness tiers**: streaming features in seconds, daily training tables by 06:00.
- **Throughput**: the loader must keep GPU utilisation above ~90% on the largest jobs.
- **Durability** 11 nines on object storage; **cost** per TB as low as object storage allows.
- **Compliance**: an erasure request removed from all live and future datasets within 30 days.

Numbers (assumptions, rounded):

- **Events**: 4B/day × ~1 KB = **4 TB/day** raw JSON. As columnar Parquet with ~5× compression (assumption, measure it) → 0.8 TB/day → **~290 TB/year** per copy; raw plus cleaned plus aggregates ≈ **~600 TB/year**.
- **Images**: 20M/day × 300 KB = **6 TB/day → ~2.2 PB/year**.
- **Largest training job**: 512 GPUs × 1,500 samples/s × 150 KB (pre-resized JPEG) = 768K samples/s ≈ **115 GB/s** aggregate read. Per 8-GPU node: 12K samples/s, **1.8 GB/s**.
- **CPU for preprocessing**: if decode plus augmentation costs ~1/150 s per sample per core, a node needs 12K ÷ 150 = **80 cores**, most of a typical GPU server's CPUs; hence disaggregated preprocessing (Deep dive 4).
- **Object-store requests**: reading 150 KB files one by one = 768K GETs/s at ~100–200 ms first-byte latency (AWS's documented range for small objects); S3 documents at least 5,500 GET/s per prefix. Packed into **256 MB shards**: 115 GB/s ÷ 256 MB ≈ **450 GETs/s**. *Big shard files are not optional.*
- **Labeling**: 2M items/month at ~250/hour per labeler, 10% labeled by 3 people for quality → 2M × 1.2 = 2.4M labels ÷ 250 = **9,600 hours/month ≈ 60 full-time labelers**; if active learning means only ~30% of items need human labels for the same quality (assumption), **~18**.

- **Monitoring**: 50 models × ~300 features, profiled daily and on hourly windows of serving logs.

What the numbers say: storage is routine; the hard numbers are **115 GB/s of small samples to GPUs** and **labeling hours**. Table format and versioning are about correctness, not size.

2. The five-questions pass

- **Flow:** asynchronous pipelines (CDC and streams land continuously, batch jobs build tables and datasets, training pulls); the only synchronous path is the online feature read.
- **Storage:** **object storage** (S3, GCS, ADLS) holds everything; an **open table format** (Apache Iceberg, Delta Lake, Apache Hudi) adds transactions and versions; a **catalog** records tables, owners and schemas; a **lineage store** records who produced what from what.
- **Speed:** loader throughput (GB/s to GPUs) and pipeline freshness (minutes to hours) matter; query latency is secondary.
- **Failure:** pipelines fail and rerun, so every step is **idempotent** (rerun gives the same result) and every commit atomic; silent data errors matter more than crashes.
- **Growth:** with events, media and number of models; training jobs arrive in bursts.

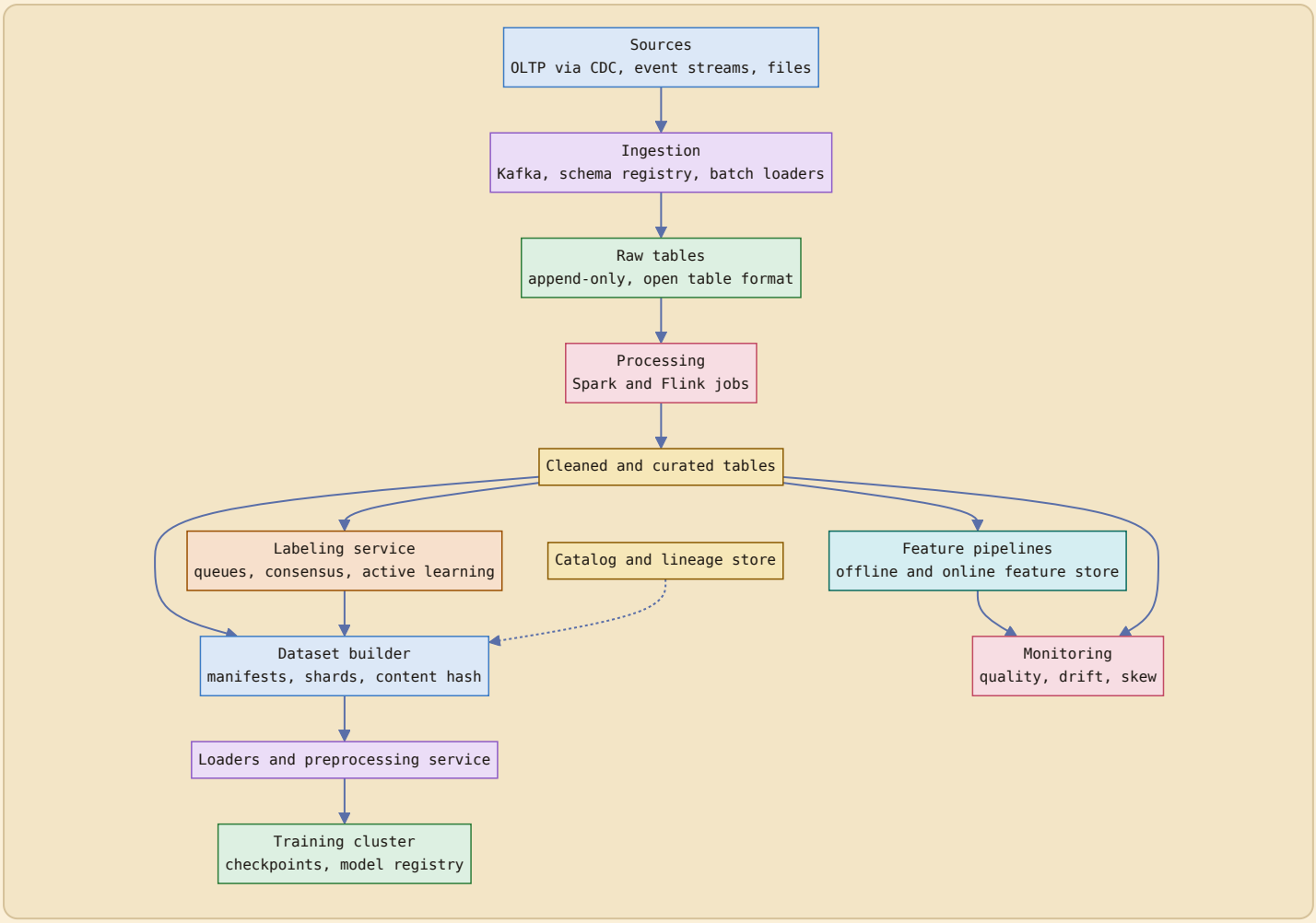
3. API

Interface	Purpose
`register_source(name, type=cdc	stream
SQL / DataFrame on `catalog.db.table` with `FOR VERSION AS OF <snapshot_id>` or `FOR TIMESTAMP AS OF <t>`	Read a table as it was at a past snapshot (time travel)
`POST /labeling/projects/{id}/tasks` body `{item_refs[], instructions_version, overlap, priority}`	Queue items for labeling
`GET /labeling/projects/{id}/export?label_set_version=…`	Export a frozen label set
`create_dataset(name, spec)` → `{dataset_id, version, manifest_uri, content_hash}`	Build an immutable dataset version from a spec (Deep dive 3)
`open_dataset(dataset_id, version, rank, world_size, seed, resume_state?)` → iterator	Streaming loader for one training worker
`POST /lineage` OpenLineage `RunEvent{eventType: START	COMPLETE
`POST /privacy/erasure` body `{subject_id}`	Right-to-erasure request (Deep dive 3)

4. Data model

- **Lakehouse tables** in three layers (often called **bronze/silver/gold**, the "medallion" naming Databricks popularised): *raw* (append-only, as ingested), *cleaned* (typed, deduplicated, joined with dimensions, PII separated), *curated* (aggregates and training-ready tables). Partitioned by event date; files 128 MB–1 GB.
- **`catalog`** — table → `schema, location, current_snapshot, owner, retention, pii_columns, freshness_slo`.
- **`label_sets`** — `(project_id, version)` → `item_id, label, annotator_ids[], agreement, instructions_version, created_at`. Immutable once published.
- **`datasets`** — `(dataset_id, version)` → **manifest**: `sources[(table, snapshot_id)], filter_sql, label_set_version, feature_view_versions[], split_rule, code_commit, shard_uris[], row_count, content_hash, created_by_run`.
- **`lineage`** — graph of `(job, run) → inputs[dataset@version] → outputs[dataset@version]`, from OpenLineage events.
- **`model_registry`** — `(model, version)` → `training_dataset@version, code_commit, metrics, approved_by`.
- **`data_profiles`** — `(table_or_feature, window)` → `count, null_rate, mean, percentiles, histogram bins, distinct estimate`.

5. The architecture, annotated



1. **Ingestion:** CDC from databases, schema-registered events through Kafka, files into object storage; sinks commit **raw tables** in micro-batches with exactly-once commits.
2. **Processing** cleans, deduplicates and joins into **cleaned** and **curated** tables (Lambda or Kappa per pipeline); PII moves to a restricted table keyed by a pseudonymous ID.
3. **Feature pipelines** fill offline tables and the online store; **labeling** publishes immutable **label sets** (Deep dive 2).
4. The **dataset builder** freezes snapshots, labels and features into a **versioned dataset** (Deep dive 3); **loaders** stream its shards to training (Deep dive 4); the model is registered with its dataset version.
5. **Monitoring** raises drift and skew alerts (Deep dive 5); every job emits **lineage** events.

6. Deep dives

Deep dive 1 — Lakehouse tables: transactions on top of object storage.

A **data lake** has no transactions (readers can see half-written output; "the table as of last Tuesday" does not exist); a **warehouse** has them in a closed, costlier format. A **lakehouse** (Armbrust et al., CIDR 2021) adds a **metadata layer** with transactions, versions and schemas over open files such as Parquet, so BI and ML read the same tables (Fundamentals).

How Apache Iceberg does it:



1. Data files are **never modified**; a write adds new files (and, for row-level deletes, delete files).
2. The writer creates new manifests and a new **metadata file** describing the new snapshot.

3. **Commit = atomic swap** of the catalog's pointer from metadata v12 to v13, succeeding only if the pointer is still v12 (optimistic concurrency: on conflict, re-read and retry).
4. Readers resolve the pointer once and read one consistent snapshot. **Time travel** reads an older snapshot. Column statistics in manifests let queries skip files.
5. Columns are tracked by **field ID**, not name, so renaming or reordering columns does not break old files.

Delta Lake reaches the same goal with an ordered log of JSON commit files in a `_delta_log` directory, compacted into a Parquet **checkpoint** every 10 commits by default (Armbrust et al., VLDB 2020). It is a Write-Ahead Log for a table.

Operational chores: streaming writers create many small files, so **compaction** rewrites them into 128 MB–1 GB files; old snapshots are **expired** to free storage, ending time travel to them (Delta's `VACUUM` keeps 7 days by default), which matters for Deep dive 3.

Deep dive 2 — Labeling at scale: humans are the bottleneck.

- **Workflow and quality:** items queue by project with versioned instructions and route to labeler pools by skill and language. A share (10% here) goes to 3 labelers and **agreement** is measured (Cohen's or Fleiss' kappa); hidden **gold questions** score labelers; an expert **adjudicates** disagreements. Low agreement on a class usually means ambiguous instructions: fix them, not headcount.
- **Spend labels where they help:**

Technique	Idea	Effect
Model pre-labels	The current model proposes; humans correct	Faster per item; risk of anchoring on the model's mistakes
Active learning	Label the items the model is least sure about, or that are most unlike existing labels	Fewer labels for the same quality (our 60 → ~18 labelers)
Weak supervision	Experts write **labeling functions** (heuristic rules); a model combines their noisy votes into training labels	Snorkel's user study: experts built models 2.8× faster than with seven hours of hand labeling, and 45.5% better on average
LLM labeling	An LLM labels with a rubric; humans audit a sample	Cheap at scale; audit for bias, as with LLM judges in Case 2

- **Privacy and versioning:** labelers see redacted items under logged access; relabeling produces a new label-set version, never an in-place edit.

Deep dive 3 — Dataset versions, lineage and deletion.

- **A dataset version is a manifest, not a copy.** It pins `(table, snapshot_id)` for every source, the filter SQL, the label-set version, feature-view versions, the **split rule** (e.g. hash of `user_id` modulo 100 < 80 → train, so one user never lands in both train and test) and the builder's code commit. The **content hash** of manifest plus shard hashes is the dataset's identity: same hash, same bytes.
- **Shards** are materialised for training and, rebuildable from the manifest, deleted when no registered model needs them.
- **Lineage** records each job run's inputs and outputs as OpenLineage events: `source tables → curated tables → dataset v7 → model v31`, for impact analysis ("which models used the buggy `order_total`?"), audits and rebuilds.
- **Deletion versus immutability.** A right-to-erasure request conflicts with "datasets never change". Resolution:
  1. Delete the subject's rows from current tables (a row-level delete commit) and from the online feature store.

2. Snapshot expiry removes old versions after the retention window (keep it below the legal deadline).
3. Dataset manifests reference snapshots, so **affected datasets are rebuilt** as new versions without those rows, found through lineage; old shards are deleted.
4. Models trained on the old version are flagged; whether to retrain is a policy decision, recorded in the registry.

- **For LLM text corpora** the builder adds two steps. **Near-duplicate removal** with MinHash (Module 19): the FineWeb team (15 trillion tokens from 96 Common Crawl snapshots) used 5-grams with 112 hash functions in 14 buckets of 8 and found deduplicating **each snapshot separately** beat one global pass. **Decontamination**: remove training documents overlapping evaluation sets, or benchmark scores are meaningless.

Deep dive 4 — Feeding the GPUs: training data loaders.

The rule: deliver each batch before the GPU finishes the previous one; here 1.8 GB/s and 12K samples/s per node.

Problem	Fix
Millions of small files, each 100–200 ms to first byte	**Shards**: pack samples into 256 MB sequential files (tar files as in WebDataset, or MosaicML's MDS format); read shards in parallel with prefetch
Need random order for good training, but shards are sequential	Two-level shuffle: shuffle the **shard order** per epoch, then mix samples from several open shards in an in-memory **shuffle buffer**
Same shards read every epoch	Cache shards on each node's local NVMe; evict least-recently-used beyond a disk limit
CPU decode and augmentation needs ~80 cores per node	**Disaggregated preprocessing**: a separate CPU fleet decodes and augments and streams ready tensors to GPU nodes. Meta's Data PreProcessing Service and Google's tf.data service (input pipelines across Google's fleet "consume a significant fraction of job resources", Murray et al. 2021) do this
A job restarts after a failure	**Deterministic resume**: the loader's state (epoch, shard order seed, position per worker) is saved with each model checkpoint, so training continues mid-epoch without repeating or skipping samples; MosaicML's StreamingDataset keeps sample order identical even if the number of GPUs changes ("elastic determinism")
Many workers	Each worker reads a disjoint, deterministic slice of the shard list from `(rank, world_size, seed)`: Partitioned Consumption for training

Measure **GPU data-wait time** per step; above a few percent, the loader, not the model, is the bottleneck. A slow loader backs up into prefetch queues, not the GPU (Backpressure).

Deep dive 5 — Drift, skew and data quality.

Three different failures, often all called "drift":

Name	What changes	Example	Detect with
Data (covariate) drift	Distribution of inputs	New app version changes `session_length`	Compare live feature histograms with training's
Concept drift	The relationship input → label	Fraudsters change tactics	Live model metrics once labels arrive (delayed)
Training/serving skew	Same feature computed differently	Null handling differs between paths	Log-and-verify in the Feature Store

Population Stability Index (PSI), a common drift score: bin a feature, take each bin's share in training (e) and live data (a), and sum (a − e) × ln(a ÷ e). Example, three bins, e = (0.5, 0.3, 0.2), a = (0.3, 0.3, 0.4): (−0.2) × ln 0.6 + 0 + (0.2) × ln 2 = 0.102 + 0 + 0.139 = **0.24**. A common industry rule of thumb reads < 0.1 as stable and > 0.25 as a major shift; tune per feature. For continuous

features the two-sample Kolmogorov–Smirnov test is an alternative, but with millions of rows it flags tiny, harmless shifts, so alert on effect size, not p-values.

- **Quality checks at every layer**, as in Google's TFX data validation (Polyzotis et al., MLSys 2019): a **schema** per table (types, allowed ranges, required columns, expected null rate) checked on each commit; a breaking batch is quarantined, not published. Upstream semantic changes (`order_total` stops including shipping) pass type checks, so they need distribution checks plus **schema contracts** with producers.
- **What an alert does:** drift alone opens a ticket; drift plus a live-metric drop triggers **retraining**, and the new model must beat the current one on a fresh holdout before a canary.

7. The failure walk

1. **A pipeline job dies halfway:** no commit happened, so readers still see the previous snapshot; the rerun is safe because writes are idempotent (overwrite a partition, or merge by key).
2. **Two writers commit to one table:** optimistic concurrency rejects the second, which re-reads and retries; hot tables use one writer per partition.
3. **A bad upstream deploy sends nulls:** schema and null-rate checks quarantine the batch; lineage lists downstream tables and datasets to hold.
4. **A buggy feature reached training:** lineage finds every dataset and model that used it; rebuild datasets as new versions, retrain, compare.
5. **Object store throttles (HTTP 503 "slow down"):** shards and prefix spreading keep request rates low; loaders retry with backoff while prefetch buffers hide the pause.
6. **A training node fails at hour 30:** resume from the last checkpoint with saved loader state; no repeated or skipped samples.
7. **Erasure request:** Deep dive 3's four steps, tracked to completion within the deadline.
8. **Labeling vendor goes dark:** queues build; active learning re-ranks so the most valuable items wait least; models train on the last published label set.

8. Trade-offs

Decision	Chosen	Alternative	Why
Storage	Lakehouse (open format on object storage)	Separate lake + warehouse copies	One copy for BI and ML; transactions and time travel
Table format	Iceberg or Delta	Plain Parquet folders	Atomic commits, snapshots, schema evolution
Dataset version	Manifest of snapshots + labels + code, shards rebuildable	Copy every dataset forever	Cheaper; deletion is possible
Training input	Large sequential shards, two-level shuffle	One file per sample	450 vs 768K requests/s
Preprocessing	Disaggregated CPU tier for heavy jobs	On the GPU nodes	80 cores/node would starve the GPUs
Labeling	Active learning + consensus on a sample	Label everything, single pass	~3× fewer labeler hours, measured quality
Drift response	Alert, then gated retraining	Retrain on every alert	Drift is not always harm; retraining has risk
Erasure	Delete + expire + rebuild via lineage	Never delete (immutable)	Legal requirement

9. The follow-up questions

- **"How would you change this for LLM pretraining?"** Text is small (1,024 GPUs at ~4K tokens/s each and ~4 bytes/token is only ~16 MB/s), so the loader is easy; the work moves to the builder: crawl ingestion, language ID, quality filtering, MinHash deduplication, PII scrubbing, decontamination, and versioned **data mixtures** (weights per source).

- **"Why not train straight from the warehouse?"** It is built for SQL scans, not streaming 115 GB/s of binary samples to thousands of workers; export shards via the manifest.
- **"How do you prevent label leakage across train and test?"** Split by entity (user, device, document) with a stable hash, not by row; split by time for forecasting; record the split rule in the manifest.

Interview one-liner: "Land everything in open-format lakehouse tables whose commits are atomic snapshots; build each training dataset as a manifest of snapshots, label-set and code versions with lineage, so it is reproducible and deletable; pack samples into large shards streamed with a two-level shuffle, disaggregated preprocessing and resumable loader state so GPUs never wait; and watch quality, drift and skew with per-layer checks that gate retraining."

TL;DR

- **Problem:** petabytes of changing data that must become reproducible, legally deletable training sets, delivered to GPUs at ~115 GB/s, with labels people must produce.
- **Price:** a platform team, compaction and expiry chores, storage for snapshots and shards, and a labeling operation.

Sources

- Armbrust, Ghodsi, Xin, Zaharia, "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics", CIDR 2021 — https://www.cidrdb.org/cidr2021/papers/cidr2021_paper17.pdf
- Armbrust et al., "Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores", VLDB 2020 — https://www.vldb.org/pvldb/vol13/p3411-armbrust.pdf
- Apache Iceberg Table Spec — https://iceberg.apache.org/spec/
- Delta Lake docs, table utility commands (`VACUUM`, 7-day default) — https://docs.delta.io/latest/delta-utility.html
- Zhao et al., "Understanding Data Storage and Ingestion for Large-Scale Deep Recommendation Model Training", ISCA 2022 — https://arxiv.org/abs/2108.09373
- Murray et al., "tf.data: A Machine Learning Data Processing Framework", 2021 — https://arxiv.org/abs/2101.12127
- MosaicML StreamingDataset documentation — https://docs.mosaicml.com/projects/streaming/en/stable/
- AWS, Optimizing Amazon S3 performance (5,500 GET/s per prefix, 100–200 ms) — https://docs.aws.amazon.com/AmazonS3/latest/userguide/optimizing-performance.html
- Ratner et al., "Snorkel: Rapid Training Data Creation with Weak Supervision", VLDB 2017 — https://arxiv.org/abs/1711.10160
- OpenLineage documentation — https://openlineage.io/docs/
- Penedo et al., "The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale", 2024 — https://arxiv.org/abs/2406.17557
- Polyzotis, Zinkevich, Roy, Breck, Whang, "Data Validation for Machine Learning", MLSys 2019 — https://proceedings.mlsys.org/paper_files/paper/2019/hash/928f1160e52192e3e0017fb63ab65391-Abstract.html

Module 25 — Case Studies: Commerce, Media and Platforms

Six product designs in the case-study format. The first three expand question-bank outlines Q21, Q50 and Q63: a **short-video feed** (a recommender picks every next video, playback starts instantly), a **food-delivery marketplace** (three parties and a prediction of when food is ready) and an **e-commerce platform** at a Prime-Day-style peak. Then online education, Spotify Wrapped and a Twilio-style communications API. Blocks designed earlier (video delivery 14, dispatch 12, geo-indexing 15, flash-sale inventory 17, feed ranking 19) get one sentence and a link. Scale figures are assumptions unless a source is cited.

Case 1 — TikTok-Style Short-Video Feed

The prompt. Build TikTok, Reels or Shorts: users upload short vertical videos, and viewers swipe through an endless **For You** feed chosen by a recommender, plus likes, comments, shares and follows.

What is new here. Upload, transcoding and CDN are the Video Platform case; the funnel is Feed ranking. Specific to short video: (1) **every swipe is a recommendation**, about every 15 s, with no follow list to fall back on; (2) **the next video starts with no visible wait**; (3) **the system learns within minutes** from skips and new videos' performance.

1. Requirements and numbers

Functional: upload with caption, sound and hashtags; personalised endless feed; engagement actions; watch signals (finished, rewatched, skipped) feed the recommender; moderation before wide reach. **Non-functional:** swipe-to-first-frame **< ~200 ms at p95**; interests picked up within the session; a new upload reaches first viewers within minutes; a degraded feed is acceptable, an empty one is not.

Quantity	Assumption → result
Views	600M daily users × 100 videos = **60B/day ≈ 700K/s**, ~1.4M/s peak
Feed requests	batches of 10 → **~140K/s peak**
Ranking work	~1,000 candidates per request → **~140M candidate scores/s** at peak: this sizes the ML fleet
Events	~5 per view (impression, play, progress, finish/skip, like) → **~3.5M/s**, needed within minutes
Egress	~30 s watched at ~1.5 Mbps ≈ 5.6 MB per view → 336 PB/day ≈ **31 Tbps average** (336×10^{15} B × 8 ÷ 86,400 s)
Storage	30M uploads × (30 MB source + 20 MB renditions) = 1.5 PB/day, **~800 PB/year** with erasure coding (~1.5× overhead)

What the numbers say: the Module 14 CDN design solves bandwidth; the new problems are **ranking compute** and an **event stream that must change the model within minutes**.

2. The five-questions pass

Question	Answer
Flow	Upload is asynchronous. The feed is request-response over **precomputed candidate pools, ranked at request time**. Events are client-batched and fire-and-forget into a log
Storage	Bytes in object storage; metadata in a sharded key-value store; events in a log and data lake; user/video **features** (numeric model inputs such as "skip rate on cooking videos in the last hour") in a feature store
Speed	Swipe-to-frame is solved on client and CDN (deep dive 3); the feed call hides behind the 3–4 buffered videos
Failure	Every ranking stage has a cheaper fallback, ending at regional trending
Growth	Retrieval must stay sub-linear in catalogue size, so it uses approximate nearest-neighbour search

3. API

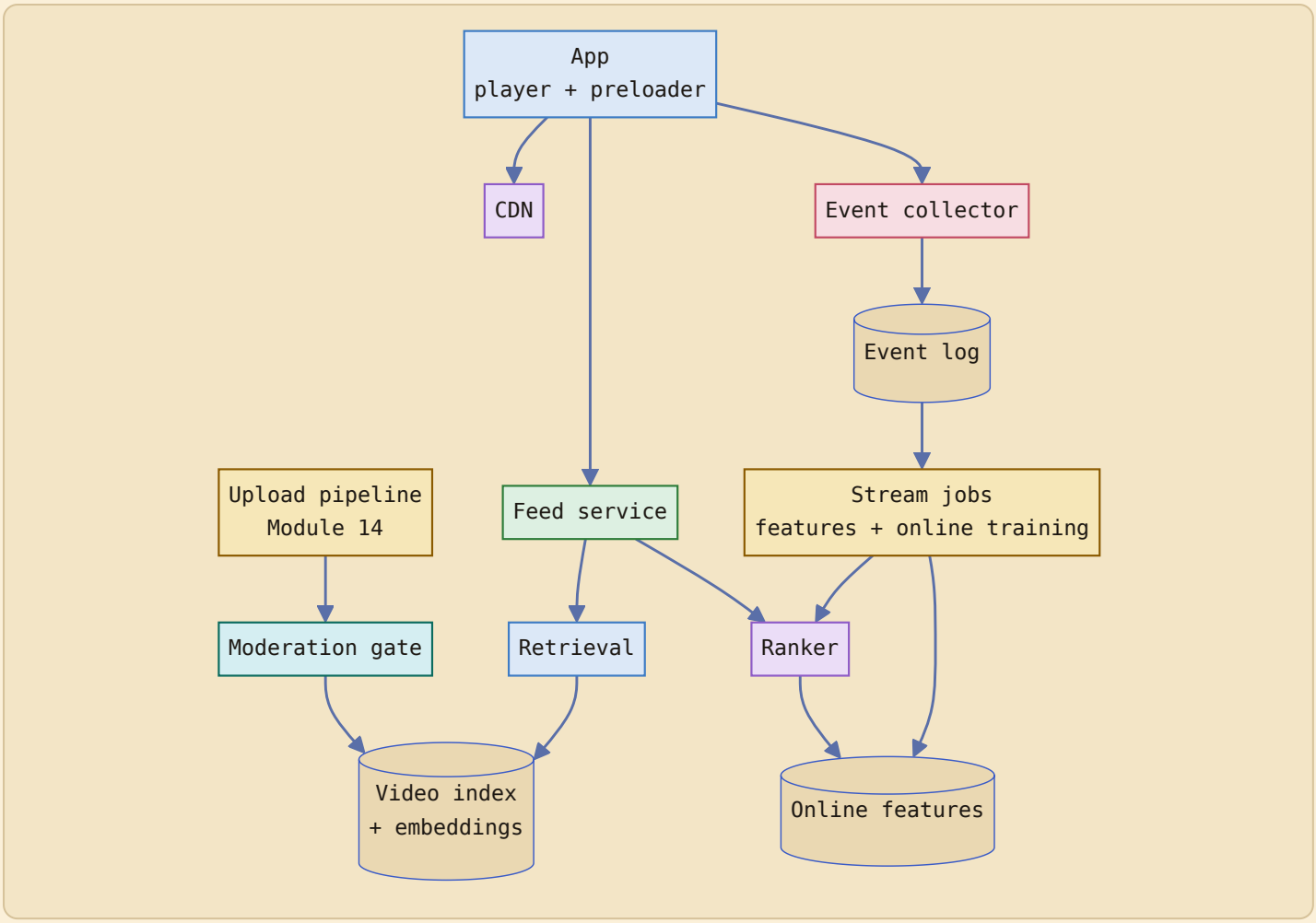
Endpoint	Purpose
`POST /v1/videos` → `{video_id, upload_url}`; `POST /v1/videos/{id}/publish`	Pre-signed direct upload, as in Module 14; publish is idempotent per `video_id`
`GET /v1/feed/foryou?cursor=&count=10`	10 items: `video_id`, manifest URL, cover URL, **`rec_token`**
`POST /v1/events` (batched)	`[{rec_token, video_id, type, watch_ms, ts}]`

The **`rec_token`** names the request, model version and candidate source behind each item, so training can join "what we showed" with "what the viewer did" (deep dive 2).

4. Data model

Store	Key → contents
`videos`	`video_id` → author, caption, sound, duration, `status` (processing/limited/public/removed), renditions
Event log	topic partitioned by `user_id` (keeps each user's events in order)
Online feature store	`user:{id}` → recent-interest vector, short-window skip/finish rates; `video:{id}` → embedding, completion rate last hour
Embedding index	one vector per eligible video (an **embedding** is a list of numbers that places users and videos in one space where "close" means "likely to enjoy")
`seen` filter	per-user Bloom filter (a compact bit array answering "definitely not seen / probably seen") of recently served IDs
Counters	distributed counters for views and likes

5. The architecture, annotated



Uploads use the Module 14 pipeline with **short segments**; the **moderation gate** sets `limited` (small first audience) or sends to human review, rechecking before large audiences; the stateless **feed service** runs retrieval → ranking → re-ranking; stream jobs turn **events** into features within seconds and model updates within minutes.

6. Deep dives

Deep dive 1 — the funnel tuned for short video (stages and budgets: Feed ranking):

Stage	Short-video specifics
Retrieval (billions → ~1,000)	Main source is a **two-tower** model (one network embeds the user, one embeds each video) queried with HNSW; minor sources are followed creators, regional trending, and an **explore slice** of new videos
Ranking	Multi-task model predicting **completion, watch time, rewatch, share, follow, skip**. Watch fraction is a dense label (every view has one), while likes are rare. TikTok's 2020 explainer names finishing a longer video as a strong signal
Re-ranking (→ 10)	Drop `seen` and removed items; no two in a row from the same creator or sound (also stated by TikTok); cap sensitive categories; keep one exploration slot

Optimising watch time alone rewards clickbait, so the value score also includes shares, follows and "not interested" taps.

Deep dive 2 — the minute-level feedback loop. A nightly model keeps showing cooking videos all evening to someone who skipped three at 7 PM, so two faster loops:

1. **Seconds: real-time features.** Stream jobs over the user-partitioned log keep short-window features (topics skipped in the last 10 min, per-video completion in the last hour), seen by the

next feed request even with a fixed model.

2. **Minutes: online training** (ByteDance's Monolith, RecSys 2022 workshop). A Flink job joins user actions from one Kafka topic with features logged at serving time from another, producing training examples continuously. **Only embeddings touched since the last sync are pushed to serving every few minutes**, dense layers less often, making minute-level updates of a multi-terabyte model affordable. Memory is bounded by a collisionless (cuckoo-hash) embedding table, frequency filtering (an ID gets a vector only after a few sightings) and expiry of inactive IDs. Training state is snapshotted only **daily**; a lost day is relearned from the stream.

The join: a like can arrive minutes after the impression, so the joiner buffers impressions by `rec_token` for a window, emitting positives as actions arrive and a negative when the window closes empty: windowed stream processing with late events.

Deep dive 3 — instant start is moved before the swipe. DNS, TLS, the manifest fetch, the first segment and decoder start-up cannot fit in 200 ms once the swipe has happened.

Technique	Effect
Preload manifest + first 1–2 s of the next 2–3 videos; pre-decode the first frame of N+1	The swipe shows a frame already in memory
1–2 s segments (RFC 8216 calls 10 s "typical" for HLS)	A small first download; adaptive bitrate can step up quickly
One warm HTTP/2 or HTTP/3 connection to the edge	No new handshake per video
Preload less on cellular data or low battery; cancel when the viewer swipes fast	Limits wasted bytes (10–20% of egress if left unchecked)
Edge request coalescing for fresh viral videos	One origin fetch per region for a hot object (CDN)

Deep dive 4 — cold start by staged exposure (explore/exploit: spend a little traffic learning about unknowns). A new video gets a few hundred impressions via the explore slot, targeted by content embedding; its completion, rewatch, share and early-skip rates are compared with **similar videos** (length, topic), not raw likes; beating that cohort moves it to a pool 10× larger, and so on, with a second moderation check before very large pools. Follower count is not a direct factor (TikTok), so unknown creators can go viral. New users start from an optional interest picker plus regional popular videos until real-time features take over after a few dozen swipes.

7. The failure walk

Failure	Response
Ranker slow or down	150 ms deadline → light model → cached last pool → regional trending (Graceful Degradation); the 3–4 buffered videos hide short outages
Feature store down	Rank with defaults plus the features from the last response
Bad model push	Parameters ship as a canary with guardrails (watch time per session, skip rate); auto-rollback to the last good snapshot
Event pipeline lag	The feed adapts more slowly; shed progress pings before finish/like/follow (Backpressure)
Harmful video goes viral	`status=removed` is checked on every response from a fast cache, and the CDN URLs are purged; staged exposure limits its reach before the second check

8. Trade-offs

Decision	Chosen	Alternative	Why
Feed source	Interest graph	Follow graph	Discovery of unknown creators; follow feed remains a tab (news feed)
When to rank	Request time over precomputed pools	Fully precomputed feed	Precomputed feeds go stale within a session
Model freshness	Minute-level sparse sync	Daily retrain	Trends and interests move in hours
Training durability	Daily snapshot	Frequent checkpoints	A lost day is relearned from the stream
Start-up	Preload 2–3 videos	Fetch on swipe	Meets 200 ms at the cost of some wasted egress

9. The follow-up questions

- **"Filter bubbles?"** Exploration and diversity slots in re-ranking, per-session caps on one topic, and a long-term metric (7-day return rate) next to watch time.
- **"How do you ship a new model?"** Offline replay on logged data, then an A/B test (a random share of users gets the new model) with guardrails; the online result decides.
- **"Re-uploads of someone else's video?"** Perceptual fingerprints at upload plus near-duplicate detection; serve one copy and credit the original.
- **"The client?"** Module 18 App 6: player pool, preload queue, offline event buffer.

> **Interview one-liner:** "Reuse the video pipeline and CDN. Run a two-tower ANN retrieval, multi-task ranking and diversity re-ranking per request. Let streamed events update real-time features in seconds and push only touched embeddings to serving every few minutes. Preload the next videos so a swipe never waits, and let new videos earn reach through staged exposure."

The price: ~140M candidate scores/s, a 3.5M/s event firehose, ~800 PB/year of mostly cold video, and wasted preload bytes.

Sources

- TikTok Newsroom, "How TikTok recommends videos #ForYou" (18 June 2020): https://newsroom.tiktok.com/en-us/how-tiktok-recommends-videos-for-you
- Liu et al. (ByteDance), "Monolith: Real Time Recommendation System With Collisionless Embedding Table", ORSUM @ ACM RecSys 2022: https://arxiv.org/abs/2209.07663
- Covington, Adams, Sargin, "Deep Neural Networks for YouTube Recommendations", ACM RecSys 2016: https://research.google/pubs/deep-neural-networks-for-youtube-recommendations/
- R. Pantos, W. May, RFC 8216 "HTTP Live Streaming" (2017): https://www.rfc-editor.org/rfc/rfc8216
- B. H. Bloom, "Space/Time Trade-offs in Hash Coding with Allowable Errors", CACM 13(7), 1970: https://dl.acm.org/doi/10.1145/362686.362692

Case 2 — Food Delivery (Uber Eats / DoorDash style)

The prompt. Eaters browse nearby restaurants, order and pay. The restaurant accepts and cooks, a courier picks the food up and delivers it, and the eater tracks the order live.

What is new here. Location streams, cell sharding, atomic offers with a TTL, ETAs and surge are Ride-Sharing Dispatch; discovery is the Proximity Service; payments follow How Money Moves. Specific to food: (1) **the start time is a prediction**: the courier should arrive when the unseen *food* is ready; (2) **a courier carries several orders**, so assignment is route planning; (3) **the restaurant is a third, unreliable party** with its own states; (4) **demand comes in waves** (meals, bad weather, when couriers also drop).

1. Requirements and numbers

Non-functional: no lost orders and no double charges; the checkout ETA is a range the platform actually meets; tracking refreshes every few seconds; a problem in one city stays in that city.

Quantity	Assumption → result
Orders	3M/day, 30% in a 2-h dinner peak → **~125 orders/s peak**
Browse	30M sessions × 20 calls = 600M/day, 30% in the 2-h peak → **~25K req/s**, ~200× the order writes
Courier GPS	300K online, every 5 s → **60K writes/s**, worthless after seconds
Orders in flight	Little's law: 125/s × 2,400 s ≈ **300K active orders**
Menus	500K stores × 150 items × 1 KB ≈ **75 GB**, which fits in cache

What the numbers say: the volume is modest. The difficulty is **prediction and coordination** among three parties.

2. The five-questions pass

Question	Answer
Flow	Checkout is synchronous until the order is placed; after that, events (`OrderPlaced → Accepted → CourierAssigned → PickedUp → Delivered`)
Storage	Orders are durable, relational, sharded by city, with an append-only event history; courier positions live only in memory
Speed	Discovery and menus come from cache and a search index; after checkout only tracking freshness matters
Failure	Positions may be lost; orders and payments may not; a silent restaurant is a *normal* case
Growth	City and zone are the shard key and the bulkhead

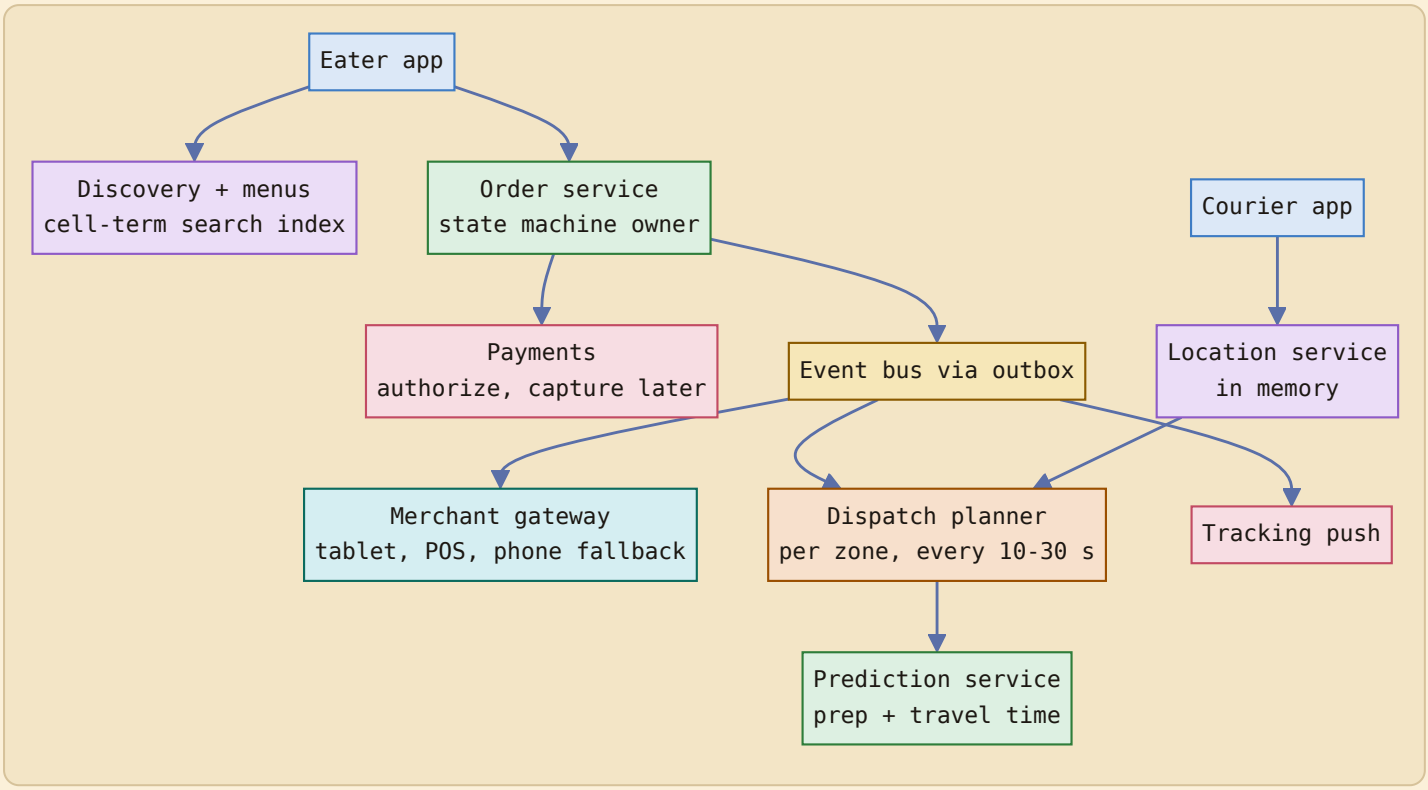
3. API

Caller	Endpoint	Note
Eater	`GET /v1/stores?lat=&lng=`	Only stores that deliver here, open now, with an ETA range
Eater	`POST /v1/checkout {cart_id, address_id, payment_method_id, idempotency_key}`	Returns `order_id` and an ETA range
Eater	`WS /v1/orders/{id}/track`	State + courier position
Restaurant	`POST /v1/merchant/orders/{id}/accept {prep_minutes}` / `reject` / `ready`	Tablet or POS (the restaurant's till system)
Restaurant	`PATCH /v1/merchant/items/{id} {available}`, `POST /v1/merchant/pause`	Mark an item sold out, or "too busy, pause orders"
Courier	`POST /v1/courier/location`, `POST /v1/courier/offers/{id}/accept`, `POST /v1/courier/tasks/{id}/status`	Accept is idempotent per offer

4. Data model

Table	Key → contents
`stores`	`store_id` → location, **delivery area as a set of H3 cells**, hours, status (open/paused)
`menus`	versioned document per store; `item_availability` is a separate small overlay, so marking an item sold out does not invalidate the whole menu
`orders`	`order_id` (sharded by city) → item **snapshot** with prices at checkout, state, `version`, quoted ETA range, payment ref, route ID
`order_events`	`(order_id, seq)` → append-only audit trail and outbox source
`courier_state` (memory)	`courier_id` → position, status, current route, remaining capacity

5. The architecture, annotated



1. **Discovery:** the address maps to an H3 cell (Uber's hexagonal grid; resolution 9 averages ~0.1 km²); stores are indexed by the cells they deliver to, so "who delivers here" is an exact lookup, ranked partly by predicted delivery time.
2. **Order service:** validates the cart against the menu version, **authorizes** the card, writes `PLACED` plus an outbox row in one transaction.
3. **Merchant gateway** pushes the order and runs the silent-store fallback (§7); the per-zone **dispatch planner** uses predictions (deep dives 1–2) and ride-sharing's atomic claim with TTL.
4. **Tracking** pushes state and courier position only while the screen is open (1–3 watchers per courier).

6. Deep dives

Deep dive 1 — dispatch at the food-ready time. Dispatch at order time leaves the courier waiting at the counter. Uber Eats engineers described an early fixed rule ("25 min prep + 5 min trip → dispatch 20

min after the order") that ignored order size and the actual courier. The fix: plan for `courier_arrival_at_store ≈ predicted_ready_time`.

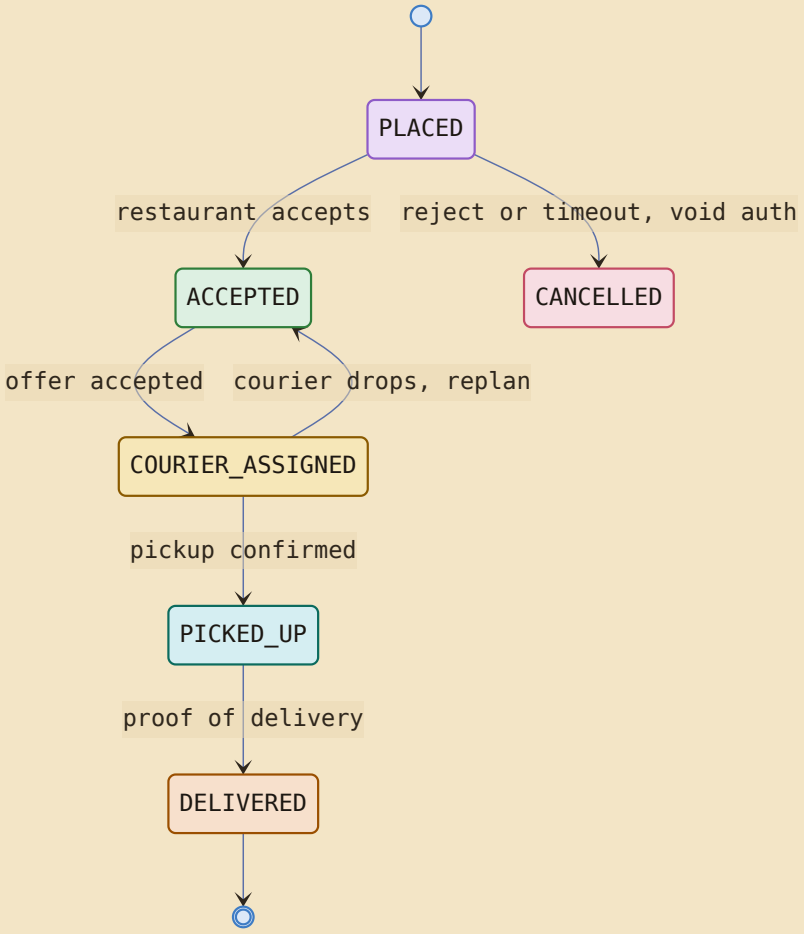
Piece	How
Prep-time model	Uber (InfoQ, 2019): gradient-boosted trees (XGBoost) on feature-store-style *historical* features (store's average prep time over the past week), *near-real-time* features (average over the last 10 min, which captures a slammed kitchen) and *real-time* features (time, day, order size), plus menu-item embeddings
Ground truth	"Ready" is inferred from the restaurant's ready tap (often late) and courier departures: noisy labels, retrained continuously
Travel time	Routing plus learned correction, as in the ride-sharing ETA deep dive, per vehicle type
Dispatch moment	When `predicted_ready − now ≤ best travel time + buffer`; the buffer grows with prediction uncertainty, because cold food costs more than a few minutes of courier waiting
Order release	For fast kitchens, the reverse: hold the order back from the kitchen until a courier is near (DoorDash has written about tuning this)

Deep dive 2 — batching orders into routes. The unit of assignment is a **route**: an ordered list of pickups and drop-offs.

1. **Window:** every 10–30 s per zone (a few H3 resolution-7 cells, ~5 km² each), take soon-dispatchable orders and couriers who are idle or a few minutes from finishing a drop-off.
2. **Candidates:** insert each order, or a pair from nearby stores, into each courier's route at every feasible position. Hard rules: pickup before drop-off, promise windows kept, a cap on minutes added to any order already in the bag.
3. **Score:** courier minutes + lateness against the quote + courier waiting + food time in the bag, using predicted means *and* variances.
4. **Solve:** pick candidates so each order and courier is used at most once. DoorDash (2020) solves this as a **mixed-integer program** (yes/no decision variables), choosing the commercial Gurobi solver after benchmarking open-source CBC; its 2021 post on its dispatch system, DeepRed, describes the split: ML predicts how each order would unfold, an optimiser decides.
5. **Offer:** atomic claim with TTL; a declined order returns to the next window.

Zones keep each solve to about a second; a few seconds of deliberate waiting beat greedy "nearest courier now" on total cost.

Deep dive 3 — the order state machine across three parties.



- **One owner, conditional transitions:** the order service applies an event only if legal from the current `state` and `version`, so duplicate or late events cannot move an order backwards.
- **Saga with the pivot at pickup:** before pickup every step can be undone (void the authorization, release the courier, notify the kitchen); after pickup, recovery only goes forward (redeliver, refund a missing item).
- **Authorize at checkout, capture at delivery** for the final amount (substitutions, tip changes). Cancellations become voids, not refunds.
- **Durable timers** ("no accept in 3 min → phone the store; 7 min → cancel and void") live in a workflow engine (DoorDash described adding Cadence as a fallback for delivery creation) or a job scheduler, never in one process's memory.

Honest ETAs. Quote a range from prep, wait and travel distributions, re-estimate on each event, push only changes over a minute, and track "delivered inside the quoted range" per city.

7. The failure walk

Failure	Response
Restaurant tablet silent	Durable timer → automated call → auto-cancel + void; repeat offenders are auto-paused so new orders stop
Kitchen slammed	The 10-minute prep features raise ETAs and lower the store's rank; the store can self-pause or is throttled to a maximum number of open orders
Zone planner crashes	A standby takes the zone lease and rebuilds from the orders DB and the next courier pings; assigned routes are durable, so there is no double assignment
Payment provider slow	Idempotency key and a reconciler that asks the provider by key (Hardening a Payment Flow); browsing continues
Storm: demand ×2, couriers −30%	Zone fees and courier incentives up, quoted ranges wider, delivery radius shrunk, then per-zone rate limiting at checkout rather than accepting orders that would arrive cold

8. Trade-offs

Decision	Chosen	Alternative	Why
When to dispatch	At predicted ready time	At order time	Courier idle time is the biggest waste
Assignment	Batched optimisation per zone	Greedy nearest courier	Lower total cost; a few seconds of delay
Orders per trip	2–3 with a freshness cap	One	More deliveries per courier-hour at peak
Consistency	Saga + durable timers	2PC	Restaurants and card networks cannot join a transaction
Payment	Authorize, then capture at delivery	Charge at checkout	Cheap cancellations, final amount can change

9. The follow-up questions

- **"Grocery delivery?"** Add a picking step with substitutions the eater approves, inventory data that is often wrong, and capture after picking.
- **"Scheduled orders?"** A durable timer injects the order at `target time − predicted lead time`, after which it runs the normal flow.
- **"Menu updates from 500K stores?"** A new menu version, then cache-aside invalidation and CDC into search; availability goes through the fast overlay.

> **Interview one-liner:** "Reuse ride-sharing's location stream and atomic offers, but dispatch to the predicted food-ready time. Every few seconds per zone, solve a small optimisation that can put two or three orders on one route. Run each order as a saga owned by one service, with durable timers for silent restaurants and pickup as the pivot. Authorize at checkout, capture at delivery, and quote ETA ranges you keep."

The price: per-city models and solvers to retrain and monitor, plus human fallbacks (phone calls, support, refunds).

Sources

- Zi Wang (Uber), "Predicting Time to Cook, Arrive, and Deliver at Uber Eats", InfoQ, 20 Nov 2019: https://www.infoq.com/articles/uber-eats-time-predictions
- Holly Jin (DoorDash), "Next-Generation Optimization for Dasher Dispatch at DoorDash", 28 Feb 2020: https://careersatdoordash.com/blog/next-generation-optimization-for-dasher-dispatch-at-doordash/
- Alex Weinstein (DoorDash), "Using ML and Optimization to Solve DoorDash's Dispatch Problem", 17 Aug 2021: https://careersatdoordash.com/blog/using-ml-and-optimization-to-solve-doordashs-dispatch-

problem/

- Alan Lin (DoorDash), "Building Reliable Workflows: Cadence as a Fallback for Event-Driven Processing", 14 Aug 2020: https://careersatdoordash.com/blog/building-reliable-workflows-cadence-as-a-fallback-for-event-driven-processing/
- H3 documentation, cell statistics by resolution (res 7 ≈ 5.16 km², res 9 ≈ 0.105 km²): https://h3geo.org/docs/core-library/restable/
- J. D. C. Little, "A Proof for the Queuing Formula L = λW", Operations Research 9(3), 1961: https://doi.org/10.1287/opre.9.3.383

Case 3 — E-Commerce Platform and Peak-Day (Prime Day) Readiness

The prompt. Build an online store (Amazon, Flipkart, BigBasket): catalogue and search, product pages, cart, checkout, orders, returns, and third-party sellers. Then the favourite follow-up: *"Prime Day is in four months, traffic will be 10× normal with spikes at every deal launch. How do you get ready?"*

What is new here. The parts have lessons: catalog caching, the order saga, holds and the waiting room, search. New: **isolating** the paths that must never go down (browse, cart) from the one that must never be wrong (checkout); **hot deal stock**; and **peak-day readiness**, an operations programme rather than a diagram.

1. Requirements and numbers

Non-functional: browse and cart at 99.99% availability (under ~53 min of downtime a year), p99 page latency under ~1 s, staleness acceptable; checkout never wrong (no money without an order, no duplicate orders, no overselling of limited deals); same behaviour at 10× load.

Quantity	Assumption → result
Normal page views	2B/day ≈ 23K/s, ~50K/s daily peak; ~30 backend calls per page → **~1.5M backend req/s**
Normal orders	5M/day ≈ 58/s average
Peak day	10× daily volume, plus ~5× spikes at deal launch → plan for **~500K page views/s (~15M backend req/s) and ~3,000 orders/s**
Catalogue	500M products × 10 KB = **5 TB**; images ≈ 1.2 PB on a CDN
Carts	100M × 2 KB = **200 GB**

Calibration from published figures: AWS reported Amazon's DynamoDB use peaking at **151M requests/s** and SQS at **166M messages/s** over the four-day Prime Day 2025, after **over 6,800** fault-injection experiments; for Prime Day 2026 (23–26 June) **192M requests/s**, **213M messages/s** and **over 44,000** experiments. Shopify peaked at **284M requests/min at its edge** during BFCM 2024 (Black Friday–Cyber Monday). Alibaba reported a peak of **544,000 orders/s** on Singles Day 2019.

What the numbers say: reads are the volume problem (caches); writes are only thousands per second but valuable and **contended**; plan for the deal-launch spike, not the average.

2. The five-questions pass

Question	Answer
Flow	Browse: synchronous over caches. Checkout: synchronous up to a **durable order acceptance**, then everything (capture, picking, emails, analytics) runs asynchronously from events
Storage	Catalogue in a document store → search index by CDC; price/offers strongly consistent with caches; inventory per `(sku, fulfilment centre)`; carts in a highly available key-value store; orders in a relational store sharded by customer
Speed	CDN and page shell at the edge; parallel page composition with per-widget deadlines
Failure	Every dependency is tagged **critical** (price, buy box, checkout) or **optional** (recommendations, reviews, ads)
Growth	Planned peaks mean capacity is set on the calendar, with reactive autoscaling only on top

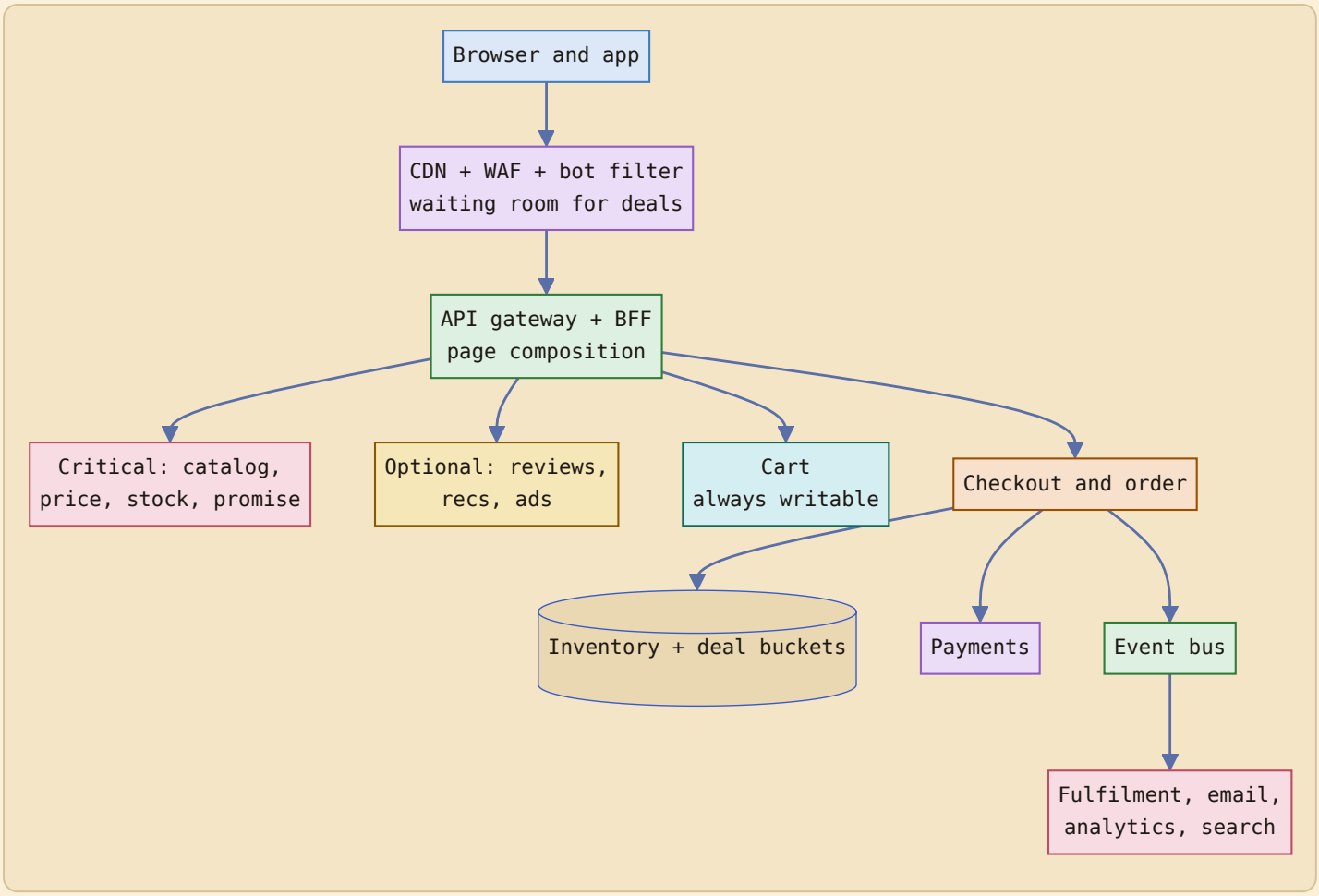
3. API

Endpoint	Note
`GET /v1/products/{id}`	Composed: static parts long-cached; price, stock badge and delivery promise short-TTL
`POST /v1/cart/items {sku, qty}`	Always accepted (deep dive 2)
`POST /v1/orders {cart_version, address_id, payment_method_id, idempotency_key}`	→ `order_id`, `ACCEPTED`; safe to retry (Idempotency)
`POST /v1/deals/{id}/claim`	Hot deals only: → claim token valid ~15 min (deep dive 4)

4. Data model

Table	Key → contents
`products`	product ID → attributes (vary by category), variants (**SKU** = stock-keeping unit, one sellable variant), images
`offers`	`(sku, seller_id)` → price, deal price and window, `version`
`inventory`	`(sku, fc_id)` → `on_hand, reserved` (FC = fulfilment centre, a warehouse)
`deal_buckets`	`(deal_id, bucket)` → `remaining` (deep dive 4)
`carts`	`customer_id` → lines `{sku, qty, price_seen}`, `version`
`orders`	sharded by `customer_id`; unique `(customer_id, idempotency_key)`; line snapshot with prices; state

5. The architecture, annotated



The **edge** (CDN) serves assets and a cacheable page shell, drops bots (WAF) and hosts the waiting room; the **BFF** composes critical and optional calls in parallel (deep dive 1); the **cart** accepts every write (deep dive 2); **checkout** runs dedupe → reserve with TTL → authorize → `ACCEPTED` + outbox as in the Module 07 capstone, and everything downstream catches up from the bus.

6. Deep dives

Deep dive 1 — a page is only as available as its critical path. Waiting for all 30 calls makes a page as slow as the slowest and as fragile as the weakest.

Mechanism	Setting
Critical vs optional	Critical: title, images, price, buy box, stock badge, delivery promise. Optional: everything else
Deadlines	Critical ~300 ms with one hedged retry (a duplicate request to another replica if the first is slow); optional ~150 ms, then render without the widget and fill it in client-side later (Timeout)
Bulkheads	Separate connection pools per dependency, so a slow recommender cannot use up pricing's threads
Circuit breakers	On optional dependencies; when open, render without them

Availability then depends on four or five services, not thirty.

Deep dive 2 — the always-writable cart (the motivating example of Amazon's Dynamo paper, SOSP 2007, with a 300 ms target at the 99.9th percentile). Any reachable replica accepts "add to cart"; conflicting versions merge later:

- **Merge, don't reject:** the union of items wins. A deleted item may reappear (harmless); a lost "add" is a lost sale (Quorum replication with vector clocks).
- **A cart is not a reservation**, or idle carts would lock up deal stock; `price_seen` only warns "price changed" at checkout.
- **Guest-cart merge at sign-in** is a union with the max quantity per SKU, made idempotent by `version`.

Deep dive 3 — the overselling budget. Ordinary items: reserve with a conditional update (`… WHERE on_hand − reserved ≥ :q`), falling back to the next fulfilment centre, and accept a tiny oversell risk: stock counts are already imperfect (damaged and misplaced units) and an apology plus cancellation is cheap. Limited deals: budget **zero** (deep dive 4).

Deep dive 4 — the lightning deal (hot stock). 10,000 units at 12:00 and hundreds of thousands of buyers in the first seconds all hit one row.

1. **Pre-split** the stock into, say, 50 `deal_buckets` of 200 units; a claim decrements a random non-empty bucket (`… SET remaining = remaining − 1 WHERE remaining > 0`), cutting per-row contention 50×: Module 17's bucketed counter and the distributed counter.
2. **Claim, then check out:** a successful decrement issues a **claim token** valid ~15 minutes, consumed by normal checkout; expired tokens return the unit to a bucket. Winning a unit (contended, tiny) is separated from paying (slow, uncontended).
3. **Waiting room** at the edge for the largest launches, admitting buyers at the claim service's measured rate (Module 17).
4. **Display is not truth:** "73% claimed" comes from a lagging stream; only the decrement decides. Per-account limits are enforced inside the claim transaction.

Deep dive 5 — peak-day readiness, a four-month programme. Answer with a timeline that has exit criteria, not "add servers".

When	Activity	Exit criterion
T−16 wk	**Forecast:** business orders/day and deal schedule → per-service peak req/s through the call graph (page views × calls per page), × 1.5–2 margin	Every owner signs a target
T−14 wk	**Critical journeys** (Shopify's term: browse, cart, checkout, payment, order creation) mapped to dependencies, including third parties (payment, tax, SMS) and their quotas	Owner + dependency map per journey
T−12→4 wk	**Production load tests** with tagged synthetic shoppers (browse → cart → checkout), ramped until something breaks. Shopify ran five scale tests for BFCM 2025 with its Genghis load generator, one reaching 146M req/min and 80K+ checkouts/min, the last 200M req/min	Each break fixed or documented with a mitigation
T−10→2 wk	**Fault injection and game days under load:** kill instances, inject dependency latency, fail an availability zone, exhaust pools. AWS reported 6,800+ experiments for Prime Day 2025 and 44,000+ for 2026, many run inside CI/CD	Each journey survives loss of one zone and of any optional dependency
T−6 wk	**Pre-provision** compute, raise database, queue and third-party limits, pre-split hot partitions and deal buckets, warm the CDN. Size for peak *with one zone lost* (**static stability**: nothing new needs to be launched during a failure)	Load test passes at target × margin
T−4 wk	**Degradation switches** as feature flags: recommendations off, personalisation off, static homepage, non-critical emails deferred; each tested in production	Runbook with switch order and load saved per switch
T−2 wk	**Change freeze** on critical journeys; dashboards on journey success rates (checkouts/min vs forecast, payment authorization rate), not CPU	War room and on-call staffed
Day 0	Incident commander with pre-agreed triggers ("checkout p99 > 2 s for 3 min → flip switches 1–2")	
T+1 wk	Review forecast vs actual and near misses	Feeds next year's forecast

Priority load shedding ties it together: the edge tags each request's priority and overloaded services reject low priorities first, early and cheaply (bots, optional widgets, deep browse, then cart and checkout last); fast rejection keeps accepted work fast (Amazon Builders' Library, "Using load shedding to avoid overload").

Why it matters (2018). CNBC reported, citing internal documents, that minutes into Prime Day 2018 Amazon ran short of server capacity, switched to a simplified fallback homepage, temporarily cut international traffic and added servers by hand; the trouble was traced to an internal compute and storage system many services shared. Call-graph forecasts, load tests and pre-provisioning target exactly this; the ready-made fallback page limited the damage.

7. The failure walk

Failure	Response
Recommender down	Breaker opens, pages render without it, switch removes it from the homepage
Search overloaded	Drop facet counts and personal re-ranking, serve cached results for top queries, cap depth
Unplanned hot SKU	Move units from the main row into new bucket rows in one transaction, then route claims there
Payment provider slow	Reservations hold for their TTL; unknown results go to `PAYMENT_PENDING` plus a reconciler that asks by key; a second provider if one exists
Zone lost	Static stability: survivors were already sized for it; databases fail over; checkouts retry with the same keys
Event bus lag	Orders are already durable; confirmation emails arrive late
Traffic beyond plan	Waiting room on deal pages, priority shedding, static homepage: slower and smaller, never down (Graceful Degradation)

8. Trade-offs

Decision	Chosen	Alternative	Why
Cart	Always writable, merged	Strongly consistent	A lost add is a lost sale
Reserve stock	At checkout / claim	At add-to-cart	Idle carts would lock stock
Oversell	Small budget for ordinary items, zero for deals	Strict everywhere	Strict locks cost latency for little benefit
Deal stock	Buckets + claim token	One counter row	Removes the single hot row
Peak capacity	Pre-provisioned on the calendar + autoscaling	Autoscaling alone	Deal spikes outrun instance start-up
Near peak	Change freeze	Normal release cadence	Most incidents start with a change

9. The follow-up questions

- **"Delivery promise ('Get it Tuesday')?"** Per centre with stock: pick-and-pack time + next carrier cut-off + transit to the postcode; precompute tables per `(fc, postcode region)` and take the best.
- **"Many sellers per product?"** Offers are `(sku, seller_id)` rows; the buy box picks one by price, speed and seller rating.
- **"Multi-region?"** Catalogue replicated everywhere; orders have a home region per customer; cells (independent stack copies, each serving a slice of customers) limit blast radius.
- **"Testing at 10× without a 10× copy?"** In production at low-traffic hours with tagged synthetic traffic and a test payment endpoint, ramped in steps, stopping at the first error-budget breach.

> **Interview one-liner:** "Separate a read path that must always answer (CDN, caches, parallel composition where only critical widgets can fail the page) from a write path that must never be wrong (an always-writable cart, then reserve → authorize → durable accept with an outbox). Split hot deal stock into buckets with claim tokens behind a waiting room. Prepare for peak day with a call-graph forecast, production load tests, fault-injection game days, capacity pre-provisioned for a lost zone, tested kill switches, a change freeze and priority load shedding."

The price: peak-only capacity, many caches and derived copies, and months of readiness work every year.

Sources

- DeCandia et al., "Dynamo: Amazon's Highly Available Key-value Store", SOSP 2007: https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf
- AWS News Blog, "AWS services scale to new heights for Prime Day 2025: key metrics and milestones": https://aws.amazon.com/blogs/aws/aws-services-scale-to-new-heights-for-prime-day-2025-key-metrics-and-milestones/
- AWS News Blog, "All the numbers: Amazon Prime Day 2026 powered by AWS" (6 Oct 2026): https://aws.amazon.com/blogs/aws/all-the-numbers-amazon-prime-day-2026-powered-by-aws/
- Shopify Engineering, "How we prepare Shopify for BFCM (2025)" (20 Nov 2025): https://shopify.engineering/bfcm-readiness-2025
- CNBC, Amazon internal documents on the Prime Day 2018 outage (19 July 2018): https://www.cnbc.com/2018/07/19/amazon-internal-documents-what-caused-prime-day-crash-company-scramble.html
- David Yanacek, "Using load shedding to avoid overload", Amazon Builders' Library: https://aws.amazon.com/builders-library/using-load-shedding-to-avoid-overload/
- TechNode, "Alibaba's Singles Day sales reach $26.4 billion in first 12 hours" (11 Nov 2019; 544,000 orders/s peak): https://technode.com/2019/11/11/alibabas-singles-day-sales-reach-26-4-billion-in-first-12-hours/

Case 4 — Online Education Platform (Coursera / Udemy style)

The prompt (in short). Instructors publish **courses** of videos, readings, quizzes and programming assignments. Learners enroll (free or paid), watch on any device, take graded quizzes, submit auto-graded code, track progress against deadlines, and earn a **certificate** an employer can verify.

What makes it hard. Video, sandboxed execution and queues are solved elsewhere; the new problems are in the seams: (1) **progress** is almost all the writes, and no retry or second device may move a learner backwards; (2) **grades carry weight**: keys never reach the browser, timed exams need an honest clock, every score is auditable; (3) **load follows deadlines** (the last hour before Sunday 23:59); (4) **courses change mid-run** without silently changing earned grades.

1. Requirements and numbers

Functional: catalog; enrollment and **entitlements** (the record that a learner may access paid content, via purchase, subscription or employer); video with resume; quizzes, timed exams, auto-graded code, peer review; progress and gradebook; deadlines and reminders; verifiable certificates; versioned instructor authoring.

Non-functional: progress never lost or reversed; grade writes durable and auditable; code feedback p95 < 2 min at deadline peak; playback and catalog stay up when grading is degraded.

Numbers (assumptions; Coursera reported 191M registered learners in September 2025):

Quantity	Arithmetic	Result
Daily active learners	assumption	10M
Video egress	10M × 40 min × ~11 MB/min (1.5 Mbps)	~4.4 PB/day, all on the CDN
Peak concurrent viewers	15% of DAU	1.5M
Progress heartbeats	1.5M players / one per 30 s	**50K writes/s**
Quiz submits, deadline hour	~10% of a week's 140M in one hour	~4K/s
Code grading	1M submissions/day × 60 vCPU-s	~700 vCPUs average, **~7,000** (10×) in the deadline hour

What the numbers say: video is big but bought from a CDN; the design load is **50K tiny progress writes/s** and a **weekly 10× grading spike**; grade data is small but must be exactly right.

2. The five-questions pass

- **Flow:** playback reads from the CDN; progress and quiz submits are fast synchronous writes; code grading is asynchronous; a "course passed" event triggers certificates.
- **Storage:** relational DB for enrollments, attempts and grades (transactions); wide-column store for progress; object storage for video and submissions.
- **Speed:** "course home" (outline + my progress + my deadlines) is the hot read: cached outline per immutable version + one progress partition read.
- **Failure:** a lost heartbeat is fine; a lost completion or grade is not; grading retries must be idempotent.
- **Growth:** grading load grows with cohort-based courses whose deadlines align.

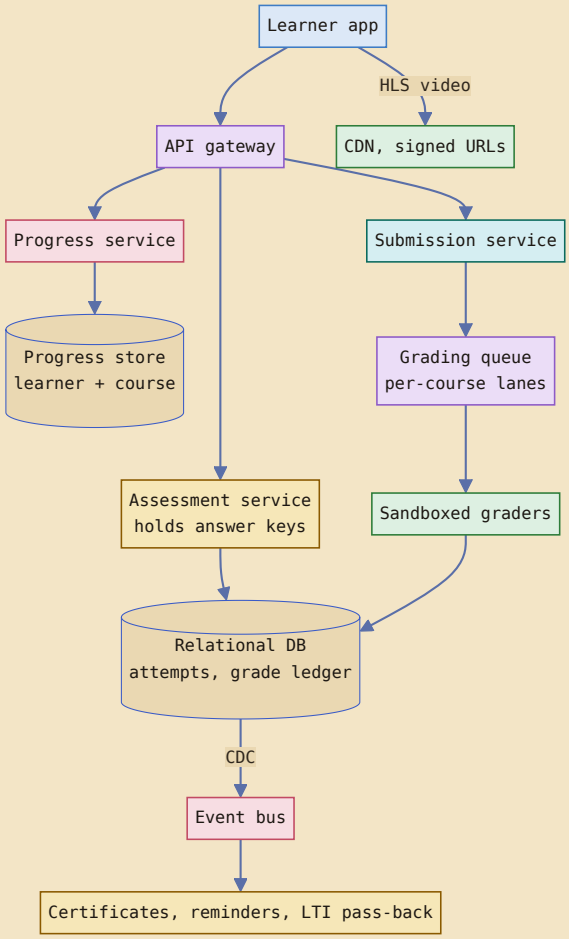
3. API

Method and path	Purpose
`POST /enrollments {course_id, mode}`	Enroll; pins the learner to the current course version
`GET /items/{id}/playback`	Entitlement check, then a short-lived **signed URL** (expiry + signature the CDN verifies without calling us)
`PUT /progress/{course}/{item} {position_s, watched_bitmap, client_seq}`	Heartbeat; merged, never overwritten
`POST /quizzes/{item}/attempts` → `{attempt_id, questions, expires_at}`	Server picks questions and starts the clock
`POST /attempts/{id}/submit` + `Idempotency-Key`	Graded server-side, returns score
`POST /assignments/{item}/submissions` → `202 {submission_id}`	Async code grading
`GET /certificates/{id}` (public)	Signed credential and revocation status

4. Data model

Table	Key	Notes
`course_versions`	`(course_id, version)`	Published versions are **immutable**; items referenced as `(item_id, item_version)`
`enrollments`	`(learner_id, course_id)`	`course_version, session_id` (a **session** is one run of a course with its own deadline calendar), `mode`
`progress`	partition `(learner_id, course_id)`, row per item	`max_position_s, watched_bitmap` (bit *i* = 10-second slice *i* watched), `resume_pos, client_seq, completed_at`
`question_bank`	`(item_id, item_version, question_id)`	Holds answer keys; readable only by the assessment service
`attempts`	`attempt_id`	`item_version, seed, started_at, expires_at, answers, score`
`grade_ledger`	append-only	`(learner, course, item, score, source_ref, reason, at)`; the gradebook is derived from it (Event Sourcing)
`certificates`	`cert_id`, unique `(learner, course, version)`	`signature, revoked_at`

5. The architecture, annotated



1. **Video** is the CDN + Video Platform pipeline plus an entitlement check before signing the URL; **progress** merges heartbeats (deep dive 1); **assessment** sends questions without answers and grades on the server (deep dive 2).
2. **Code submissions** go to object storage and a queue; workers run the instructor's grader image in the Online Judge sandbox (Coursera's open-source grader tooling: Docker images with 1, 2 or 4 vCPUs, up to 16 GB memory, 300–3,600 s timeout).
3. Grade changes leave by CDC to certificates, reminders, analytics and university pass-back.

6. Deep dives

Deep dive 1 — progress that never goes backwards. Overwriting (`SET position = ?`) lets a delayed retry or the second device replace newer data with older. Instead every field merges monotonically:

Field	Merge rule	Why
`max_position_s`	`max(old, new)`	Furthest point reached
`watched_bitmap`	`old OR new`	Never loses a watched slice; offline weeks merge cleanly
`completed_at`	first non-null wins	Completion can't be undone
`resume_pos`	last writer by `client_seq`	"Resume" should follow your latest device

These merges are **commutative and idempotent** (order and repeats don't matter), the property of a CRDT (conflict-free replicated data type), so duplicates, reordering and multi-region writes are harmless. "Complete" means ≥ 80% of bitmap bits set, computed server-side, so skipping to the end doesn't count. At 50K/s the server may buffer each partition for a second (Write-Behind), losing at most a second of position on a crash; completions are written through synchronously.

Deep dive 2 — assessments you can trust.

- **Keys stay on the server**; shipping them to the browser for instant feedback leaks them within hours.
- **Pools and seeds:** each attempt draws N questions from a larger pool, shuffled by a seed stored on the attempt, so papers differ and a regrade reproduces exactly what the learner saw.
- **Server clock:** `expires_at` is set at attempt start; submits after it plus ~30 s network grace are rejected; the client timer is display only.
- **Idempotent submit:** `Idempotency-Key` plus `status = in_progress` check means a double-click never burns a second attempt (Idempotency).
- **Regrades:** fixing a wrong key creates a new item version and a regrade job that **appends** ledger rows with `reason = key_fix`; history stays for disputes.
- **Peer review:** after the deadline a batch job assigns each essay to 3–4 peers; the score is the median, robust to one harsh or lazy reviewer.

Deep dive 3 — the Sunday-night spike. Deadlines are known weeks ahead, so graders are **pre-scaled from the deadline calendar** and topped up by queue-depth autoscaling. **The deadline applies to upload time, not grading time** ("received 23:58, grading…"). Per-course lanes stop one slow grader starving the rest (Bulkhead); a submission that crashes graders three times goes to a DLQ.

Deep dive 4 — certificates and university integration. A certificate is a JSON credential signed with the platform's key, verifiable offline, in the open **Open Badges 3.0** format (1EdTech), which models badges as W3C **Verifiable Credentials**. Issuance is idempotent on `(learner, course, version)`; revocation sets `revoked_at`, checked on verification. Universities' LMSs (e.g. Canvas) connect via **LTI 1.3**: launches carry a signed JWT through an OpenID Connect flow and grades return through Assignment and Grade Services, so one learner maps to many external identities and pass-back is one more ledger consumer.

7. The failure walk

1. **CDN degraded:** player fails over to an alternate CDN and drops bitrate (Graceful Degradation).
2. **Progress store slow:** clients buffer and retry; idempotent merges mean nothing double-counts.
3. **Grading backlog at deadline:** submissions still accepted and timestamped; only feedback is late.
4. **Buggy grader gives everyone zero:** alert on per-item score distribution; fix, then regrade by appending ledger rows; certificate eligibility is re-evaluated from the same events.
5. **Region outage:** catalog, playback and progress run active-active (CRDT merges are safe across regions); the grade ledger has one writer region, so quiz submits pause for minutes during failover rather than forking grade history (Multi-Region).

8. Trade-offs

Decision	Chosen	Alternative	Why
Progress	Monotonic merge in a wide-column store	Overwrite in the grades DB	Multi-device + retries; keep 50K/s off the grades DB
Grades	Append-only ledger	Mutable column	Regrades and disputes need history
Course edits	Immutable versions, learners pinned	Edit in place	Edits must not change earned grades
Quiz grading	Server-side	Client-side	Keys leak
Certificates	Signed Open Badges 3.0 credential	PDF + lookup page	Verifiable without us, for decades
Live classes	Integrate a meeting provider	Own WebRTC stack	Rare, bursty, not the core product

9. The follow-up questions

- **"Stop answer sharing?"** Large pools, per-attempt seeds, attempt limits with cool-downs, similarity fingerprinting on code and text, proctoring for high stakes; none is perfect alone.
- **"'Continue learning' row?"** A per-learner read model (next incomplete item per enrollment) updated on progress events, read with one key lookup (CQRS).
- **"Deadline reminders across timezones?"** Deadlines stored in UTC; a scheduler scans hourly buckets and enqueues notifications (Job Scheduler, Notification capstone).
- **"An AI tutor?"** RAG over the course version's transcripts and readings, never over the question bank; rate-limited and logged for integrity review.

> **Interview one-liner:** "Video is a CDN with entitlement-signed URLs; progress is a monotonic CRDT-style merge so retries and devices never move anyone backwards; quizzes are graded server-side against immutable item versions with server clocks; code goes through pre-scaled, per-course grading lanes with the deadline stamped at upload; grades are an append-only ledger whose events drive certificates and pass-back."

The price: two stores (relational + wide-column), a grading fleet idle most of the week, versioning for every item.

Sources

- Coursera, Q3 2025 results (191M registered learners): https://www.businesswire.com/news/home/20251023606409/en/Coursera-Reports-Third-Quarter-2025-Financial-Results
- Coursera autograder tooling (Docker graders, CPU/memory/timeout limits): https://github.com/coursera/coursera_autograder
- 1EdTech, LTI Core Specification 1.3: https://www.imsglobal.org/spec/lti/v1p3
- 1EdTech, Open Badges 3.0 as W3C Verifiable Credentials: https://www.1edtech.org/1edtech-article/new-open-badges-30-standard-provides-enhanced-security-and-mobility/411060
- Shapiro, M. et al., "Conflict-free Replicated Data Types", SSS 2011: https://inria.hal.science/inria-00609399

Case 5 — Spotify Wrapped: a Yearly Batch Personalisation Pipeline

The prompt (in short). Once a year every active listener gets a personalised story (top artists, top songs, minutes listened, genres, podcasts) with shareable cards. Compute it for hundreds of millions of people and serve it on launch day.

What makes it hard. (1) One giant batch over a year of history with an immovable public deadline. (2) Serving is idle 364 days, then tens of millions arrive in hours: Spotify's load-testing post names the first three to four hours as heaviest, and Spotify reported over 200M engaged users (viewed at least one story) in the first 24 hours of Wrapped 2025. (3) Wrong numbers get screenshotted by millions.

1. Requirements and numbers

Functional: per-user yearly "data stories", localized; share images; global launch; fix one story late without recomputing all. **Non-functional:** compute done days early with room for a rerun; launch p99 of a few hundred ms; nobody sees another user's data.

Quantity	Arithmetic (assumptions)	Result
Input	300M users × 30 plays/day × 365 × ~100 B	~3.3T events, ~330 TB
Serving data	300M × 20 stories × ~1 KB	~6 TB, one row per user
Launch load	60M users × 10 calls / 14,400 s, ×2–3 peak	~42K/s avg, plan ~100K/s
Share images	2019 actual: pre-rendered	~535M images in a 42-hour job

Spotify's 2022 post expected "tens of thousands requests per second" per region, consistent with the estimate.

2. The five-questions pass

- **Flow:** batch weeks ahead → bulk load → read-only API on launch day.
- **Storage:** immutable event lake in; row per user, column family per story out.
- **Speed:** serving is one key lookup plus cached metadata and translations.
- **Failure:** batch re-runnable per story; serving degrades story-by-story.
- **Growth:** input with users; launch peak with marketing success.

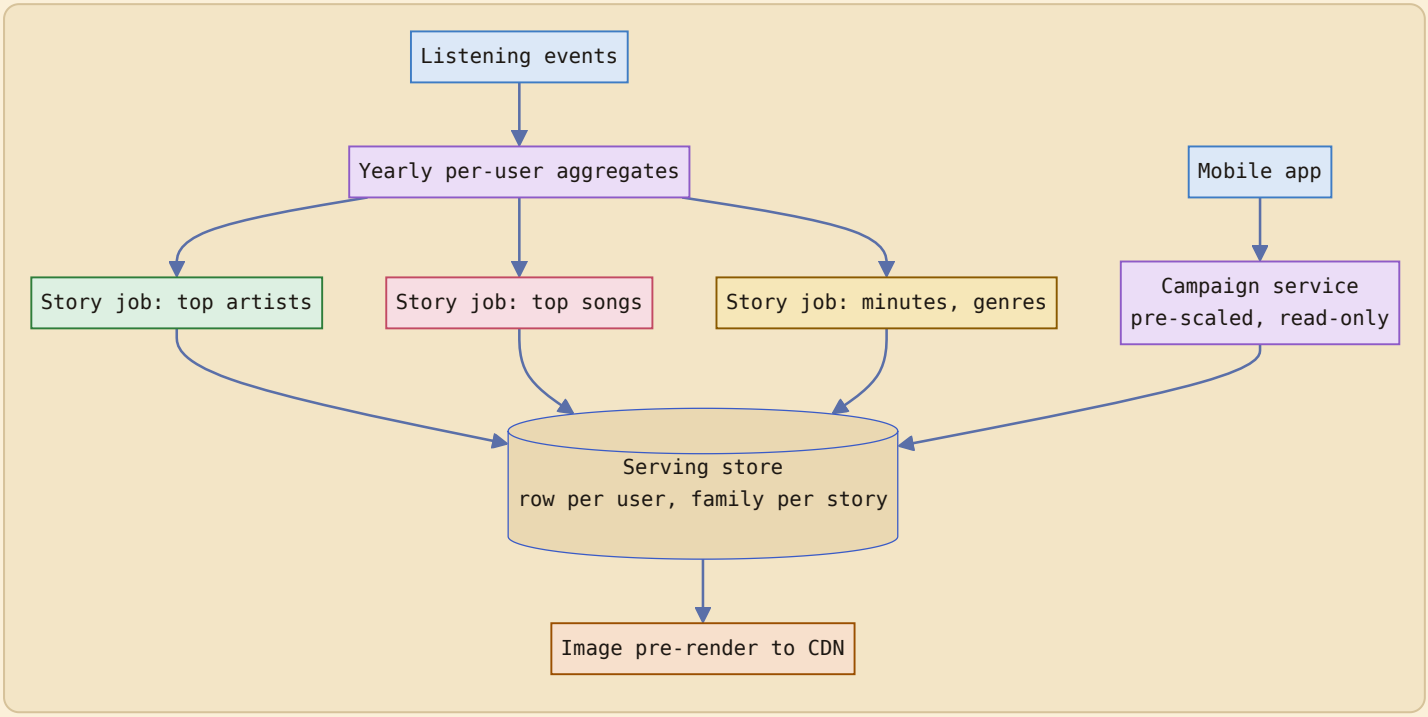
3. API

`GET /wrapped/{year}` returns the caller's stories (no `user_id` parameter, so one user cannot request another's); `GET /wrapped/{year}/cards/{story_id}` returns a CDN URL for the share image.

4. Data model

Input: daily-partitioned events `(user_id, track_id, ms_played, ts)`. Intermediate: per-user, per-year aggregates. Serving: **one row per user, one column family per story** (a column family is a named group of columns stored together) in Bigtable, as Spotify describes; each story job writes only its own family.

5. The architecture, annotated



1. **Aggregate once, reuse.** Grouping by user needs a **shuffle** (moving records so equal keys meet on one machine), the costly part of any MapReduce-style job. Spotify's 2018 pipeline was one of the largest Dataflow jobs on Google Cloud and hit shuffle limits; for 2019's decade view it combined per-year aggregates already in Bigtable, processing ~5× the data for 25% less spend.
2. **One job per story**, so a late rule change reruns one job and jobs run in parallel.
3. **Row-per-user output** is pre-grouped, so serving is one read.

4. **Campaign service** (Spotify's backend that returns personalised, localised data) joins the row with cached metadata and translations.

6. Deep dives

Deep dive 1 — a safe once-a-year batch. Story jobs overwrite their own family, so reruns are idempotent; outputs are tagged by run ID and serving points at a run, so switching is a pointer flip (Idempotency). Test on a fixed sub-sample and sanity-check with scripts reading the store directly (Spotify wrote a small Bigtable reader). Before flipping, compare distributions with last year (median minutes, share with a podcast story); a sudden shift blocks release. A listening cut-off well before launch leaves time for one emergency rerun.

Deep dive 2 — launch as a planned stampede.

Lever	What it does
Pre-scale to forecast	Autoscaling reacts in minutes; launch arrives in seconds (Auto-Scaling)
Load-test the real path	Spotify drove HTTP/gRPC load with Moshpit (its Backstage load-testing plugin) against the campaign service and its dependencies (metadata, translation, image generation, personalisation) in US, EU and Asia; requests with employee flags skipped caches and had to be fixed to mirror production. In 2019, tests three days out forced the forecast up
Regional waves behind a flag	Turns one global spike into several, with an instant off switch (Feature Flags)
Cache shared data	Metadata and translations are the same for millions (Cache Stampede Prevention)

Deep dive 3 — share images. Pre-rendering everything is predictable but wastes work on unshared cards; rendering on demand puts CPU-heavy work on the launch path. Spotify did both in 2019: a real-time renderer (HTML templates via CEF, the Chromium Embedded Framework) plus a precautionary batch of ~535M images finished two hours before launch. Default: pre-render for the early wave, render-and-cache the tail.

7. The failure walk

1. **Wrong numbers in one story:** caught by the distribution gate; rerun that job, flip its pointer.
2. **Batch late:** launch without that story (hidden by flag); the pointer never moves to a partial run.
3. **Metadata slow at launch:** serve stale from cache; show the story without an image (Graceful Degradation).
4. **Renderer saturated:** fall back to pre-rendered or generic cards.

8. Trade-offs

Decision	Chosen	Alternative	Why
Compute	Yearly batch, saved intermediates	Streaming counters all year	Rules change late; batch over immutable events can be recomputed, a year of stream state cannot
Job split	One job per story	Monolith	Isolated reruns, parallelism
Layout	Row per user	Table per story	One read at launch
Images	Pre-render early wave + on-demand	All one way	Bounded risk, bounded waste

9. The follow-up questions

- **"Top-K per user efficiently?"** Sum `(user, artist) → ms_played` with a combiner (partial sums before the shuffle), then a size-K heap per user in the reducer.

- **"Privacy?"** Token-scoped reads only, share images contain only what the user shares, serving data deleted after the campaign.

> **Interview one-liner:** "Batch-then-serve: aggregate a year of immutable events once, compute each story in its own re-runnable job into a row-per-user store, pre-render cards for the early wave, and serve read-only lookups from a pre-scaled, load-tested service launched in flagged regional waves."

The price: a big one-off compute bill and a fleet sized for a few hours.

Sources

- Spotify Engineering, "Spotify Unwrapped: How we brought you a decade of data" (2020): https://engineering.atspotify.com/2020/2/spotify-unwrapped-how-we-brought-you-a-decade-of-data
- Spotify Engineering, "Spotify Unwrapped 2019: How We Built an In-App Experience Just for You" (2020): https://engineering.atspotify.com/2020/9/spotify-unwrapped-2019-how-we-built-an-in-app-experience-just-for-you
- Spotify Engineering, "Load Testing for 2022 Wrapped" (2023): https://engineering.atspotify.com/2023/3/load-testing-for-2022-wrapped
- TechCrunch, "Spotify says Wrapped 2025 is its biggest yet, with 200M users in its first day" (4 Dec 2025): https://techcrunch.com/2025/12/04/spotify-says-wrapped-2025-is-its-biggest-yet-with-200m-users-in-its-first-day

---

Case 6 — A Twilio-Style Communications API Platform

The prompt (in short). A developer platform where `POST /Messages {to, from, body}` sends an SMS through hundreds of carriers worldwide, and the platform calls the customer back with **webhooks** for delivery status and incoming messages.

What makes it hard. (1) It is a giant rate-matching buffer **between two systems it doesn't control**: customers sending a million messages at once, carriers accepting a few per second per number. (2) **Every message costs money**: a duplicate charges twice and pesters a person. (3) Callbacks go to thousands of customer servers; one slow one must not delay the rest.

1. Requirements and numbers

Functional: send SMS/MMS; status lifecycle `queued → sending → sent → delivered / undelivered / failed`; status and inbound webhooks; number provisioning; usage billing. **Non-functional:** accept in < 200 ms and never lose an accepted message; honour per-sender limits exactly; tenant isolation; signed, retried webhooks.

Quantity	Value	Note
Volume	5B msgs/day ≈ 58K/s, ~300K/s peak	assumption
US sender limits (Twilio docs)	long code ~1 segment/s, toll-free 3/s, short code 100/s	higher by account agreement
Segment	160 GSM-7 chars (153 when split); one emoji forces UCS-2: 70 (67)	the unit sent and billed
One campaign	1M SMS from 10 toll-free numbers = 30/s	**~9.3 h** to drain
Queue limit (Twilio docs)	messages queued over 10 h fail with error 30001	so queue age must be visible
Webhooks	~2 per message	~600K/s peak

What the numbers say: the API is easy; the hard part is **millions of slow, independently rate-limited queues**.

2. The five-questions pass

- **Flow:** accept → durable log → `queued` returned → async dispatch → async callbacks.
- **Storage:** message records, an append-only usage ledger, per-sender queues.
- **Speed:** one durable write on the request path.
- **Failure:** carriers, customer servers and our hosts all fail, so retries and idempotency everywhere. Twilio's early principles (InfoQ, 2011) agree: the unit of failure is a single host; short timeouts with quick retries; idempotent service interfaces; small stateless services; relaxed consistency where possible.
- **Growth:** queue cardinality (accounts × senders) grows faster than volume.

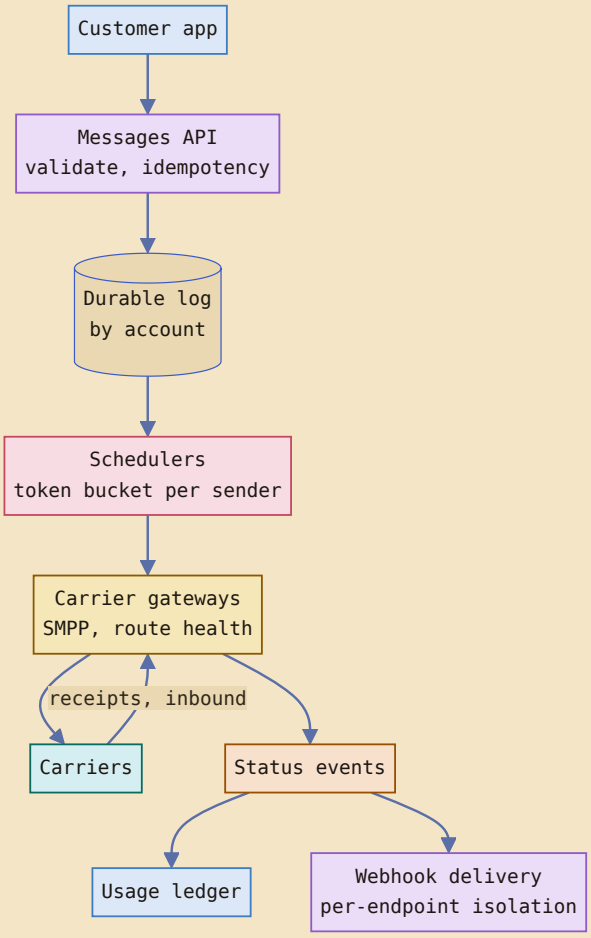
3. API

`POST /Accounts/{sid}/Messages` + `Idempotency-Key` → `{sid, status: queued, num_segments}`; `GET …/Messages/{sid}` for status, error and price; outbound webhooks for status and inbound SMS, each carrying `X-Twilio-Signature`, an HMAC-SHA1 (a keyed hash only the key holder can produce) of the URL and parameters keyed by the account's auth token. The webhook mechanics are those of Webhooks.

4. Data model

Table	Key	Notes
`messages`	`(account_id, day)`, `msg_sid`	segments, encoding, status history, carrier route and ID, price, error
`idempotency_keys`	`(account_id, key)` → `msg_sid`	TTL 24 h
`sender_queues`	`(account_id, sender, channel)`	token-bucket rate (Rate Limiting)
`usage_ledger`	append-only	balance is derived; any cached balance is a hint
`webhook_jobs`	`job_id`	url, payload, attempts, next_attempt_at, expires_at

5. The architecture, annotated



1. **API** counts segments, checks the idempotency key, writes the durable log (Message Queue), returns `queued` (Idempotency).
2. **Schedulers** (sharded by account) release each sender queue at its rate. Twilio's docs describe two levels: an account-level queue per channel and sender type, feeding phone-number queues.
3. **Carrier gateways** pick a route by country, cost and health, and hold SMPP connections (Short Message Peer-to-Peer, the carriers' binary SMS protocol) with a cap on unacknowledged messages.
4. **Status events** feed the message record, the ledger and webhook delivery.

6. Deep dives

Deep dive 1 — millions of tiny queues. One shared FIFO lets one 1M-message campaign at 30/s block everyone behind it (**head-of-line blocking**); one broker topic per sender fails because brokers don't handle millions of topics. So queues are **logical** (rows or in-memory structures on the owning scheduler shard), each with a token bucket, and the scheduler **round-robins across queues that have tokens** under an account-level ceiling (Backpressure). Messages past their validity period fail explicitly instead of arriving hours late.

Deep dive 2 — webhooks that isolate bad endpoints. Segment's Centrifuge post (2018) is the reference: a shared queue let one slow endpoint back up everyone, and per-destination queues still let one big customer block others on the same endpoint. Centrifuge kept delivery jobs in MySQL (each database owned by exactly one "director"), retried with exponential backoff until jobs expired after 4 hours, then archived them to S3 for replay. At ~400K outbound requests/s, one degraded partner API absorbed ~85M events during its outage, which drained within ~30 minutes afterwards without affecting other integrations. Take-aways: per-endpoint isolation (Bulkhead); short timeouts (Twilio webhooks

default to 5 s connect, 15 s total, overridable per URL); bounded backoff with jitter; a replayable dead-letter archive; and receivers dedupe on a retry token (Twilio sends `I-Twilio-Idempotency-Token`).

Deep dive 3 — billing must never act on missing data. Twilio's July 2013 post-mortem: balances lived in a Redis cluster (an in-memory store). A network blip made all replicas resync at once and overloaded the primary; restarted with the wrong config, it came back empty and read-only, so balances read as zero. Usage then **auto-recharged** customer cards over and over and some accounts were suspended; **1.4% of customers** were affected while calls and SMS kept working. Fixes: promote a replica instead of restarting the primary; never charge or suspend when balance data is missing or unwritable; reconcile in real time against the independent double-entry ledger. Rule: **the ledger is truth, the cache is a hint, money actions fail safe** (see the Digital Wallet design).

Deep dive 4 — regions and cells. Customers choose an **edge location** (Twilio's webhook default is `ashburn`); accounts are partitioned into **cells** (independent full stacks, each serving a subset of accounts) so a bad deploy or noisy tenant hurts one cell (Cell-Based Architecture).

7. The failure walk

1. **Carrier route degrades:** route health drops and new traffic shifts to an alternate; handed-off messages are re-sent only on explicit rejection, avoiding visible duplicates.
2. **Scheduler shard dies:** queues move by consistent hashing; the new owner rebuilds from the log; unacked sends retry with the same internal ID and the gateway dedupes.
3. **Customer endpoint down:** its webhooks back off and archive; nobody else notices.
4. **Customer floods the API:** `429`; accepted messages drain at that account's rate only.
5. **Billing anomaly:** pause money actions, keep messaging.

8. Trade-offs

Decision	Chosen	Alternative	Why
API semantics	Accept-and-queue	Synchronous send	Carriers are slow and rate-limited
Queues	Logical per-sender + fair scheduler	Shared FIFO / topic per sender	Head-of-line blocking / broker limits
Webhooks	Per-endpoint isolation, bounded retry, archive	Shared queue, infinite retry	One bad endpoint must not hurt others
Billing	Ledger + fail-safe cache	Balance only in cache	The 2013 incident
Carrier delivery	At-least-once + dedupe by ID	Fire and forget	Lost paid messages are worse

9. The follow-up questions

- **"Exactly-once to the customer?"** Not across carriers; offer idempotent acceptance (same key → same SID) and at-least-once internally with dedupe by SID.
- **"STOP / opt-out?"** An inbound STOP writes an opt-out record per (sender, recipient) and the API rejects later sends: the platform enforces compliance, not customers.
- **"Toll fraud?"** Per-account velocity limits by destination country, anomaly checks on new accounts, premium-rate prefixes blocked by default.

> **Interview one-liner:** "Accept and durably log every message, return `queued`, and drain millions of logical per-sender token-bucket queues to carriers with a fair scheduler; status events feed an append-only ledger and webhook delivery that isolates each endpoint with short timeouts, bounded retries and a replayable archive; idempotency everywhere, and billing never acts on missing data."

The price: a stateful scheduling tier and heavy per-tenant bookkeeping.

Sources

- InfoQ, "Twilio's Cloud Architecture Principles" (2011): https://www.infoq.com/news/2011/04/twilio-cloud-architecture
- Twilio, "Billing Incident Post-Mortem: Breakdown, Analysis and Root Cause" (2013): https://www.twilio.com/en-us/blog/company/communications/billing-incident-post-mortem-breakdown-analysis-and-root-cause-html
- Twilio Segment, "Centrifuge: a reliable system for delivering billions of events per day" (2018): https://www.twilio.com/en-us/blog/archive/2018/introducing-centrifuge/
- Twilio Docs, Account Based Throughput Overview: https://www.twilio.com/docs/messaging/guides/account-based-throughput-overview
- Twilio Docs, Webhooks Connection Overrides: https://www.twilio.com/docs/usage/webhooks/webhooks-connection-overrides
- Twilio Docs, Webhooks security (X-Twilio-Signature): https://www.twilio.com/docs/usage/security
- Twilio Docs, SMS character limit and segments: https://www.twilio.com/docs/glossary/what-sms-character-limit
- Twilio Support, Rate Limits and Message Queues: https://support.twilio.com/hc/en-us/articles/115002943027-Understanding-Twilio-Rate-Limits-and-Message-Queues

Module 26 — When Systems Fail: Postmortems as Patterns

Every pattern in this volume exists because something once broke. This module works backwards from seven well-documented outages (Facebook 2021, AWS us-east-1 in 2020, 2021 and 2025, GitLab 2017, Cloudflare 2019, CrowdStrike 2024): timeline, root cause, fix, and links to the patterns that would have contained it (not re-explained here). Facts come from the companies' own postmortems or primary observers such as Cloudflare's network data. In an interview, "this is the outage that design choice prevents" is one of the clearest signs you have run production systems.

How to read a postmortem

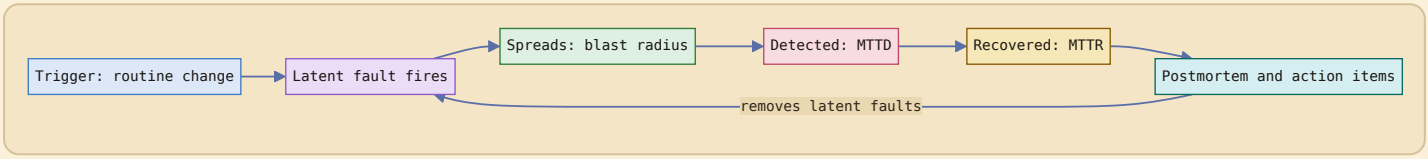
A **postmortem** is the written record of an **incident** (an unplanned event that hurts users or puts them at risk): impact, mitigation, root causes and follow-up actions (alerting and on-call, Module 0). Teams decide in advance which incidents need one, so nobody argues about it at 3 AM. Typical triggers (Google's SRE book): user-visible downtime or degradation beyond a threshold; any data loss; on-call having to intervene (e.g. a rollback); a long time to resolve; a monitoring failure, where a human noticed before the alerts did.

Anatomy

Section	Answers	What a designer looks for
Summary and impact	What broke, for whom, for how long	The **blast radius** (the share of users or systems one failure reached)
Timeline	What happened when	Three gaps: trigger to detection, detection to diagnosis, diagnosis to recovery
Trigger	The event that started it	Usually a routine change: deploy, config push, capacity addition
Root cause	The weakness that let the trigger do harm	Usually **latent**: present for months, waiting for a trigger
Contributing factors	What made it worse or longer	The design lessons live here
Action items	Fixes, each with an owner and a priority	Sorted into *prevent*, *detect faster*, *recover faster*

Five terms appear in every story below:

- **Trigger vs. root cause.** The trigger starts the failure; the root cause is the flaw that let it do harm. Fix only the trigger ("don't add capacity") and the flaw waits for the next one.
- **Latent fault.** A *fault* is a defect; a *failure* is when users see it. A latent fault exists but has not yet fired.
- **Control plane vs. data plane.** The control plane creates and changes things (launches servers, updates DNS records or routes); the data plane serves requests and forwards packets. A **statically stable** system keeps its data plane running on its last known state when the control plane or another dependency fails (AWS's term; Module 6).
- **MTTD / MTTR.** Mean time to *detect* and to *recover*. Most action items shrink one of these, because preventing every failure is impossible.
- **Shared fate.** One failure takes out both things: a status page hosted on the platform it reports on, or repair tools that need the network they repair.



Blameless culture

A **blameless** postmortem assumes everyone acted in good faith and asks what in the *system* made the mistake easy and its consequences large: fix systems and processes, not people. It is an engineering choice: punishment teaches people to hide near-misses, the best data an organisation has about where it is fragile. GitLab's postmortem below is the model: one sentence on the command that wiped the database, pages on why five recovery mechanisms failed.

To read any postmortem as a designer, ask: (1) What changed? (2) Why did it spread? (3) Why was detection slow? (4) Why was recovery slow (shared fate? recovery that is itself a load spike?) (5) Which pattern contains it?

> **In the interview**
>
> **Q:** An engineer ran the command that caused the outage. What do you do about them?
>
> **A:** Nothing punitive. Ask why one command could do that much damage, why no guard rail stopped it, and why recovery was slow; make an action item for each answer.

Sources

- Google, *Site Reliability Engineering*, ch. 15 "Postmortem Culture": https://sre.google/sre-book/postmortem-culture/
- AWS Builders' Library, "Static stability using Availability Zones": https://builder.aws.com/content/3K9SiqiJ369TGkkGzqPM95BuYWI/static-stability-using-availability-zones

2021 — Facebook BGP and DNS outage

On 4 October 2021, Facebook, Instagram, WhatsApp and Messenger disappeared from the internet for about six hours (Cloudflare saw `facebook.com` unresolvable from about 15:50 to 21:20 UTC). **BGP** (Border Gateway Protocol) is how networks announce "these IP addresses are reachable through me"; withdrawing an announcement makes them unreachable. DNS maps names to IPs, and a domain's **authoritative name servers** hold its official answers.

What happened

Times are UTC, from Cloudflare's external measurements. Meta's posts give the sequence of events but no clock times.

Time (UTC)	Event
~15:40	Spike of BGP updates from Facebook's normally quiet network
15:50	`facebook.com` stops resolving on 1.1.1.1. Public resolvers return SERVFAIL for Facebook, Instagram and WhatsApp
15:58	Facebook is no longer announcing routes to its DNS server prefixes (e.g. 185.89.218.0/23, 129.134.30.0/23)
During	Retrying clients push DNS resolvers to ~30x usual query volume. Internal tools and out-of-band access are down; engineers go to the data centres in person
~21:00–21:17	BGP activity from Facebook resumes and peaks
21:20–21:28	Names resolve and Facebook is reconnected; services return gradually

Why it happened

- **Trigger.** During routine maintenance, a command meant to *assess* global backbone capacity instead took down every connection in the **backbone**, Facebook's private fibre network between its data centres.
- **A failed guard.** An audit tool meant to block such commands had a bug that let it through.

- **Root cause of the disappearance.** Facebook's DNS servers withdraw their own BGP announcements when they cannot reach the data centres, so an unhealthy site stops attracting traffic. With the backbone down, *every* DNS site judged itself unhealthy at once and withdrew. The name servers were still running, but nothing could reach them.
- **Contributing factors.** Internal diagnosis tools depended on the same network and DNS; out-of-band management access was also down; strict physical and hardware security slowed on-site work (a trade-off Meta judges worth it for everyday security).

How it was fixed

Engineers on site restored the backbone, and the DNS announcements returned. Turning everything back on at once was its own risk: data centres had dropped power draw by **tens of megawatts**, and a sudden return to full load could stress electrical systems and caches. Meta's **"storm" drills** (planned exercises that take services or regions offline) had prepared teams to ramp traffic back gradually, and no further system-wide failure followed. The drills had never simulated a global backbone failure; Meta said it would explore doing so.

The patterns that would have contained it

- Staged rollout of network changes, so one command reaches one failure domain first: Canary Deployment, CI/CD — Module 9.
- Independent failure domains: Multi-Region and Cell-Based Architecture — Module 6.
- Health checks that never remove *all* capacity; if every site looks unhealthy, distrust the signal: Health Check Endpoint — Module 9.
- Gradual recovery and client backoff, so the restart is not a second outage: Retry with Exponential Backoff — Module 5.
- Out-of-band access that does not share fate with production, exercised in drills.

In the interview

Q: The DNS servers were up. Why was Facebook unreachable?

A: Each DNS site withdrew its BGP routes when it lost contact with the data centres, and with the backbone down everywhere, all withdrew at once. A local "remove myself when unhealthy" rule needs a global floor that never lets the health signal remove all capacity.

Sources

- Meta, "More details about the October 4 outage": https://engineering.fb.com/2021/10/05/networking-traffic/outage-details/
- Meta, "Update about the October 4th outage": https://engineering.fb.com/2021/10/04/networking-traffic/outage/
- Cloudflare, "Understanding how Facebook disappeared from the Internet": https://blog.cloudflare.com/october-2021-facebook-outage/

2020 — AWS Kinesis outage (us-east-1)

On 25 November 2020, **Amazon Kinesis Data Streams** (managed ingestion of ordered record streams split into **shards**) failed in us-east-1 for most of a day. Its **back-end** was already divided into cells; its **front-end** was a single fleet of "many thousands" of servers that authenticate, throttle and route requests to shards. Each front-end server caches a **shard-map** (fleet membership and shard ownership), built partly by talking to peers, and **creates one OS thread for every other server in the front-end fleet.**

What happened

Time (PST)	Event
2:44–3:47 AM	Small capacity addition to the front-end fleet; peers take up to an hour to learn of new members
5:15 AM	First alarms on Kinesis put/get errors. CloudWatch and EventBridge errors begin
7:51 AM	Every candidate cause needs a full front-end restart; the new capacity is removed
9:39 AM	Root cause confirmed: every server has exceeded the OS thread limit
10:07 AM	First restarted servers take traffic, at a few hundred servers per hour
10:23 PM	Kinesis fully recovered, about 17 hours after the first alarm

Why it happened

- **Root cause.** The design was all-to-all: in a fleet of N servers, each needs about N peer threads. Every capacity addition pushed every server closer to an OS thread limit, and no alarm watched that margin. Once it was crossed, shard-map construction failed and servers could not route requests.
- **Trigger.** The capacity addition; AWS states plainly it was the trigger, not the root cause.
- **Why recovery was slow.** Building a shard-map competes with serving traffic for CPU, so servers restarted too fast would perform badly, be marked unhealthy and be removed. The team also chose not to raise the thread limit fleet-wide untested.
- **Contributing factors: hidden dependencies.**
  - **Cognito** (AWS sign-in) sent analytics to Kinesis through a buffer; a latent bug made its web servers block when the buffer backed up.
  - **CloudWatch** lost metrics, alarms went to "insufficient data", and metric-driven **Auto Scaling** was delayed until 5:47 PM.
  - **Lambda** saw memory contention from buffered metrics; **EventBridge** delays hurt ECS and EKS provisioning.
  - The **status page** posting tool depended on Cognito; the minimal-dependency backup tool was manual and unfamiliar to staff.

How it was fixed

AWS removed the new capacity, let servers bootstrap from the authoritative metadata store instead of from peers, and restarted the fleet slowly. Action items: larger servers (so fewer of them); fine-grained alarms on thread use; testing a higher OS thread limit; a dedicated fleet for the front-end cache; a separate, partitioned front-end for large internal users such as CloudWatch; and **faster cellularisation of the front-end** to match the back-end. Cognito now tolerates Kinesis errors without exhausting its buffers, CloudWatch keeps three hours of metrics locally, and support staff are trained on the backup status-page tool.

The patterns that would have contained it

- Cells, so each server tracks only peers in a bounded, tested cell: Multi-Region and Cell-Based Architecture — Module 6.
- A bulkhead per large tenant, such as CloudWatch's own partition: Bulkhead — Module 5.
- Non-critical side paths (Cognito's analytics buffer) that degrade instead of blocking: Graceful Degradation and Timeout — Module 5.
- Alarms on usage as a percentage of a limit, not only on errors, and a designed cold-start path: Module 9.

In the interview

Q: How can adding servers break a system?

A: When per-server cost grows with fleet size: each server held a thread per peer, so adding servers pushed *every* server past the OS thread limit at once.

Q: How would you redesign it?

A: Bounded-size cells for the front-end; routing metadata from an authoritative store, not fleet-wide peer traffic; own partitions for big internal tenants; alarms on thread headroom.

Sources

- AWS, "Summary of the Amazon Kinesis Event in the Northern Virginia (US-EAST-1) Region": https://aws.amazon.com/message/11201/

2021 — AWS us-east-1 internal network congestion

On 7 December 2021, congestion inside AWS's network impaired the us-east-1 control plane for about seven hours. AWS runs two networks in the region: the **main AWS network** (customer workloads and most services) and a separate **internal network** hosting foundational services such as monitoring, internal DNS, authorisation and parts of the EC2 control plane. Networking devices between them route and translate traffic.

What happened

Time (PST)	Event
7:30 AM	Automated scaling of a service on the main network triggers unexpected behaviour in many internal-network clients; a connection surge overwhelms the devices between the networks
7:30–7:33 AM	EC2 API errors and latency begin, and Route 53 APIs are impaired. Support case creation is down until 2:25 PM
8:22 AM	The Service Health Dashboard posts updates; its failover to a standby region had itself been impaired
9:28 AM	Internal DNS moved off the congested paths; DNS errors recover, congestion persists
1:34 PM	Congestion significantly improved
2:22 PM	All network devices fully recovered
2:40 PM – evening	EC2 launches recover; API Gateway, Fargate and EventBridge tails last until 4:37, 5:00 and 6:40 PM

Why it happened

- **Root cause.** Internal clients' back-off should slow retries under congestion, but a latent bug in a long-standing code path stopped it engaging. Delays caused retries, and retries kept the devices congested: a self-sustaining **retry storm**.
- **Trigger.** A routine automated capacity-scaling activity.
- **Contributing factors.** The congestion knocked out the real-time monitoring operators needed, so they worked from logs; internal deployment systems ran on the same network, slowing every fix; the status page's failover and the support contact centre both depended on the internal network.
- **What held.** The data plane was statically stable: running EC2 instances, existing DNS answers, existing load balancers and Lambda invocations were unaffected. What failed were the control-plane APIs (launches, provisioning, logins) and services that needed the internal network, such as API Gateway, STS, Connect, VPC endpoints for S3 and DynamoDB, and container restarts in ECS, EKS and Fargate.

How it was fixed

Operators moved DNS traffic off the congested paths, isolated the largest traffic sources, turned off heavy network services (EventBridge delivery was paused from 12:35 to 2:35 PM), and added capacity. Action items: disable the scaling activities until all fixes ship; fix the back-off bug (expected within two weeks); add network configuration protecting the devices against similar surges; keep API Gateway servers healthy when the internal network is unavailable; build a new status dashboard and a support system running actively across several regions.

The patterns that would have contained it

- Backoff with jitter, tested under real congestion, and a retry budget: Retry with Exponential Backoff, Circuit Breaker — Module 5.
- Load shedding and admission control at shared chokepoints: Rate Limiting — Module 8, Backpressure — Module 10.
- Static stability, why running workloads survived: Multi-Region and Cell-Based Architecture — Module 6.
- Monitoring, deploy tooling and status communication that do not share fate with the network they observe: Module 9.

In the interview

Q: Why did the congestion last for hours instead of clearing itself?

A: The clients' back-off failed to engage, so every timeout produced a retry that kept the devices overloaded: a classic retry storm, broken only by shedding load from outside.

Sources

- AWS, "Summary of the AWS Service Event in the Northern Virginia (US-EAST-1) Region" (Dec 2021): https://aws.amazon.com/message/12721/

2025 — AWS us-east-1 DynamoDB DNS outage

On 19–20 October 2025, the DNS record for DynamoDB's regional endpoint, `dynamodb.us-east-1.amazonaws.com`, was left **empty**, so every client of AWS's managed key-value database, including many AWS services, lost the ability to find it. Recovery then set off further failures in EC2 and Network Load Balancer (NLB). Impact lasted from 11:48 PM to 2:20 PM PDT, about 14.5 hours.

Two parts of DynamoDB's DNS automation matter: the **DNS Planner** watches load-balancer health and capacity and produces numbered **plans** (sets of load balancers and weights per endpoint); the **DNS Enactor** runs independently in three Availability Zones, applies plans to Route 53 and then cleans up old ones.

What happened

Time (PDT)	Event
11:48 PM (Oct 19)	DynamoDB endpoint DNS resolution fails; EC2's server-management system (DWFM) starts losing leases
12:38 AM	Engineers identify DynamoDB's DNS state as the source
2:25 AM	All DNS information restored
2:40 AM	Cached DNS entries expire; customers can connect again
2:25–5:28 AM	DWFM collapses under congestion re-leasing every server; recovers after throttling and selective restarts
5:30 AM – 2:09 PM	NLB connection errors from health checks that flip back and forth
10:36 AM	EC2 network propagation is back to normal
1:50 PM / 2:20 PM	EC2 APIs and launches normal / last services recovered

Why it happened

- **Root cause: a latent race condition** in the DNS management system. The sequence:
  1. One Enactor was badly delayed while applying an old plan.
  2. A second Enactor applied the newest plan and then started its cleanup, which deletes plans much older than the one just applied.

3. The delayed Enactor finally wrote its old plan over the new one. Its "is my plan newer?" check had run only *once, at the start*: a **time-of-check to time-of-use** bug (a decision made on a fact that changed before it was acted on).
4. Cleanup then deleted that old plan, now the active one. Every IP address for the endpoint was removed, in a state that blocked further automatic updates; engineers repaired it by hand.

- **Second failure: congestive collapse in EC2.** The **DropletWorkflow Manager (DWFM)** manages the physical servers ("droplets") that host instances, holding a **lease** on each, renewed by state checks that depend on DynamoDB. The leases expired during the outage; afterwards DWFM tried to re-lease the whole fleet at once, and work timed out and was re-queued faster than it could finish. Without a lease a droplet cannot accept launches, so launches failed with "insufficient capacity".
- **Third failure: health-check flapping in NLB.** New instances came up before their network configuration had propagated, so NLB health checks failed, passed and failed them again. The extra load triggered **automatic AZ DNS failover**, which removed healthy capacity.

How it was fixed

Operators repaired the DynamoDB DNS record by hand, throttled DWFM's incoming work and selectively restarted hosts (from 4:14 AM), and disabled NLB's automatic health-check failover at 9:36 AM, which brought healthy nodes back. Action items: disable the DNS Planner and Enactor worldwide until the race is fixed and protections against incorrect plans are added; a **velocity control** limiting how much capacity one NLB can remove through failover; a test suite for DWFM's recovery workflow; queue-size-driven throttling in EC2's data propagation.

The patterns that would have contained it

- Check freshness at commit time, not at start: conditional writes with a version or fencing token (a number that rejects a stale writer): Consensus, leader election and distributed locks — Module 0.
- Refuse to publish an empty or drastically smaller DNS plan, keeping the last good state (static stability): Graceful Degradation — Module 5.
- Admission control by queue size, so recovery cannot collapse under its backlog: Backpressure — Module 10, Rate Limiting — Module 8.
- Rate limits on automated failover, so health checks can never remove all capacity: Health Check Endpoint — Module 9.
- Regional cells, so one regional endpoint is not a single point of failure: Multi-Region and Cell-Based Architecture — Module 6.

In the interview

Q: How can three redundant DNS Enactors produce an *empty* record?

A: Redundancy without coordination: one slow Enactor wrote a stale plan after its freshness check had gone stale, and another's cleanup deleted it as obsolete. Fix: conditional writes at commit time, plus a guard that never publishes an empty answer.

Q: DynamoDB was back at 2:40 AM. Why did EC2 launches fail until 1:50 PM?

A: Recovery was a load spike: DWFM tried to renew leases for the whole fleet at once, timed out, retried and collapsed; a network-configuration backlog and NLB health-check flapping followed. Recovery paths need their own throttling and tests.

Sources

- AWS, "Summary of the Amazon DynamoDB Service Disruption in the Northern Virginia (US-EAST-1) Region" (Oct 2025): https://aws.amazon.com/message/101925/

2017 — GitLab.com database deletion

On 31 January 2017, a GitLab.com engineer rebuilding a broken **replica** (a copy that follows the primary) wiped the **primary's** data directory instead. The site was down about 18 hours and about six hours of database changes were lost; Git repositories and wikis, stored separately, were unaffected.

What happened

Time (UTC)	Event
~17:20	An engineer takes a manual LVM snapshot (Linux volume manager; a point-in-time disk copy) of production to load into staging
~19:00	Database load spikes, probably from spam plus a background job removing a flagged employee's data
~23:00	Replication lag: the secondary needs WAL (PostgreSQL's write-ahead log) segments the primary already deleted. No WAL archive, so the secondary must be rebuilt by hand
~23:00–23:30	`pg_basebackup` hangs silently. Settings are changed: `max_wal_senders` from 3 to 32, `max_connections` from 8000 to 2000
~23:30	The engineer wipes the primary's data directory, meaning to target the secondary; stopped within a second or two, ~300 GB already gone
Feb 1, ~18:00	Restore from the 6-hour-old staging snapshot is complete, after an ~18-hour copy over disks throttled to about 60 Mbps

Why it happened

The deletion was the trigger; the real story is that **none of the recovery mechanisms worked**:

Mechanism	Why it failed
Daily `pg_dump` to S3	pg_dump 9.2 run against PostgreSQL 9.6 failed; the S3 bucket was empty
Failure emails	Rejected by the receiver because DMARC (an email authentication standard) was not enabled for them, so nobody knew
Azure disk snapshots	Enabled for NFS servers but not for database servers
LVM snapshots	Daily, for staging, not designed for disaster recovery; the manual 17:20 snapshot saved six hours
Replication	Built for failover, not recovery: copies a deletion as faithfully as a write

Contributing factors from GitLab's "five whys": a manual replica rebuild, neither automated nor well documented; no WAL archiving, so lag could break replication; an abuse workflow that made it easy to hard-delete an account by mistake; and **no owner for testing backups**. About 5,000 projects, 5,000 comments and 700 user accounts were affected.

How it was fixed

The team copied the staging LVM snapshot to production, restored webhooks separately, and bumped ID sequences by 100,000 so IDs would not be reused. Tracked action items include hourly LVM snapshots; Azure snapshots for the database; a sensible `max_connections`; point-in-time recovery with continuous WAL archiving; **automated testing of backup restores**; better replication runbooks; an owner for data durability; monitoring and a public dashboard for backups.

The patterns that would have contained it

- Continuous WAL archiving and point-in-time recovery (recovery point in minutes, not hours): Write-Ahead Log, Primary-Replica — Module 3.
- Restore drills with a stated **RPO** (how much data you may lose) and **RTO** (how long recovery may take). A backup does not count until it has been restored: Disaster recovery — Module 0.
- Alerts on the *absence* of a successful backup, not on emailed failures: Module 9.
- Automated, runbook-driven replica rebuilds, with clearly different prompts and hostnames for primary and replica, so manual destructive work is rare and hard to misaim.

In the interview

Q: What is the single most valuable action item?

A: Automated, regular restore tests with an owner and an alert on failure; they would have revealed every other gap long before.

Sources

- GitLab, "Postmortem of database outage of January 31": https://about.gitlab.com/blog/2017/02/10/postmortem-of-database-outage-of-january-31/

2019 — Cloudflare WAF regex outage

On 2 July 2019, one new rule in Cloudflare's **WAF** (web application firewall: checks requests against attack signatures) pushed CPU to nearly 100% on every HTTP-serving server worldwide. Cloudflare lost about 80% of its traffic for 27 minutes; users saw 502 errors.

What happened

Time (UTC)	Event
13:31	A pull request adding a cross-site-scripting detection rule is merged after review
13:37	Build and tests pass; tests do not measure CPU use
13:42	The rule is deployed globally in seconds through Quicksilver, Cloudflare's config store (p99 propagation 2.29 s)
~13:45	First page, then global traffic-drop and CPU-exhaustion alerts
14:00	WAF identified as the cause; attack ruled out
14:07	A **global terminate** (kill switch) disables the WAF everywhere
14:09	Traffic and CPU are back to normal
14:52	Fix verified in one city; WAF re-enabled globally

Why it happened

- **Root cause.** The rule contained `.*(?:.*=.*)`. The WAF's regex engine (PCRE under Lua) uses **backtracking**: when a match fails, it tries every other way of splitting the input, so with two greedy `.*` before an `=` the steps grow faster than linearly with input length. Cloudflare's example: 20 characters with no `=` take 4,067 steps just to conclude nothing matches. Nothing limited the work one match could do.
- **Trigger.** The rule was meant to run in "simulate" mode (log, don't block), but a simulated rule still executes, so it still burned CPU.
- **Contributing factors (Cloudflare's own list).** A protection against excessive regex CPU had been removed by accident weeks earlier in a refactor; tests could not detect CPU use; WAF rule changes could go global without a staged rollout; rollback required two full WAF builds; the traffic-drop alert was slow; engineers' access to internal tools went through Cloudflare's own edge and failed with it (the bypass was rarely used, and some credentials had timed out for security reasons); customers' dashboard and API also ran through the failing edge.

How it was fixed

The global kill switch restored service at 14:07–14:09. Action items: restore the CPU protection (done); inspect all 3,868 WAF rules for backtracking (done); add performance profiling to rule tests; switch to RE2 or Rust's regex engine, which guarantee linear-time matching; **stage WAF rule rollouts** like other software, keeping a global emergency path for active attacks; a way to move the dashboard and API off the edge; automated status-page updates.

The patterns that would have contained it

- Config and rules are code: staged rollout with automatic checks (Canary Deployment, Feature Flags — Module 9); the kill switch, a feature flag, ended the outage.
- A per-request work budget, so one expensive operation cannot take a whole core: Timeout, Bulkhead — Module 5.
- Algorithms with guaranteed cost: a linear-time regex engine removes this whole class of failure.
- Management and recovery access that does not share fate with the edge it manages.

In the interview

Q: What two changes give the most protection?

A: Staged rollout of rule and config changes (small initial blast radius), and a linear-time matching engine or a hard per-match budget.

Sources

- Cloudflare, "Details of the Cloudflare outage on July 2, 2019": https://blog.cloudflare.com/details-of-the-cloudflare-outage-on-july-2-2019/

2024 — CrowdStrike Channel File 291

On 19 July 2024, a content update for CrowdStrike's Falcon security sensor crashed Windows machines worldwide with a blue screen. Microsoft estimated **8.5 million** devices, under 1% of all Windows machines, but enterprises running many critical services, so the harm was large. Mac and Linux were not affected. (Module 0 uses it as the canonical staged-rollout lesson.)

Terms from CrowdStrike's root cause analysis (RCA):

- The **sensor** includes a Windows **kernel driver** (`csagent.sys`) that loads early during boot; a fault in the kernel crashes the whole machine.
- **Rapid Response Content** is configuration data, not code, pushed from the cloud to change detections without a sensor release.
- A **Template Type** is a detection capability compiled into the sensor, with a fixed set of input fields. **Template Instances** are its matching rules, delivered in numbered **channel files**.
- The **Content Validator** checks content before publishing; the **Content Interpreter** on the sensor evaluates it.

What happened

Date / time (UTC)	Event
28 Feb 2024	Sensor 7.11 introduces the IPC Template Type (detects abuse of Windows named pipes); its content arrives in Channel File 291
5 Mar 2024	Template Type passes a stress test; first instance ships
8–24 Apr 2024	Three more instances ship without problems
19 Jul, 04:09	Two new instances ship to every online Windows sensor (7.11+); machines crash on the next named-pipe event, many again at every boot
19 Jul, 05:27	Content reverted; machines that stay up long enough to fetch it recover, the rest need manual repair

Why it happened

- **Root cause.** The IPC Template Type was *defined* with 21 input fields, but the sensor code calling the Content Interpreter supplied only **20**. Earlier instances and test cases all used a wildcard for field 21, so it was never read. A 19 July instance was the first to use a real value there; the interpreter read past the end of the 20-entry array (an **out-of-bounds read**), and the kernel crashed.

- **Contributing factors (CrowdStrike's six findings).**
  1. The field count was not validated at compile time.
  2. There was no runtime bounds check in the interpreter.
  3. Tests used wildcards for every field.
  4. The Content Validator had a logic error: it checked against 21 fields.
  5. Validation did not run instances through the real interpreter.
  6. Template Instances were not deployed in stages, so the update reached every customer at once.
- **Why recovery was slow.** The kernel crashed during boot, before the fix could download. Repair was manual, machine by machine: boot into Safe Mode or the recovery environment and delete `C-00000291*.sys`; BitLocker-encrypted disks needed recovery keys first.

How it was fixed

CrowdStrike reverted the content at 05:27 UTC and published manual and cloud-VM remediation steps and bootable recovery images; Microsoft released a recovery tool. Mitigations from the RCA: field-count validation at compile time (in production from 27 July); runtime bounds checks in the Content Interpreter (backported to 7.11+); testing every field with non-wildcard values; new Content Validator checks; testing every Template Instance in the interpreter; **staged deployment rings with canaries and bake time**; **customer control over when content is deployed**; two independent third-party code and process reviews.

The patterns that would have contained it

- Configuration is code and needs staged rings and bake time (Canary Deployment, CI/CD — Module 9); even a 1% first ring would have shown crashes within minutes.
- Defensive parsing at the consumer: check bounds, reject bad content and fall back to the last good content instead of crashing: Graceful Degradation — Module 5.
- Customer-controlled update rings (a test fleet first) give each customer a bulkhead against vendor pushes: Bulkhead — Module 5.
- Automatic safe mode: a boot-time agent that crashes repeatedly disables its latest content itself, because a fix you cannot download is not a rollback.

In the interview

Q: If the bad file was reverted after 78 minutes, why did recovery take days?

A: The file crashed the kernel at boot, so many machines never stayed up long enough to fetch the revert; each needed hands-on repair, often with BitLocker recovery keys. Rollback only works if the broken system can still receive it.

Sources

- CrowdStrike, "External Technical Root Cause Analysis — Channel File 291" (6 Aug 2024): https://www.crowdstrike.com/wp-content/uploads/2024/08/Channel-File-291-Incident-Root-Cause-Analysis-08.06.2024.pdf
- CrowdStrike, Preliminary Post Incident Review: https://www.crowdstrike.com/en-us/blog/falcon-content-update-preliminary-post-incident-report/
- CrowdStrike, "Technical Details: Falcon Content Update for Windows Hosts": https://www.crowdstrike.com/blog/falcon-update-for-windows-hosts-technical-details/
- CrowdStrike, Remediation and Guidance Hub: https://www.crowdstrike.com/en-us/blog/statement-on-falcon-content-update-for-windows-hosts/
- Microsoft, "Helping our customers through the CrowdStrike outage" (8.5 million devices): https://blogs.microsoft.com/blog/2024/07/20/helping-our-customers-through-the-crowdstrike-outage/
- Microsoft, KB5042421 (manual recovery steps): https://support.microsoft.com/en-us/help/5042421

Patterns across all the outages

Incident	Trigger	Latent root cause	Why it spread	Why recovery was slow	Containing patterns
Facebook 2021	Backbone maintenance command	Audit-tool bug, and DNS sites that withdraw when unhealthy	One command reached the whole backbone, and every DNS site withdrew together	Tools and out-of-band access shared fate. On-site security. Load had to be ramped	Staged change, failure domains, fail-open health checks, independent repair path
AWS Kinesis 2020	Capacity addition	One thread per peer, near the OS limit	One un-celled front-end fleet. Dependents blocked	Restart paced by cache rebuild	Cells, bulkheads per tenant, graceful degradation, limit alarms
AWS Dec 2021	Automated scaling activity	Client back-off bug	Retry storm on shared network devices	Monitoring and deploy tools on the congested network	Backoff with jitter, load shedding, static stability, independent tooling
AWS Oct 2025	Delayed DNS Enactor	Time-of-check to time-of-use race in plan apply and cleanup	Regional endpoint used by many AWS services	Lease storm in EC2, NLB health checks flapping	Conditional writes, never publish empty state, queue-based throttling, failover velocity limits
GitLab 2017	Command run on the wrong host	Untested, broken backups	Single primary, and replication copies deletions	Only a slow-disk staging snapshot worked	WAL archiving and PITR, restore drills, backup alerting
Cloudflare 2019	WAF rule deploy	Backtracking regex, CPU guard removed	Global push in seconds	Access and dashboards ran through the failed edge	Staged config rollout, kill switch, per-request budgets, linear-time algorithms
CrowdStrike 2024	Content update	21 fields defined, 20 supplied, no bounds check	Global push to every sensor	Boot loop blocked the revert. Manual repair per machine	Staged rings, defensive parsing, last-known-good fallback, customer-controlled rollout

Six lessons recur, and each maps to a pattern taught elsewhere in this volume:

1. **Configuration changes are deploys.** None of the seven began with an ordinary code deploy: a command, a capacity change, an automated workflow, a rule or a content file. Give them canaries, rings and kill switches (Module 9).
2. **Blast radius is a design choice.** Cells, regions and per-tenant bulkheads decide in advance how far a fault can spread (Module 6, Module 5).
3. **Safety checks are code and can fail.** Facebook's audit tool, CrowdStrike's validator and Cloudflare's removed CPU guard each had bugs. Layer defences, and make the consumer safe too.
4. **Recovery is a load event.** Retry storms, lease storms, cache rebuilds and power ramps hit at the worst moment: use backoff, throttling, backpressure and gradual ramps (Module 5, Module 10).
5. **Repair paths must not share fate.** Monitoring, deploy tools, admin access and status pages failed with the systems they serve at Facebook, Cloudflare and in two AWS events.
6. **Test the recovery, not just the system.** Untested backups (GitLab), cold restart (Kinesis) and lease recovery (DWFM), and an undrilled global failure (Facebook), turned short faults into long outages.

Module 22 — Interview Question Bank

How to use this bank

This module collects the system design questions that come up again and again in interviews, worded the way they usually appear on LeetCode Discuss interview-experience posts, in the big public GitHub lists and on popular question sites (see the sources at the end). Use it as a checklist so that nothing in the room surprises you.

- ⋆ marks a question that is asked very often. Do those first.
- **Part A** is the "Design X" prompts (URL shortener, news feed, Uber, Ticketmaster …). When this book has a full case study, the entry is a two-minute outline plus a link to the full design. When it doesn't, the entry is a longer outline that stands on its own, with links to the modules that teach each building block.
- **Part B** is the concept questions ("What is consistent hashing?", "SQL vs NoSQL?", "How does a circuit breaker work?"). Each has a short answer you can say out loud and a link to the full explanation. Part B ends with **More design prompts** (Q213–Q219), design questions added after the numbering was fixed.
- **Part C** is a minute-by-minute plan for a 45-minute system design round.
- **Part D** is how to prepare and practise: a 4/8/12-week plan, a mock-interview rubric, traps that sink candidates, API-design rounds and what each level is expected to show.

How to drill: read the question, cover the answer, say your answer out loud (or sketch the boxes), then compare. Mark the ones you stumbled on and come back to them the next day. For Part A, time yourself: two minutes for the outline, then pick one deep dive.

Part A — "Design X" questions

These are the open-ended "Design X" prompts, worded the way candidates report them. ⋆ marks a question that shows up in three or more independent lists or again and again in interview-experience posts. *Asked at* lists companies only where a source reported them; those tags are self-reported, so treat them as hints, not facts.

Two kinds of answer follow. When this book already has a full design, you get a short outline you can say in two minutes, then a link to the full design. When the book has no dedicated design, you get a longer outline that stands on its own, with links to the modules that teach each building block. Each outline follows the same order you would use at the whiteboard: requirements, then components, then the decisions the interviewer is likely to push on, then scale.

Building blocks

Q1. ⋆ Design a URL Shortener (TinyURL / bit.ly)

- **Requirements:** turn a long URL into a short one (about 7 characters), redirect fast, optional custom alias and expiry; reads outnumber writes by roughly 100 to 1.
- **Components:** write API, key generator, key-value store mapping short code to long URL, read cache (Redis), redirect service behind a CDN.
- **Probe 1, making the key:** base62-encode a unique counter or Snowflake ID (no collisions; 62^7 is about 3.5 trillion codes) versus hashing the URL and retrying on collision.
- **Probe 2, 301 or 302:** a 301 (permanent) lets browsers cache the redirect and cuts load; a 302 (temporary) sends every click to you, which you need for click analytics.
- **Probe 3:** the same long URL submitted twice: return the existing code (needs a reverse index) or mint a new one.
- **Scale:** the read path is a cache lookup; shard the store by short code; push click events to a queue for analytics.

→ *Full design:* Module 13 — Case 2: Design a URL Shortener · Module 0 — Worked example: designing a URL shortener

Q2. Design Pastebin

- **Requirements:** upload a text blob, get a short link, read it back; optional expiry and private pastes; pastes can be up to a few MB.
- **Components:** the URL-shortener key path, plus object storage (S3-style) for the text body, a metadata table (key, owner, size, expiry, storage pointer), a CDN for popular pastes.
- **Probe 1:** keep large bodies out of the database; store a pointer to the object store.
- **Probe 2:** expiry: a lazy check at read time plus a background sweeper that deletes expired objects.
- **Scale:** read-heavy, so CDN and cache absorb most traffic; rate-limit anonymous uploads to stop abuse.

→ *Full design:* Module 13 — Case 2, Variant: Pastebin

Q3. ⋆ Design an API Rate Limiter

Asked at: Amazon, Atlassian, Uber, Microsoft, Stripe, Pinterest, Patreon

- **Requirements:** cap requests per user, API key or IP (for example 100 per minute); answer with HTTP 429 and a `Retry-After` header; add under a millisecond of latency; keep working when the limiter store is down.
- **Components:** limiter middleware in the API gateway, a shared counter store (Redis), a rules config service.
- **Probe 1, algorithm:** token bucket (allows short bursts and is cheap) versus sliding-window log (exact but memory-heavy) versus sliding-window counter (a good approximation).
- **Probe 2, races:** make check-and-increment atomic with a Redis Lua script or `INCR` plus expiry, so two gateway nodes cannot both let the last request through.
- **Probe 3, store outage:** fail open (allow traffic) or fail closed (block it). Most public APIs fail open and keep a local in-memory limit as a backstop.
- **Scale:** shard counters by key; for global limits across regions, accept slight overshoot or sync counts asynchronously.

→ *Full design:* Module 23 — Case 2: Design a Rate Limiter · Module 8 — Rate Limiting · Module 0 — Rate limiting and throttling

Q4. ⋆ Design a Unique ID Generator in distributed systems (Twitter Snowflake)

- **Requirements:** 64-bit IDs that are unique, roughly sortable by time, and generated at over 10,000 per second per node with no coordination on the hot path.
- **Components:** an ID library or service on each node; a way to hand out worker IDs (ZooKeeper/etcd or config).
- **Probe 1, layout:** 1 sign bit, 41 bits of milliseconds since a custom epoch (about 69 years), 10 bits of worker ID, 12 bits of sequence (4,096 IDs per millisecond per worker).
- **Probe 2, clock going backwards:** refuse to issue IDs (or wait) until the clock passes the last timestamp used.
- **Probe 3, alternatives:** UUIDv4 (no coordination, but 128 bits and random, so it hurts B-tree index locality), UUIDv7 (time-ordered), a database ticket server (simple, but a bottleneck).
- **Scale:** each node is independent, so throughput grows linearly with nodes.

→ *Full design:* Module 13 — Case 1: Design a Unique ID Generator

Q5. ⋆ Design a (Distributed) Key-Value Store

Asked at: Apple, Google, Canva, Rubrik, Avalara, OpenDoor

- **Requirements:** `get(key)` / `put(key, value)`, small values, always writable, tunable consistency, horizontal scale, survive node and rack loss.
- **Components:** consistent-hash ring with virtual nodes, N replicas per key (N = 3), a coordinator node per request, an LSM-tree storage engine (write-ahead log, memtable, SSTables), gossip membership.
- **Probe 1, quorums:** choose R and W with R + W > N to read your latest write; lower them for speed when stale reads are fine.

- **Probe 2, conflicts:** last-write-wins (simple, can lose data) versus vector clocks with client-side merge.
- **Probe 3, failures:** hinted handoff for short outages, Merkle-tree anti-entropy to repair replicas after long ones.
- **Scale:** add nodes and let virtual nodes move small slices of data; a hot key needs caching or key splitting.

→ *Full design:* Module 13 — Case 3: Design a Distributed Key-Value Store

Q6. ⋆ Design a Distributed Cache (Redis / Memcached)

- **Requirements:** sub-millisecond get/set with TTL, eviction when memory is full, horizontal scale, survive node loss without a stampede on the database.
- **Components:** cache nodes holding in-memory hash tables, a partitioning scheme (client-side consistent hashing, or Redis Cluster's 16,384 hash slots), replicas, a cluster manager.
- **Probe 1, eviction:** LRU or LFU per node; the TTL choice sets how stale data can get.
- **Probe 2, hot keys:** replicate a hot key to many nodes, or keep a small local cache in the app.
- **Probe 3, consistency with the DB:** cache-aside with delete-on-write, and request coalescing so one miss does not become a thousand DB queries.
- **Scale:** resharding moves slots one at a time while clients follow redirects.

→ *Full design:* Module 21 — Case 4: Distributed Cache Cluster

Q7. ⋆ Design a Web Crawler

Asked at: Amazon

- **Requirements:** crawl billions of pages, stay polite (obey robots.txt, limit requests per host), skip duplicates, refresh pages that change, and be robust to traps and broken HTML.
- **Components:** seed URLs, URL frontier (priority queues plus one queue per host), DNS cache, fetchers, parser and link extractor, URL-seen filter (Bloom filter), content dedup (fingerprints), object storage for pages.
- **Probe 1, politeness:** map each host to one queue served by one worker, with a delay between hits.
- **Probe 2, dedup:** Bloom filter for URLs seen; a content hash or SimHash for near-duplicate pages.
- **Probe 3, traps:** cap depth, URL length and pages per host.
- **Scale:** partition the frontier by host hash across many machines; checkpoint the frontier so a crash does not restart the crawl.

→ *Full design:* Module 13 — Case 5: Design a Web Crawler

Q8. ⋆ Design Search Autocomplete / Typeahead Suggestion

- **Requirements:** top 5 to 10 suggestions for a prefix within about 100 ms, ranked by popularity, refreshed daily or hourly.
- **Components:** query-log collector, offline aggregator (counts per query), trie builder, trie servers with the top-k results cached at each node, browser and CDN caching.
- **Probe 1:** precompute top-k per prefix so a lookup costs O(prefix length), not a subtree scan.
- **Probe 2:** rebuild the trie offline and swap it in, instead of updating it on every keystroke.
- **Probe 3:** shard by prefix range (uneven: many words start with "s") or by hash with a lookup map; filter offensive terms.
- **Scale:** debounce in the client, cache short prefixes everywhere, add a small real-time layer for trending terms.

→ *Full design:* Module 13 — Case 6: Design Search Autocomplete

Q9. ⋆ Design a Distributed Message Queue (Kafka / RabbitMQ)

Asked at: Amazon, Microsoft, Apple, Instacart, Wise, Confluent

- **Requirements:** producers publish to topics, consumer groups read independently, ordering per partition, durable storage with retention, at-least-once delivery.
- **Components:** brokers storing partitioned append-only logs, a controller quorum (KRaft; ZooKeeper only in versions before Kafka 4.0) for metadata and leader election, replication with in-sync replicas (ISR), consumer group coordinator, offset storage.

- **Probe 1:** ordering exists only within a partition, so choose the partition key carefully.
- **Probe 2, durability:** `acks=all` plus a minimum ISR size, versus `acks=1` for speed.
- **Probe 3, delivery:** at-least-once (commit the offset after processing) plus idempotent consumers; exactly-once needs idempotent producers and transactions.
- **Scale:** add partitions and brokers; sequential disk writes and zero-copy reads keep throughput high.

→ *Full design:* Module 13 — Case 4: Design a Distributed Message Queue · Module 10 — Inside Kafka

Q10. ⋆ Design a Notification Service (at scale)

Asked at: Google, Pinterest, Amazon, Airbnb, Instacart, StubHub, OCI, Flipkart

- **Requirements:** send push, SMS, email and in-app messages, triggered by events or by campaigns; respect user preferences and quiet hours; no duplicates; millions per minute at peak.
- **Components:** notification API, preference and template services, a queue per channel, channel workers that call providers (APNs, FCM, SMS gateway, email provider), a delivery-status store, retry queue and dead-letter queue.
- **Probe 1:** dedup with an idempotency key per (user, event) so retries do not send twice.
- **Probe 2:** separate queues per channel and priority, so a slow SMS provider or a big marketing blast cannot delay OTPs.
- **Probe 3:** per-user rate limits and digesting, so you do not spam people.
- **Scale:** campaign fan-out runs as a batch job that feeds the queues at a controlled rate.

→ *Full design:* Module 2 — Capstone: Designing a Notification System

Q11. Design a CDN

- **Requirements:** serve static and cacheable content from close to users, absorb traffic spikes, protect the origin.
- **Components:** edge points of presence (PoPs) with cache servers, request routing (anycast or GeoDNS), a mid-tier "origin shield" cache, the origin, a purge/invalidation API, TLS at the edge.
- **Probe 1:** pull (fetch from origin on a miss) versus push (upload ahead of time, good for big releases).
- **Probe 2:** cache key design and TTLs; versioned file names beat purges.
- **Probe 3:** routing users to the nearest healthy PoP and failing over when one dies.
- **Scale:** the origin shield collapses many edge misses into one origin fetch.

→ *Full design:* Module 23 — Case 1: Design a CDN · Module 8 — CDN · Module 0 — CDNs and anycast

Q12. Design a load balancer

- **Requirements:** spread traffic over healthy backends, add little latency, have no single point of failure, support TLS and sticky sessions.
- **Components:** an L4 layer (routes by IP and port, very fast) in front of an L7 layer (reads HTTP, routes by path or header), health checker, config store, a redundant pair or anycast group.
- **Probe 1, algorithms:** round robin, least connections, consistent hashing (Maglev) for affinity.
- **Probe 2:** active versus passive health checks and connection draining during deploys.
- **Probe 3:** making the balancer itself redundant with a floating IP (VRRP) or anycast plus ECMP.
- **Scale:** L4 scales by spreading flows over many stateless nodes; L7 scales horizontally behind it.

→ *Full design:* Module 6 — Load Balancing · Module 0 — Load balancers

Q13. Design a distributed locking service

- **Requirements:** mutual exclusion across machines, locks that expire if the holder dies, safety even under process pauses and network delays.
- **Components:** a consensus-backed store (etcd, ZooKeeper or Consul), lock = key plus lease (a lock that expires unless renewed), a client library with renew and release.
- **Probe 1, fencing tokens:** every grant carries an increasing number; the protected resource rejects writes with an older number, so a paused old holder cannot do damage.
- **Probe 2:** Redis `SET NX PX` is fine for efficiency locks; for correctness use a consensus store. Redlock's safety is disputed.
- **Probe 3:** lease length trades fast recovery against false expiry.

- **Scale:** shard locks by name; avoid one hot global lock.

→ *Full design:* Module 0 — Consensus, leader election and distributed locks

Q14. Design an authentication / login system (OTP, OAuth, SSO)

Asked at: Google, Visa, Gusto

- **Requirements:** sign up, log in with password, OTP or social login; sessions across devices; logout and revoke; MFA; resistance to brute force.
- **Components:** identity service, credential store (passwords hashed with bcrypt or Argon2), token service (short-lived access token plus refresh token), OTP service, OAuth 2.0 / OpenID Connect provider integration, rate limiter.
- **Probe 1:** server sessions (easy to revoke) versus JWTs (stateless, hard to revoke, so keep them short-lived).
- **Probe 2:** refresh-token rotation and detecting reuse of a stolen refresh token.
- **Probe 3:** lockouts and rate limits per account and per IP; MFA for risky logins.
- **Scale:** the token check is stateless at the gateway; only refresh and login hit the identity database.

→ *Full design:* Module 8 — Authorization Service · Module 0 — OAuth 2.0, OpenID Connect, SSO and API keys

Q15. Design a webhook dispatcher

- **Requirements:** deliver events to customer URLs reliably, in order per subscription where possible, retry for up to days, let customers verify the sender.
- **Components:** event outbox in the producing service, a dispatch queue partitioned per endpoint, delivery workers, a retry scheduler with exponential backoff, dead-letter store, a delivery log the customer can see and replay.
- **Probe 1:** sign each payload with HMAC and a timestamp so the receiver can check it is real and fresh.
- **Probe 2:** isolation: one slow or dead endpoint must not block others (per-endpoint queues and concurrency caps, a circuit breaker per endpoint).
- **Probe 3:** at-least-once delivery, so include an event ID that receivers use to deduplicate.
- **Scale:** partition by endpoint; disable endpoints that keep failing and alert the owner.

→ *Full design:* Module 2 — Webhooks

Q16. Design an SMS / OTP service

- **Requirements:** send a one-time code within seconds, valid for about 5 minutes, limited attempts, high delivery rate, resistant to abuse.
- **Components:** OTP API, code generator (a random 6-digit code from a secure random source), store holding a hash of the code with TTL and attempt counter (Redis), SMS router choosing among several providers, delivery-receipt handler.
- **Probe 1:** never store the plain code; compare hashes; lock after 3 to 5 wrong tries.
- **Probe 2:** route by country and fail over to a second provider when delivery receipts show problems.
- **Probe 3:** stop "SMS pumping" fraud with per-number, per-IP and per-country limits and CAPTCHAs.
- **Scale:** OTPs get their own high-priority queue, separate from marketing messages.

→ *Full design:* Module 2 — Capstone: Designing a Notification System (the OTP pieces are the add-on) · Module 25 — Case 6: Twilio-style platform (per-sender queues, carrier limits)

Q17. Design a key-value store / query cache for a search engine

- **Requirements:** cache results for popular queries to cut search-backend load; keep latency low; drop stale entries when the index changes.
- **Components:** query normalizer (lowercase, trim, sort filters) to build the cache key, a sharded in-memory cache (hash of key picks the shard), LRU eviction, TTL.
- **Probe 1:** query popularity follows a long tail, so a small cache catches a large share of traffic.
- **Probe 2:** invalidation: short TTLs, or put an index version number in the key so a new index naturally misses.

- **Scale:** replicate hot shards; coalesce identical concurrent misses.

→ *Full design:* Module 4 — Cache-Aside · Module 13 — Case 3: Design a Distributed Key-Value Store

Social and content

Q18. ⋆ Design Facebook News Feed

- **Requirements:** a home feed of posts from friends and pages, ranked, loaded in under 200 ms; posting is fast; feeds update within seconds.
- **Components:** post service, social graph service, fan-out workers, feed cache (a list of post IDs per user), ranking service, media on a CDN.
- **Probe 1, fan-out on write vs on read:** pushing post IDs into followers' feeds at write time makes reads cheap but is costly for celebrities; pulling at read time is the opposite. Use a hybrid: push for normal users, pull for accounts with millions of followers.
- **Probe 2:** ranking: candidates, then a scoring model, then diversity rules.
- **Probe 3:** cursor-based pagination and handling deleted or hidden posts.
- **Scale:** keep feed caches only for active users; rebuild inactive users' feeds on demand.

→ *Full design:* Module 12 — Design 1: The News Feed · Module 18 — App 1: News Feed App

Q19. ⋆ Design Twitter (timeline + search)

- **Requirements:** post short tweets, follow users, home and user timelines, likes and retweets, search, trends; heavily read-heavy.
- **Components:** tweet service and store (sharded by tweet ID), follow graph, fan-out service writing into per-user timeline caches (Redis lists), search indexer, trends pipeline.
- **Probe 1:** the hybrid fan-out for celebrity accounts (merge their tweets at read time).
- **Probe 2:** Snowflake IDs give a time order, so timelines sort by ID.
- **Probe 3:** search uses a real-time inverted index fed from the tweet stream.
- **Scale:** counters (likes, retweets) are sharded and updated asynchronously.

→ *Full design:* Module 20 — Case 5: Microblogging (Twitter/X-like)

Q20. ⋆ Design Instagram / photo sharing (Flickr, Google Photos)

Asked at: Google, DoorDash, Amazon, Uber, Facebook

- **Requirements:** upload photos and videos, follow users, feed, likes and comments; uploads are durable; images load fast worldwide.
- **Components:** upload service issuing pre-signed URLs (the client uploads straight to object storage), media pipeline (thumbnails, several resolutions, format conversion), metadata DB, feed service, CDN.
- **Probe 1:** keep big bytes out of your app servers; store metadata separately from blobs.
- **Probe 2:** feed generation, with the same fan-out trade-off as the news feed.
- **Probe 3:** sharding metadata by user ID and the hot-celebrity problem.
- **Scale:** the CDN serves most reads; process media asynchronously after upload.

→ *Full design:* Module 20 — Case 4: Photo Sharing (Instagram-like)

Q21. Design TikTok / short-video feed

- **Requirements:** upload short videos; an endless "For You" feed picked by a recommender, not by follows; next video starts in under ~200 ms; new uploads reach viewers within minutes.
- **Numbers:** 600M daily users × 100 views ≈ 700K views/s; ~140K feed requests/s at peak, each ranking ~1,000 candidates.
- **Components:** the video upload/transcode/CDN pipeline; a funnel of embedding retrieval → multi-task ranker → diversity re-ranker over precomputed candidate pools; an event log feeding real-time features.
- **Probe 1:** instant start: preload the first seconds of the next 2–3 videos, short segments, warm connections.

- **Probe 2:** learning in minutes: streamed features plus online training that pushes only updated embeddings.
- **Probe 3:** cold start by staged exposure; fall back to trending if the ranker fails.

→ *Full design:* Module 25 — Case 1: TikTok-Style Short-Video Feed

Q22. Design Tinder

Requirements. Functional: create a profile with photos, set preferences (age, distance, gender), see a stack of nearby candidates, swipe right (like) or left (pass), get a match when both people like each other, then chat. Non-functional: the swipe deck loads quickly, a match shows up instantly, and nobody sees the same profile twice in a short window.

Estimates. 75M monthly users and 10M daily; around 1B swipes a day, so about 12k swipes a second on average and 3 to 4 times that at evening peak. Photos dominate storage (about 6 per user at 200 KB each).

API. `GET /recs?count=20`, `POST /swipe {target_id, direction}` (returns `{match: true}` when it is mutual), `GET /matches`, plus the chat APIs.

Data model. Profiles in a document store with a geo index; swipes as `(swiper, target) -> direction`, sharded by swiper; matches as rows indexed for both users; a "seen" set per user (a Bloom filter or a recent-ID set).

Architecture. Profile service; location service (geohash cells updated when the app opens); recommendation service builds a deck by querying candidates in nearby cells that pass the preference filters, then ranks them; swipe service; match service; chat service over WebSockets; push notifications.

Deep dives.

1. *Match detection:* when A likes B, check whether B already liked A. Store likes keyed by `(target, swiper)` so the check is one key lookup; write the match atomically and notify both users.
2. *Deck generation:* precompute decks in the background and keep them in a cache, so a swipe pops the next profile locally. Exclude anyone already swiped.
3. *Ranking:* mix attractiveness or desirability signals, activity recency and distance; keep fairness so popular profiles do not get all the exposure.

Failures and trade-offs. Swipes can be accepted into a queue and processed a moment later, so the app never blocks; the match check must still be correct, so make it idempotent. Location is coarse on purpose (a geohash cell) for privacy.

→ *Building blocks:* Module 15 — Design 1: Proximity Service · Module 14 — Case 1: Chat System · Module 3 — Sharding · Module 0 — Probabilistic data structures and Merkle trees

Q23. Design Reddit / Quora (Q&A platform)

- **Requirements:** post questions or links, answers and nested comments, up/down votes, ranked listings (hot, top, new), topics or subreddits, search.
- **Components:** post and comment service, vote service, ranking jobs, listing caches per topic, search index, notification fan-out.
- **Probe 1:** votes are very write-heavy, so aggregate counts asynchronously and store one row per (user, item) to stop double voting.
- **Probe 2:** "hot" ranking mixes score and age; recompute it for active items only.
- **Probe 3:** storing comment trees (a parent ID plus a path, or a materialized path) and loading the top branches first.
- **Scale:** cache listing pages; most traffic is anonymous reads.

→ *Full design:* Module 20 — Case 3: Q&A Platform (Quora-like)

Q24. ⋆ Design a search engine / distributed search (Google, Elasticsearch)

- **Requirements:** keyword search over billions of documents with results in under 200 ms, relevance ranking, near-real-time indexing of new documents.

- **Components:** ingestion and tokenizer, inverted index (term to list of documents), index shards plus replicas, a query broker doing scatter-gather, a ranker (BM25 plus signals), a result cache.
- **Probe 1, partitioning:** by document (each shard holds a full index for its documents; every query hits every shard) versus by term (a query touches fewer shards, but multi-term queries are hard). Most systems partition by document.
- **Probe 2:** tail latency in scatter-gather: hedge requests and return partial results.
- **Probe 3:** near-real-time updates using small new segments that merge later (LSM-like).
- **Scale:** add replicas for query load and shards for data size.

→ *Full design:* Module 20 — Case 1: Distributed Search · Module 19 — Inverted index

Q25. Design Twitter search

- **Requirements:** find tweets by keyword or hashtag within seconds of posting, newest and top results, very high ingest.
- **Components:** the tweet stream feeds indexers, a real-time index partitioned by time and then by tweet ID, an archive index for older tweets, a broker merging results.
- **Probe 1:** time partitioning lets most queries hit only the recent, in-memory partitions.
- **Probe 2:** ranking "top" results needs engagement signals that keep changing after indexing.
- **Scale:** the same scatter-gather as Q24, with an in-memory recent tier.

→ *Full design:* Module 20 — Case 1: Distributed Search (Twitter/Elasticsearch-style)

Q26. Design the data structures for a social network / Facebook graph search

Requirements. Functional: store people and friendships; answer "friends of X", "mutual friends of X and Y", "shortest path (degrees of separation) between X and Y", and graph-search queries such as "friends of mine who live in Pune and like hiking". Non-functional: friend lists in under 10 ms, 2-hop queries in under 100 ms, billions of edges.

Estimates. 2B users, about 300 friends each, so about 600B directed edges. At 16 bytes per edge that is around 10 TB, too big for one machine but small enough to spread across a few hundred.

API. `GET /users/{id}/friends`, `GET /users/{a}/mutual/{b}`, `GET /path?from=a&to=b`, `GET /search?q=...`.

Data model. Adjacency lists: `user_id -> sorted list of friend IDs`, sharded by user ID (this is what Facebook's TAO roughly does: objects plus typed edges, cached heavily). Attributes such as city and likes live in their own indexes (city -> users).

Architecture. A graph store sharded by user, a read-through cache tier in front (graph data is read far more than written), a person lookup service, and a search service that turns a graph query into set operations.

Deep dives.

1. *Mutual friends:* intersect two sorted friend lists (a linear merge, or bitmaps for big lists).
2. *Shortest path:* bidirectional breadth-first search from both ends, stopping when the frontiers meet. Each hop is a batch of cross-shard lookups, so cap the depth (3 is plenty) and the fan-out.
3. *Graph search:* compute "my friends" ∩ "people in Pune" ∩ "people who like hiking", starting with the smallest set; this is the same plan a query optimizer would pick.

Failures and trade-offs. Sharding by user means any multi-hop query crosses machines; some systems co-locate friend clusters to reduce that, at the cost of complex rebalancing. Store each friendship as two directed edges so each side reads locally, and accept brief inconsistency while both are written.

→ *Building blocks:* Module 3 — Sharding · Module 4 — Read-Through · Module 20 — Case 1: Distributed Search

Q27. Design a tagging service (Atlassian-style: tag Jira / Confluence items)

Asked at: Atlassian, PhonePe

Requirements. Functional: add and remove tags on items owned by many products (Jira issues, Confluence pages, Bitbucket PRs); list the tags of an item; list the items with a given tag across

products; show popular tags; autocomplete tag names. Non-functional: multi-tenant (one customer never sees another's tags), reads under 50 ms, tag changes visible within seconds, products stay loosely coupled.

Estimates. 10M tenants, 10B tagged items, 3 tags per item on average, about 30B item-tag links. Writes are perhaps 5k per second; reads are 10 to 20 times that.

API. `PUT /tenants/{t}/items/{product}:{itemId}/tags/{tag}`, `DELETE` the same, `GET .../items/{id}/tags`, `GET /tenants/{t}/tags/{tag}/items?product=&cursor=`, `GET /tenants/{t}/tags?prefix=`.

Data model. Tag table: `(tenant, tag_id, name, usage_count)`. Two link tables, one per direction: `item -> tags` and `tag -> items`, both keyed with the tenant first so a tenant's data sits together. Items are referenced by a global ID (`product:id`) so the service does not need to understand any product's schema.

Architecture. The tagging API writes the `item -> tags` row (the source of truth) and publishes an event; a consumer updates the `tag -> items` index, the counts and a search index. Products call the API and also get events when tags change; when an item is deleted, the product sends an event so its links are removed.

Deep dives.

1. *Two indexes:* each query direction gets its own table; keep them in sync through the event stream or an outbox.
2. *Permissions:* the tag service does not know who may see a Jira issue, so it returns item IDs and the product filters them (or a permission service checks them in a batch).
3. *Hot tags:* a tag such as "bug" can have millions of items, so paginate with cursors and keep counts in sharded counters.

Failures and trade-offs. Eventual consistency between the two directions is acceptable for tags; tag names need normalising (case, spaces) to avoid near-duplicates.

→ *Building blocks:* Module 3 — CQRS · Module 2 — Event-Driven Architecture · Module 8 — Cursor-Based Pagination · Module 16 — Design 4: Distributed Counter

Q28. Design "recently viewed items"

Requirements. Functional: record each product or page a user views; show the last N (say 20) distinct items, newest first, on every device the user signs in from; let the user clear the list. Non-functional: recording must never slow the page (fire and forget); reads under 20 ms; losing the last view or two in a failure is acceptable.

Estimates. 50M daily users and 30 views each is 1.5B views a day (about 17k per second, 50k at peak). Storage is small: 20 IDs of about 16 bytes per user is about 1 KB with timestamps, so 100M users need about 100 GB.

API. `POST /users/{id}/views {item_id}` (asynchronous; 202 Accepted), `GET /users/{id}/recent?limit=20`, `DELETE /users/{id}/recent`.

Data model. One Redis sorted set per user (`member = item_id`, `score = timestamp`). Adding an item already present just updates its score, which gives you dedup for free; trim to the last N after each add. Back it with a key-value store (DynamoDB or Cassandra) holding the same list, for durability and for users whose data was evicted.

Architecture. The client or the web tier sends view events to a stream (Kafka); a consumer updates Redis and writes through to the durable store. The read API serves from Redis and falls back to the store on a miss. Anonymous users keep the list in local storage and merge it into the account at sign-in.

Deep dives.

1. *Write volume:* batch events per user in the consumer and apply one `ZADD` plus `ZREMRANGEBYRANK` per batch.
2. *Showing items:* the list holds only IDs; hydrate names, prices and stock in one batch call, and drop items that were deleted.

3. *Privacy:* honour "clear history" and retention limits; never use the list for ads without consent.

Failures and trade-offs. If Redis loses data, rebuild from the store; if the stream lags, the newest view shows up a few seconds late, which nobody notices.

→ *Building blocks:* Module 4 — Write-Through · Module 2 — Event-Driven Architecture · Module 13 — Case 3: Design a Distributed Key-Value Store

Counting, ranking and analytics

Q29. ⋆ Top K problem: most shared articles / heavy hitters (1 h / 24 h windows)

Asked at: LinkedIn, Facebook, Twitter, Amazon, Bloomberg, Pinterest

- **Requirements:** top K items by count over sliding windows (last hour, last day), refreshed every minute, over millions of events per second.
- **Components:** event stream (Kafka) partitioned by item, stream processors keeping per-window counts, a merge step for the global top K, a results cache behind the API.
- **Probe 1, exact vs approximate:** a count-min sketch plus a small heap (or the Space-Saving algorithm) uses fixed memory and gives slightly high counts; exact counts need a hash map per window.
- **Probe 2, windows:** keep 1-minute buckets and sum the last 60 (tumbling buckets) instead of tracking every event.
- **Probe 3:** a fast approximate stream path plus a slower exact batch path (the Lambda pattern).
- **Scale:** each partition computes a local top K; merge them (take somewhat more than K per partition to stay accurate).

→ *Full design:* Module 19 — Heavy hitters: Misra-Gries, Space-Saving and streaming top-k

Q30. Design a trending topics system (Twitter)

- **Requirements:** find hashtags and phrases whose usage is rising unusually fast, per region, refreshed every few minutes.
- **Components:** tweet stream, tokenizer and entity extractor, windowed counters, a scorer comparing the current rate with a baseline, spam filters, a results cache.
- **Probe 1:** "trending" means growth, not volume: use exponentially decayed counts and a z-score against the usual rate, so "good morning" does not trend every day.
- **Probe 2:** filter bots and coordinated spam before counting.
- **Scale:** the same streaming top-K pipeline as Q29, keyed by (region, term).

→ *Full design:* Module 19 — Trending detection with decayed counts and z-scores

Q31. Design Amazon's sales ranking by category

- **Requirements:** rank products by recent sales within each category and parent category, updated hourly.
- **Components:** order events, a stream or hourly batch job computing sales per (category, product) over a decayed window, a rank table per category, a cache for product pages.
- **Probe 1:** a product sits in many categories: count once, then emit to each ancestor category.
- **Probe 2:** batch (MapReduce) versus streaming; hourly freshness makes batch enough.
- **Scale:** per-category top lists are small; only the counting job is large.

→ *Full design:* Module 19 — Heavy hitters · Module 10 — MapReduce

Q32. Count Facebook likes / distributed counter (views, likes at scale)

Asked at: Facebook, Amazon, Twitter, Coinbase

- **Requirements:** increment counts for likes and views at huge rates, show a close-enough count fast, and make sure each user's like counts once.
- **Components:** a like table `(user, item)` as the source of truth, an event stream, aggregators that batch increments, sharded counters (N sub-counters per hot item, summed on read), a cache.

- **Probe 1:** a hot item turns one counter row into a write hotspot: split it into sub-counters or pre-aggregate in the stream.
- **Probe 2:** exact (likes) versus approximate (unique views with HyperLogLog).
- **Scale:** batch writes every second or so; readers tolerate a slightly stale count.

→ *Full design:* Module 16 — Design 4: Distributed Counter

Q33. Design "N users viewing this page"

Asked at: Booking.com

- **Requirements:** show roughly how many people are viewing a page right now; the number may be a few seconds old.
- **Components:** clients send a heartbeat every 10 to 30 s; a presence store holds a per-page set with expiring entries (Redis sorted set scored by last-seen time); a reader counts entries newer than the cutoff.
- **Probe 1:** heartbeats versus WebSocket connect/disconnect (disconnects get lost, so expiry is still needed).
- **Probe 2:** for huge pages, use HyperLogLog per time bucket instead of exact sets.
- **Scale:** shard by page ID; cache the count for a few seconds.

→ *Full design:* Module 16 — Design 4: Distributed Counter

Q34. Design Facebook likes with live updates

Asked at: Facebook, Coinbase

- **Requirements:** like counts on a post update live for everyone currently viewing it.
- **Components:** the distributed counter from Q32, plus a subscription service: each viewer's connection (SSE or WebSocket) subscribes to the post IDs on screen; a pub/sub fan-out pushes new counts.
- **Probe 1:** throttle pushes (at most one update per post per second or two) and send totals, not each like.
- **Probe 2:** track subscriptions per gateway server so you only push to servers that have viewers of that post.
- **Scale:** viral posts get very large subscriber sets; batch and sample updates.

→ *Full design:* Module 16 — Design 4: Distributed Counter · Module 2 — Server-Sent Events

Q35. Design ad click event aggregation

- **Requirements:** count clicks per ad per minute for billing and dashboards, exactly-once for billing, results within minutes, queries by ad and time.
- **Components:** click log (Kafka), a stream aggregator with windowing, an OLAP store for results, a nightly reconciliation batch job, dedup by click ID.
- **Probe 1:** event time versus processing time, with watermarks for late events.
- **Probe 2:** exactly-once through checkpoints plus idempotent sinks.
- **Probe 3:** fraud filtering before counts reach billing.
- **Scale:** partition by ad ID; pre-aggregate hot ads.

→ *Full design:* Module 20 — Case 6: Ad-Click Aggregation

Q36. Design an ads management and display system for a social feed

Asked at: Facebook, Google, Amazon, Pinterest

- **Requirements:** advertisers create campaigns with budgets and targeting; the feed requests ads and gets an ad within about 50 ms; budgets are not overspent; impressions and clicks are counted.
- **Components:** campaign management (OLTP database), an ad index of active ads by targeting attributes, an ad server running a quick auction (bid times predicted click rate), pacing and budget service, event logging into the aggregation pipeline of Q35.
- **Probe 1:** budget pacing spreads spend over the day; budgets are enforced from near-real-time counters with a small safety margin.
- **Probe 2:** the auction ranks by expected value, not bid alone.
- **Scale:** replicate the ad index to every ad-server node; the counters are the only shared state.

→ *Full design:* Module 20 — Case 6: Ad-Click Aggregation (the serving side is the add-on)

Q37. Design Google Analytics / an analytics platform (metrics & logging)

Asked at: Microsoft, Facebook, Qualtrics, Google

- **Requirements:** a JavaScript tag collects page views and events from millions of sites; dashboards show sessions, funnels and breakdowns by any dimension; near-real-time views plus accurate daily reports.
- **Components:** collection endpoint (stateless, at the edge), Kafka, stream jobs for real-time counts, a data lake, batch jobs that build sessions, a columnar OLAP store (ClickHouse, Druid or BigQuery) for queries.
- **Probe 1:** pre-aggregated rollups versus querying raw events (flexible but costly); usually both.
- **Probe 2:** unique visitors with HyperLogLog; sampling for very large sites.
- **Scale:** partition by site ID; isolate huge tenants.

→ *Full design:* Module 10 — Capstone: Clickstream to Dashboard

Messaging and real-time

Q38. ⋆ Design a Chat System (WhatsApp / Facebook Messenger)

- **Requirements:** one-to-one and group chat, online presence, delivered and read receipts, offline delivery, history on several devices, low latency.
- **Components:** WebSocket gateways (long-lived connections), a session registry (which user is on which gateway), chat service, message store (wide-column, keyed by conversation and time), presence service, push notifications for offline users.
- **Probe 1:** ordering: a per-conversation sequence number assigned by the server.
- **Probe 2:** delivery guarantees: client message IDs for dedup, acks, and resending on reconnect.
- **Probe 3:** group fan-out: write once per group and fan out at read time, or copy to each member's inbox for small groups.
- **Scale:** gateways scale on connection count; route messages through a pub/sub layer between gateways.

→ *Full design:* Module 14 — Case 1: Chat System · Module 18 — App 2: Chat App

Q39. Design an email service (Gmail)

- **Requirements:** send and receive email over SMTP, store mail forever, search, labels and threads, spam filtering, attachments.
- **Components:** inbound SMTP servers, spam and virus filtering, a mailbox store (metadata in a database, bodies and attachments in object storage), a search index per user, outbound SMTP with queues and retries, web and IMAP front ends.
- **Probe 1:** shard everything by user; a mailbox is a natural unit.
- **Probe 2:** sender reputation and authentication (SPF, DKIM, DMARC) for deliverability.
- **Scale:** dedupe attachments sent to many recipients; move cold mail to cheaper storage.

→ *Full design:* Module 14 — Case 2: Email Service

Q40. Design a live comments feature (Facebook Live)

Asked at: Facebook

Requirements. Functional: viewers of a live video post comments; every viewer sees new comments within about 2 seconds; viewers who join late see recent comments; the streamer can moderate. Non-functional: a single stream may have millions of viewers, comments must not block the video, and dropping or sampling some comments on huge streams is fine.

Estimates. A big stream: 1M viewers and 5k comments per second. Sending every comment to every viewer would mean 5 billion deliveries a second, which is impossible and unreadable, so the design must sample.

API. `POST /streams/{id}/comments {text}`; a subscription over WebSocket or SSE: `SUBSCRIBE stream_id`; `GET /streams/{id}/comments?before=` for history.

Data model. Comments `(stream_id, comment_id, user_id, text, ts)` in a wide-column store partitioned by stream and time bucket; a short recent buffer per stream in Redis.

Architecture. The comment service validates, filters (spam, banned words) and stores the comment, then publishes it to a pub/sub topic for that stream. Edge gateway servers hold viewer connections; each gateway subscribes to the topics of the streams its viewers watch, so one message per gateway, not per viewer, crosses the network. The gateway then fans out to its local connections.

Deep dives.

1. *Two-level fan-out:* comment service, then pub/sub, then a few thousand gateways, then millions of sockets. The subscription registry maps a stream to the gateways that need it.
2. *Sampling and ranking:* above a threshold, send each viewer a sample (friends' comments first, then highly ranked ones) at a readable rate, for example 10 a second.
3. *Late joiners:* the recent buffer serves the last 50 comments at connect time.

Failures and trade-offs. If a gateway dies, clients reconnect to another one and fetch recent comments; brief gaps are acceptable. Ordering is per stream and approximate; exact global order is not worth the cost here.

→ *Building blocks:* Module 14 — Case 5: Live Streaming · Module 2 — Publish-Subscribe · Module 2 — Server-Sent Events · Module 5 — Graceful Degradation

Q41. Design Zoom / video conferencing

Requirements. Functional: create and join meetings, audio and video for 2 to 1,000 people, screen sharing, chat, recording, waiting room. Non-functional: end-to-end delay under about 150 ms for natural conversation, adapt to bad networks, and drop a little quality rather than freeze.

Estimates. 300M daily meeting participants and 3M meetings at the same time at peak. Each participant sends about 1.5 Mbps of video and receives several streams, so the media servers carry multiple Tbps at peak; signalling traffic is tiny by comparison.

API. `POST /meetings` (returns an ID and join URL), `POST /meetings/{id}/join` (returns a token plus the media server address), and a signalling WebSocket for joining, leaving, muting and sharing events.

Data model. Meetings and participants in a relational database; live meeting state (who is in, who is speaking) in memory on the meeting's media server, with a copy in Redis; recordings in object storage.

Architecture. A signalling service handles login and session setup (WebRTC-style offer and answer). Media flows over UDP (RTP), because a late packet is useless and TCP retransmits would add delay. Use a Selective Forwarding Unit (SFU): each client sends its video once to a media server, which forwards it to the others without decoding it. NAT traversal uses STUN, with TURN relays as the fallback. Media servers sit in many regions; a meeting is placed near most of its participants, and servers can be chained (cascaded) across regions.

Deep dives.

1. *SFU versus MCU versus mesh:* mesh (everyone sends to everyone) breaks above about 4 people; an MCU mixes streams into one but costs a lot of CPU and adds delay; an SFU is the usual middle ground.
2. *Simulcast:* each sender uploads 2 or 3 resolutions; the SFU sends each receiver the one that fits its bandwidth and layout (thumbnail or full screen).
3. *Congestion control:* receivers report loss and delay; senders lower bitrate; audio gets priority over video.

Failures and trade-offs. If a media server fails, clients re-join through signalling within seconds. End-to-end encryption conflicts with server-side recording and transcription, which need the server to see the media.

→ *Building blocks:* Module 0 — TCP, UDP and QUIC · Module 2 — Bidirectional Streaming · Module 6 — Multi-Region and Cell-Based Architecture · Module 14 — Case 5: Live Streaming

Q42. Design a real-time gaming leaderboard

- **Requirements:** update a player's score and show top 10 and "my rank" instantly, for millions of players per game or season.
- **Components:** score service, Redis sorted sets (`ZINCRBY` to update, `ZRANGE ... REV` (formerly `ZREVRANGE`) for top N, `ZREVRANK` for my rank; each O(log n), plus the items returned), a durable score store for recovery, a scheduled reset per season.
- **Probe 1:** ties: break them by timestamp encoded into the score.
- **Probe 2:** beyond one Redis node: shard by score range, or keep exact ranks for the top and approximate percentiles for everyone else.
- **Scale:** rebuild the sorted set from the durable store after a failure.

→ *Full design:* Module 15 — Design 4: Real-Time Gaming Leaderboard

Q43. Design an online multiplayer card game

Requirements. Functional: players log in, join matchmaking or a private table, play turn-based card games (poker, rummy, bridge) with 2 to 8 players, see others' moves instantly, reconnect after a drop, and see their history. Non-functional: game state is server-authoritative (no client can cheat), dealing must be provably fair, moves show up in under about 200 ms, and no game is lost when a server dies.

Estimates. 1M concurrent players at about 5 per table is 200k live tables. A move every few seconds per table gives about 70k moves a second, and each move is a small message.

API. `POST /matchmaking {game, stakes}` (returns a table ID and server), a WebSocket on the game server with messages such as `JOIN`, `PLAY_CARD`, `BET` and `FOLD`, and server events `STATE`, `TURN` and `RESULT`.

Data model. In memory on the game server: table state (deck, hands, pot, turn, timer). Durably: an append-only move log per table (for recovery and audits), a players and wallet database, and game results.

Architecture. Matchmaking groups players by skill, stakes and region. A table lives on exactly one game server (its owner), which runs the game as a state machine and is the only one that changes it. A lobby service and a routing table map table IDs to servers. The wallet service holds chips; bets are ledger entries.

Deep dives.

1. *Server authority and hidden information:* the server shuffles with a secure random generator and sends each client only its own cards; clients send intentions, and the server validates every move.
2. *Turn timers and reconnects:* each turn has a timeout that auto-folds or auto-plays; a dropped player can reconnect and receive a full state snapshot.
3. *Crash recovery:* replay the move log onto a new server; snapshot periodically to shorten replay.

Failures and trade-offs. One owner per table keeps the logic simple but makes that server a failure point for its tables, which the move log covers. With real money, collusion detection and audits matter as much as latency.

→ *Building blocks:* Module 2 — Bidirectional Streaming · Module 3 — Event Sourcing · Module 17 — Design a Digital Wallet · Module 15 — Design 4: Real-Time Gaming Leaderboard

Location

Q44. ⋆ Design Uber / Lyft (ride sharing)

Asked at: Facebook, Uber, Google, Microsoft

- **Requirements:** riders request rides, nearby drivers are matched within seconds, both see live location, pricing, trip state and payment.
- **Components:** location service (drivers send GPS every few seconds into an in-memory geo index), dispatch and matching service, trip service (a state machine), pricing service, payments, notifications.

- **Probe 1:** the geo index: geohash, quadtree or H3 cells, kept in memory and sharded by region.
- **Probe 2:** matching: nearest by ETA, not by straight-line distance; offer to one driver at a time with a timeout, or batch matching.
- **Probe 3:** never double-assign a driver: a lock or conditional update on the driver's state.
- **Scale:** location writes are the heaviest load; keep them in memory and write history asynchronously.

→ *Full design:* Module 12 — Design 2: Ride-Sharing Dispatch

Q45. Design an ETA service / location sharing between driver and rider

Asked at: Uber

- **Requirements:** show the driver's live position to the rider and a continuously updated arrival time.
- **Components:** location ingest, a pub/sub channel per trip pushing positions to the rider over WebSocket, a routing engine on a road graph with live traffic weights, an ML correction on top of the route ETA.
- **Probe 1:** throttle and smooth GPS points; snap them to roads (map matching).
- **Probe 2:** precomputed routing (contraction hierarchies) to answer shortest-path queries in milliseconds.
- **Scale:** partition the road graph by region; cache ETAs for popular origin and destination cells.

→ *Full design:* Module 15 — Design 2: Nearby Friends · Module 15 — Design 3: Maps & Navigation

Q46. Design surge pricing (Uber)

Asked at: Uber, Lyft

- **Requirements:** raise prices in areas where demand exceeds supply, update every minute or two, explain the multiplier to riders, avoid wild swings.
- **Components:** demand events (app opens, requests) and supply events (free drivers) bucketed by hexagon cell; a stream job computing a demand/supply ratio per cell; a pricing model turning it into a multiplier; a per-cell price cache read by the quote service.
- **Probe 1:** smooth over time and across neighbouring cells to avoid flicker.
- **Probe 2:** lock the quoted price for a few minutes so the rider pays what was shown.
- **Scale:** cells are independent, so partition by city.

→ *Full design:* Module 12 — Design 2: Ride-Sharing Dispatch (surge is part of it)

Q47. ⋆ Design Yelp / proximity service (nearby places)

- **Requirements:** find businesses within a radius, filtered and ranked; business data changes rarely; read-heavy.
- **Components:** search service with a geospatial index (geohash prefixes or a quadtree), business service and database, read replicas, cache by geohash cell.
- **Probe 1:** geohash (simple, fits a key-value or SQL index; check neighbouring cells at edges) versus quadtree (adapts to density).
- **Probe 2:** rebuild the index nightly versus updating it live, since data changes slowly.
- **Scale:** the index fits in memory; replicate it everywhere.

→ *Full design:* Module 15 — Design 1: Proximity Service

Q48. Design Nearby Friends

- **Requirements:** opted-in users see friends within a few km, updated every few seconds.
- **Components:** WebSocket servers, a location cache with TTL, a pub/sub channel per user; each user subscribes to friends' channels and filters by distance.
- **Probe 1:** the push model through per-user channels versus querying a geo index.
- **Probe 2:** inactive users expire automatically through the TTL.
- **Scale:** the pub/sub layer is the bottleneck; shard channels with consistent hashing.

→ *Full design:* Module 15 — Design 2: Nearby Friends

Q49. ⋆ Design Google Maps

- **Requirements:** render maps, search places, route between points with live traffic and ETA, turn-by-turn navigation.

- **Components:** map tiles pre-rendered per zoom level on a CDN (or vector tiles), geocoding service, routing service on a road graph, traffic service from aggregated phone locations, navigation session service.
- **Probe 1:** routing at continent scale: precomputed hierarchies (contraction hierarchies) instead of a plain Dijkstra search.
- **Probe 2:** live traffic turns into edge weights that change every few minutes.
- **Scale:** tiles are static and cached; routing is sharded by region.

→ *Full design:* Module 15 — Design 3: Maps & Navigation

Q50. ⋆ Design a food delivery app (DoorDash / Swiggy / Zomato / Uber Eats)

Asked at: Swiggy, Flipkart

- **Requirements:** browse nearby restaurants, order and pay, restaurant accepts, courier assigned, live tracking; no lost or double-charged orders; survive meal peaks.
- **Numbers:** 3M orders/day → ~125 orders/s at dinner peak; 300K couriers × GPS every 5 s = 60K location writes/s, the heaviest write load.
- **Components:** discovery by delivery-area cells, order service owning the state machine, payments (authorize, capture at delivery), merchant gateway, per-zone dispatch planner, location and tracking services.
- **Probe 1:** dispatch to the predicted food-ready time, not the order time.
- **Probe 2:** batch 2–3 orders per courier with a per-zone optimiser.
- **Probe 3:** the order as a saga with durable timers for silent restaurants; pickup is the point of no return.

→ *Full design:* Module 25 — Case 2: Food Delivery

Q51. Design a heat map of delivery agents (Swiggy/Zomato)

Asked at: Swiggy

- **Requirements:** show operations teams the density of free and busy agents and of open orders per area across a city, refreshed every 30 to 60 s.
- **Components:** the agent location stream, a stream job counting agents and orders per hexagon cell (H3) per time bucket, a store of cell counts, a map UI that draws colour-coded cells.
- **Probe 1:** fixed hexagon cells make counting a simple group-by and neighbours easy to find.
- **Probe 2:** zoom levels: roll small cells up into larger parent cells.
- **Scale:** partition the stream by city; the output is small.

→ *Full design:* Module 15 — Design 1: Proximity Service (geo cells) · Module 10 — Stream Processing

Media

Q52. ⋆ Design YouTube / video streaming

- **Requirements:** upload videos, transcode them, stream them smoothly on any device and network, search, comments and views; huge storage and bandwidth.
- **Components:** resumable upload to object storage, a transcoding pipeline (a job graph that splits the video into chunks and encodes them in parallel into many bitrates), metadata database, CDN, adaptive bitrate player (HLS/DASH), view counter.
- **Probe 1:** adaptive bitrate: the player picks the next segment's quality based on its measured bandwidth.
- **Probe 2:** CDN cost: popular videos at the edge, the long tail served from origin or regional caches.
- **Probe 3:** parallel chunk transcoding with retries per chunk.
- **Scale:** storage and egress dominate cost; transcode rarely watched videos to fewer formats.

→ *Full design:* Module 14 — Case 3: Video Platform (YouTube-like) · Module 18 — App 6: Video Streaming App

Q53. Design Netflix

- **Requirements:** a catalog of licensed titles, personalised home rows, playback start under 2 s, global audience with evening peaks.
- **Components:** control plane in the cloud (catalog, accounts, recommendations, playback authorisation and DRM licences) and a data plane of its own CDN appliances placed inside ISPs (Open Connect), filled overnight.
- **Probe 1:** the catalog is known in advance, so push content to caches before demand, unlike YouTube's pull model.
- **Probe 2:** per-title and per-scene encoding to save bandwidth.
- **Scale:** resilience practices (chaos testing, fallback rows when recommendations fail).

→ *Full design:* Module 14 — Case 3: Video Platform (Netflix is discussed in the body)

Q54. Netflix: limit the number of screens each user can watch

Asked at: "Some FAANG"

Requirements. Functional: each plan allows K streams at once (for example 1, 2 or 4); starting stream K+1 is refused with a clear message and an option to stop another device; a stream that stops (closed app, lost network) frees its slot quickly. Non-functional: the check runs on the playback-start path, so it must be fast (under 50 ms) and highly available; small, brief over-allowance is acceptable, while wrongly blocking a paying user is worse.

Estimates. 300M accounts and about 30M concurrent streams at peak. Heartbeats every 60 s give about 500k heartbeats a second, and starts are perhaps 10k a second.

API. `POST /playback/start {account, device, title}` returns a stream lease `{lease_id, ttl}` or `409 LIMIT_REACHED {active_devices}`; `POST /playback/heartbeat {lease_id}`; `POST /playback/stop {lease_id}`.

Data model. Per account, in Redis (or a strongly consistent key-value store): a hash or sorted set of active leases `lease_id -> expires_at`. The plan limit comes from the account service and is cached.

Architecture. The playback service calls a concurrency service before issuing the DRM licence. The concurrency service, in one atomic step (a Lua script or a conditional write), removes expired leases, counts the remaining ones, and adds the new lease if the count is below K. The player heartbeats while playing; licences are short-lived and renewed only with a valid lease, so a client that ignores the rule loses playback.

Deep dives.

> 1. *Leases, not counters:* a plain counter leaks when devices die without saying stop; leases with TTL (about 2 to 3 heartbeat intervals) clean themselves up.
> 2. *Atomicity:* two devices starting at the same moment must not both take the last slot, so check-and-add must be one atomic operation per account.
> 3. *Same device restarting:* reuse the lease when the device ID matches, so a quick restart does not count twice.

Failures and trade-offs. If the store is down, fail open (allow playback) and log it; enforcement resumes when it is back. Sharding by account keeps every check on a single shard.

→ *Building blocks:* Module 0 — Consensus, leader election and distributed locks · Module 8 — Rate Limiting · Module 5 — Graceful Degradation

Q55. Design a live streaming platform (Twitch)

- **Requirements:** a streamer broadcasts; thousands to millions watch with a few seconds of delay; live chat; recordings (VODs).
- **Components:** ingest servers (RTMP or SRT), real-time transcoding into a bitrate ladder, packaging into short HLS segments, CDN distribution, a chat service, recording to object storage.
- **Probe 1:** latency versus stability: shorter segments (or low-latency HLS) cut delay but raise request load and the risk of stalls.
- **Probe 2:** transcoding only popular channels at every bitrate to save cost.
- **Scale:** the CDN fan-out tree; chat is a separate pub/sub system.

→ *Full design:* Module 14 — Case 5: Live Streaming

Q56. Design Spotify (music streaming)

Requirements. Functional: search a catalog of about 100M tracks, stream with near-instant start, playlists (personal and shared), follow artists, offline downloads, personalised mixes, play across devices. Non-functional: playback starts in under 300 ms, no gaps between tracks, licensing rules per country, and royalties need accurate play counts.

Estimates. About 750M monthly users, about 100M listening at peak. At 160 kbps average that is roughly 16 Tbps of egress at peak. The catalog at several bitrates is a few PB. Play events run at about 1M a second at peak.

API. `GET /search?q=`, `GET /tracks/{id}/stream` (returns signed CDN URLs per bitrate), `GET/PUT /playlists/{id}`, `POST /events/plays` (batched), and a connect channel (WebSocket) for controlling playback across devices.

Data model. Catalog (tracks, albums, artists, rights per country) in a relational database plus a search index; playlists in a wide-column store keyed by playlist ID with versioning for collaborative edits; play events in Kafka and then a data lake; user taste vectors in a feature store.

Architecture. Audio files are encrypted and stored in object storage, served through a CDN (popular tracks stay at the edge). The client caches recently played tracks and prefetches the next track before the current one ends. A play-event pipeline feeds royalties (batch, exact) and recommendations (stream plus batch). Recommendation jobs generate weekly and daily playlists offline; home screens are assembled at request time.

Deep dives.

1. *Instant start and gapless play:* fetch the first chunk at low bitrate, then raise quality; prefetch the next track; cache aggressively on the device.
2. *Royalties:* count a play only after about 30 seconds of listening, dedupe events by ID, reconcile daily; this pipeline must be exact, unlike recommendations.
3. *Offline mode:* downloads are encrypted with a key that expires unless the device checks in (for example every 30 days).

Failures and trade-offs. If recommendations fail, show editorial playlists; playback must keep working when everything except auth and the CDN is down.

→ *Building blocks:* Module 8 — CDN · Module 14 — Case 3: Video Platform · Module 10 — Stream Processing · Module 20 — Case 1: Distributed Search · Module 11 — Feature Store

Q57. Design video upload over low bandwidth (resumable upload)

- **Requirements:** upload large files over flaky mobile networks and continue where it stopped, even after an app restart.
- **Components:** an upload session API (create a session, get an ID and chunk size), chunked upload of 5 to 8 MB parts with a checksum each, a session store tracking received parts, then completion that assembles the object (S3 multipart upload works this way).
- **Probe 1:** the client asks "which parts do you have?" after reconnecting and sends only the missing ones.
- **Probe 2:** adapt chunk size to the network; compress or transcode on the device first to cut bytes.
- **Scale:** chunks go straight to object storage through pre-signed URLs; sessions expire after a day or so.

→ *Full design:* Module 14 — Case 4: File Sync and Storage (chunked upload)

Files and collaboration

Q58. ⋆ Design Dropbox / Google Drive (file sync)

Asked at: Dropbox, Facebook, Google, Amazon, Microsoft, OCI

- **Requirements:** upload, download and sync files across devices, share with others, version history, work offline; files up to many GB.

- **Components:** a client sync engine, block service (files split into about 4 MB chunks addressed by their hash), object storage for blocks, metadata service (file tree, versions, block lists) on a strongly consistent database, a notification channel telling devices to pull changes.
- **Probe 1:** chunking plus content hashes give dedup and delta sync: only changed chunks upload.
- **Probe 2:** conflicts: two offline edits produce a "conflicted copy" rather than a silent overwrite.
- **Probe 3:** metadata needs strong consistency; blocks are immutable, so they cache easily.
- **Scale:** shard metadata by user or namespace; long polling or WebSockets for change notifications.

→ *Full design:* Module 14 — Case 4: File Sync and Storage · Module 18 — App 5: File Sync App

Q59. ⋆ Design Google Docs (collaborative editor)

Asked at: Google, Flipkart, Notion, Amazon

- **Requirements:** many people edit the same document at once, see each other's cursors, edits appear within about 100 ms, offline edits merge, full history.
- **Components:** WebSocket collaboration servers, one session owner per document (it orders operations), an operation log plus periodic snapshots, a presence service, document storage.
- **Probe 1, OT versus CRDT:** operational transformation needs a central server to order and transform operations; CRDTs merge without one and suit offline and peer-to-peer use, at the cost of extra metadata.
- **Probe 2:** route all editors of a document to the same server (consistent hashing on document ID).
- **Scale:** snapshot plus log replay for loading; compact old operations.

→ *Full design:* Module 16 — Design 5: Collaborative Document Editor

Q60. Design Google Calendar

Asked at: Google, LinkedIn

Requirements. Functional: create, edit and delete events (one-off and recurring), invite attendees and track RSVPs, view day, week and month, see free/busy for others, reminders, time zones, shared calendars. Non-functional: views load in under 200 ms, edits sync to all devices and attendees within seconds, reminders are never missed and fire within a minute of their time, and recurrence must be correct across daylight-saving changes.

Estimates. 500M users, about 5 events a day each in views, and around 100M event writes a day (about 1.2k a second). Reminders run at about 50M a day, with sharp peaks on the hour and half hour.

API. `POST /calendars/{id}/events {title, start, end, tz, rrule, attendees}`, `GET /calendars/{id}/events?from=&to=`, `PATCH /events/{id}?scope=this|following|all`, `POST /events/{id}/rsvp`, `POST /freebusy {users, range}`.

Data model. Events table: `(event_id, calendar_id, start_utc, end_utc, tz, rrule, organizer, version)`. A recurring event is stored once with an RRULE (the iCalendar recurrence rule, for example "every Tuesday until June") plus exception rows for edited or cancelled instances. Attendees table `(event_id, user_id, rsvp)`. A per-user index of instances expanded for, say, the next year makes range queries fast.

Architecture. Event service on a relational database sharded by calendar or user; an expansion worker that turns RRULEs into concrete instances in the index; an invite service that copies the event to each attendee's calendar view and sends email (iTIP/iCalendar for external users); a free/busy service reading the instance index; a reminder scheduler; sync to devices through push notifications plus incremental sync tokens.

Deep dives.

1. *Recurrence:* store the rule in the event's own time zone, not UTC, so "9 am every Monday" stays at 9 am after daylight saving changes; expand lazily for far-future ranges.
2. *Editing "this and following":* split the series into two rules at that date, so history is untouched.
3. *Reminders:* a sharded delay queue or timing wheel by fire time, with at-least-once delivery and dedup by (event instance, reminder).

Failures and trade-offs. Concurrent edits use optimistic concurrency with the event version; a conflict asks the user. Attendee copies are eventually consistent.

→ *Building blocks:* Module 16 — Design 3: Distributed Job Scheduler · Module 19 — Hierarchical timing wheels · Module 3 — Sharding · Module 2 — Capstone: Designing a Notification System

Q61. Design BitTorrent / P2P file transfer

Asked at: Google, Atlassian, Twitch

Requirements. Functional: share a large file (or a set of files) with many people, where downloaders also upload pieces to each other; verify integrity; resume after interruption; find peers. Non-functional: total throughput grows with the number of peers, the original seeder's bandwidth is not a bottleneck, and it tolerates peers coming and going all the time. Internal variants (Facebook, and Twitter with its BitTorrent-based "Murder" tool, have used P2P to push large binaries to thousands of servers) add speed and trust.

Estimates. A 10 GB file sent to 10,000 machines is 100 TB of transfer. From a single server at 10 Gbps that takes about a day; with P2P, every machine also uploads, so it takes minutes.

API / protocol. A metadata file (`.torrent`) holds the piece size (for example 1 to 4 MB), the SHA-1 or SHA-256 hash of each piece, and tracker addresses. Tracker: `announce(info_hash, peer_id, port, downloaded, left)` returns a list of peers. Peer wire protocol: handshake, `bitfield` (which pieces I have), `have`, `interested`, `choke/unchoke`, `request(piece, offset, length)`, `piece`.

Data model. The tracker keeps `info_hash -> set of (peer, last_seen)` with TTL. Each peer keeps a bitmap of the pieces it holds and verifies every piece against its hash.

Architecture. Tracker (or a trackerless DHT based on Kademlia) for peer discovery; each peer connects to about 30 to 50 others, exchanges bitfields and downloads different pieces from different peers in parallel.

Deep dives.

1. *Rarest first:* request the pieces fewest peers have, so rare pieces spread before their only holder leaves.
2. *Tit-for-tat choking:* upload to the peers that give you the best download rate, plus one "optimistic unchoke" to discover better partners and help newcomers.
3. *Integrity:* per-piece hashes catch corrupt or malicious data; a bad piece is downloaded again from someone else.

Failures and trade-offs. The tracker is a single point of failure, so use several trackers or a DHT. Inside a data centre, add rack awareness so most traffic stays within a rack.

→ *Building blocks:* Module 3 — Consistent Hashing · Module 0 — Probabilistic data structures and Merkle trees · Module 0 — Heartbeats, gossip and failure detection

Q62. Design a file downloader library

Asked at: Facebook

Requirements. Functional: an API that apps call to download files by URL; parallel downloads, pause, resume and cancel; progress callbacks; retries; downloading one file in several ranges at once; optional checksum checks. Non-functional: limits on total connections and bandwidth, no corrupt files written, resume after an app restart, and low memory use for files larger than RAM.

Estimates. On a phone: maybe 3 to 6 concurrent downloads, files up to several GB. On a server: hundreds of concurrent downloads, so the design should not use one thread per download.

API.

```
id = downloader.enqueue(url, destPath, options{priority, checksum, maxRetries, headers})
downloader.pause(id) / resume(id) / cancel(id)
downloader.onProgress(id, (bytesDone, total) -> ...)
downloader.onComplete(id, result) / onError(id, error)
```

Data model. A persistent task table (SQLite on a device): `id, url, dest, state (QUEUED, RUNNING, PAUSED, DONE, FAILED), total_size, etag, segments[{start, end, done}]`. Data is written to a `.part` file and renamed on success.

Architecture. A scheduler with a priority queue and a cap on concurrent tasks; a worker pool (or async I/O event loop) runs segments; each segment does an HTTP `GET` with a `Range: bytes=start-end` header and streams the body to disk at its offset; a progress aggregator throttles callbacks to a few per second; a retry policy with exponential backoff and jitter.

Deep dives.

1. *Resume:* store bytes done per segment; on resume send a Range request with `If-Range: <etag>`; if the server says the file changed (a full 200 response or a different ETag), restart from zero.
2. *Segmented download:* only when the server answers a `HEAD` request with `Accept-Ranges: bytes` and a content length; split into N parts, each written with positioned writes.
3. *Integrity and atomicity:* verify the checksum, then rename `.part` to the final name in one atomic step, so readers never see half a file.

Failures and trade-offs. More segments can mean more speed but also more load on the server; respect `Retry-After` on 429 and 503 responses. Keep the core independent of platform specifics (network, storage, threads) through interfaces, which also makes it testable.

→ *Building blocks:* Module 5 — Retry with Exponential Backoff · Module 5 — Timeout · Module 14 — Case 4: File Sync and Storage

Commerce and payments

Q63. ⋆ Design an e-commerce site (Amazon / BigBasket)

- **Requirements:** catalog, search, product pages, cart, checkout, orders, returns, sellers; browsing always fast and available; checkout never wrong (no money without an order, no overselling deals); sale-day peaks at 10× normal.
- **Numbers:** 2B page views/day × ~30 backend calls per page ≈ 1.5M backend req/s; 5M orders/day ≈ 60/s, thousands/s at deal launches.
- **Components:** CDN and BFF page composition (critical vs optional widgets), always-writable cart, checkout saga (reserve → authorize → durable accept with outbox), inventory per (SKU, warehouse), search fed by CDC.
- **Probe 1:** hot deal stock: split into buckets, claim tokens, waiting room.
- **Probe 2:** a slow recommender must not break the page: deadlines, bulkheads, circuit breakers.
- **Probe 3:** peak-day readiness: forecast, load tests, game days, pre-provisioning, kill switches, change freeze.

→ *Full design:* Module 25 — Case 3: E-Commerce Platform and Peak-Day Readiness

Q64. Design Facebook Marketplace

Asked at: Facebook, Roblox

Requirements. Functional: individuals list items (photos, price, category, location), buyers browse and search nearby listings with filters, message the seller, mark as sold; saved searches with alerts; reporting of scams. Non-functional: local search under 300 ms, new listings searchable within a minute, strong moderation, mostly reads.

Estimates. 1B monthly users, 50M new listings a day (about 600 a second), 5 photos each. Searches at about 50k a second at peak.

API. `POST /listings` (with pre-signed photo uploads), `GET /listings/search?q=&lat=&lng=&radius=&category=&price_max=`, `POST /listings/{id}/messages`, `PATCH /listings/{id} {status: sold}`, `POST /saved-searches`.

Data model. Listings in a sharded store (by listing ID) with status and TTL (auto-expire after 30 to 60 days); a search index with text, category, price and a geo field (geohash); photos in object storage behind a CDN; conversations reuse the messenger stack.

Architecture. Listing service, then a moderation pipeline (ML classifiers for banned items and scam patterns, plus human review), then an indexer (through CDC or events) into the geo-aware search

cluster; a feed or browse service mixing nearby and personalised listings; saved-search matcher; messaging.

Deep dives.

1. *Geo plus text search:* filter by geohash cells covering the radius, then rank by text relevance, distance, freshness and seller rating. Shard the index by region so a query hits only local shards.
2. *Saved-search alerts:* run each new listing against stored queries (a percolator, or a "reverse search" index of saved queries by area and category) instead of re-running every saved query.
3. *Trust and safety:* hold risky listings for review, rate-limit new accounts, and use seller reputation in ranking.

Failures and trade-offs. Index lag means a just-sold item can still appear for a minute; the listing page checks the current status. Moderation before publishing slows sellers, after publishing exposes buyers; a risk score decides which path an item takes.

→ *Building blocks:* Module 15 — Design 1: Proximity Service · Module 20 — Case 1: Distributed Search · Module 20 — Case 4: Photo Sharing · Module 14 — Case 1: Chat System

Q65. Design Shopify

Requirements. Functional: merchants create stores (theme, custom domain, products, inventory), customers browse and check out, apps extend stores through APIs and webhooks, merchants see orders and analytics. Non-functional: many tenants on shared infrastructure, one merchant's flash sale must not hurt others, checkout is highly available and correct, and pages are fast worldwide.

Estimates. Millions of merchants; most are tiny, a few are huge. Black Friday peaks reach tens of thousands of checkouts a minute across the platform, with single merchants spiking 100 times their normal load.

API. Storefront API (product and cart reads), Checkout API, Admin API (products, orders, inventory; REST and GraphQL), webhooks to apps (`orders/create`, etc.), all rate-limited per app and per store.

Data model. Every table has a `shop_id`; shops are grouped into "pods" (cells), each with its own database cluster, cache and workers. A routing table maps shop to pod.

Architecture. Edge and CDN cache rendered storefront pages per shop; the request router finds the pod from the shop's domain; the pod runs the application (the same code everywhere), its own MySQL shards, Redis, and job queues. Checkout talks to payment gateways; webhooks go out through a dispatcher.

Deep dives.

1. *Multi-tenancy with pods:* a pod is a full, isolated copy of the stack for a subset of shops. A noisy shop affects only its pod, and a big shop can be moved to a quieter pod (online data migration).
2. *Flash sales:* queue checkouts with a throttle, keep inventory counters correct with conditional updates, and serve storefront pages from cache.
3. *Extensibility:* apps use public APIs with per-app rate limits (a leaky bucket); webhooks retry with backoff.

Failures and trade-offs. A pod outage hits only its shops. Shared pods are cheaper than one database per merchant, but moving a shop between pods is complex.

→ *Building blocks:* Module 6 — Multi-Region and Cell-Based Architecture · Module 3 — Sharding · Module 8 — Rate Limiting · Module 2 — Webhooks · Module 17 — Design an Event Ticketing and Flash-Sale System

Q66. Design a coupon / promo-code system

Requirements. Functional: marketing creates campaigns with codes (one shared code like `DIWALI20`, or millions of unique single-use codes); rules for eligibility (new users, minimum cart value, categories, region, payment method), discount type (flat, percent with a cap, buy-X-get-Y), validity window, global

budget and per-user limits; apply at checkout; best-offer suggestions. Non-functional: validation under 50 ms on the checkout path, never exceed usage limits even under a burst, and a full audit trail.

Estimates. 10k active campaigns, 100M unique codes; at sale peaks around 5k redemptions a second, with a single viral code taking most of them.

API. `POST /campaigns`, `POST /campaigns/{id}/codes:generate {count}`, `POST /coupons/validate {code, user, cart}` (returns the discount, read-only), `POST /coupons/redeem {code, user, order_id}` (idempotent), `POST /coupons/release {order_id}`.

Data model. `campaigns(id, rules_json, discount, starts, ends, total_limit, per_user_limit)`; `codes(code, campaign_id, status, max_uses, used)`; `redemptions(code, user_id, order_id, status, ts)` with a unique key on `order_id` and on `(campaign, user)` where the per-user limit is 1.

Architecture. Campaign admin service; a rules engine (evaluates eligibility conditions in order, returning the reason when a code fails); a redemption service; a cache of active campaigns and rules on every node; a counter store (Redis) for hot usage counts, reconciled with the database.

Deep dives.

1. *Limits under concurrency:* redeem with an atomic conditional decrement (`used < max_uses`, or Redis `DECR` checked against zero) plus a unique constraint on the per-user row. Reserve at order creation, then confirm on payment success or release on failure or timeout.
2. *Unique code generation:* random codes from an unambiguous alphabet (no O/0, I/1), long enough that guessing is hopeless, pre-generated in bulk and checked for collisions; or an HMAC-signed code that can be checked without a lookup.
3. *Stacking and best offer:* define the order (item discounts, then cart discounts) and stacking groups; to find the best offer, evaluate the eligible campaigns and pick the largest discount.

Failures and trade-offs. If the counter store fails, fail closed for limited campaigns (budget matters) and fail open only for unlimited ones. Abuse controls: per-device and per-payment-instrument limits, and rate-limited validation so codes cannot be brute-forced.

→ *Building blocks:* Module 5 — Idempotency · Module 17 — Design an Event Ticketing and Flash-Sale System · Module 7 — Saga · Module 8 — Rate Limiting

Q67. Design a parts-compatibility feature for an e-commerce site

Asked at: "Some FAANG"

Requirements. Functional: a customer saves their vehicle or device ("2019 Honda City VX", "Dell XPS 13 9310"); product pages show "Fits your vehicle" or "Does not fit"; search and browse can filter to compatible parts; sellers and data teams maintain the fitment data. Non-functional: the fit check adds under 20 ms to a page, the data is large but changes slowly, and wrong "fits" answers cost returns, so accuracy beats coverage.

Estimates. 50k vehicle configurations (make, model, year, trim, engine) and 20M parts, with an average part fitting 200 configurations: about 4B (part, config) pairs. Stored as compressed bitmaps (about 2 bytes per pair), that is roughly 10 to 20 GB for both directions.

API. `GET /garage` / `POST /garage {config}`, `GET /compat?part_id=&config_id=` (returns fits, does not fit or unknown, plus notes), `GET /search?q=&config_id=`.

Data model. A configuration taxonomy as a tree (make, model, year, trim); fitment rules stored at the most general level where they hold ("fits all 2018 to 2021 City trims"), with explicit exclusions; a precomputed index `config_id -> bitmap of part IDs` (Roaring bitmaps) and, the other way, `part_id -> list of config IDs`.

Architecture. Ingest of fitment data from manufacturer catalogs (in the US, the ACES/PIES standards) and seller input, then validation and conflict detection, then the rule store, then a batch job that expands rules into the bitmap index, published to the compatibility service and to the search index as a filterable field.

Deep dives.

1. *Expanding rules:* store compactly as ranges and hierarchy, serve fast from an expanded index; rebuild incrementally when a rule changes.
2. *Search filter:* intersect the query's results with the config's part bitmap, which is fast with bitmap intersection.
3. *Unknown is not "no":* show "check fitment" when no data exists, and learn from returns that cite "did not fit".

Failures and trade-offs. If the compatibility service is down, the page still renders without the badge. Data quality is the real challenge, so add confidence scores and human review for disputed entries.

→ *Building blocks:* Module 20 — Case 1: Distributed Search · Module 10 — MapReduce · Module 4 — Cache-Aside

Q68. Design a flash sale system

- **Requirements:** a limited stock (say 10k units) goes on sale at a fixed time to millions of users; never oversell; stay up; be fair.
- **Components:** a virtual waiting room (a queue with tokens to enter), static pages on the CDN, an inventory counter in Redis decremented atomically, an order queue processed asynchronously, payment with a hold expiry.
- **Probe 1:** reject early: once the counter hits zero, answer "sold out" at the edge.
- **Probe 2:** bots: CAPTCHA, per-account limits, signed and time-limited tokens.
- **Scale:** the hot path is one counter; shard it into buckets if one key is too hot.

→ *Full design:* Module 17 — Design an Event Ticketing and Flash-Sale System

Q69. Design a backend to give away 6M free burgers in one hour

Asked at: Google, Deliveroo

- **Requirements:** exactly 6M vouchers, at most one per person, a huge burst at the start, an hour-long window.
- **Components:** a waiting room at the edge, the claim service, a pre-sharded voucher pool (for example 600 shards of 10k vouchers each, an atomic decrement per shard), a uniqueness check per user (unique key), voucher delivery by email or push.
- **Probe 1:** pre-split the stock so no single counter is hot; route users to random shards and try another when one is empty.
- **Probe 2:** one per person: the identity check (account plus phone or card) matters more than the counter.
- **Scale:** about 1,700 claims a second on average, with 10 to 100 times that in the first minutes, so queue and smooth.

→ *Full design:* Module 17 — Design an Event Ticketing and Flash-Sale System

Q70. ⋆ Design Ticketmaster / BookMyShow

Asked at: Meta, Flipkart

- **Requirements:** browse events and shows, see a seat map, hold seats for a few minutes, pay, get a ticket; no double booking; survive huge on-sale spikes.
- **Components:** event and venue catalog, seat inventory service, a hold service (TTL holds), booking and payment service, waiting room, ticket delivery.
- **Probe 1:** holding seats: a conditional update `status = HELD WHERE status = AVAILABLE` (or a Redis key with TTL) so only one user wins each seat.
- **Probe 2:** hold expiry and release if payment fails; payment idempotency.
- **Probe 3:** the waiting room lets in only as many users as inventory can handle.
- **Scale:** shard by event; seat maps are cached and seat status is pushed or polled.

→ *Full design:* Module 17 — Design an Event Ticketing and Flash-Sale System (Ticketmaster-like)

Q71. ⋆ Design a hotel booking system (Airbnb / Booking.com)

Asked at: Amazon, Square, Booking.com

- **Requirements:** search availability by place and dates, book a room type, pay, cancel; no overbooking (or controlled overbooking).
- **Components:** search service (index plus availability cache), inventory table per (hotel, room type, date) with total and reserved counts, reservation service, payment.
- **Probe 1:** concurrency: optimistic locking with a version column, or a database constraint `reserved <= total`.
- **Probe 2:** idempotent reservation requests so a double click does not book twice.
- **Scale:** search is read-heavy and cacheable; bookings are low-volume but must be correct; shard by hotel.

→ *Full design:* Module 17 — Design a Hotel Reservation System · Module 18 — App 4: Hotel Reservation App

Q72. Design a flight booking system

- **Requirements:** search flights (with connections) across airlines, hold a seat, pay, issue a ticket.
- **Components:** search over fares cached from global distribution systems (GDS) or airline APIs, an itinerary builder, a booking service that creates a reservation (PNR) with the airline, payment, ticketing.
- **Probe 1:** fares are volatile and expensive to query: cache searches briefly and re-price before payment.
- **Probe 2:** booking spans external systems, so run it as a saga (hold, then pay, then ticket, with cancellation as compensation).
- **Scale:** the search cache absorbs the "look-to-book" ratio (thousands of searches per booking).

→ *Full design:* Module 17 — Design a Hotel Reservation System (same reservation pattern)

Q73. ⋆ Design a payment system / payment gateway (Stripe, Razorpay)

Asked at: Stripe, Paytm, PayPal, Databricks, Capital One

- **Requirements:** accept payments by card, UPI or wallet, authorise and capture, refund; never charge twice; full audit; payouts to merchants.
- **Components:** payment API, payment state machine (created, authorised, captured, refunded, failed), PSP and card-network connectors, a double-entry ledger, risk and fraud service, reconciliation jobs, webhooks to merchants.
- **Probe 1:** idempotency keys on every request and on calls to PSPs.
- **Probe 2:** double-entry ledger: every movement is a debit and a credit that sum to zero; never update balances in place.
- **Probe 3:** reconciliation against PSP and bank settlement files to catch mismatches.
- **Scale:** shard by merchant; the outbox pattern for reliable events.

→ *Full design:* Module 12 — Design 3: The Payment Platform · Module 5 — Capstone: Hardening a Payment Flow · Module 17 — How Money Moves (primer)

Q74. Design a digital wallet

- **Requirements:** hold balances, top up, transfer between users, pay merchants, show history; balances must never be wrong.
- **Components:** wallet API, ledger (event-sourced account entries), balance projection, transfer orchestrator, top-up through payment gateways.
- **Probe 1:** transfers across shards: a saga (debit, then credit, with compensation) or a single-shard transaction when both accounts live together.
- **Probe 2:** event sourcing plus snapshots for auditability and rebuilding balances.
- **Scale:** partition by account; hot merchant accounts get sub-accounts.

→ *Full design:* Module 17 — Design a Digital Wallet

Q75. Design UPI (Unified Payments Interface)

- **Requirements:** instant bank-to-bank transfers by a virtual payment address (VPA, like `name@bank`), from any app, at very high volume.

- **Components:** payer app and its payment service provider (PSP) bank, the NPCI central switch, the payer's (issuer) bank and the payee's bank, VPA resolution by the PSP that owns the handle.
- **Probe 1:** the flow: the payer PSP sends the request to NPCI, which resolves the payee, asks the issuer bank to debit (the PIN is captured in a secure library and only the issuer bank can verify it), then asks the payee bank to credit.
- **Probe 2:** timeouts and "deemed" or pending states, resolved by status checks and reconciliation; an automatic reversal if the credit fails.
- **Scale:** the central switch plus idempotent transaction IDs end to end.

→ *Full design:* Module 17 — How Money Moves (primer)

Q76. Design a wire transfer API

Asked at: Google, Capital One, Revolut

- **Requirements:** a customer sends money to an external bank account; the API is idempotent; status updates over hours or days; compliance checks.
- **Components:** transfer API, a state machine (created, screened, funds held, submitted, settled, failed or returned), sanctions and fraud screening, a ledger hold on the source account, a connector to the payment rail (SWIFT, ACH, SEPA), webhooks for status.
- **Probe 1:** hold funds first, then release or post them when the rail confirms.
- **Probe 2:** the rail is slow and asynchronous: poll or receive files and messages, then reconcile.
- **Scale:** volume is modest; correctness and audit dominate.

→ *Full design:* Module 17 — Design a Digital Wallet (holds and ledger)

Q77. Design Mint.com

Requirements. Functional: users link bank and card accounts; the service imports transactions daily, categorises them (groceries, rent), shows budgets and trends, and alerts on overspending or unusual charges. Non-functional: bank credentials and data are highly sensitive, data is fresh within a day (hours preferred), imports run in bulk, reads are light and per user.

Estimates. 10M users with 4 linked accounts each is 40M accounts. 5 new transactions per account per day is 200M transactions a day to import and categorise, almost all in batch.

API. `POST /links` (starts an OAuth or aggregator flow), `GET /transactions?from=&category=`, `PATCH /transactions/{id} {category}`, `POST /budgets`, `GET /reports/monthly`.

Data model. `accounts(id, user, institution, token_ref, last_sync)`; `transactions(id, account, date, amount, merchant_raw, merchant_norm, category, source_id)` with a unique key on `(account, source_id)` so re-imports do not duplicate; `budgets(user, category, monthly_limit)`; monthly spend aggregates per (user, category).

Architecture. Link service (uses a data aggregator such as Plaid, or open-banking APIs, and stores tokens in a secrets vault, never passwords in the application database); a sync scheduler that spreads refreshes across the day per institution, with rate limits; import workers that write to a queue; a categorisation service (merchant normalisation, rules, then an ML classifier, plus user overrides that also train the model); an aggregation job; budget and alert evaluators; notifications.

Deep dives.

1. *Import idempotency:* banks re-send and amend transactions (pending, then posted); dedupe on source ID and update in place.
2. *Categorisation:* first normalise merchant strings ("AMZN Mktp IN*2X3" becomes Amazon), then apply user rules, then the model; a user's correction applies to their future transactions from that merchant.
3. *Aggregates:* precompute per-month, per-category sums, updated incrementally on import, so dashboards are cheap.

Failures and trade-offs. Bank APIs are flaky, so retry per institution with backoff and show "last updated" times. Security: encryption at rest, scoped read-only tokens, audit logging.

→ *Building blocks:* Module 10 — MapReduce · Module 16 — Design 3: Distributed Job Scheduler · Module 5 — Idempotency · Module 0 — Encryption, hashing, passwords and secrets

Q78. ⋆ Design a stock exchange (NASDAQ / Binance)

- **Requirements:** accept orders (limit, market, cancel), match them fairly with price-time priority, publish trades and market data, with microsecond-level latency and strict ordering.
- **Components:** gateway, risk checks, a sequencer that stamps every order with a global order, an in-memory single-threaded matching engine per symbol (order book), market data publisher, reporting and settlement.
- **Probe 1:** why single-threaded: deterministic and lock-free; replicas replay the same sequenced input for failover.
- **Probe 2:** the order book: price levels in a sorted map with a FIFO queue of orders at each level.
- **Probe 3:** market data fan-out over multicast or many gateways.
- **Scale:** partition by symbol; the event log enables recovery by replay.

→ *Full design:* Module 17 — Design a Stock Exchange · Module 18 — App 3: Stock Trading App

Q79. Design a price alert system (Amazon prices / stock prices)

Asked at: Facebook, Bloomberg, Coinbase, Swyftx, Trade Republic

- **Requirements:** users set "notify me when X crosses Y"; alerts fire once, quickly, for millions of alerts.
- **Components:** alert store; per-symbol (or per-product) sorted structures of thresholds; a price stream; a matcher that, on each price tick, finds the crossed thresholds by range query; the notification service.
- **Probe 1:** index alerts by threshold so each tick is a range scan between the old and new price, not a scan of all alerts.
- **Probe 2:** exactly one notification: mark the alert as fired atomically before sending.
- **Scale:** partition by symbol; product prices change rarely, so they can be checked in batch.

→ *Full design:* Module 18 — App 3: Stock Trading App

Q80. Design a system to view the latest stock prices worldwide

Asked at: Google, Bloomberg, Amazon

- **Requirements:** millions of viewers see live prices for the symbols they watch, with sub-second freshness in every region.
- **Components:** exchange feed handlers, normalisation, a global pub/sub backbone, regional edge servers holding WebSocket or SSE connections, a latest-price snapshot cache per symbol.
- **Probe 1:** conflation: send the newest price at most a few times a second per symbol instead of every tick.
- **Probe 2:** a new subscriber gets the snapshot first, then deltas.
- **Scale:** two-level fan-out (backbone to regions, then regions to clients), the same as live comments.

→ *Full design:* Module 17 — Design a Stock Exchange (market-data fan-out)

Infrastructure

Q81. ⋆ Design a system that scales to millions of users (on AWS)

- **Requirements:** start from one server and grow step by step to millions of users, explaining what breaks at each stage.
- **Components, in order of adoption:** separate the database; a load balancer plus several stateless app servers; read replicas; a cache; a CDN; async work through queues; database sharding; multi-region.
- **Probe 1:** keep app servers stateless (sessions in Redis or tokens) so autoscaling works.
- **Probe 2:** the database is the usual bottleneck: replicas for reads, caching, then sharding by a well-chosen key.
- **Scale:** measure first, then fix the current bottleneck; do not jump straight to microservices.

→ *Full design:* Module 6 — Capstone: From 100 to 1,000,000 Requests per Second · Module 3 — Capstone: Scaling a Database to 100 Million Users

Q82. Design a system that serves data from multiple data centers

- **Requirements:** low latency for users everywhere, survive the loss of a whole region, a clear consistency story.
- **Components:** GeoDNS or anycast routing, a full stack per region, data replication between regions, a global control plane.
- **Probe 1:** active-passive (simple, slower failover) versus active-active (needs conflict handling or partitioning by user's home region).
- **Probe 2:** synchronous cross-region writes cost about 100 ms or more each; most designs replicate asynchronously and accept a small loss window (RPO).
- **Scale:** cell-based architecture limits the blast radius of any failure.

→ *Full design:* Module 6 — Multi-Region and Cell-Based Architecture

Q83. Design S3-like object storage

- **Requirements:** store and fetch immutable objects by bucket and key, from bytes to terabytes, with 11 nines of durability and cheap storage.
- **Components:** API front ends, a metadata service (bucket and key to object location, versions), data nodes storing chunks, a placement service, background repair and garbage collection.
- **Probe 1:** replication (3 copies, simple) versus erasure coding (for example 8 data plus 4 parity chunks: 1.5× storage instead of 3×, and it survives any 4 lost chunks instead of 2, but repairs read more data and cost more CPU).
- **Probe 2:** multipart upload for big objects; listing by prefix needs a sorted metadata index.
- **Scale:** metadata is the hard part: shard it by bucket and key range.

→ *Full design:* Module 16 — Design 1: S3-like Object Storage

Q84. ⋆ Design metrics monitoring and alerting (collect metrics from thousands of servers)

Asked at: Google, Datadog, Amazon, eBay, LinkedIn

- **Requirements:** collect time-series metrics, query dashboards, evaluate alert rules every minute, keep a year of history.
- **Components:** agents or exporters; collection (pull, Prometheus-style, or push through Kafka); a time-series database with compression; a query service; an alert evaluator; a notifier with dedup and routing.
- **Probe 1:** pull versus push (pull detects dead targets easily; push suits short-lived jobs).
- **Probe 2:** cardinality: each label combination is a new series; unbounded labels such as user ID blow up storage.
- **Probe 3:** downsampling old data (for example 1-minute, then 1-hour rollups).
- **Scale:** shard by series hash; the alerting path must not depend on the system it monitors.

→ *Full design:* Module 16 — Design 2: Metrics Monitoring & Alerting

Q85. Design a system to monitor the health of a cluster

Asked at: Uber, Lacework, Amazon, Google

- **Requirements:** detect failed or unhealthy nodes within seconds, avoid false alarms, show cluster status.
- **Components:** heartbeats or gossip between nodes, a failure detector (phi-accrual gives a suspicion level instead of a yes/no timeout), health-check endpoints, metric collection, an aggregator and a dashboard.
- **Probe 1:** central monitor (simple, but a single point of failure) versus gossip such as SWIM (scales, no single point).
- **Probe 2:** "dead" versus "slow" versus "partitioned": confirm from several observers before acting.
- **Scale:** hierarchical aggregation by rack and zone.

→ *Full design:* Module 16 — Design 2: Metrics Monitoring & Alerting · Module 19 — Failure detection: heartbeats, phi-accrual and SWIM

Q86. Design an on-call escalation system (PagerDuty)

Asked at: Uber

Requirements. Functional: receive alerts from monitoring tools; group duplicates into incidents; find who is on call from rotation schedules; notify by push, SMS, phone call and email; escalate to the next level if nobody acknowledges within N minutes; acknowledge and resolve; overrides and handoffs. Non-functional: extremely high availability (it must work during everyone else's outage), notification within seconds, never drop an alert, never page twice for the same incident.

Estimates. 50k customer organisations; normally about 1k alerts a second, but a large outage can produce 100 times that in a minute, most of them duplicates.

API. `POST /v1/events {routing_key, dedup_key, severity, summary, action: trigger|resolve}`; `POST /incidents/{id}/ack`; schedule and escalation-policy CRUD.

Data model. Services, each with an escalation policy (levels: who to notify and timeouts); schedules (rotation layers plus overrides) that resolve "who is on call at time T"; incidents `(id, service, dedup_key, state, level, next_escalation_at)` with a unique open incident per `(service, dedup_key)`; a notification log.

Architecture. A multi-region event intake that writes to a durable queue before answering; a dedup and grouping service; the incident state machine (triggered, acknowledged, resolved); an escalation scheduler (a durable delay queue keyed on `next_escalation_at`); a notification dispatcher with several SMS, voice and push providers; mobile apps for acknowledging.

Deep dives.

1. *Escalation timers:* every incident has a scheduled check; on firing, if it is still unacknowledged, notify the next level and schedule the next check. The update is conditional on state, so an ack that races the timer wins cleanly.
2. *Dedup:* the `dedup_key` folds alert storms into one incident; related incidents can be grouped further.
3. *Reliability:* active-active across regions; provider failover; the system's own monitoring runs on separate infrastructure.

Failures and trade-offs. Sending a duplicate page is better than missing one, so delivery is at-least-once with dedup at the incident level, never by dropping attempts.

→ *Building blocks:* Module 16 — Design 3: Distributed Job Scheduler · Module 2 — Capstone: Designing a Notification System · Module 6 — Multi-Region and Cell-Based Architecture · Module 0 — Dashboards, alerting, on-call and SLO-based alerts

Q87. ⋆ Design a distributed logging system (Splunk-like)

Asked at: Google, Facebook, Amazon, eBay, Datadog, Atlassian

- **Requirements:** collect logs from thousands of services, search recent logs within seconds, keep months of history cheaply, alert on patterns.
- **Components:** agents (tail files, batch, compress), Kafka as a buffer, parsers, an index tier (Elasticsearch-style) for hot data, object storage for cold data, a query service, alerting.
- **Probe 1:** index everything (fast, expensive) versus index only labels and scan the content (Loki-style, cheap).
- **Probe 2:** backpressure: the buffer absorbs spikes; sample or drop debug logs under load.
- **Scale:** hot, warm and cold tiers with retention per tier; partition by tenant and time.

→ *Full design:* Module 20 — Case 2: Distributed Logging and Error Monitoring

Q88. Design a distributed tracing system (Zipkin)

Asked at: Uber, Amazon

- **Requirements:** follow one request across many services and show where time was spent.
- **Components:** instrumentation libraries that propagate trace context (W3C `traceparent` header with trace ID and span ID), span reporting to collectors, storage keyed by trace ID, a UI showing the span tree as a timeline.
- **Probe 1:** sampling: head-based (decide at the start, cheap) versus tail-based (keep slow or failed traces, needs buffering).

- **Probe 2:** clock skew between hosts when drawing the timeline.
- **Scale:** collectors behind a queue; short retention for full traces.

→ *Full design:* Module 9 — Distributed Tracing

Q89. ⋆ Design a distributed job scheduler

Asked at: Google, Amazon, Microsoft, DoorDash, Netflix, Atlassian, Flipkart

- **Requirements:** schedule one-off and cron jobs, run each due job at least once (ideally once) on a worker fleet, retries, visibility into status.
- **Components:** job definition store, a scheduler that finds due jobs (partitioned by time bucket or job ID), a queue, workers that take leases on jobs, an execution-history store.
- **Probe 1:** avoiding double runs: a lease with a fencing token plus idempotent jobs; exactly-once is not realistic.
- **Probe 2:** finding due jobs efficiently: an index on next run time, or timing wheels.
- **Probe 3:** a worker dies mid-job: the lease expires and the job is retried.
- **Scale:** shard schedulers by job ID range, with leader election per shard.

→ *Full design:* Module 16 — Design 3: Distributed Job Scheduler

Q90. Design a system for sorting large data sets

Asked at: Google, Microsoft

- **Requirements:** sort data far larger than one machine's memory (terabytes to petabytes).
- **Components:** on one machine, an external merge sort (sort chunks that fit in memory, write sorted runs, then a k-way merge with a heap); across machines, a MapReduce-style sort: sample keys to choose range boundaries, shuffle each record to its range's reducer, sort locally, and concatenate the outputs in order.
- **Probe 1:** sampling avoids skewed partitions.
- **Probe 2:** stragglers and failures: speculative execution and re-running failed tasks.
- **Scale:** the network shuffle is the cost; compress and combine.

→ *Full design:* Module 10 — MapReduce

Q91. Design a system to migrate large data to the cloud

Asked at: Google, OCI

Requirements. Functional: move petabytes of files and database tables from an on-premises data centre to cloud storage and databases; keep the source live during the move; verify that nothing was lost or corrupted; cut over with minimal downtime. Non-functional: limited network bandwidth, resumable after failures, no impact on production performance, encryption in transit.

Estimates. 2 PB over a dedicated 10 Gbps link at 80 % use is about 2 × 10^15 × 8 / (8 × 10^9) = 2 × 10^6 s, roughly 23 days. If that is too slow, ship physical storage for the bulk (for example Google Transfer Appliance, or AWS Data Transfer Terminal now that Snowball is closed to new customers) and use the network only for changes.

API. A control-plane API: `POST /migrations {source, target, filters, bandwidth_cap}`, `GET /migrations/{id}/progress`, `POST /migrations/{id}/cutover`.

Data model. A manifest per migration: every object or table chunk with size, checksum, state (pending, copying, verified) and attempts. This makes the job resumable and auditable.

Architecture. A scanner builds the manifest; a scheduler hands chunks to many parallel transfer agents near the source; agents throttle to the bandwidth cap, upload in parts and record checksums; a verifier compares source and target checksums. For databases: an initial bulk copy, then change data capture (CDC) streams changes until the target catches up, then a short write freeze and cutover.

Deep dives.

1. *Bulk plus delta:* copy a snapshot, then repeatedly copy what changed (by modification time or CDC) until the remaining delta is small enough for a brief cutover window.
2. *Verification:* per-chunk checksums plus row counts and sample queries for databases.

3. *Cutover and rollback:* dual-write or reverse replication for a while, so you can go back if the cloud side misbehaves.

Failures and trade-offs. Agents retry chunks idempotently; the manifest prevents gaps and duplicates. More parallelism finishes sooner but can hurt production, so schedule heavy transfers off-peak.

→ *Building blocks:* Module 9 — Capstone: The Zero-Downtime Migration · Module 10 — Change Data Capture · Module 10 — Backpressure · Module 5 — Idempotency

Q92. Design a control plane for a distributed database

Asked at: Netflix

Requirements. Functional: create and delete database clusters; add and remove nodes; rebalance shards; run rolling upgrades and config changes; detect failures and replace nodes; backups and restores; expose cluster state to clients (which node owns which shard). Non-functional: never lose data during operations, the data plane keeps serving even if the control plane is down, every change is auditable and can be paused, and it manages thousands of clusters.

Estimates. 1,000 clusters with 6 to 100 nodes each is about 30k nodes. Node failures happen several times a day across the fleet; operations take minutes to hours.

API. Declarative: `PUT /clusters/{id} {version, node_count, instance_type, replication_factor}`; read status with `GET /clusters/{id}`; operations are resources: `GET /operations/{id}`.

Data model. Desired state per cluster (what the user asked for), observed state (reported by node agents), the shard map with an epoch number, and an operation log with step-by-step progress.

Architecture. A consensus-backed metadata store (etcd or ZooKeeper) holds desired state and the shard map. Reconciler loops compare desired with observed and plan steps (the Kubernetes operator pattern). A workflow engine runs long, resumable operations. Node agents report health and perform local actions. Clients and routers watch the shard map.

Deep dives.

1. *Safe rolling changes:* one node or one availability zone at a time, check health and replication lag before moving on, stop automatically when error rates rise.
2. *Shard moves:* copy data to the new node, catch up through the log, then flip ownership by bumping the shard-map epoch; old owners reject requests carrying an older epoch (a fencing token).
3. *Failure replacement:* confirm a failure from several signals before acting, and rate-limit automated replacements so a network blip does not trigger mass rebuilds.

Failures and trade-offs. Keep the control plane off the data path, so its outage only pauses changes. Automation speeds recovery but can amplify mistakes, hence the rate limits and a pause switch.

→ *Building blocks:* Module 0 — Consensus, leader election and distributed locks · Module 9 — Containers and Kubernetes · Module 3 — Consistent Hashing · Module 19 — Leases and lease-based leader election

Q93. Design a configuration management system

Requirements. Functional: store configuration per service and environment (dev, staging, prod) with optional per-region overrides; version every change; validate against a schema; review and approve changes; roll out gradually; roll back in one click; services receive changes without a restart. Non-functional: reads never block service startup (services must start with the last known config if the system is down), propagation within seconds, full audit.

Estimates. 5k services and 100k instances, each watching a few config keys; changes are rare (thousands a day) but every instance must see them quickly. Reads are served from local copies, so the total load is mostly watch connections.

API. `PUT /configs/{service}/{env} {data, schema_version}` (creates a new version, pending approval); `POST /configs/.../versions/{v}/rollout {stages}`; `POST .../rollback {to_version}`; a client SDK: `get(key)` and `onChange(key, callback)`.

Data model. Immutable versions `(service, env, version, data, author, approved_by, created_at)` and a "current pointer" per (service, env, rollout stage); secrets referenced by a pointer to a secret store, never

stored inline.

Architecture. An admin UI and API with schema validation and approvals (often backed by Git as the source of truth); a consistent store for the current pointers (etcd or a database); a distribution layer that pushes changes (watch streams or long polling) to a sidecar or SDK on every instance; a local file cache on each host.

Deep dives.

1. *Safe rollout:* release to canary instances first, watch error metrics, then widen; roll back automatically if health drops.
2. *Push versus pull:* long-polling or watch streams for speed, plus periodic full syncs to repair missed updates.
3. *Static versus dynamic config:* some settings need a restart; mark them and handle them through deploys instead.

Failures and trade-offs. The SDK always serves the last good local copy, so a config outage cannot take services down. A bad config is a common outage cause, so validation and staged rollout matter more than propagation speed.

→ *Building blocks:* Module 9 — Feature Flags · Module 9 — Canary Deployment · Module 8 — Sidecar · Module 0 — Consensus, leader election and distributed locks

Q94. Design a garbage collection system

This one is outside the book's scope (it is a language-runtime topic, not a distributed-systems one), so here is a short answer. It sometimes also means "distributed GC" of unreferenced data, which is covered at the end.

Requirements. Functional: automatically free heap memory that the program can no longer reach. Non-functional: short pauses (latency), high throughput (little CPU spent on GC), and low memory overhead; you can usually optimise two of these three at the expense of the third.

Core ideas.

- *Reachability:* start from roots (stack variables, globals, registers) and mark everything reachable; the rest is garbage. Reference counting frees objects immediately but cannot free cycles without extra help.
- *Mark-sweep* frees unmarked objects but leaves the heap fragmented; *mark-compact* slides live objects together; *copying* collectors move live objects into a fresh space (fast allocation, but only half the space is usable).
- *Generational hypothesis:* most objects die young. A small young generation is collected often and cheaply by copying; survivors are promoted to an old generation collected rarely. A write barrier records old-to-young pointers (a card table) so a young collection does not scan the old generation.
- *Concurrent and incremental collection:* mark while the program runs, using tri-colour marking (white, grey, black) and barriers so the program cannot hide a live object from the collector. Java's G1 splits the heap into regions; ZGC and Shenandoah also compact concurrently, keeping pauses to about a millisecond or less.

Deep dives an interviewer may push on. Why stop-the-world pauses exist and how to shorten them; how the collector finds roots safely (safepoints); how heap size trades memory for fewer collections.

Distributed variant. For "delete blobs nothing references" (object stores, Git, content-addressed storage), run a periodic mark phase over the metadata, then sweep after a grace period, so uploads in progress are not deleted.

→ *Building blocks:* Module 16 — Design 1: S3-like Object Storage (background garbage collection of chunks)

Developer and AI platforms

Q95. Design an online judge (LeetCode)

Asked at: Flipkart

- **Requirements:** submit code in many languages, run it against hidden tests in a sandbox, return a verdict in seconds, run contests with a live leaderboard.
- **Components:** submission API, a queue (priority for contests), judge workers running each submission in an isolated sandbox (containers with seccomp and cgroups, or microVMs such as Firecracker), a test-data store, a results store, a leaderboard.
- **Probe 1:** security: no network, CPU, memory and process limits, read-only filesystem.
- **Probe 2:** fairness: pinned CPUs and repeatable timing.
- **Scale:** autoscale workers ahead of contest start; queue depth drives scaling.

→ *Full design:* Module 21 — Case 1: Online Judge (LeetCode-like)

Q96. Design an online code editor / CodePair

Requirements. Functional: an interviewer and a candidate share a code editor in the browser; both see each other's edits and cursors live; run code in 30+ languages and see output; chat or video; replay the session afterwards. Non-functional: edits appear within about 100 ms, running code is fully sandboxed, a session survives a page reload, and code from one session can never reach another.

Estimates. 50k concurrent sessions at peak with 2 to 4 people each; about 5 edit operations a second per active typist, so about 250k operations a second overall; code runs at about 1 per session per minute, roughly 1k a second at peak.

API. `POST /pads` (returns a pad ID and invite link), a WebSocket `/pads/{id}/collab` carrying operations and presence, `POST /pads/{id}/run {language, code, stdin}` (returns a run ID, output streamed back), `GET /pads/{id}/playback`.

Data model. Pad metadata (owner, language, participants); an operation log per pad (for replay and recovery) with periodic snapshots; run records (code hash, output, exit code, duration).

Architecture. The editor client runs an OT or CRDT engine. Collaboration servers own each pad (routed by consistent hashing on the pad ID) and order and broadcast operations. An execution service queues runs onto sandboxed workers (microVMs or gVisor containers with CPU, memory and time limits and no network), streaming stdout back over the socket. Optional language servers give autocomplete.

Deep dives.

1. *Real-time sync:* operational transformation with a server-side sequencer per pad, or a CRDT such as Yjs; presence (cursors, selections) is sent separately and is never stored.
2. *Sandboxed execution:* pre-warm a pool of sandboxes per language to cut start time to under a second; destroy each sandbox after the run.
3. *Playback:* the operation log with timestamps lets you replay the whole session keystroke by keystroke.

Failures and trade-offs. If a collaboration server dies, clients reconnect to the new owner, which rebuilds from snapshot plus log; unacknowledged local edits are re-sent. Execution is the costly part, so rate-limit runs per pad.

→ *Building blocks:* Module 16 — Design 5: Collaborative Document Editor · Module 21 — Case 1: Online Judge · Module 2 — Bidirectional Streaming

Q97. Design a code deployment system

- **Requirements:** build artifacts once and deploy them to thousands of machines across regions, safely, with fast rollback.
- **Components:** build service producing immutable, versioned artifacts; an artifact store with regional replicas (or P2P distribution); a deployment orchestrator; agents on hosts; health gates.
- **Probe 1:** rollout strategy: canary, then waves by zone and region, with automatic rollback on bad metrics.
- **Probe 2:** distributing large artifacts fast: regional caches or P2P.
- **Scale:** the orchestrator stores deployment state durably so it can resume after its own failure.

→ *Full design:* Module 21 — Case 2: Global Deployment System · Module 9 — Canary Deployment

Q98. Design ChatGPT / a conversational AI service

- **Requirements:** multi-turn chat with streamed answers, conversation history, file and tool use, fair sharing of scarce GPUs across users.
- **Components:** API gateway with per-user token-based rate limits, a conversation service and store, a prompt builder (history truncation or summarisation, retrieval), an LLM gateway routing to model-serving clusters, streaming over SSE, safety filters.
- **Probe 1:** GPU efficiency: continuous batching, KV-cache reuse (prompt caching), and routing by model and context length.
- **Probe 2:** streaming tokens back with SSE, and resuming a dropped stream.
- **Probe 3:** queuing and priority under overload: shed or degrade free-tier traffic first.
- **Scale:** GPU capacity is the limit; autoscale on queue depth and tokens per second.

→ *Full design:* Module 21 — Case 3: Conversational AI Service (ChatGPT-like) · Module 11 — Capstone: A Production RAG Chatbot at Scale

Q99. Design an A/B testing system (Optimizely)

Asked at: Affirm

- **Requirements:** define experiments with variants and traffic splits, assign users consistently, log exposures, compute results with statistical significance.
- **Components:** experiment config service, an assignment SDK (hash of user ID plus experiment ID into buckets; deterministic, so no lookup is needed), exposure event logging, a metrics pipeline, a stats engine.
- **Probe 1:** consistent assignment without a database: hashing; layers keep concurrent experiments independent.
- **Probe 2:** log an exposure only when the user actually sees the variant; check that the observed split matches the planned one (sample-ratio mismatch).
- **Scale:** assignment is local in the SDK; the analytics pipeline does the heavy lifting.

→ *Full design:* Module 9 — Feature Flags (assignment and rollout; the stats pipeline is the add-on)

Q100. Design a recommendation system

Requirements. Functional: recommend items (products, videos, articles) on the home page, "because you watched X" rows and "similar items" rows, personalised for each user; react to recent behaviour; support new users and new items. Non-functional: under 100 ms to serve, millions of users and items, measurable impact through A/B tests.

Estimates. 100M users and 10M items. Ranking every item for every request is impossible (10M scores in 100 ms), so the design must narrow the candidates first.

API. `GET /recs?user=&surface=home&count=50` (returns item IDs with a reason), `GET /items/{id}/similar`, `POST /events` (views, clicks, purchases, dwell time).

Data model. An interaction log (user, item, event, time) in Kafka and the data lake; user and item embeddings; a feature store (online for serving, offline for training) with user features (recent categories, activity) and item features (popularity, freshness); precomputed "similar items" lists.

Architecture. A two-stage (or three-stage) funnel. 1) *Candidate generation:* several sources, each returning hundreds of items, such as approximate nearest-neighbour search on embeddings (HNSW), item-to-item co-occurrence ("people who bought X bought Y"), trending, and follows. 2) *Ranking:* a model (gradient-boosted trees or a neural network) scores about 1,000 candidates using features from the feature store and predicts click or watch probability. 3) *Re-ranking:* business rules, diversity, freshness and removing items already consumed. Training runs offline (daily) with near-real-time feature updates.

Deep dives.

1. *Cold start:* new users get popular and contextual (location, device) items and onboarding choices; new items use content features (text, image embeddings) until interactions build up.
2. *Training-serving skew:* compute features once in the feature store and use the same values for training and serving.

3. *Feedback loops:* the system only learns from what it showed, so reserve a little exploration traffic.

Failures and trade-offs. If the ranker times out, return candidates ordered by a simple score; keep a cached fallback list per user. Precomputing per-user lists is cheap to serve but stale; real-time ranking is fresh but costly.

→ *Building blocks:* Module 11 — Feature Store · Module 11 — Model Serving (and Batching) · Module 19 — Vector search with HNSW, IVF and product quantization · Module 19 — Feed ranking · Module 10 — Lambda and Kappa

Q101. Design a dependency injection / IoC framework

Asked at: ADP

- **Requirements:** register components, resolve their dependencies automatically, manage lifetimes (singleton, prototype, request), detect cycles.
- **Components:** a registry of bindings (interface to implementation or factory), a resolver that builds the dependency graph by reading constructor parameters through reflection or annotations, scope managers, lifecycle hooks (init and destroy).
- **Probe 1:** circular dependencies: detect with a "currently creating" set during a depth-first resolve; fail fast, or allow setter injection to break the cycle.
- **Probe 2:** resolve when the container starts (fail fast, slower startup) or lazily on first use.
- **Scale:** cache the resolved graph; compile-time DI (generated code) avoids runtime reflection.

→ *Full design:* LLD book, Chapter 11 — 11.7.2 Dependency Injection (DI)

Other

Q102. Design a weather application

Asked at: Amazon, Chime, Facebook, HubSpot, Uber, Klaviyo

Requirements. Functional: current conditions and hourly and 10-day forecasts for a location (GPS or search); weather maps (radar); severe-weather alerts pushed to users in affected areas; favourite locations. Non-functional: very read-heavy, fast (under 200 ms), data a few minutes old is fine, and alerts must arrive quickly.

Estimates. 50M daily users opening the app 3 times a day is 150M reads a day (about 2k a second, 10k+ at morning peaks). But the data space is small: global forecast models run on grids of roughly 10 km, so about 1.5M cells cover all land, with forecasts updated every 15 to 60 minutes.

API. `GET /weather?lat=&lng=` (rounded to the grid cell), `GET /forecast?cell=&days=10`, `GET /tiles/radar/{z}/{x}/{y}.png`, `POST /subscriptions {locations}` for alerts.

Data model. Weather per grid cell `(cell_id, valid_time, temp, precipitation, wind, ...)` in a key-value or time-series store; radar map tiles in object storage; alerts as polygons with severity and expiry; user subscriptions indexed by cell.

Architecture. Ingestion pulls from providers (national weather services, commercial APIs, and model outputs such as GFS or ECMWF) on their schedules; a processing step normalises units, interpolates onto the grid and renders map tiles; results go to the store and a cache. The read API rounds coordinates to a cell, so requests share cache entries, and the CDN caches responses by cell with a short TTL. An alert service matches incoming alert polygons against subscribed cells and calls the notification service.

Deep dives.

1. *Caching by cell:* rounding to a grid cell maps endless distinct coordinates onto at most about 1.5M keys, and far fewer are hot because people cluster in cities; the TTL matches the provider's update interval.
2. *Provider failures:* use several providers, fall back to the last good data, and show "updated X minutes ago".

3. *Alert fan-out:* polygon to cells to subscribers; dedupe updates to the same alert.

Failures and trade-offs. Stale data is far better than none, so serve from cache when ingestion fails. Rounding locations improves caching and privacy at a small cost in precision.

→ *Building blocks:* Module 8 — CDN · Module 4 — Cache-Aside · Module 15 — Design 1: Proximity Service · Module 2 — Capstone: Designing a Notification System

Q103. Design a plagiarism checker

- **Requirements:** compare a new document or code submission against a large corpus and earlier submissions, and flag near-duplicates with the matching passages.
- **Components:** normalisation (lowercase, strip formatting; for code, rename identifiers and drop comments), shingling (overlapping k-word or k-token chunks), fingerprinting (winnowing picks a subset of chunk hashes), MinHash signatures with locality-sensitive hashing (LSH) to find candidate matches quickly, then exact comparison of the candidates to highlight passages.
- **Probe 1:** LSH turns "compare against everything" into "compare against a few candidates".
- **Probe 2:** paraphrasing defeats shingles, so add embeddings for semantic similarity.
- **Scale:** index fingerprints in a sharded key-value store; batch checks after contests or deadlines.

→ *Full design:* Module 19 — Near-duplicate detection with SimHash, MinHash and LSH

Q104. Design a URL / content safety filter (Amazon comments filtering)

Asked at: Amazon

Requirements. Functional: check every new comment or review (text and links) for spam, abuse, hate speech, personal data and malicious URLs before or right after it is published; allow, block, or send to human review; let users report content; let moderators override. Non-functional: inline checks under about 100 ms, so posting stays snappy; high recall for dangerous content; few false positives (blocking honest reviews angers customers); rules can be updated within minutes when a new spam wave starts.

Estimates. 10M comments a day (about 120 a second, 1k a second at peak) and about 2M URLs to check. Human review capacity is limited (for example 50k items a day), so automation must settle more than 99.5 % on its own.

API. `POST /moderate {content_id, text, urls, author_id, context}` returns `{decision: allow|block|review, reasons}`; `POST /reports`; moderator APIs for queues and decisions.

Data model. Decision log per content item (scores, rule hits, final decision; used for training and appeals); a URL reputation store (domain and URL hash mapped to a verdict and expiry); a blocklist and pattern rules; author trust scores.

Architecture. A synchronous fast path: normalisation (Unicode tricks, leetspeak), cheap checks first (blocklists, regex rules, URL reputation lookup, rate and velocity per author), then an ML text classifier served with low latency. An asynchronous slow path: deeper models (large language model classifiers), crawling unknown URLs in a sandbox, and duplicate and spam-ring detection across accounts. Borderline cases go to a human review queue; decisions feed back into training data.

Deep dives.

1. *Tiered pipeline:* run cheap, precise checks before expensive ones; most content exits early.
2. *URLs:* expand shorteners, check the domain against reputation lists (for example, Google Safe Browsing), and cache verdicts with TTLs.
3. *Adversaries adapt:* add rules quickly through a hot-reloaded rule engine, retrain often, and score accounts as well as content.

Failures and trade-offs. If the classifier is down, fail to "publish then check asynchronously" for trusted authors and "hold for review" for new ones. The block-versus-review threshold trades user friction against moderator workload.

→ *Building blocks:* Module 11 — Model Serving (and Batching) · Module 2 — Message Queue · Module 19 — Near-duplicate detection with SimHash, MinHash and LSH · Module 8 — Rate Limiting

Part B — Concept questions

These are the "explain X" and "X vs Y" questions that come up on their own or in the middle of a design round. Each answer is short enough to say in about 30 seconds. Use it to start, then go deeper if the interviewer asks.

Performance and scale

Q105. ⋆ How do you approach a system design interview? Follow a fixed order so you always know what comes next. First clarify functional requirements (features) and non-functional requirements (scale, latency, availability, consistency), and agree on what is out of scope. Then make quick estimates, define the API and the data model, and draw a simple high-level design that works end to end. Use the remaining time for deep dives on the hardest parts. Finish by naming the trade-offs, the weak spots and what you would change at 10× scale. → *More:* Module 00 — How to approach a system design interview · Module 01 — The Five Questions

Q106. ⋆ What is the difference between latency and throughput? Latency is how long one request takes from start to finish, for example 40 ms. Throughput is how many requests the system finishes per unit of time, for example 5,000 requests per second. They are related but separate: batching can raise throughput while making each request wait longer, and a fast system can still have low throughput if it handles only one request at a time. Bandwidth is the maximum data rate a link can carry, so it caps throughput but says nothing about latency. → *More:* Module 00 — Latency, throughput and bandwidth

Q107. ⋆ Why do we look at p99 latency instead of the average? A percentile such as p99 is the value that 99% of requests beat, so it shows what your slowest users actually experience. The average hides outliers: 99 requests at 10 ms and one at 5 seconds average to about 60 ms, which describes nobody. Tail latency also gets worse as you fan out, because a page that calls 100 backends in parallel hits at least one backend's p99 on about 63 % of requests (1 − 0.99^100). That is why SLOs are usually written on p95 or p99. → *More:* Module 00 — Percentiles, tail latency and why averages lie

Q108. ⋆ How do you do a back-of-the-envelope estimate (QPS, storage, bandwidth)? Start from daily active users and actions per user, then divide by about 100,000 seconds in a day to get average QPS (queries per second); multiply by 2–3 for peak. Storage is writes per day × bytes per write × days of retention × replication factor. Bandwidth is QPS × response size. Round hard (powers of ten are fine) and say what each number means for the design, for example "100:1 reads to writes, so a cache pays off." → *More:* Module 01 — Back-of-Envelope Math · Module 00 — Back-of-the-envelope estimation

Q109. ⋆ What latency numbers should every engineer know? The rough ladder is: L1 cache about 1 ns, main memory about 100 ns, an SSD random read in the tens of microseconds, a round trip inside one data centre about 0.5 ms, a spinning-disk seek about 10 ms, and a round trip across a continent or ocean 50–150 ms. The lesson is the ratios: memory is several hundred times faster than an SSD read, and the network across regions dwarfs almost everything. That is why we cache in memory, keep chatty calls inside one region and avoid sequential cross-region round trips. → *More:* Module 00 — Latency numbers every engineer should know

Q110. What is Little's law and how is it useful? Little's law says L = λ × W: the average number of requests in a system equals the arrival rate times the average time each one spends inside. If a service gets 1,000 requests per second and each takes 200 ms, about 200 are in flight at any moment. That tells you how many threads, connections or workers you need. It also explains why a slow downstream dependency suddenly exhausts your thread pool. → *More:* Module 00 — Little's law

Q111. ⋆ Vertical vs horizontal scaling: what is the difference? Vertical scaling (scaling up) means a bigger machine with more CPU, RAM or faster disks. It is simple and needs no code changes, but it has a hard ceiling, gets expensive fast and leaves a single point of failure. Horizontal scaling (scaling out) means adding more machines behind a load balancer. It has almost no ceiling and gives redundancy, but the services must be stateless or the data must be partitioned. Most systems scale up first and scale out once they hit limits. → *More:* Module 06 — Vertical Scaling · Module 00 — Vertical and horizontal scaling

Q112. What is the difference between performance and scalability? Performance is about speed: how quickly one request completes when the system is lightly loaded. Scalability is about growth: whether

you can keep that speed as traffic rises by adding machines. A quick test: if a lone request is already slow, fix performance (a bad query, a missing index); if requests are quick at low traffic but degrade as traffic climbs, fix scalability (a shared bottleneck, a hot shard). In a scalable design, doubling the hardware roughly doubles the capacity. → *More:* Module 00 — Performance vs scalability

Q113. ⋆ Stateless vs stateful services: why does it matter? What are sticky sessions? A stateless service keeps nothing about the user between requests in its own memory, so any instance can serve any request. That makes scaling out, autoscaling and replacing failed instances easy. A stateful service (a database, a WebSocket server, a game server) holds data that ties a user to a particular machine. Sticky sessions make the load balancer send the same user to the same server, which works but spreads load unevenly and loses sessions when that server dies. The usual fix is to move session state into a shared store such as Redis. → *More:* Module 00 — Stateless and stateful services

Q114. ⋆ What does "five nines" availability mean? Availability is the fraction of time the system works, usually measured as successful requests over total requests. 99.9% allows about 8.8 hours of downtime a year, 99.99% about 53 minutes, and 99.999% about 5 minutes. Components in series multiply, so two 99.9% services that both must be up give about 99.8%. Redundant copies in parallel raise availability. Each extra nine costs much more in redundancy, automation and operational work. → *More:* Module 00 — Availability and the nines

Q115. ⋆ What is the difference between an SLA, an SLO and an SLI? What is an error budget? An SLI (service level indicator) is a measurement, for example the share of requests answered under 300 ms. An SLO (objective) is the internal target for that SLI, such as 99.9% over 30 days. An SLA (agreement) is a contract with customers that promises a looser level and states the penalty if you miss it. The error budget is 100% minus the SLO. While budget is left you can ship fast; when it runs out you slow releases and fix reliability. → *More:* Module 00 — SLA, SLO, SLI and error budgets

Q116. Reliability vs availability vs durability: how do they differ? Availability means the system answers when you call it. Reliability means it does the right thing over time, and is often measured by MTBF (mean time between failures). Durability means data, once acknowledged, is never lost, and is often expressed as eleven nines per year for object stores. A system can be available but unreliable (it answers with wrong data) or durable but unavailable (the data is safe but you cannot reach it). MTTR (mean time to recovery) is usually the cheapest number to improve. → *More:* Module 00 — Reliability, availability and durability · Module 00 — MTBF, MTTR and measuring failure

Q117. ⋆ How would you scale a web application from one server to millions of users? Start with one server, then move the database to its own machine. Put several stateless app servers behind a load balancer and move sessions to a shared store. Add a cache for hot reads, a CDN for static files and read replicas for read-heavy traffic. Move slow work onto queues. When writes or data size outgrow one primary, shard the database, and finally go multi-region for latency and disaster recovery. At every step, measure first and fix the bottleneck you actually have. → *More:* Module 06 — Capstone - From 100 to 1,000,000 Requests per Second · Module 03 — Capstone - Scaling a Database to 100 Million Users

Networking and APIs

Q118. ⋆ What happens when you type a URL into the browser and press Enter? The browser parses the URL and checks its caches. DNS then turns the host name into an IP address, going through the browser, OS and resolver caches to the root, TLD and authoritative servers. The browser opens a TCP connection (or QUIC for HTTP/3), runs a TLS handshake to set up encryption and check the certificate, and sends an HTTP request. The request usually goes through a CDN or load balancer to an app server, which may call caches and databases before returning the response. Finally the browser parses the HTML, fetches CSS, JavaScript and images, and renders the page. → *More:* Module 00 — A request's journey

Q119. ⋆ How does DNS work? DNS (Domain Name System) maps names like example.com to IP addresses. The client asks a recursive resolver, usually run by the ISP or a public provider. On a cache miss the resolver asks a root server, which points to the TLD server (.com), which points to the domain's authoritative name server, which gives the answer. Each answer carries a TTL (time to live), so caches at every level keep it for a while. Common record types are A/AAAA (address), CNAME (alias), MX (mail) and NS (name server). DNS can also do simple load balancing and geo-routing. → *More:* Module 00 — DNS — the internet's phone book

Q120. ⋆ TCP vs UDP: when would you use each? Where does QUIC fit? TCP gives a reliable, ordered byte stream with a connection handshake, retransmission, flow control and congestion control. Web APIs, databases and file transfer use it. UDP just sends datagrams with no guarantees and no handshake, so it has lower overhead. It suits live video, voice, games and DNS, where a late packet is useless anyway. QUIC is built on UDP and adds reliability, encryption and many independent streams, so one lost packet does not block the other streams. HTTP/3 runs on QUIC. → *More:* Module 00 — TCP, UDP and QUIC

Q121. ⋆ What changed between HTTP/1.1, HTTP/2 and HTTP/3? HTTP/1.1 sends one request at a time per connection, so browsers open several connections, and a slow response blocks the ones behind it (head-of-line blocking). HTTP/2 multiplexes many streams over a single TCP connection and compresses headers. But one lost TCP packet still stalls every stream. HTTP/3 moves to QUIC over UDP, so streams recover from loss independently, and the combined transport and TLS handshake makes setup faster. → *More:* Module 00 — HTTP — the language of the web

Q122. ⋆ How does HTTPS / TLS work? TLS (Transport Layer Security) adds encryption, integrity and server authentication on top of TCP. In the handshake the server sends its certificate, which is signed by a certificate authority the client trusts, to prove its identity. Both sides use a key exchange (ephemeral Diffie-Hellman in TLS 1.3) to agree on a shared secret without ever sending it. After that, all traffic is encrypted with fast symmetric ciphers such as AES-GCM. TLS 1.3 needs only one round trip for the handshake, and zero on resumption. → *More:* Module 00 — HTTPS and TLS

Q123. ⋆ REST vs gRPC vs GraphQL: when would you use which? REST models resources as URLs with HTTP verbs and JSON. It is simple, cache-friendly and universal, which makes it the default for public APIs. gRPC uses Protocol Buffers over HTTP/2 with a strict schema, generated clients and streaming. It is compact and fast, so it is a good fit for internal service-to-service calls. GraphQL lets clients ask for exactly the fields they need from one endpoint. That suits varied front ends, but HTTP caching gets harder and you must guard against expensive queries. → *More:* Module 00 — API styles — REST, RPC and gRPC, GraphQL

Q124. ⋆ Polling vs long polling vs WebSockets vs Server-Sent Events: how do you choose? Short polling asks "anything new?" every few seconds. It is simple but wasteful and adds delay. Long polling holds each request open until there is data or a timeout, which gives near-real-time updates over plain HTTP. Server-Sent Events (SSE) keep one HTTP response open and stream events from server to client only, with automatic reconnect, which is ideal for feeds and notifications. WebSockets give a full two-way channel, which you need for chat, games and collaborative editing, but they are stateful connections that are harder to scale. → *More:* Module 00 — Real-time communication · Module 02 — Server-Sent Events

Q125. How do you scale WebSocket connections to millions of users? Run a fleet of connection servers that only hold sockets. Each one can carry tens of thousands of connections, and they sit behind an L4 load balancer. Keep a mapping of user to connection server in a shared store such as Redis. When a message comes in, look up the server that holds the recipient and route the message there, often through pub/sub. Clients must reconnect with backoff and jitter, and resume from a last-seen message ID, because servers will restart. Drain connections gradually during deploys so millions of clients do not reconnect at once. → *More:* Module 02 — Bidirectional Streaming

Q126. What is a webhook and how do you deliver webhooks reliably? A webhook is an HTTP callback: when an event happens, your system POSTs it to a URL the customer registered. To make delivery reliable, write the event to a durable queue first. Retry with exponential backoff for hours or days, and park repeated failures in a dead letter queue. Include an event ID so receivers can deduplicate. Sign each payload with an HMAC so receivers can check it is genuine. Do not assume events arrive in order. → *More:* Module 02 — Webhooks

Q127. ⋆ What makes a good REST API? Which HTTP methods are idempotent? Use nouns for resources (`/orders/42`), HTTP verbs for actions, correct status codes (201 created, 400 bad input, 404 missing, 409 conflict, 429 too many requests, 5xx server fault), consistent errors, pagination and versioning. GET, PUT and DELETE are idempotent: repeating them leaves the same state. POST is not, so for operations like payments you add an idempotency key. PATCH is not guaranteed to be idempotent. GET must also be safe, meaning it has no side effects. → *More:* Module 00 — API design basics

Q128. ⋆ Offset vs cursor pagination: which is better? Offset pagination (`LIMIT 20 OFFSET 10000`) is easy and lets you jump to page N. But the database still scans and throws away every skipped row, and items shift when rows are inserted, so users see duplicates or miss items. Cursor (keyset) pagination returns

an opaque token holding the last seen sort key, and the next query asks for `WHERE (created_at, id) < cursor`. It stays fast at any depth and is stable while data changes, so feeds and large lists use it. The cost is that you cannot jump to an arbitrary page. → *More:* Module 08 — Cursor-Based Pagination

Q129. ⋆ How do you version an API? The common options are a version in the URL path (`/v2/orders`), in a header or media type, or in a query parameter. Path versioning is the most visible and the easiest to route and cache. The bigger rule is to avoid breaking changes: add optional fields and never remove or rename them in place. Run old and new versions side by side, publish a deprecation schedule, and watch traffic until clients have moved before you switch the old version off. → *More:* Module 08 — API Versioning

Load balancing and caching

Q130. ⋆ What is a load balancer? L4 vs L7? Which algorithms are used? A load balancer spreads incoming requests over many servers, removes unhealthy ones and gives clients one stable address. An L4 balancer works on TCP/UDP connections (IP and port only), so it is very fast but cannot see the content. An L7 balancer reads the HTTP request, so it can route by path, header or cookie, terminate TLS and retry. Common algorithms are round robin, weighted round robin, least connections, least response time, IP or consistent hashing for affinity, and "power of two random choices." → *More:* Module 00 — Load balancers · Module 06 — Load Balancing

Q131. How does a load balancer know a server is healthy? Liveness vs readiness? The balancer calls a health-check endpoint on a schedule, and it removes a server after a few failures and adds it back after a few successes. A liveness check asks "is the process alive?", and a failure means restart it. A readiness check asks "can it serve traffic now?", covering things like a warmed cache and reachable dependencies, and a failure means stop sending traffic without restarting. Make the liveness check cheap and local. A liveness check that calls a shared database turns one brief database hiccup into a restart of the whole fleet at once. → *More:* Module 09 — Health Check Endpoint

Q132. ⋆ What is the difference between a forward proxy and a reverse proxy? A forward proxy sits in front of clients and makes requests on their behalf. Companies use one for filtering, caching or hiding client IPs, and the server sees the proxy, not the user. A reverse proxy sits in front of servers and receives requests on their behalf. It handles TLS termination, load balancing, caching, compression and protection, and the client sees only the proxy. Nginx, Envoy and CDNs are reverse proxies. → *More:* Module 08 — Reverse Proxy · Module 00 — Proxies and API gateways

Q133. ⋆ What is an API gateway? What is a Backend for Frontend (BFF)? An API gateway is a single entry point in front of many microservices. It handles cross-cutting work in one place: routing, authentication, rate limiting, request shaping, TLS and logging. That keeps every service from re-implementing these. A BFF is a separate gateway layer per client type (web, iOS, TV). It shapes and combines data to fit that client's screens, so a mobile app makes one call instead of six. The risks are a gateway that becomes a bottleneck or a place where business logic piles up. → *More:* Module 08 — API Gateway · Module 08 — Backend for Frontend

Q134. ⋆ What is a CDN and how does it work? Push vs pull? A CDN (content delivery network) is a set of edge servers around the world that cache content close to users. That cuts latency and takes load off your origin. DNS or anycast routes each user to a nearby edge. On a miss, a pull CDN fetches from the origin and caches the object for its TTL, which is easy and suits most sites. With a push CDN, you upload content ahead of time, which suits large, rarely changing files such as video releases. To invalidate, purge the object or use versioned file names (`app.3f9a.js`). → *More:* Module 08 — CDN · Module 00 — CDNs and anycast

Q135. ⋆ Explain cache-aside, read-through, write-through and write-behind. With cache-aside, the app checks the cache, reads the database on a miss and fills the cache. It is the most common pattern and only caches what is actually read. Read-through is the same idea, but the cache library loads from the database itself. Write-through writes to the cache and the database together, so the cache is fresh but writes are slower. Write-behind (write-back) writes to the cache and flushes to the database later. Writes are very fast, but you can lose data if the cache dies before the flush. → *More:* Module 04 — Cache-Aside · Module 00 — Caching strategies

Q136. ⋆ What cache eviction policies do you know? When the cache is full it must drop something. LRU (least recently used) drops what has gone unused longest. It is a good default and is built from a hash map plus a doubly linked list. LFU (least frequently used) keeps popular items even if they were not touched recently, but it adapts slowly. FIFO and random are simpler. TTL expiry removes entries after a fixed time no matter how full the cache is. Modern caches use smarter mixes such as W-TinyLFU, which resists one-off scans that would flush LRU. → *More:* Module 00 — Eviction policies · Module 19 — Cache eviction beyond LRU

Q137. ⋆ How do you handle cache invalidation and the thundering herd (cache stampede)? For invalidation, set a TTL as a safety net and delete the key whenever the source data changes. A stampede happens when a hot key expires and thousands of requests miss at once and hammer the database. The fixes are: a lock or "single flight" so only one request rebuilds the key while the others wait or get the stale value; early probabilistic refresh before expiry; serving stale data while revalidating; and random jitter on TTLs so keys do not all expire together. → *More:* Module 04 — Cache Stampede Prevention · Module 00 — Cache invalidation, stale data and stampedes

Q138. ⋆ Where can you put a cache in a system? Caches can sit at every layer: the browser or mobile app (HTTP cache headers), the CDN at the edge, the reverse proxy or gateway, in-process memory in each app server, a shared distributed cache such as Redis or Memcached, and the database's own buffer pool. The closer to the user, the faster the hit, but the harder it is to invalidate. Pick the layer by how stale each kind of data is allowed to be and how many readers share it. → *More:* Module 00 — Where caches live

Q139. Redis vs Memcached: which would you pick? Memcached is a simple, multi-threaded, in-memory key-value cache for strings. It is very fast and easy to run when all you need is plain caching. Redis offers rich data structures (lists, hashes, sets, sorted sets, streams, HyperLogLog), optional persistence, replication, Lua scripting and pub/sub. So it also serves as a rate limiter, leaderboard, session store or lightweight queue. Most new systems pick Redis (or a compatible fork such as Valkey) for that flexibility. Memcached still makes sense for huge, simple caches. → *More:* Module 21 — Case 4 — Distributed Cache Cluster

Q140. How do you handle a hot key in a cache? A hot key is one entry, such as a celebrity profile or a viral product, that gets so many reads that the single cache node holding it maxes out. Options: add a small in-process cache with a short TTL in each app server; copy the key to several nodes under suffixed names (`key#1` … `key#N`) and read a random copy; or use read replicas of that cache shard. Detect hot keys with sampling or the cache's own tooling, so you can react before the node falls over. → *More:* Module 21 — Case 4 — Distributed Cache Cluster

Q141. How do you keep a cache consistent with the database? Update the database first, then delete the cache key rather than overwrite it. A delete is safe to repeat and avoids two writers leaving an old value in the cache. A race can still put stale data back: a slow reader fills an old value just after the delete. A short TTL, a second delayed delete, or version checks shrink that window. For stronger guarantees, drive invalidation from the database's change stream (CDC), so every committed change evicts the key even if the app crashed mid-request. For data that must be exact, such as a balance at checkout, read the database. → *More:* Module 04 — Write-Through · Module 10 — Change Data Capture

Databases and storage

Q142. ⋆ SQL vs NoSQL: how do you choose? Choose SQL (relational) when data has clear relationships, you need joins, multi-row ACID transactions and flexible queries. Orders, payments and inventory are typical. Choose NoSQL when the access pattern is simple and known (lookup by key), the scale or write rate is huge, the schema changes often, or you need easy horizontal scaling and multi-region writes. Modern relational databases scale a long way, so the safe default is SQL unless a specific requirement pushes you elsewhere. Always give the reason, not just the product name. → *More:* Module 00 — SQL vs NoSQL and BASE · Module 00 — Choosing a database

Q143. What are the main types of NoSQL databases? Key-value stores (Redis, DynamoDB) do fast lookups by key, for sessions, carts and caches. Document stores (MongoDB) hold JSON-like documents with nested fields, for catalogs and user profiles. Wide-column stores (Cassandra, HBase) are partitioned rows with many columns and are tuned for heavy writes and time-ordered data. Graph databases (Neo4j) store nodes

and edges, for relationship queries such as "friends of friends." Search engines (Elasticsearch) and time-series databases are also often grouped here. → *More:* Module 00 — NoSQL database families

Q144. ⋆ What is ACID? How is it different from BASE? ACID describes transactions. Atomicity means all or nothing. Consistency means the rules and constraints still hold afterwards. Isolation means concurrent transactions do not see each other's half-done work. Durability means committed data survives crashes. BASE ("basically available, soft state, eventually consistent") describes many distributed NoSQL systems: they stay available and accept that replicas converge over time. ACID favours correctness, while BASE favours availability and scale. → *More:* Module 00 — Transactions and ACID

Q145. ⋆ What are the transaction isolation levels and the anomalies they prevent? Read uncommitted allows dirty reads, meaning you can see another transaction's uncommitted data. Read committed (the PostgreSQL default) stops dirty reads, but the same row can change between two reads (a non-repeatable read). Repeatable read or snapshot isolation (the MySQL InnoDB default is repeatable read) gives a stable view. Phantoms, which are new rows that match your query, may or may not be blocked depending on the engine. Write skew is still possible: two transactions each check a rule and then write different rows, breaking it together. Serializable prevents all of these, as if transactions ran one at a time, at the cost of more aborts or locking. → *More:* Module 00 — Isolation levels and anomalies

Q146. ⋆ What is a database index? B-tree vs LSM tree? An index is an extra sorted structure that finds rows without scanning the whole table. It speeds up reads but costs storage and slows every write. A B-tree (B+-tree) keeps sorted pages and updates them in place. Lookups and range scans are fast and predictable, which is why it is the default in PostgreSQL and MySQL. An LSM tree buffers writes in memory, flushes them as sorted files and merges those files in the background (compaction). Writes are very fast, but reads may check several files, helped by Bloom filters. Cassandra and RocksDB use LSM trees. → *More:* Module 00 — Indexes · Module 19 — LSM trees in depth

Q147. ⋆ Normalization vs denormalization: when would you denormalize? Normalization stores each fact exactly once, split across tables linked by keys. That avoids update anomalies and keeps writes simple and correct. Denormalization copies data, for example storing the author's name in every post, so that reads need fewer or no joins. Denormalize when reads vastly outnumber writes, when you are on a NoSQL store without joins, or when you build read models for a hot path. The price is that every copy must be updated when the source changes. → *More:* Module 00 — Normalization and denormalization

Q148. ⋆ Optimistic vs pessimistic locking: when do you use each? Pessimistic locking takes a lock before reading (`SELECT … FOR UPDATE`), so others wait. It is safe when conflicts are frequent, as with the last few seats in a hot event, but it lowers throughput and can deadlock. Optimistic locking reads freely and checks a version number at write time (`UPDATE … WHERE version = 7`). If someone else changed the row, the update affects zero rows and you retry. It is better when conflicts are rare, and it holds no locks across user think time. → *More:* Module 17 — Design an Event Ticketing and Flash-Sale System · Module 19 — Two-phase locking and Serializable Snapshot Isolation

Q149. What is MVCC? MVCC (multi-version concurrency control) keeps several versions of each row instead of overwriting it in place. Each transaction (or, under read committed, each statement) reads a consistent snapshot taken when it started, so readers never block writers and writers never block readers. Old versions are cleaned up later, by VACUUM in PostgreSQL or the undo log purge in InnoDB. MVCC is how most databases provide snapshot and repeatable-read isolation. → *More:* Module 19 — MVCC step by step

Q150. ⋆ Block vs file vs object storage: what is the difference? Block storage exposes raw fixed-size blocks to one machine, like a virtual disk (EBS). It is the fastest and is used for databases and OS volumes. File storage offers a shared hierarchical file system over the network (NFS, EFS), which many machines can mount. Object storage (S3) keeps whole immutable objects with metadata in a flat namespace, accessed by HTTP. It is cheap, nearly unlimited and very durable, and suits images, videos, backups and data lakes. You cannot edit part of an object in place. → *More:* Module 00 — Storage types: block, file and object

Q151. ⋆ OLTP vs OLAP; row vs column storage; data lake vs data warehouse? OLTP (online transaction processing) handles many small reads and writes on single rows, such as placing an order, and it uses row-oriented storage. OLAP (online analytical processing) runs large scans and aggregations over many rows but few columns. Columnar storage suits it because it reads only the needed columns and compresses

well. A data warehouse stores cleaned, structured data for SQL analytics. A data lake stores raw files of any format cheaply in object storage. A "lakehouse" adds table formats such as Iceberg on top to get warehouse features. → *More:* Module 00 — OLTP vs OLAP, row vs column storage, lakes and warehouses

Q152. ⋆ What is a write-ahead log (WAL) and why does it matter? A WAL is an append-only file where the database records each change before applying it to the main data files. If the machine crashes, the database replays the log to restore every committed transaction, which gives durability. Sequential appends are much faster than random page writes, so the WAL also speeds up commits. The same log is reused for replication (replicas replay it) and for change data capture. → *More:* Module 03 — Write-Ahead Log · Module 00 — Durability: WAL, fsync, backups and recovery

Q153. What is connection pooling, and what is the N+1 query problem? Opening a database connection is expensive (TCP, TLS, auth), and databases support only a limited number. A pool keeps a fixed set of open connections that requests borrow and return. With many app instances you often add an external pooler such as PgBouncer. The N+1 problem is an ORM loading a list with one query and then running one more query per item to fetch related data, which means 101 queries instead of 2. Fix it with joins, batched `IN (…)` loading or eager fetching. → *More:* Module 00 — Connection pooling, ORMs and N+1 queries · Module 06 — Database Connection Pooling

Replication and partitioning

Q154. ⋆ Replication vs sharding: what is the difference? Replication keeps full copies of the same data on several machines. It adds read capacity and availability, but every copy still holds everything and every write goes everywhere. Sharding (partitioning) splits the data so each machine holds only a subset. It adds write capacity and storage, but queries across shards become hard. Large systems do both: data is sharded, and each shard is replicated. → *More:* Module 00 — Replication · Module 00 — Partitioning and sharding

Q155. ⋆ Leader-follower vs multi-leader vs leaderless replication? In leader-follower (primary-replica) replication, all writes go to one leader, which streams changes to followers that serve reads. It is simple and has no write conflicts, but the leader is the write bottleneck and failover is needed. Multi-leader lets several nodes, often one per region, accept writes. That gives low local write latency, but concurrent writes to the same data conflict and must be resolved. Leaderless replication (Dynamo, Cassandra) sends writes and reads to several replicas and uses quorums, which gives high availability at the cost of weaker consistency. → *More:* Module 03 — Primary-Replica · Module 00 — Replication

Q156. ⋆ Synchronous vs asynchronous replication? What is replication lag? With synchronous replication the leader waits until a replica confirms before acknowledging a write. No acknowledged data is lost on failover, but every write is slower and a slow replica stalls everything. With asynchronous replication the leader acknowledges right away, so writes are fast, but recent writes can be lost if the leader dies. Replication lag is how far a replica is behind. It causes "I saved it but it is gone" moments. Fix it with read-your-writes, for example by reading your own recent data from the leader. Many systems compromise with semi-synchronous replication: one replica is synchronous and the rest are asynchronous. → *More:* Module 00 — Replication

Q157. How do multi-leader or leaderless systems resolve write conflicts? Last-write-wins (LWW) keeps the write with the highest timestamp. It is simple but silently drops data and depends on clocks. Version vectors detect true concurrency, so the system can keep both versions ("siblings") and let the app or user merge them. CRDTs (conflict-free replicated data types), such as counters, sets and text sequences, are designed so concurrent updates always merge to the same result automatically. Often the best fix is to avoid conflicts by routing all writes for one record to a single home region. → *More:* Module 07 — Vector Clocks · Module 19 — Version vectors vs vector clocks

Q158. ⋆ How do you choose a shard key? How do you handle hot partitions (the celebrity problem)? A good shard key spreads data and traffic evenly and keeps the most common queries on one shard. User ID suits user-centric apps, for example. Avoid keys that are monotonic, like timestamps, which send every new write to one shard, and keys with low cardinality, like country. Hot partitions still appear when one key is huge, such as a celebrity's posts. Fixes: add a random suffix to split that key over several

shards and merge on read, give hot keys their own partitions, and put a cache in front. → *More:* Module 03 — Sharding · Module 00 — Partitioning and sharding

Q159. ⋆ How does consistent hashing work, and why use virtual nodes? Both nodes and keys are hashed onto a circle, and each key belongs to the first node clockwise from it. When a node joins or leaves, only the keys next to it move, roughly 1/N of the data, instead of nearly all keys as with `hash(key) mod N`. With few nodes the arcs are uneven, so each physical node is placed at many points as virtual nodes. That evens out load and spreads a failed node's keys across many survivors. Variants such as rendezvous and jump hashing solve the same problem differently. → *More:* Module 03 — Consistent Hashing · Module 19 — Consistent hashing variants

Q160. How do you reshard or rebalance data without downtime? Plan for growth by creating many more logical partitions than machines at the start, so rebalancing just moves whole partitions between nodes. To move data, copy a snapshot to the new location, then stream ongoing changes (CDC) until it catches up. Next, switch reads, then writes, briefly double-writing or using a routing table with a version. Throttle the copy so it does not hurt live traffic, and verify with checksums before deleting the old copy. → *More:* Module 03 — Sharding · Module 09 — Capstone - The Zero-Downtime Migration

Q161. How do you design a system that serves users from multiple data centers or regions? Use GeoDNS or anycast to send users to the nearest healthy region, and keep stateless services in every region. The hard part is data. One choice is a single write region with read replicas elsewhere, which is simple but adds latency to remote writes. Another is to partition users by home region, so writes stay local. A third is multi-leader or globally consistent databases, which bring conflict handling or higher write latency. Plan regional failover, and test it, so a whole region can go down. Cell-based designs also cap the blast radius of a failure. → *More:* Module 06 — Multi-Region and Cell-Based Architecture

Consistency and consensus

Q162. ⋆ Explain the CAP theorem. What does PACELC add? CAP says that during a network partition, a distributed system must choose between consistency (every read sees the latest write) and availability (every request gets a non-error answer). Partitions cannot be ruled out, so the real choice is CP (refuse some requests) or AP (answer, possibly with stale data) while the partition lasts. PACELC adds the normal case: Else, when there is no partition, you trade latency against consistency. That explains why systems like Cassandra are tunable even when the network is healthy. → *More:* Module 00 — CAP theorem and PACELC

Q163. ⋆ Strong vs eventual consistency: what consistency models do you know? Strong consistency (linearizability) means every read sees the most recent completed write, as if there were one copy. Eventual consistency only promises that replicas converge once writes stop, so reads may be stale for a while. In between are session guarantees. Read-your-writes means you see your own updates. Monotonic reads means you never go back in time. Consistent prefix means you see writes in order. Causal consistency means effects never appear before their causes. Pick the weakest model each feature can tolerate, because stronger models cost latency and availability. → *More:* Module 00 — Consistency models

Q164. What is the difference between linearizability and serializability? Linearizability is about single objects and real time. Each read or write appears to take effect at one instant between its start and finish, so nobody reads an older value after a newer one was returned. Serializability is about transactions over many objects. The result equals some one-at-a-time order of the transactions, but that order need not match real time. "Strict serializability" combines both, and systems such as Spanner provide it. → *More:* Module 00 — Consistency models · Module 19 — Google TrueTime and commit-wait

Q165. ⋆ What are quorums (N, R, W)? N is the number of replicas, W is how many must confirm a write, and R is how many are asked on a read. If R + W > N, every read set overlaps every write set, so a read sees at least one copy of the latest write. N=3, W=2, R=2 is a common choice. W=1 makes writes fast and R=1 makes reads fast, at the cost of staleness. Quorums alone are not full linearizability, because of things like sloppy quorums and concurrent writes, but they are the core tuning knob in Dynamo-style stores. → *More:* Module 07 — Quorum · Module 00 — Quorums

Q166. ⋆ Two-phase commit vs saga: how do you do a distributed transaction? Two-phase commit (2PC) uses a coordinator. In phase one it asks every participant to prepare and vote. In phase two it tells them

all to commit or all to abort. That gives atomicity, but participants hold locks while waiting and block if the coordinator dies. A saga splits the work into a chain of local transactions, each with a compensating action (refund, release seat) to undo it if a later step fails. It is run by an orchestrator or by events (choreography). Microservices prefer sagas, accepting that intermediate states are visible. → *More:* Module 07 — Two-Phase Commit · Module 07 — Saga

Q167. ⋆ What is the transactional outbox pattern? It solves the "dual write" problem: updating the database and publishing an event are two separate actions, so a crash between them leaves the two out of sync. Instead, in the same local database transaction, you write the business change and an event row into an "outbox" table. A separate relay process, a poller or CDC on the outbox table, reads new rows and publishes them to the broker, then marks them as sent. Delivery is at least once, so consumers must be idempotent. → *More:* Module 10 — Change Data Capture · Module 07 — Capstone - Order Fulfillment, Built Twice

Q168. ⋆ What is consensus? How do Raft and Paxos work, and how is a leader elected? Consensus is getting several nodes to agree on one value, or one ordered log, even if some crash. In Raft, nodes start as followers. If one stops hearing heartbeats from a leader, it becomes a candidate, bumps the term number and asks for votes, and a majority makes it leader. The leader appends entries to its log and replicates them, and an entry is committed once a majority has stored it. Paxos reaches the same guarantees with proposers and acceptors, using numbered proposals and majority promises. Both need a majority alive, so 5 nodes tolerate 2 failures. etcd and ZooKeeper provide these guarantees as a service. → *More:* Module 00 — Consensus, leader election and distributed locks · Module 19 — Paxos and Multi-Paxos

Q169. ⋆ How do distributed locks work? What is a fencing token? A distributed lock makes sure only one worker at a time does something, using a shared service such as ZooKeeper, etcd or Redis. Locks always have a lease (expiry) so a crashed holder does not block forever. The danger is that a holder pauses, for example on a long GC, its lease expires, someone else takes the lock, and then the first holder wakes up and writes anyway. A fencing token is a number that increases each time the lock is granted. The storage rejects writes carrying a token older than the newest one it has seen. → *More:* Module 00 — Consensus, leader election and distributed locks · Module 19 — Leases and lease-based leader election

Q170. ⋆ How do you order events in a distributed system? Lamport clocks, vector clocks, hybrid logical clocks? Machine clocks drift, so wall-clock timestamps cannot be trusted for ordering. A Lamport clock is a counter that each node increments and that is merged with `max+1` on every message. It gives an order consistent with causality but cannot tell when two events were concurrent. A vector clock keeps one counter per node, so it can detect "happened before" versus "concurrent." It grows with the number of nodes. A hybrid logical clock combines physical time with a logical counter, giving timestamps close to real time that still respect causality. CockroachDB uses them. → *More:* Module 00 — Clocks and ordering · Module 07 — Vector Clocks · Module 19 — Hybrid logical clocks

Q171. ⋆ What is split brain and how do you prevent it? Split brain is when a network partition makes two nodes each believe they are the leader, so both accept writes and the data diverges. Prevent it by requiring a majority quorum to elect or keep a leader: only one side of a partition can hold a majority, so with an even cluster size you add a tie-breaker (witness) node. Add leases, so an old leader stops serving when its lease expires, and fencing tokens, so storage rejects the stale leader's writes. STONITH ("shoot the other node in the head") forcibly powers off the old leader. → *More:* Module 00 — Consensus, leader election and distributed locks

Q172. ⋆ How do nodes detect failures? Heartbeats, gossip, phi-accrual? With heartbeats, each node pings a monitor, or its peers, periodically, and is suspected dead after missing several. In a network you cannot tell a dead node from a slow one, so timeouts are a trade-off between fast detection and false alarms. A gossip protocol has each node share membership and health with a few random peers every round, so information spreads to all nodes in about log N rounds without a central point. Phi-accrual detectors (Cassandra) output a suspicion level based on recent heartbeat timing instead of a fixed timeout. SWIM adds indirect probes to cut false positives. → *More:* Module 00 — Heartbeats, gossip and failure detection · Module 19 — Failure detection

Messaging and streaming

Q173. ⋆ Message queue vs publish-subscribe? Kafka vs RabbitMQ? A message queue delivers each message to one consumer out of a pool, which suits distributing jobs. Pub/sub delivers each message to every subscriber, which suits broadcasting events to many services. RabbitMQ is a traditional broker: it routes messages flexibly, tracks acks per message and deletes messages once they are consumed. It is good for task queues. Kafka is a distributed append-only log. Messages are kept for days, consumers track their own offsets and can replay, and throughput is very high. It is good for event streaming, analytics and feeding many independent consumers. → *More:* Module 02 — Message Queue · Module 02 — Publish-Subscribe · Module 10 — Inside Kafka

Q174. ⋆ At-most-once vs at-least-once vs exactly-once delivery? At-most-once means you send or acknowledge before processing. Nothing is duplicated, but messages can be lost. At-least-once means you acknowledge after processing and retry on failure. Nothing is lost, but duplicates happen, and it is the usual default. True exactly-once delivery over a network is not possible. What systems provide is exactly-once *effect*: at-least-once delivery plus idempotent processing or deduplication, or transactions that commit the output and the consumer offset together, as Kafka transactions do. → *More:* Module 00 — Delivery guarantees, idempotency and retries · Module 10 — Exactly-Once Semantics

Q175. ⋆ What is idempotency, and how do you build an idempotent API? An operation is idempotent if doing it twice has the same effect as doing it once, so retries are safe. For a create or payment endpoint, the client sends a unique idempotency key, such as a UUID, with the request. The server stores the key with the result in the same transaction as the work. If the same key arrives again, it returns the stored result instead of acting again. Also use natural idempotency where possible: "set status = shipped" instead of "increment," and unique constraints. → *More:* Module 05 — Idempotency

Q176. How does Kafka guarantee ordering? What are partitions, consumer groups and offsets? A topic is split into partitions, and each partition is an ordered, append-only log. Kafka guarantees order only within a partition, so you key messages by entity (for example order ID) to keep each entity's events in order. A consumer group shares the partitions among its members, with each partition read by exactly one consumer in the group. The number of partitions therefore caps that group's parallelism. Each consumer commits an offset, its position in the partition, so it can resume after a restart or replay from an earlier point. → *More:* Module 10 — Inside Kafka · Module 10 — Partitioned Consumption

Q177. ⋆ Batch vs stream processing? Lambda vs Kappa architecture? Batch processing runs over a large bounded dataset on a schedule, for example nightly. It is simple and easy to rerun, but results are hours old. Stream processing handles events continuously as they arrive, with results in seconds, but it must deal with windows, late events and state. Lambda architecture runs both a batch layer (accurate) and a speed layer (fresh) and merges them, which means two codebases. Kappa architecture uses only a stream processor and reprocesses history by replaying the log, so there is one codebase. → *More:* Module 10 — Stream Processing · Module 10 — Lambda and Kappa

Q178. ⋆ What is MapReduce? MapReduce is a model for processing huge datasets in parallel on many machines. The map step turns each input record into key-value pairs, for example (word, 1). The framework then shuffles and sorts so that all values for the same key reach one reducer. The reduce step combines them, for example summing the counts. The framework handles splitting the input, scheduling, retrying failed tasks and moving computation to where the data lives. Spark later kept the idea but works in memory and is much faster for multi-step jobs. → *More:* Module 10 — MapReduce

Q179. ⋆ What are event-driven architecture, event sourcing and CQRS? In event-driven architecture, services publish facts ("OrderPlaced") and others react, so producers do not need to know their consumers. Event sourcing stores the full sequence of events as the source of truth instead of only the current state, and current state is rebuilt by replaying them. That gives a complete audit trail and time travel. CQRS (command query responsibility segregation) uses one model for writes and separate, read-optimized models for queries, kept up to date from events. They are often combined, but each adds complexity and eventual consistency. → *More:* Module 02 — Event-Driven Architecture · Module 03 — Event Sourcing · Module 03 — CQRS

Q180. ⋆ Synchronous vs asynchronous communication between services: when do you use each? Synchronous calls (HTTP or gRPC request-response) are simple and give an immediate answer. But they couple availability: if B is down or slow, A is too, and latencies add up along a chain. Asynchronous

messaging through a queue or event stream decouples the services in time, absorbs bursts and lets each side scale and fail separately, but the result is eventual and harder to trace. Use synchronous calls when the user needs the answer now, for example showing a price. Use asynchronous messaging for follow-up work such as emails, fulfillment and analytics. → *More:* Module 00 — Synchronous vs asynchronous communication · Module 02 — Request-Response

Q181. ⋆ What is change data capture (CDC)? CDC streams every insert, update and delete from a database as events, usually by reading the database's own replication log (WAL or binlog), for example with Debezium. Other systems, such as search indexes, caches, data warehouses and other services, can then stay in sync without the application doing fragile dual writes. Because it reads the log, it captures every committed change in order and adds little load to the database. It is also the usual way to run the outbox relay. → *More:* Module 10 — Change Data Capture

Q182. ⋆ What is a dead letter queue? A dead letter queue (DLQ) is a separate queue where messages go after they have failed processing a set number of times. Examples are malformed data or a bug that will never succeed ("poison messages"). Moving them aside stops one bad message from blocking the queue or burning resources in endless retries. You alert on DLQ growth, inspect the messages, fix the cause and then replay them to the main queue. → *More:* Module 05 — Dead Letter Queue

Q183. ⋆ Fan-out on write vs fan-out on read for a news feed? Fan-out on write (push) copies each new post into every follower's precomputed feed, for example a Redis list per user, when it is posted. Reads are instant, but a user with 50 million followers creates 50 million writes. Fan-out on read (pull) builds the feed at request time by fetching recent posts from everyone you follow. Writes are cheap, but reads are slow. The usual answer is a hybrid: push for normal users, and pull for celebrities, whose posts are merged in when the feed is read. → *More:* Module 12 — Design 1 — The News Feed · Module 19 — Feed ranking

Reliability and resilience

Q184. ⋆ What is a single point of failure? Active-active vs active-passive failover? A single point of failure (SPOF) is any component whose failure takes the whole system down: one database, one load balancer, one region. Remove SPOFs with redundancy and automatic failover. In active-passive, a standby takes over only when the primary fails. It is simple, but the standby sits idle and failover takes time. In active-active, all copies serve traffic at once. You get better use of resources and instant failover, but you must handle shared state and conflicts, and each copy must be able to absorb the others' load. → *More:* Module 00 — Redundancy, single points of failure and failover

Q185. ⋆ How should you use timeouts and retries? Why exponential backoff with jitter? Every network call needs a timeout, or a hung dependency will hold your threads forever. Set it from the dependency's p99, not a guess. Retry only transient errors, only idempotent operations, and only a few times. Exponential backoff (wait 100 ms, 200, 400…) gives the struggling service room to recover. Jitter (random spread) stops thousands of clients from retrying in sync. Cap total retries with a budget and retry at one layer only, because retries at every layer multiply into a "retry storm." → *More:* Module 05 — Timeout · Module 05 — Retry with Exponential Backoff

Q186. ⋆ What are the circuit breaker and bulkhead patterns? A circuit breaker wraps calls to a dependency and counts failures. When the failure rate crosses a threshold it "opens" and fails fast without calling, so the dependency gets time to recover and your threads are not wasted. After a cool-down it goes "half-open" and lets a few test calls through. A bulkhead isolates resources, such as a separate thread or connection pool per dependency or per tenant, so that one slow dependency cannot use up everything and sink the whole service. Pair both with a fallback, such as cached or default data. → *More:* Module 05 — Circuit Breaker · Module 05 — Bulkhead

Q187. ⋆ What are backpressure, load shedding and graceful degradation? Backpressure means a slow consumer signals producers to slow down, through bounded queues, blocking or pull-based flow, instead of letting work pile up until memory runs out. Load shedding means deliberately rejecting some requests when overloaded, for example with fast 503s or by dropping low-priority traffic first, so the requests you do accept still succeed. Graceful degradation means turning off non-essential features under stress, such as recommendations or live counts, while the core path keeps working. Together they keep

an overload from turning into a full outage. → *More:* Module 10 — Backpressure · Module 05 — Graceful Degradation · Module 00 — Load shedding and graceful degradation

Q188. ⋆ What are RPO and RTO in disaster recovery? RPO (recovery point objective) is how much data you can afford to lose, measured in time. An RPO of 5 minutes means backups or replication must never be more than 5 minutes behind. RTO (recovery time objective) is how long you can be down before service is restored. The strategies, from cheap to expensive, are: backup and restore (hours), pilot light, warm standby, and multi-region active-active (near zero for both). Test restores and failovers regularly, because you only know a backup works once you have restored from it. → *More:* Module 00 — Disaster recovery

Q189. ⋆ What rate-limiting algorithms do you know? Token bucket adds tokens at a fixed rate up to a maximum, and each request spends one. It allows short bursts and is the most common. Leaky bucket processes requests at a fixed outflow rate and smooths traffic. Fixed window counts requests per clock minute. It is cheap, but it allows double bursts at window edges. Sliding window log stores timestamps, which is exact but memory-heavy. Sliding window counter adds a weighted share of the previous window's count to the current one; it is cheap and a close approximation, because it assumes the previous window's requests were evenly spread. In a cluster, keep counters in Redis with atomic operations, and return HTTP 429 with a `Retry-After` header. → *More:* Module 08 — Rate Limiting · Module 00 — Rate limiting and throttling

Security

Q190. ⋆ Authentication vs authorization? Sessions vs JWT? Authentication checks who you are (login, MFA). Authorization decides what you may do (roles, permissions). With server-side sessions, the server stores session data and the client holds only a random ID in a cookie. Logout and revocation are easy, but every request needs a session-store lookup. A JWT (JSON Web Token) is a signed, self-contained token carrying the user's claims, so any service can verify it without a lookup. That scales well, but a JWT cannot easily be revoked before it expires. So keep access tokens short-lived, use refresh tokens, and remember the payload is only signed, not encrypted. → *More:* Module 00 — Authentication and authorization · Module 00 — Sessions, cookies, tokens and JWT

Q191. ⋆ How do OAuth 2.0, OpenID Connect and SSO work? OAuth 2.0 is for delegated authorization. It lets an app get an access token to call an API on a user's behalf without seeing their password. The usual flow is the authorization code flow with PKCE: redirect to the identity provider, the user logs in and consents, the app gets a code and exchanges it for tokens. OpenID Connect adds authentication on top through an ID token, a JWT that says who the user is. SSO (single sign-on) uses OIDC or SAML so one login at a central identity provider works across many apps. → *More:* Module 00 — OAuth 2.0, OpenID Connect, SSO and API keys · Module 08 — Authorization Service

Q192. ⋆ Explain CORS, CSRF, XSS and SQL injection. CORS (cross-origin resource sharing) is a browser rule: a page from one origin cannot read responses from another origin unless that server allows it with headers. It relaxes a browser restriction and does not protect your server. CSRF (cross-site request forgery) tricks a logged-in user's browser into sending an unwanted request. Defend with SameSite cookies and anti-CSRF tokens. XSS (cross-site scripting) injects scripts into your pages. Defend by escaping output and using a Content Security Policy. SQL injection smuggles SQL through user input. Defend with parameterized queries, never string concatenation. → *More:* Module 00 — Common web vulnerabilities: CORS, CSRF, XSS and SQL injection

Q193. ⋆ How do you protect a system against DDoS attacks? What is a WAF? A DDoS (distributed denial of service) attack floods you with traffic from many sources. Network-level floods are absorbed by large edge networks: CDNs, anycast and cloud DDoS protection spread the load across many locations. Application-level floods, which look like real requests, need rate limiting, bot detection, challenges and autoscaling. A WAF (web application firewall) inspects HTTP traffic at the edge and blocks known attack patterns such as SQL injection, bad bots and abusive IPs. Also keep origin servers hidden so attackers cannot bypass the edge. → *More:* Module 00 — Network security: firewalls, WAF and DDoS

Q194. How do you store passwords and secrets? Encryption at rest vs in transit? Never store passwords in plain text or with fast hashes like SHA-256. Use a slow, salted password hash such as Argon2id, bcrypt or scrypt, so stolen hashes are expensive to crack. Encryption in transit (TLS everywhere,

including inside the network) protects data on the wire. Encryption at rest protects disks and backups, with keys held in a KMS (key management service) and often rotated. Secrets such as API keys and database passwords belong in a secrets manager or vault and are injected at runtime, never committed to code. → *More:* Module 00 — Encryption, hashing, passwords and secrets

Q195. What are least privilege and zero trust? Least privilege means every user, service and key gets only the permissions it needs, and only for as long as it needs them, so a compromise does limited damage. Zero trust means you never trust a request just because it comes from inside your network. Every call is authenticated and authorized, for example with mTLS (mutual TLS) between services, short-lived credentials and per-request policy checks. Together they shrink the blast radius when, not if, something is breached. → *More:* Module 00 — Least privilege and zero trust

Operations

Q196. ⋆ Monolith vs microservices: when do you choose which? A monolith is one deployable application. It is simple to build, test, debug and deploy, and calls are fast in-process function calls. Microservices split the system into independently deployable services that own their own data. Teams can ship and scale separately, but you take on network failures, distributed data, tracing and operational overhead. Start with a well-structured (modular) monolith and split services out when team size, deploy friction or very different scaling needs justify it. Splitting too early creates a "distributed monolith." → *More:* Module 00 — Architectural styles

Q197. ⋆ What are service discovery, a sidecar and a service mesh? Service discovery lets services find each other's current addresses as instances come and go. It works either through a registry such as Consul, etcd or Kubernetes DNS, or through a load balancer that does the lookup. A sidecar is a helper process deployed next to each service instance, handling things like proxying, TLS or log shipping, so the app does not have to. A service mesh (Istio, Linkerd) is a fleet of sidecar proxies plus a control plane. It gives every service-to-service call mTLS, retries, timeouts, traffic splitting and metrics without code changes, at the cost of extra latency and operational complexity. → *More:* Module 08 — Sidecar · Module 08 — Service Mesh · Module 00 — Service discovery, sidecars and service mesh

Q198. ⋆ Blue-green vs canary deployments? What are feature flags? Blue-green runs two identical environments. You deploy to the idle one, test it, then switch all traffic over. Rollback is instant by switching back, but you need double capacity for a while. A canary sends a small share of traffic, say 1% and then 10%, to the new version, compares error and latency metrics, and widens or rolls back. That limits the blast radius of a bad release. Feature flags separate deploying code from releasing a feature: you turn features on per user or percentage at runtime, and you can switch one off without a redeploy. → *More:* Module 09 — Blue-Green Deployment · Module 09 — Canary Deployment · Module 09 — Feature Flags

Q199. ⋆ What is observability? Logs vs metrics vs traces? Observability is being able to explain what a system is doing from the outside, including problems you did not predict. Metrics are cheap numeric time series (request rate, errors, latency percentiles, saturation) for dashboards and alerts. Logs are detailed timestamped records of individual events, best kept structured (JSON) and searchable. Traces follow one request across all services as a tree of timed spans, showing where time went. Tie them together with a shared trace ID so you can jump from an alert to the exact traces and logs. → *More:* Module 00 — Observability: logs, metrics and traces

Q200. How does distributed tracing work? When a request enters the system it gets a trace ID. Every service creates a span (operation name, start, duration, tags) and passes the trace ID and its span ID to downstream calls in headers, using the W3C `traceparent` format via OpenTelemetry. Spans are sent to a collector, and a backend such as Jaeger or Zipkin stitches them into a timeline. Tracing every request is costly, so systems sample: either a fixed share at the start (head sampling) or after the fact, keeping slow and failed traces (tail sampling). → *More:* Module 09 — Distributed Tracing

Q201. What should you alert on? Alert on symptoms users feel, not on every cause. Error rate, latency and availability measured against your SLOs are good examples. CPU at 80% on its own is not. Use SLO burn-rate alerts: page when the error budget is being used up fast, and open a ticket when it drains slowly. Every page should be urgent, actionable and linked to a runbook. Noisy alerts train people to

ignore them. Dashboards per service usually follow the four golden signals: latency, traffic, errors and saturation. → *More:* Module 00 — Dashboards, alerting, on-call and SLO-based alerts

Q202. Virtual machines vs containers? What does Kubernetes do? A virtual machine virtualizes hardware and runs a full guest OS. It is strongly isolated but heavy and slow to boot. A container shares the host kernel and packages only the app and its dependencies, so it starts in seconds and many fit on one machine. The isolation is weaker. Kubernetes orchestrates containers across a cluster: it schedules pods onto nodes, restarts failed ones, scales them, rolls out new versions, and provides service discovery, configuration and secrets. → *More:* Module 09 — Containers and Kubernetes · Module 00 — Virtual machines, containers and Docker

Q203. How does autoscaling work and what can go wrong? Autoscaling adds or removes instances based on a metric, such as CPU, requests per instance or queue depth, or on a schedule for predictable peaks. Watch out for slow startup: new instances take minutes, so scale on leading signals and keep headroom. Avoid flapping by using cooldowns. Remember the database behind the app tier usually does not autoscale, so more app servers can just mean more connections. Always set a maximum, so a bug or attack cannot scale your bill without limit. → *More:* Module 06 — Auto-Scaling · Module 00 — Autoscaling

Q204. How do you change a database schema or migrate data with zero downtime? Use the expand-and-contract pattern. First expand: add the new column or table so that old code still works. Deploy code that writes to both old and new, then backfill existing rows in small batches. Switch reads to the new structure and check that the results match. Only then contract: stop writing the old one and drop it. Each step can be rolled back on its own, and you never ship a change that breaks the version currently running. → *More:* Module 09 — Capstone - The Zero-Downtime Migration

Algorithms and data structures

Q205. ⋆ What are Bloom filters, HyperLogLog and count-min sketch used for? They are probabilistic structures that trade a little accuracy for huge memory savings. A Bloom filter answers "have I seen this?" with no false negatives but some false positives. It skips disk reads for absent keys in LSM stores and catches already-crawled URLs. HyperLogLog estimates the number of distinct items, such as unique visitors, in about 12 KB with roughly 1% error. A count-min sketch estimates how often each item appears, and it may overcount but never undercounts. It is used for heavy hitters and rate counts. → *More:* Module 00 — Probabilistic data structures and Merkle trees · Module 19 — HyperLogLog internals · Module 19 — Count-min sketch with conservative update

Q206. ⋆ What is a Merkle tree and what is anti-entropy? A Merkle tree is a tree of hashes: leaves hash data blocks, and each parent hashes its children, so the root summarizes everything. Two replicas compare root hashes, and if they differ they walk down only the branches that differ. That finds the out-of-sync keys while transferring very little data. Anti-entropy is the background repair process that uses this comparison to make replicas match again, in Dynamo and Cassandra for example. Git and blockchains use the same structure to verify content. → *More:* Module 00 — Probabilistic data structures and Merkle trees

Q207. ⋆ Geohash vs quadtree: how do you find nearby places? A geohash encodes latitude and longitude into a short string, where a longer shared prefix means a closer location. You can index it in any database and search by prefix. Points near a cell edge can be close but have different prefixes, so you also search the 8 neighbouring cells. A quadtree recursively splits a square into four whenever it holds too many points. It adapts to density, with small cells in cities and big ones in deserts, and it usually lives in memory. Other common choices are S2 (Google) and H3 (Uber hexagons). → *More:* Module 00 — Geospatial indexing: geohash and quadtree · Module 15 — Design 1 — Proximity Service

Q208. ⋆ How do you generate unique IDs in a distributed system? UUID vs Snowflake vs database sequence? A database auto-increment is simple, compact and ordered, but one database is a bottleneck and a single point of failure. You can ease that with ranges handed out in blocks. A random UUIDv4 needs no coordination, but at 128 bits it is large and random, which scatters B-tree inserts. UUIDv7 puts a time prefix first to fix the ordering. A Snowflake ID is 64 bits: a 41-bit millisecond timestamp, a 10-bit machine ID and a 12-bit sequence. That gives roughly time-sorted IDs, up to 4,096 per millisecond per machine, with no coordination, as long as clocks do not go backwards. → *More:* Module 13 — Case 1 — Design a Unique ID Generator · Module 00 — Unique ID generation

Q209. How do you find the top K most frequent items in a stream (heavy hitters)? For exact results at moderate scale, count with a hash map and keep a min-heap of size K, giving O(n log K). At very high scale, use a count-min sketch for approximate counts plus a heap, or the Space-Saving algorithm, which tracks only a fixed number of counters. Shard the counting by key, pre-aggregate in each window (per minute), and merge the partial top-K lists. For "last hour" or "last 24 hours" questions, use sliding windows in a stream processor, and run a batch job if exact numbers are needed later. → *More:* Module 19 — Heavy hitters: Misra-Gries, Space-Saving and streaming top-k

Q210. What is an inverted index, and how does a search engine rank results? An inverted index maps each term to a posting list of the documents (and positions) that contain it, like the index at the back of a book. A query looks up each term's list and intersects or unions them. Ranking scores each match, classically with TF-IDF or BM25: a term counts more if it appears often in this document and rarely across all documents. Large search systems shard the index by document, query all shards in parallel and merge the top results. → *More:* Module 19 — Inverted index · Module 19 — Ranking with TF-IDF and BM25

Q211. What is a skip list, and why does Redis use one for leaderboards? A skip list is a sorted linked list with extra "express lane" levels. Each node is promoted to higher levels at random, so search, insert and delete take O(log n) on average. Redis sorted sets combine a skip list, ordered by score, with a hash table, member to score. That makes "update score," "get rank" and "top 10" all fast, which is exactly what a real-time leaderboard needs. Skip lists are also easier to make concurrent than balanced trees, and LSM memtables often use them. → *More:* Module 19 — Skip lists · Module 15 — Design 4 — Real-Time Gaming Leaderboard

Q212. How does vector search work (for embeddings / RAG)? Text or images are turned into embeddings, which are vectors where similar meaning gives nearby points. Search then means finding the nearest vectors to the query vector. Exact search compares against everything, which is too slow at millions of items. So systems use approximate nearest neighbour (ANN) indexes. HNSW builds a layered graph that is walked greedily. IVF clusters vectors and searches only the closest clusters. Product quantization compresses vectors to save memory. You trade a little recall for big speed gains, and you shard the index as it grows. → *More:* Module 19 — Vector search with HNSW, IVF and product quantization · Module 11 — Vector Database Sharding

More design prompts

Designs added to the book after Part A was numbered. Because numbering runs through Parts A and B, they continue after Q212 here instead of joining their topic group above.

Q213. Design a pub-sub service (Google Cloud Pub/Sub / Amazon SNS)

- **Core:** store each message once in a replicated per-topic log (ack after a majority write); each subscription is ack state over that log (floor, ranges, leases), so fan-out to thousands of subscriptions costs no copies.
- **Probes:** per-message redelivery and dead-lettering; push windows (AIMD) isolating slow endpoints; ordering keys; seek/replay. When to choose Kafka instead.

→ *Full design:* Module 23 — Case 3: Design a Pub-Sub Service

Q214. Design an online learning platform (Coursera / Udemy)

- **Core:** CDN video behind entitlement-signed URLs; progress as monotonic merges (max, OR of a watched bitmap) so retries and devices never go backwards; server-side quizzes with pools, seeds and server clocks; async sandboxed code grading.
- **Probes:** the Sunday-deadline spike (pre-scale, deadline = upload time); an append-only grade ledger for regrades; verifiable certificates.

→ *Full design:* Module 25 — Case 4: Online Education Platform

Q215. Design Spotify Wrapped (a yearly personalised recap)

- **Core:** batch over a year of immutable listening events with reusable yearly aggregates; one re-runnable job per story writing a row-per-user store; read-only launch-day service.

- **Probes:** shuffle cost, distribution checks before release, pre-scaling and load tests for the first hours, pre-rendered vs on-demand share images.

→ *Full design:* Module 25 — Case 5: Spotify Wrapped

Q216. Design a communications API platform (Twilio)

- **Core:** accept, durably log and return `queued`; drain millions of logical per-sender token-bucket queues to carriers with a fair scheduler; idempotency keys on every send.
- **Probes:** webhook delivery that isolates bad endpoints; billing that never acts on missing balance data; cells per account group.

→ *Full design:* Module 25 — Case 6: Twilio-Style Communications API Platform

Q217. Design an AI code assistant (GitHub Copilot / Cursor)

- **Core:** a ~250 ms keystroke-to-suggestion loop: debounce and cancel on the client, fill-in-the-middle prompts with stable context first for prefix-cache hits, a small regional model with speculative decoding.
- **Probes:** repository index kept fresh by Merkle sync; tenant isolation including caches; acceptance and retention telemetry gating every change.

→ *Full design:* Module 24 — Case 1: AI-Powered Code Assistant

Q218. Design an LLM customer support bot

- **Core:** RAG over versioned policies plus tools that act with the customer's delegated token; one-owner conversation state machine; warm handoff to human agents with the CRM as case of record.
- **Probes:** PII redaction before the model; promises only from tool results; resolution rate without repeat contact; LLM-judge plus human-audited evaluation loop.

→ *Full design:* Module 24 — Case 2: LLM Customer Support Bot

Q219. Design the data platform for ML training (lakehouse, labeling, loaders)

- **Core:** lakehouse tables with atomic snapshot commits; datasets as manifests of snapshots, label sets and code (reproducible and deletable); large shards streamed with a two-level shuffle so GPUs never wait.
- **Probes:** labeling cost and quality; right-to-erasure vs immutable datasets; drift (PSI) and training/serving skew.

→ *Full design:* Module 24 — Case 3: Data Infrastructure for AI/ML

Part C — How to answer a system design question in 45 minutes

A system design round is a structured conversation, not a quiz. The interviewer wants to see you turn a vague prompt into a working design, make trade-offs out loud and go deep where it matters. A fixed plan gets you through it, so you never wonder what to do next. Module 00's interview section walks through the steps in detail. Module 01's Five Questions give you a checklist for the design itself.

The minute-by-minute plan

Minutes 0–5: requirements. Do not draw yet. Ask who the users are and which three or four features matter most, and write them down as functional requirements. Then ask about the qualities: how many users, read-to-write ratio, latency target, availability, and whether stale data is acceptable. Say what you will leave out. *Say:* "To keep this focused, I'll cover posting and the home timeline, and leave out search and ads. Does that match what you want?"

Minutes 5–10: estimates. Spend just a few minutes on rough QPS, peak QPS, storage per year and bandwidth. Then connect each number to a decision. *Say:* "That's about 50,000 reads a second against 500 writes, so the read path needs a cache, and one database primary can handle the writes."

Minutes 10–15: API and data model. List the three to five main endpoints with inputs and outputs. Name the core entities, their keys and how they are accessed, then choose a store that fits the access pattern and give the reason.

Minutes 15–25: high-level design. Draw the simplest design that meets every functional requirement: clients, load balancer, stateless services, cache, database, queue, object storage. Walk one write and one read through it end to end. Check it against the requirements before you add anything clever.

Minutes 25–40: deep dives. Choose the two or three hardest parts, or ask the interviewer which they care about. Typical ones are the shard key and hot keys, fan-out, consistency at the critical step, idempotent payments, and what happens when each dependency fails. Go through every arrow and ask "what if this is slow, fails once, or is down for ten minutes?" Give at least two options for each choice and say why you picked one.

Minutes 40–45: wrap-up. Summarize the design in a few sentences. Name its weak spots, what you would monitor, and what changes at 10× scale. Interviewers often remember this part best.

Phrases that help

- "Let me check I've got the requirements right before I design anything."
- "There are two options here. A is simpler, B scales further. Given our numbers I'd pick A, and here's when I'd switch."
- "This box is a single point of failure. Here's how I'd make it redundant."
- "I'm assuming X. Let me know if that's wrong."

Common mistakes

- Drawing boxes before agreeing on requirements and scale.
- Thinking in silence. The interviewer can only grade what you say.
- Naming products without reasons ("use Kafka"). Tie each choice to a requirement or a number.
- Over-engineering: microservices, three databases and a service mesh for 100 requests a second.
- Going deep on one component and never finishing the end-to-end design.
- Ignoring failure, consistency and the non-functional requirements, which usually decide the design.
- Ignoring hints. When the interviewer steers you, follow.
- Not going back to the requirements and estimates at the end to show the design meets them.

Practise with a timer. Take one prompt from Part A, say each phase out loud, and stop at 45 minutes. After a few rounds the plan becomes automatic, and your attention goes to the design itself.

→ *More:* Module 00 — How to approach a system design interview · Module 01 — The Five Questions · Module 01 — Back-of-Envelope Math

Part D — Preparing and practising

Part C covers the 45 minutes; this part covers everything around them, linking to Part C and Module 01 rather than repeating them. Two terms: a **signal** is evidence the interviewer writes down ("asked for the read:write ratio before choosing a store"); a **rubric** is the fixed list of qualities every candidate is scored on, so interviewers grade alike (Google's hiring guide calls this structured interviewing). You never see the rubric, so leave a signal on every likely line of it.

Our method vs the RESHADED-style checklist

Courses often teach a lettered checklist, best known **RESHADED** from *Grokking the Modern System Design Interview* (only its step names are used here). Our method is Step 0 plus the five questions (move, live, fast, fail, grow) from Module 01 inside Part C's timed plan, and every case study (Modules 12–25) uses the same headings.

Course step (name only)	What it means	Our equivalent	Case-study heading
Requirements	Features, qualities (scale, latency, availability, consistency), out of scope	Part C min 0–5	`1. Requirements and numbers`
Estimation	Rough QPS, peak, storage, bandwidth, machines, each tied to a decision	Step 0, Module 01 "Back-of-Envelope Math"	`1. Requirements and numbers`
Storage schema (optional)	Entities, keys, access patterns, the store that fits	Five Questions Q2 (live)	`4. Data model`
High-level design	Simplest boxes and arrows meeting every feature	Q1 (move) gives the arrows	`5. The architecture, annotated`
APIs	The few client calls with inputs, outputs, key headers	Part C min 10–15	`3. API`
Detailed design	Deep dives on the hardest parts, options compared	Q3 (fast), Q4 (fail), Q5 (grow)	`6. Deep dives`, `7. The failure walk`
Evaluation	Check the design against the agreed requirements, name weak spots	Part C wrap-up, "every pattern is a trade"	`8. Trade-offs`
Distinctive component	The one thing that makes this problem unlike a generic web app	Module 01 "find the problems"	Named in `2. The five-questions pass`

Where we differ, and why:

- **API before the diagram.** Three to five calls take two minutes and pin down what the system does. If the interviewer prefers the other order, follow them; nothing depends on it.
- **Failure is its own step** (Q4 and the failure walk), not folded into "detailed design". It is the most-skipped step and the most-probed one.
- **Questions, not stages.** The five questions say *what to look for* and work inside any stage, so the method survives an interviewer who reorders the round.
- **Name the distinctive component early** (around minute 10, from the five-questions pass), not at the end, so deep-dive time goes to it.

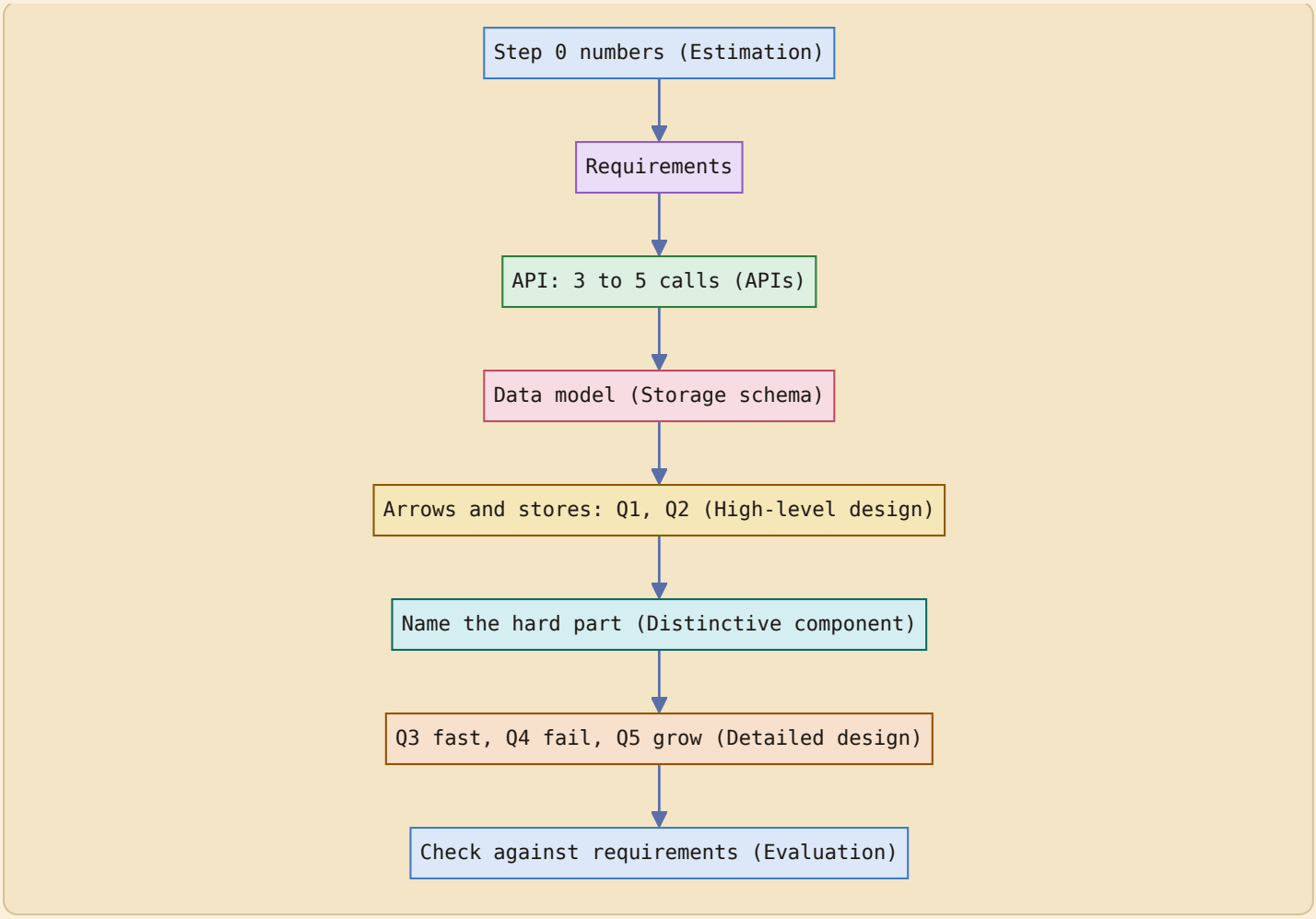


Figure — Our flow, with the course step names in brackets.

Do's and don'ts per phase

Part C's "Common mistakes" covers the general errors (silence, products without reasons, over-engineering, ignoring hints). This table adds the phase-specific ones and the signal each phase is meant to produce.

Phase	Do	Don't	Signal collected
Before the round	Ask the recruiter the round type (product, infrastructure, API, client, ML) and the drawing tool. Practise twice in that tool. Prepare a 60-second story of a system you built, with one number and one trade-off.	Assume all design rounds are alike (client rounds: Module 18). Cram a new topic the night before.	—
Requirements	Restate the prompt. Ask five to eight questions whose answers change the design (scale, staleness, money or safety involved). Propose assumptions for the rest. Write them where you can point to them later.	Ask questions that change nothing ("which language?"). Ask twenty questions.	Can you turn a vague prompt into a clear problem?
Estimates	One significant figure. Estimate peak (state the factor, often 2–5× average). End each number with a conclusion.	Spend ten minutes. Insist on estimates after the interviewer says skip them.	Feel for scale; numbers drive choices.
API and data model	Three to five calls with one key detail each (cursor, idempotency key). Name keys and the main access pattern, then choose the store from it.	Write every column. Say "SQL or NoSQL, both work".	Clean interfaces matched to access patterns.
High-level design	Label each arrow sync or async. Walk one read and one write. Say the distinctive component aloud.	Add a box no requirement or number asked for.	A complete, working, explainable system.
Deep dives	Offer the interviewer a menu of two or three hard parts. Reuse your numbers to size shards and caches. Time-box each dive to about 7 minutes.	Let one dive run 15 minutes without a decision.	Depth, trade-offs, design for failure.
Wrap-up	Name two real weak spots, what you would monitor, what breaks first at 10×.	Add a major component in the last two minutes. Claim there are no weaknesses.	Judgment and self-awareness.

Check in every 8–10 minutes ("Is this the right level of detail?"), and when you do not know something, reason aloud instead of bluffing.

Why engineers fail: trap patterns

Strong engineers rarely fail for lack of knowledge; they fall into one of these patterns:

Trap	What the interviewer sees	Fix
Early dive	Partitioning a table in minute two.	Requirements and Step 0 first; breadth before depth.
Memorised architecture	A famous blog diagram that collapses when one requirement changes.	For every case you study, write which requirement caused each box; practise changed requirements.
Happy path only	Every call succeeds, nothing is slow.	Run Q4 on every arrow: slow, fails once, down ten minutes, what the user sees (Module 05).
Trade-off dodge	"Either would work."	"A is simpler, B scales further; given our numbers, A, and I switch when X."
Rabbit hole	25 minutes on the ID generator, no end-to-end design.	7-minute dives; park items out loud.
Hint deflection	"That's handled," then moving on.	Every question points to a rubric line: stop and answer fully.
Forgotten requirement	"Never double-charge" agreed in minute 3, never mentioned again.	Read each requirement in the wrap-up and point to what meets it.
Numbers that decide nothing	A neat estimate followed by an unrelated design.	Every number ends in "so…".
Level mismatch	A staff candidate with no cost, migration or ownership view; a mid candidate who never finishes.	See "Level expectations" below.
Stress collapse	Good start, one hard question, then guessing.	Timed mocks with strangers; a recovery line: "Let me restate what we have, then answer."

Candidates who pass drive the conversation, make every box and number earn its place, state each choice's cost with the choice, spend their depth on the distinctive component, and close by tying the design to the requirements.

4-, 8- and 12-week prep plans

Pick a plan. 12 weeks if you cannot yet explain, unaided, what a load balancer, cache, replica, shard and queue each do and cost. 8 weeks if you know the blocks but have done fewer than five full designs. 4 weeks if you have prepared before and need to sharpen.

All plans assume 8–10 hours a week and rest on two findings: **retrieval practice** (recalling before looking) beats re-reading for long-term memory (Roediger and Karpicke, 2006), and **deliberate practice** at the edge of your ability with quick feedback builds skill (Ericsson et al., 1993), which is what a scored mock is. So each week mixes reading with recall drills (Part B answered aloud first) and, from mid-plan, timed designs and scored mocks. "Q" numbers refer to Part A.

4 weeks

Week	Read	Practise	Checkpoint
1	Module 01; TL;DRs of 02–06	Part B: Performance, Load balancing and caching, Databases. Timed Q1 (vs Module 13 Case 2), Q3 (vs Module 08 Rate Limiting)	Five-questions pass on any prompt in under 3 min
2	Module 12 Designs 1–3; Module 07 Saga, Quorum	Part B: Replication, Consistency, Reliability. Timed Q18, Q44, Q73. One mock	Defend a shard key, a fan-out choice, an idempotent payment
3	By target company: 14, 17 or 21	Q38, Q52, Q70 (or the company's ★ questions). Two scored mocks	No rubric line below 2
4	Weakest rubric line; Module 09 skim; level table below	Three mocks with different people; redo the worst prompt	Two mocks in a row at target level

8 weeks

Week	Read	Practise	Checkpoint
1	Module 01; Module 00 Part 1	Ten estimation drills on Part A prompts	Each under 3 min, each ending in a conclusion
2	02, 03; Module 00 Part 3	Part B: Databases, Replication. Untimed Q1, Q2 vs 13 Case 2	When to replicate, when to shard, which key
3	04, 05	Part B: Caching, Reliability. Timed Q6 (vs 21 Case 4)	Failure walk without being asked
4	06, 07, 08	Timed Q3, Q9 (vs 13 Case 4). First scored mock	Baseline rubric score
5	12 Designs 1–3; 14 Cases 1, 3	Timed Q18, Q38, Q52. Scored mock	Distinctive component named by minute 10
6	Two cases each from 15, 16, 17	Timed Q47, Q84, Q70. Scored mock	No rubric line below 2
7	10, 11 TL;DRs; 20 Cases 1, 5; 21 Case 3; API section below	Timed Q35, Q98, one API-design prompt. Two scored mocks	One data, one AI, one API prompt at target level
8	Two weakest areas; level table	Three mocks, different partners; redo two worst prompts	Two mocks in a row at target level

12 weeks

Weeks	Read	Practise	Checkpoint
1–2	Module 00 Parts 1–2	Part B: Performance, Networking and APIs, five a day aloud	Trace a request browser → database → back, naming every hop
3	Module 00 Parts 3–4; Module 01	Part B: Databases. Ten estimation drills	Choose a store from an access pattern, with reasons
4	02, 03	Untimed Q1, Q2 vs 13 Case 2	First complete end-to-end design
5	04, 05	Untimed Q4 (vs 13 Case 1), Q6	Failure walk on every arrow
6	06, 07	Timed Q5 (vs 13 Case 3). Part B: Replication, Consistency	Explain quorums and when a saga is needed
7	08, 09	Timed Q3, Q7, Q9. First scored mock	Baseline rubric score
8	12 all	Timed Q18, Q44, Q73. Scored mock	Defend fan-out, dispatch, payment choices
9	14, 15	Timed Q38, Q52, Q47, Q49. Scored mock	No rubric line below 2
10	16, 17; 19 algorithms your targets use	Timed Q83, Q89, Q70, Q78. Two scored mocks	Two levels deep on one algorithm
11	10, 11, 20, 21; 18 if a client round is possible; API section below	Timed Q24, Q35, Q98, one API prompt. Two scored mocks	Data, AI and API prompts at target level
12	Weak areas; level table; Part C	Three or four mocks; redo three worst prompts; rest the last day	Two mocks in a row at target level

Tune to the company: swap timed designs for its ★ and "asked at" questions in Part A. Payments companies lean on Module 17; social products on 12, 14, 20; infrastructure teams on 13, 16, 19; AI product teams on 11 and 21.

Keep an error log: one line per mistake after every design or mock (prompt, what went wrong, trap from the table above, fix). Read it before each mock; most people repeat the same three mistakes.

Mock-interview protocol and self-scoring rubric

Partners, best first: someone who has interviewed candidates; a peer preparing for the same round (swap roles, since interviewing teaches what scores); a recording of yourself, scored the next day; an AI model as interviewer (good for follow-up drills at any hour, but its praise is not a reliable score, so still self-score).

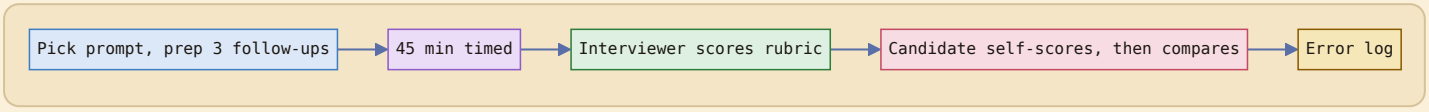


Figure — One 60-minute mock session.

Step	Interviewer	Candidate
Prepare (5 min)	Pick a Part A prompt unseen by the candidate; read its outline and full case. Prepare three follow-ups: 10× scale, a dependency down for ten minutes, one changed requirement.	—
Run (45 min)	Give a one-sentence prompt. Answer questions with sensible numbers, "your call" otherwise. Interrupt at least three times with the follow-ups. Note positive and negative signals with the minute. Stop at 45 even if unfinished.	Run the round with a visible timer.
Score (5 min)	Score the rubric, then give feedback: the one thing that helped most, the one that hurt most, the rest.	Score yourself *before* seeing their score; large gaps show where your self-image is off.
Log (5 min)	—	One error-log line per mistake.

Interviewer prompts for untrained partners: "Why that rather than [alternative]?" (choice without a reason) · "What if [component] is down for ten minutes?" (arrow without failure story) · "What's the simplest thing that would work?" (stuck over a minute) · "Ten minutes left — where do you want to spend them?" (minute 35, no wrap-up) · "With another hour, what would you change?" (close).

Rubric. Score each line 1–4. No company publishes its full rubric; this one reflects commonly scored qualities and is a practice tool.

Line	1	2	3	4
Requirements	Draws at once	Misses the deciding quality	Clear lists, scope agreed	Finds and confirms the requirement that decides the design
Estimation	None or unused	Errors, no conclusions	Quick, roughly right, each tied to a decision	Reused in deep dives to size shards, caches, machines
High-level design	Incomplete	Works, with unexplained boxes	Simple, complete, read and write walked	Minimal, sync/async labelled, hard part named
API and data	Store by habit, no API	Weakly tied to access patterns	Matches access patterns	Keys, indexes, pagination, idempotency chosen with reasons
Depth	Box level only	Deep on an unimportant part	Deep on the hard part, two options	Mechanisms explained, limits quantified
Trade-offs	No alternatives	Alternatives, no choice	Choice with reasons	Plus the condition that would change it
Failure and operations	Happy path	"Add redundancy"	Failure walk with user impact	Plus monitoring, rollout, recovery
Communication	Silent or chaotic	Hard to follow, ignores hints	Clear agenda, follows hints	Drives, adapts, ends on time with a summary

Reading the total (8–32), this book's guide, not any company's: under 16, keep studying; 16–23, around mid-level if no line is 1; 24–28, senior; above 28 with depth, trade-offs and failure at 4, staff. The **lowest line** matters more than the total, because panels often reject for one clear weakness. Fix it first.

How AI changed system design interviews and roles

New prompts and priorities. ChatGPT-like services, RAG bots, code assistants, support agents, recommendation systems and ML data platforms joined the standard list (Part A Q98, Q100; Module 11; Module 21 Case 3). They shift the priorities:

New priority	What to discuss	Where taught
Cost per request	GPU scarcity, batching, semantic caching, routing easy requests to small models, capacity in GPU-hours	Module 11: Model Serving, GPU Auto-Scaling, Semantic Caching
Latency shape	**Time to first token** (delay before text starts) and tokens per second; stream the response	Module 02 Server-Sent Events
Non-determinism	**Evaluation**: a fixed graded test set run before every model or prompt change, plus quality monitoring in production	Module 21 Case 3
New failure modes	**Prompt injection** (text that tries to override instructions), private-data leaks, hallucination, provider limits and outages: guardrail filters, a gateway with fallbacks, circuit breakers	Module 11 LLM Gateway; Module 05
Data pipelines	Fresh chunked embeddings for retrieval, versioned datasets for training	Module 10, Module 11 RAG Pipeline, Feature Store

Classic prompts remain, often with an AI feature added ("add smart replies to the chat app"). The method is unchanged: the AI part usually becomes the distinctive component.

New interview formats. Real-time AI help undermined remote rounds, and companies went different ways (as reported in 2025): Google said engineering candidates would again have at least one in-person interview; Meta was reported to be piloting coding rounds with a built-in AI assistant; Canva expects candidates in several engineering roles to use AI tools and assesses how; others ask candidates to declare they will not use AI. For design rounds this means:

- **Reasoning outweighs recall.** A model can draw a textbook architecture, so interviewers push on judgment: why this choice for these numbers, what breaks, what changes if one requirement does.
- **If AI is allowed,** use it for mechanical work (arithmetic, an API draft, a list of failure cases), then check its output aloud. Accepting a wrong suggestion unnoticed is a strong negative signal.
- **If AI is not allowed,** hidden use is misconduct. Ask the recruiter which format applies.

New roles. Loops for ML engineers, ML-infrastructure engineers and **AI engineers** (who build products on top of models) now include design rounds: classic systems, ML systems (training pipeline, feature store, serving), or LLM applications (retrieval, agents, evaluation, guardrails). A separate **ML system design** round adds problem framing (what to predict, how to measure it), training data, offline and online evaluation, and model monitoring. For these roles add Modules 10, 11 and 21 Case 3 to your plan and do at least two AI prompts timed. On the job, as assistants write more routine code, choosing designs, reviewing generated code against system invariants, and owning reliability and cost become a larger share of the work, and interviews follow.

API-design interview vs system design interview

Companies whose product is an API (payments, communications, developer platforms) often run a separate **API-design round**.

	System design round	API-design round
Question	"Design a system that does X at scale."	"Design the interface developers use to do X."
Main output	Components, stores, scaling, failure plans	Resources, endpoints, payloads, errors, evolution rules
The "user"	End users, the business	Developers outside your company who cannot read your code
"Scale" means	Requests per second, data size	Many clients, many versions, years of compatibility
Deep dives	Sharding, caching, consistency, failure	Pagination, partial updates, idempotency, concurrency, errors, auth, rate limits, versioning, webhooks
Fails on	No trade-offs, no failure handling	Verbs in URLs, inconsistent names, no pagination, unsafe retries, breaking changes, unactionable errors
Taught in	Modules 01–07, 12–21	Module 00 "API design basics", "API styles"; Module 08 Gateway, Rate Limiting, Cursor-Based Pagination, Versioning, Authorization; Module 05 Idempotency; Module 02 Webhooks

Round checklist: use cases and callers → resources and relationships → standard methods (custom actions only when a state change does not fit an update) → field shapes → pagination, filtering, sorting → safe retries and concurrency → errors → auth, scopes, rate limits → versioning → events. Requirements are **use cases** ("a partner lists open tasks assigned to a user, newest first"), not traffic. A short backend sketch is welcome but must not take over.

Worked answer: public REST API for a team task tracker

Prompt: "Design the public REST API for a team task tracker. Workspaces have projects, projects have tasks, tasks have comments. Partners build integrations on it."

1. Use cases. Partner servers list projects and tasks (filter by status and assignee, sort by last update); create tasks without duplicates on retry; change a few fields without overwriting a colleague's concurrent edit; add and list comments; learn about changes without polling. Out of scope: attachments, cross-workspace search, billing. Auth: **OAuth 2.0** access tokens (a user grants an app limited, revocable access).

2. Resources. Nested by ownership; IDs are opaque server-generated strings (`tsk_8Kq2…`), not sequential integers, so they leak no counts, cannot be enumerated, and let the ID scheme change later. Because task IDs are globally unique, `/v1/tasks/{t}` is a short alias.

```
GET    /v1/workspaces/{ws}/projects
GET    /v1/workspaces/{ws}/projects/{p}/tasks?status=open&assignee=usr_42&order_by=-
updated_at&page_size=50&page_token=...
POST   /v1/workspaces/{ws}/projects/{p}/tasks       Idempotency-Key header
GET    /v1/tasks/{t}
PATCH  /v1/tasks/{t}                                If-Match header
DELETE /v1/tasks/{t}
GET    /v1/tasks/{t}/comments?page_size=50&page_token=...
POST   /v1/tasks/{t}/comments                       Idempotency-Key header
POST   /v1/workspaces/{ws}/webhooks
```

Conventions: nouns in paths, HTTP methods as verbs, plural collections, one casing everywhere (`snake_case`), UTC timestamps in RFC 3339 (`2026-10-15T09:30:00Z`). A custom action such as `POST /v1/tasks/{t}:complete` is justified only if completing does more than set one field (closes subtasks, notifies watchers).

3. The task resource.

```
{
  "id": "tsk_8Kq2...", "project_id": "prj_3Lm...",
  "title": "Fix login timeout", "description": "...",
  "status": "open",              // open | in_progress | done, more may be added
  "assignee_id": "usr_42", "due_date": "2026-11-01", "labels": ["auth"],
  "created_at": "...", "updated_at": "...", "etag": "\"v7\""   // read-only
}
```

Documenting that enum values may be added (clients must tolerate unknown ones) prevents a whole class of breaking changes.

4. Lists. Cursor pagination (Module 08): the client sends `page_size` and the previous `page_token`; the server returns `next_page_token`, empty on the last page. The token is opaque and signed (it encodes the last item's sort key and ID plus the filter), so clients cannot edit it. Default page size 50, maximum 200, larger values clamped rather than rejected. Offsets (`?page=7`) skip or repeat items when tasks are inserted mid-scan and get slow at depth; a cursor resumes exactly after the last item seen. Google's AIP-158 sets the same rules for its APIs. Allow only a few `order_by` keys, each backed by an index.

5. Safe creates. If a POST times out, the client cannot know whether the task exists. It sends an `Idempotency-Key` (a UUID generated once per logical operation and reused on retries).

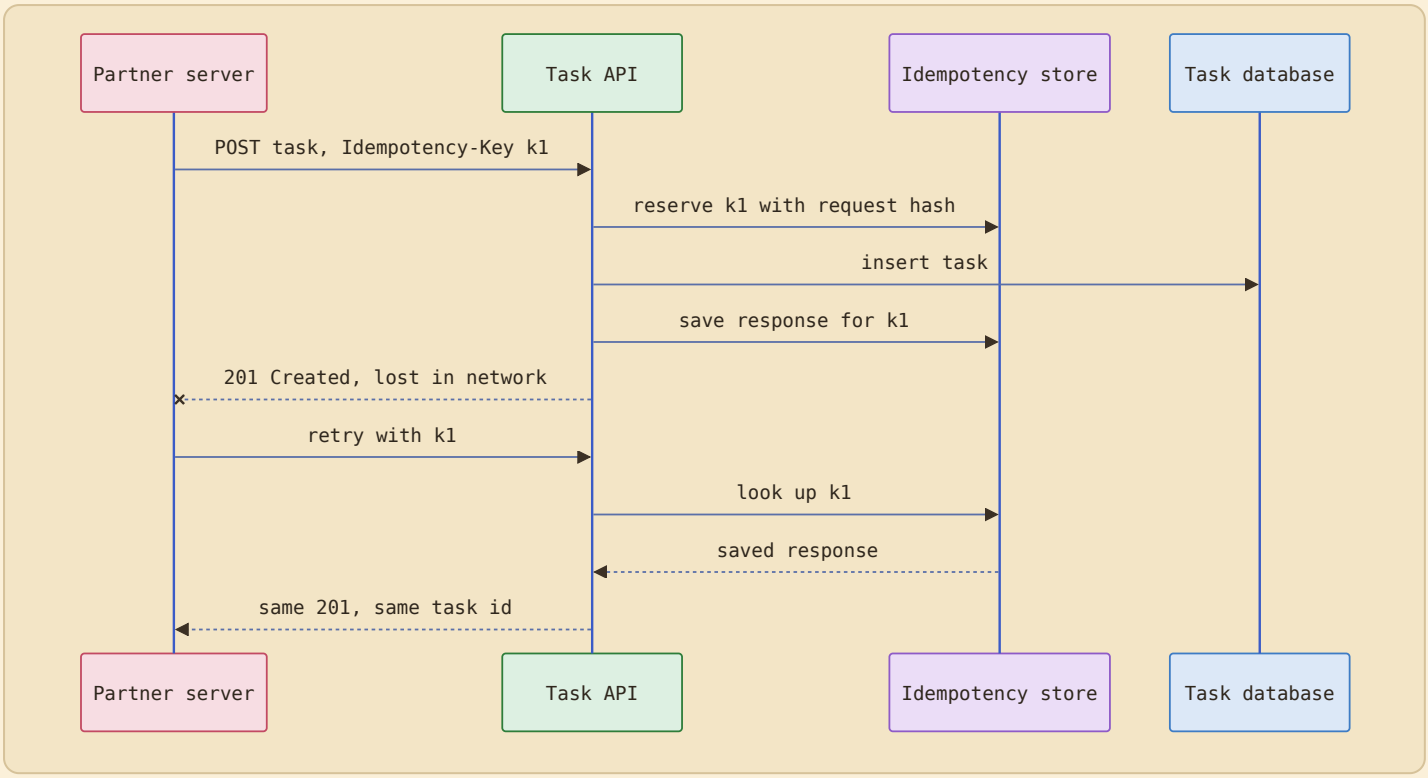


Figure — A retried create returns the stored response instead of a second task.

Server rules: keep key, request hash and response for at least 24 hours; same key and body → stored response; same key, different body → error (client bug); first request still running → 409, retry shortly. Stripe works this way (keys kept at least 24 hours, mismatched parameters rejected); an IETF Internet-Draft is standardising the header. GET, PUT and DELETE are idempotent by HTTP definition and need no key. Storage side: Module 05 Idempotency.

6. Updates without lost changes. `PATCH` with a **JSON Merge Patch** body (RFC 7396): only the fields to change, `null` clears a field, so changing status cannot wipe a description the client never loaded (Google-style APIs use an `update_mask` instead). Every task has an **ETag** (a version tag that changes on every write); the client sends it in `If-Match`, and a stale one gets `412 Precondition Failed`, after which the client re-reads and retries. This is HTTP's conditional-request mechanism (RFC 9110) and needs no locks.

7. Errors. One body shape everywhere, the **problem details** format (RFC 9457): `type` (a URI naming the error kind), `title`, `status`, `detail` ("due_date must not be in the past"), a per-field `errors` list, and a `request_id` for support. Codes: 400 malformed · 401 unauthenticated · 403 forbidden (or 404 when

revealing existence would leak data) · 409 conflict · 412 stale ETag · 422 validation · 429 rate limited · 5xx server fault, safe to retry with backoff.

8. Auth and limits. Tokens carry **scopes** (named permissions such as `tasks:read`, `tasks:write`) so a reporting integration cannot delete. Every request is checked against the token's workspace. A per-token token bucket (Module 08 Rate Limiting) answers `429` with `Retry-After` (RFC 6585, RFC 9110), and responses report remaining budget so clients slow down early.

9. Webhooks. Partners register a URL and event types (`task.created`, `task.updated`, `comment.created`). Each delivery is signed with a per-partner secret, retried with exponential backoff for about a day, carries an event ID for de-duplication (delivery is at-least-once) and `updated_at` to discard stale events (no ordering promise). A "list recent events" endpoint lets a partner catch up after downtime (Module 02 Webhooks).

10. Versioning. Major version in the path (`/v1/`), easy to route and spot in logs (Module 08 API Versioning). Within a version only additive changes: new optional fields, endpoints, enum values. Removing or renaming a field, changing a type or making a field required means `/v2/`, run in parallel through an announced deprecation period. Date-pinned per-client versions are the alternative to mention.

11. Behind the API (if time allows). Stateless servers behind a gateway doing auth and rate limits; a relational store sharded by `workspace_id` (every query is workspace-scoped), indexed on `(project_id, status, updated_at, id)`; a key-value idempotency store with 24-hour TTL; task changes written to an outbox table in the same transaction and relayed to the webhook dispatcher, so no webhook fires for a rolled-back change (Part A Q15).

Scoring. Interviewers look for consistent naming, resources that match the use cases, correct methods and status codes, pagination that cannot skip items, safe retries, no lost updates, actionable errors and a clear compatibility rule. With ten minutes left, spend them on idempotency, concurrency and versioning before the database.

Level expectations: mid, senior, staff

Rough equivalents: **mid** ≈ Google L4 / Meta E4 / Amazon SDE II; **senior** ≈ L5 / E5 / SDE III; **staff** ≈ L6 / E6 / Amazon Principal. Cross-company mappings are approximate (sources such as Levels.fyi disagree on some rows), so ask the recruiter which level the loop targets; published ladders such as Dropbox's show what each level owns.

	Mid	Senior	Staff
Who drives	Interviewer guides; candidate completes the design	Candidate drives; interviewer asks follow-ups	Candidate drives and reshapes the problem: which constraints are real, what to cut
Requirements	The obvious functional and non-functional ones	Finds the deciding requirement ("never oversell a seat")	Phasing, build vs buy, business definition of success
Breadth	Complete design with standard components	Right components for the numbers, nothing extra	Fits the organisation: team ownership, existing systems, migration path
Depth	One component, when prompted	Self-chosen depth on the hard part, options compared with numbers	Two or three areas, at least one cross-cutting (consistency model, multi-region, migration)
Trade-offs	Knows them when asked	Raises them unprompted with switch conditions	Ties them to cost, risk, team effort and time
Failure and operations	Replication and retries	Full failure walk, idempotency, degradation, monitoring	Plus blast radius, cells, disaster recovery, on-call load, safe evolution over years
Typical miss	Does not finish or needs heavy help	Correct but generic, no depth where it mattered	An excellent senior answer with no organisational, cost or long-term view

Practice per level. Mid: speed and completeness. Senior: make the deep dive, the trade-off sentence and the failure walk automatic. Staff: add a framing opening (problem, phases) and a closing on migration, cost, operations and first-year risks. Staff roles take several shapes (tech lead, architect, solver, right hand, in Will Larson's terms), but all reach beyond a single component, and your wrap-up should show it. A higher-level candidate giving a lower-level answer may be down-levelled or simply rejected, depending on the company, so prepare for the level you want.

Sources

- Google re:Work, "A guide to structured interviewing for better hiring practices" — https://rework.withgoogle.com/intl/en/guides/a-guide-to-structured-interviewing-for-better-hiring-practices
- Educative, *Grokking the Modern System Design Interview* (step names only) — https://www.educative.io/courses/grokking-the-system-design-interview
- H. L. Roediger III and J. D. Karpicke, "Test-Enhanced Learning", *Psychological Science* 17(3), 2006.
- K. A. Ericsson, R. T. Krampe and C. Tesch-Römer, "The Role of Deliberate Practice in the Acquisition of Expert Performance", *Psychological Review* 100(3), 1993.
- S. Newton, "Yes, You Can Use AI in Our Interviews", Canva Engineering Blog, 2025 — https://canva.dev/blog/engineering/yes-you-can-use-ai-in-our-interviews
- Meta AI-assisted coding interview pilot (404 Media report, July 2025, via Business Today) — https://www.businesstoday.in/amp/technology/news/story/meta-to-test-job-applicants-with-ai-assisted-coding-interviews-amid-ai-expansion-plans-487200-2025-07-31
- CNBC (syndicated by NBC New York), "Meet the 21-year-old helping coders use AI to cheat in Google and other tech job interviews" (2025; covers Google's return to in-person rounds) — https://www.nbcnewyork.com/news/business/money-report/meet-the-21-year-old-helping-coders-use-ai-to-cheat-in-google-and-other-tech-job-interviews/6178911/
- Google AIP-158 "Pagination" — https://google.aip.dev/158 · AIP-134 "Standard methods: Update" — https://google.aip.dev/134
- Stripe API reference, "Idempotent requests" — https://docs.stripe.com/api/idempotent_requests
- IETF HTTPAPI WG, "The Idempotency-Key HTTP Header Field" (Internet-Draft) — https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/
- RFC 9110 *HTTP Semantics* (2022); RFC 7396 *JSON Merge Patch* (2014); RFC 9457 *Problem Details for HTTP APIs* (2023); RFC 6585 *Additional HTTP Status Codes* (2012); RFC 6749 *OAuth 2.0* (2012); RFC 3339 *Timestamps* (2002).
- Dropbox Engineering Career Framework — https://dropbox.github.io/dbx-career-framework/
- W. Larson, "Staff archetypes" — https://staffeng.com/guides/staff-archetypes/
- Levels.fyi (approximate level comparisons) — https://www.levels.fyi/

Sources of the question lists

Question wording and frequency only; every answer above is written for this book.

- donnemartin/system-design-primer — "System design interview questions with solutions" and "Additional system design interview questions"
- ashishps1/awesome-system-design-resources — "System Design Interview Problems" (Easy / Medium / Hard)
- checkcheckzz/system-design-interview — "Products and systems" question list
- systemdesign.io 45-question list (posted on r/leetcode and HackerNoon, with company tags from LeetCode Discuss threads)
- LeetCode Discuss interview-experience posts (Flipkart, Swiggy, Atlassian, PhonePe, Meta and others)
- Course and book tables of contents (titles only): Grokking the System Design Interview; Alex Xu, *System Design Interview* Vol 1 and 2